



$$\psi_k = \sum_{\mu} c_{\mu k} \chi_{\mu} \quad (\text{LCAO})$$

$$\hat{F} \varphi_i = \varepsilon_i \varphi_i$$

$$S_{\mu\nu} = \int \chi_{\mu}^*(\mathbf{r}) \chi_{\nu}(\mathbf{r}) d\mathbf{r}$$

С. М. Пономаренко

Квантово-механічні методи обчислення Використання PySCF

$$\left[-\frac{1}{2} \nabla_i^2 + \sum_{j \neq i} \frac{1}{r_{ij}} + V_{\text{nuc}}(i) \right] \varphi_i = \varepsilon_i \varphi_i$$
$$\Psi = \det \begin{pmatrix} \varphi_1(1) & \varphi_2(1) & \cdots & \varphi_N(1) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_1(N) & \varphi_2(N) & \cdots & \varphi_N(N) \end{pmatrix}$$

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

С. М. Пономаренко

Квантово-механічні методи обчислення Використання PySCF

*Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського як
навчальний посібник для здобувачів ступеня магістра за спеціальностями
Е6 «Прикладна фізика та наноматеріали»*

КИЇВ
КПІ ім. Ігоря Сікорського
2025

Зміст

1	Знайомство з PySCF	8
1.1	Що таке PySCF: можливості та переваги	8
1.1.1	Основні можливості PySCF	8
1.1.2	Переваги використання PySCF	8
1.1.3	Порівняння з іншими програмами	9
1.2	Встановлення PySCF	9
1.2.1	Системні вимоги	9
1.2.2	Встановлення в Linux	10
1.2.3	Встановлення в Windows	10
1.2.4	Встановлення в macOS	10
1.2.5	Перевірка встановлення	11
1.3	Базова структура програми на PySCF	11
1.3.1	Загальна схема розрахунку	11
1.3.2	Мінімальний приклад	11
1.3.3	Структура для атома Гелію	12
1.3.4	Розрахунок з аналізом результатів	12
1.4	Система одиниць та конвенції	12
1.4.1	Атомні одиниці	12
1.4.2	Конвертація одиниць	13
1.4.3	Системні константи	13
1.4.4	Конвенції для атомів	13
1.5	Документація та ресурси	14
1.5.1	Офіційна документація	14
1.5.2	Основні публікації	14
1.6	Резюме	15
2	Базові об'єкти PySCF	16
2.1	Молекулярний об'єкт (Mole)	16
2.1.1	Клас Mole — серце PySCF	16
2.1.2	Створення об'єкта Mole	16
2.1.3	Основні атрибути об'єкта Mole	17
2.1.4	Визначення атома: різні способи	19
2.1.5	Робота з іонами	20
2.1.6	Налаштування вербальності виводу	20

2.2	Базисні набори для атомів	21
2.2.1	Типи базисних наборів	21
2.2.2	Вибір базисного набору: рекомендації	23
2.2.3	Власні базисні набори	23
2.2.4	Комбінування базисів	24
2.2.5	Інформація про базисний набір	25
2.3	Симетрія в атомних розрахунках	25
2.3.1	Точкові групи симетрії атомів	26
2.3.2	Використання симетрії	26
2.3.3	Симетрія орбіталей	27
2.3.4	Коли вимикати симетрію	28
2.4	Спін та мультиплетність	28
2.4.1	Визначення спінового стану	28
2.4.2	Розподіл альфа- та бета-електронів	29
2.4.3	Вибір правильного спіну	30
2.4.4	Енергетичне підтвердження спінових станів	30
2.4.5	Практичні зауваження	31
2.5	Практичний приклад: комплексний аналіз атома	31
2.6	Корисні функції та методи	34
2.6.1	Інформація про атом	34
2.6.2	Маніпуляції з базисом	34
2.6.3	Енергетичні компоненти	35
2.6.4	Резюме	35
2.6.5	Контрольні запитання	36
2.6.6	Завдання для самостійної роботи	36

3 Метод Хартрі-Фока 37

3.1	Теоретичні основи методу Хартрі-Фока	37
3.1.1	Рівняння Хартрі-Фока	37
3.1.2	Енергія Хартрі-Фока	37
3.1.3	Матриця густини та енергетичні складові	38
3.1.4	Варіанти методу Хартрі-Фока	39
3.2	Розрахунок атома Гідрогену	40
3.2.1	Особливості одноелектронної системи	40
3.2.2	Залежність від базисного набору	41
3.2.3	Аналіз орбіталей	42
3.2.4	Висновки	44
3.3	Розрахунок атома Гелію	44
3.3.1	Двоелектронна система	44
3.3.2	Порівняння RHF та UHF	45
3.3.3	Збуджені стани Гелію	46
3.4	Атоми другого періоду (Li–Ne)	47
3.4.1	Літій (Li, Z=3)	47
3.4.2	Берилій (Be, Z=4)	49

3.4.3	Бор–Неон: систематичне дослідження	50
3.5	Аналіз енергій та орбіталей	52
3.5.1	Енергетична діаграма орбіталей	52
3.5.2	Енергії іонізації	54
3.5.3	Електронна густина та її інтерпретація	58
3.5.4	Порівняння електронних густин різних атомів	62
3.5.5	Зв'язок із електронними оболонками	62
3.5.6	Практичні завдання	62
3.5.7	Методичні рекомендації	63
3.6	Порівняння методів: RHF vs UHF vs ROHF	63
3.6.1	Спінове забруднення	64
3.6.2	Тестовий випадок: атом Вуглецю	67
3.7	Складні випадки та збіжність	69
3.7.1	Причини поганої збіжності	70
3.7.2	Перехідні метали	70
3.7.3	Вироджені орбіталі та дробові заповнення	71
3.7.4	Стратегія досягнення збіжності SCF	72
3.8	Резюме	75
3.8.1	Ключові висновки	75
4	Метод Хартрі-Фока для простих молекул	76
4.1	Орбіталі і електронні стани молекул	76
4.1.1	Класифікація молекулярних орбіталей	76
4.1.2	Класифікація електронних станів молекули	77
4.2	Початкові модельні молекули: H_2^+ та H_2	79
4.3	Іон молекули водню H_2^+	79
4.3.1	Теоретичні основи	79
4.3.2	Визначення молекули в PySCF	80
4.3.3	Інтерпретація результатів	81
4.3.4	Крива потенційної енергії (PES)	82
4.3.5	Оптимізація геометрії	84
4.3.6	Аналіз молекулярних орбіталей	87
4.3.7	Вплив базисного набору	90
4.4	Молекула водню H_2	93
4.4.1	Двоелектронна система	93
4.4.2	RHF розрахунок	94
4.5	Проблема дисоціації в методі Хартрі-Фока	96
4.5.1	Фізична картина	98
4.5.2	Візуалізація проблеми	98
4.5.3	Відсутність кореляції як джерело проблеми	102
4.6	Резюме	103

5	Теорія функціоналу густини (DFT)	105
5.1	Основи теорії функціоналу густини	105
5.1.1	Теореми Хоенберга–Кона	105
5.1.2	Рівняння Кона–Шема	105
5.1.3	Обмінно-кореляційна енергія	106
5.1.4	Порівняння HF та DFT	106
5.2	Функціонали обміну-кореляції	106
5.2.1	Класифікація функціоналів: драбина Якова	106
5.2.2	LDA та LSDA функціонали	107
5.2.3	GGA функціонали	108
5.2.4	Meta-GGA функціонали	109
5.2.5	Гібридні функціонали	110
5.2.6	Порівняння різних рівнів теорії	112
5.2.7	Вибір функціоналу: рекомендації	114
5.2.8	Практичний приклад: вплив функціоналу на властивості	114
5.3	DFT розрахунки атомів	116
5.3.1	Базова структура DFT розрахунку	116
5.3.2	Систематичний розрахунок атомів другого періоду . .	116
5.3.3	Розрахунок атомів перехідних металів	117
5.3.4	Порівняння спінових станів	118
5.3.5	Аналіз d-орбіталей перехідних металів	120
5.3.6	Розрахунок важких атомів	122
5.3.7	Числові сітки в DFT	122
5.3.8	Паралелізація DFT розрахунків	124
5.4	Порівняння HF та DFT результатів	124
5.4.1	Систематичне порівняння енергій	124
5.4.2	Порівняння енергій іонізації	127
5.4.3	Електронна спорідненість	129
5.4.4	Порівняння орбітальних енергій	130
5.4.5	Забруднення спіном: HF vs DFT	132
5.4.6	Час обчислень: HF vs DFT	133
5.4.7	Загальні висновки HF vs DFT	134
5.5	Вибір функціоналу для атомних систем	135
5.5.1	Тестовий набір даних	135
5.5.2	Рекомендації для різних елементів	137
5.6	Практичні завдання	137
5.6.1	Завдання 1: Систематичне дослідження	137
5.6.2	Завдання 2: Функціональна залежність	138
5.6.3	Завдання 3: Конвергенція до базисної межі	138
5.6.4	Завдання 4: Перехідні метали	138
5.7	Резюме	138
5.7.1	Ключові висновки	139
5.7.2	Типові помилки	139

5.7.3	Корисні посилання	139
5.8	DFT для діатомних молекул	139
5.8.1	Базові DFT розрахунки молекул	139
5.8.2	Оптимізація геометрії	140
5.8.3	Криві потенційної енергії	141
5.8.4	Порівняння функціоналів для зв'язків	142
5.8.5	Спектроскопічні константи	143
5.8.6	Гетероядерні молекули	145
5.8.7	Молекули з кратними зв'язками	146
5.8.8	Порівняння HF та DFT для молекул	147
5.8.9	Молекули перехідних металів	148
5.8.10	Відкритошкаралупні молекули	149
5.8.11	Дисперсійні корекції для слабких зв'язків	150
5.8.12	Аналіз хвильових функцій	151
5.8.13	Вплив базисного набору	152
5.8.14	Дисоціація зв'язків	154
5.8.15	Збуджені стани: TD-DFT	156
5.8.16	Порівняльна таблиця результатів	157
5.8.17	Практичні рекомендації	157
5.8.18	Типові помилки при розрахунках молекул	158
5.8.19	Задачі для самостійної роботи	158
5.8.20	Типові помилки при розрахунках молекул	159
5.8.21	Систематичне дослідження галогенів	159
5.8.22	Benchmark: час обчислень	160
5.8.23	Тестовий набір G2	161
5.8.24	Використання симетрії	163
5.8.25	Резюме секції	165

6 Пост-Хартрі-Фоківські методи 166

6.1	Вступ до електронної кореляції	166
6.1.1	Що таке електронна кореляція?	166
6.1.2	Типи електронної кореляції	166
6.1.3	Кореляційна енергія атомів	167
6.1.4	Концепція одно- та багатоконфігураційних методів	167
6.1.5	Коли потрібні post-HF методи?	169
6.2	Теорія збурень Møller-Plesset (MP2)	169
6.2.1	Теоретичні основи MP2	169
6.2.2	Формула MP2 кореляційної енергії	169
6.2.3	MP2 розрахунок атома Неону	170
6.2.4	MP2 для відкритих оболонок (UMP2)	171
6.2.5	Систематичне порівняння HF vs MP2	171
6.2.6	Залежність MP2 від базисного набору	173
6.2.7	Переваги та недоліки MP2	175
6.2.8	Рекомендації по використанню MP2	176

6.3	Coupled Cluster методи (CCSD, CCSD(T))	176
6.3.1	Теоретичні основи Coupled Cluster	176
6.3.2	CCSD розрахунок атома Гелію	177
6.3.3	CCSD для відкритих оболонок (UCCSD)	178
6.3.4	Порівняння ієрархії методів	179
6.3.5	T1 діагностика та багатоконфігураційний характер	181
6.3.6	Обчислювальна складність та масштабування	183
6.4	Configuration Interaction (CI) та Full CI	183
6.4.1	Основи Configuration Interaction	183
6.4.2	Full CI — точний розв’язок	184
6.4.3	CISD розрахунки	186
6.4.4	Порівняння CI та CC методів	187
6.4.5	Size-extensivity проблема CI	189
6.4.6	Коли використовувати CI vs CC	190
6.5	CASSCF — багатоконфігураційний метод	191
6.5.1	Ідея Complete Active Space Self-Consistent Field	191
6.5.2	Позначення CASSCF(n,m)	191
6.5.3	CASSCF розрахунок атома Берилію	192
6.5.4	CASSCF для перехідних металів	193
6.5.5	Вибір активного простору	195
6.5.6	State-averaged CASSCF	195
6.5.7	CASPT2 — динамічна кореляція поверх CASSCF	196
6.5.8	Переваги та недоліки CASSCF	197
6.5.9	Ієрархія методів	198
6.5.10	Важливість кореляції для різних властивостей	198
6.5.11	Порівняння методів для He	200
6.6	Практичні завдання	202
6.6.1	Завдання 1: Порівняння методів для He	202
6.7	Резюме	204
6.7.1	Основні методи та їх характеристики	204
6.7.2	Ключові висновки	204
6.7.3	Рекомендації по використанню	205
6.7.4	Типові помилки	205
6.7.5	Практичні поради	206
6.7.6	Обчислювальні ресурси	206
6.7.7	Подальше вивчення	207
6.7.8	Що далі?	207

1

Знайомство з PySCF

1.1. Що таке PySCF: можливості та переваги

PySCF (Python-based Simulations of Chemistry Framework) — це сучасний програмний пакет з відкритим вихідним кодом для квантово-хімічних розрахунків, написаний переважно на мові Python з критичними для продуктивності частинами на C. Розроблений групою під керівництвом професора Garnet Chan, PySCF надає потужний та гнучкий інструментарій для проведення *ab initio* розрахунків молекулярних та атомних систем.

1.1.1. Основні можливості PySCF

PySCF підтримує широкий спектр квантово-хімічних методів:

- *Метод Хартрі-Фока (HF)*: restricted (RHF), unrestricted (UHF), та restricted open-shell (ROHF) варіанти
- *Теорія функціоналу густини (DFT)*: з великою бібліотекою функціоналів обміну-кореляції через інтеграцію з `libxc`
- *Post-Hartree-Fock методи*: MP2, CCSD, CCSD(T), CASSCF, CASPT2, NEVPT2
- *Configuration Interaction (CI)*: CISD, full CI для малих систем
- *Явна кореляція*: F12 методи
- *Багаторівневі методи*: ONIOM, embedding
- *Періодичні системи*: через модуль PBC (Periodic Boundary Conditions)

1.1.2. Переваги використання PySCF

Відкритий код та безкоштовність. PySCF розповсюджується під ліцензією Apache 2.0, що робить його повністю безкоштовним для академічного та комерційного використання. Доступ до вихідного коду дозволяє глибоко зрозуміти реалізацію методів та за потреби модифікувати програму.

Гнучкість та розширюваність. Завдяки використанню Python як основної мови, PySCF надзвичайно гнучкий. Користувачі можуть легко:

- Комбінувати різні методи в одному скрипті.

- Створювати власні workflow для складних розрахунків.
- Інтегрувати PySCF з іншими Python бібліотеками (NumPy, SciPy, matplotlib).
- Розробляти власні модулі та методи.

Сучасна архітектура. PySCF використовує об'єктно-орієнтований підхід, що робить код зрозумілим та легким для модифікації. Модульна структура дозволяє використовувати тільки потрібні компоненти.

Активна розробка. Проект активно розвивається, регулярно додаються нові функції та покращується продуктивність. Спільнота користувачів надає підтримку через GitHub та форуми.

Продуктивність. Незважаючи на використання Python, критичні обчислювальні частини написані на C та оптимізовані. PySCF ефективно використовує сучасні бібліотеки лінійної алгебри (BLAS, LAPACK) та може працювати на багатоядерних системах.

1.1.3. Порівняння з іншими програмами

У таблиці 1.1 наведено порівняння PySCF з іншими популярними квантово-хімічними пакетами.

Таблиця 1.1. Порівняння квантово-хімічних програм

Програма	Ліцензія	Мова	Гнучкість	Навчання
PySCF	Відкрита	Python/C	Висока	Середнє
Gaussian	Комерційна	Fortran	Низька	Легке
ORCA	Академічна	C++	Середня	Легке
Q-Chem	Комерційна	C/C++	Середня	Середнє
Psi4	Відкрита	C++/Python	Висока	Середнє

1.2. Встановлення PySCF

1.2.1. Системні вимоги

Для роботи з PySCF необхідно:

- Python 3.7 або новіший
- NumPy версії 1.13.0 або новішої
- SciPy версії 1.0.0 або новішої
- H5py (для збереження великих масивів даних)
- 4–8 ГБ оперативної пам'яті (мінімум)

1. Знайомство з PySCF

- Сучасний процесор (бажано багатоядерний)

1.2.2. Встановлення в Linux

Найпростіший спосіб встановлення PySCF у Linux — використання `pip`:

```
# Оновлення pip
pip install --upgrade pip

# Встановлення PySCF
pip install pyscf
```

Для встановлення з додатковими можливостями:

```
# З підтримкою ХС функціоналів
pip install pyscf[geomopt,dftd3,dmrgscf]
```

Альтернативно, можна встановити з вихідного коду:

```
git clone https://github.com/pyscf/pyscf.git
cd pyscf
pip install -e .
```

1.2.3. Встановлення в Windows

Для Windows рекомендується використання Anaconda:

```
# Створення нового середовища
conda create -n pyscf_env python=3.10
conda activate pyscf_env

# Встановлення PySCF
conda install -c pyscf pyscf
```

1.2.4. Встановлення в macOS

Для macOS процес аналогічний до Linux:

```
# Встановлення через pip
pip3 install pyscf

# Або через Homebrew + pip
brew install python3
pip3 install pyscf
```

1.2.5. Перевірка встановлення

Після встановлення перевіримо, чи PySCF працює коректно:

code_1.py

```
import pyscf
# Вивід версії
print(pyscf.__version__)
# Простий тест
from pyscf import gto, scf
mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
mf = scf.RHF(mol)
energy = mf.kernel()
print(f'Енергія H2: {energy:.6f} Ha')
```

Якщо програма виводить версію та обчислює енергію молекули H_2 , встановлення виконано успішно.

1.3. Базова структура програми на PySCF

1.3.1. Загальна схема розрахунку

Типовий розрахунок у PySCF складається з трьох основних етапів:

1. **Створення молекулярного об'єкта** — визначення геометрії системи, базисного набору, заряду та спіну
2. **Вибір та налаштування методу** — вибір квантово-хімічного методу (HF, DFT, CC тощо)
3. **Виконання розрахунку** — запуск обчислень та отримання результатів

1.3.2. Мінімальний приклад

Розглянемо найпростіший приклад розрахунку атома Гідрогену:

code_2.py

```
from pyscf import gto, scf
# Етап 1: Створення молекулярного об'єкта
mol = gto.M(
    atom='H 0 0 0',      # Координати атома
    basis='sto-3g',      # Базисний набір
    spin=1,              # 2S (1 неспарений електрон)
)
# Етап 2: Вибір методу (UHF для відкритої оболонки)
mf = scf.UHF(mol)
# Етап 3: Виконання розрахунку
energy = mf.kernel()
# Виведення результатів
print(f'Енергія атома H: {energy:.8f} Hartree')
print(f'Енергія атома H: {energy * 27.211386:.6f} eV')
```

1.3.3. Структура для атома Гелію

Для атома з двома електронами у замкненій оболонці:

[code_3.py](#)

```
from pyscf import gto, scf
# Атом Гелію
mol = gto.M(
    atom='He 0 0 0',
    basis='6-31g',
    spin=0,                # Всі електрони спарені
    charge=0               # Нейтральний атом
)
# RHF для замкненої оболонки
mf = scf.RHF(mol)
energy = mf.kernel()
print(f'Енергія He: {energy:.8f} Ha')
```

1.3.4. Розрахунок з аналізом результатів

Більш детальний приклад з виведенням додаткової інформації:

[code_4.py](#)

```
from pyscf import gto, scf
# Атом Літію
mol = gto.M(
    atom='Li 0 0 0',
    basis='cc-pvdz',
    spin=1                # Один неспарений електрон
)
# UHF розрахунок
mf = scf.UHF(mol)
mf.verbose = 4           # Детальний вивід
energy = mf.kernel()
# Аналіз результатів
print('\n=== Результати розрахунку ===')
print(f'Енергія: {energy:.8f} Ha')
print(f'Кількість базисних функцій: {mol.nao_nr()}')
print(f'Кількість електронів: {mol.nelectron}')
print(f'Спін: {mol.spin}')
# Заселеності Малікена
from pyscf import lo
pop = mf.mulliken_pop()
print('\nАналіз заселеностей Малікена:')
print(pop)
# Енергії орбіталей
print('\nЕнергії орбіталей (альфа):')
for i, e in enumerate(mf.mo_energy[0][:5]):
    print(f' MO {i+1}: {e:.6f} Ha ({e*27.211386:.4f} eV)')
```

1.4. Система одиниць та конвенції

1.4.1. Атомні одиниці

PySCF використовує атомну систему одиниць (atomic units, a.u.), де:

$$\begin{aligned}\hbar &= 1 \\ m_e &= 1 \quad (\text{маса електрона}) \\ e &= 1 \quad (\text{елементарний заряд}) \\ 4\pi\epsilon_0 &= 1 \quad (\text{електрична константа})\end{aligned}$$

У цій системі:

- Енергія вимірюється в Hartree (Ha): $1 \text{ Ha} = 27.211386 \text{ eV} = 627.509 \text{ kcal/mol}$
- Відстань вимірюється в Bohr (a_0): $1 \text{ Bohr} = 0.529177 \text{ \AA}$
- Час вимірюється у а.о.: $1 \text{ a.u.} = 2.4189 \times 10^{-17} \text{ s}$

1.4.2. Конвертація одиниць

PySCF надає модуль для конвертації:

[code_5.py](#)

```
from pyscf import lib
# Конвертація енергії
energy_ha = -2.85516
energy_ev = energy_ha * lib.param.HARTREE2EV
energy_kcal = energy_ha * lib.param.HARTREE2KCAL
print(f'{energy_ha:.6f} Ha = {energy_ev:.4f} eV')
print(f'{energy_ha:.6f} Ha = {energy_kcal:.2f} kcal/mol')
# Конвертація відстані
dist_bohr = 2.0
dist_angstrom = dist_bohr * lib.param.BOHR
print(f'{dist_bohr:.2f} Bohr = {dist_angstrom:.4f} \AA')
```

1.4.3. Системні константи

Доступ до фізичних констант:

[code_6.py](#)

```
from pyscf.data import nist
print(f'Швидкість світла: {nist.LIGHT_SPEED} a.u.')
print(f'Константа Планка: {nist.PLANCK} J.s')
print(f'Число Авогадро: {nist.AVOGADRO} mol^-1')
```

1.4.4. Конвенції для атомів

При визначенні атомів у PySCF:

Координати. Координати задаються у формі рядка або списку. За замовчуванням використовуються ангстрєми ($\text{\AA}$), але можна вказати одиниці:

[code_7.py](#)

```
# Координати в Ångström (за замовчуванням)
mol = gto.M(atom='C 0 0 0')
# Явне вказання одиниць
mol = gto.M(atom='C 0 0 0', unit='Angstrom')
# Координати в Bohr
mol = gto.M(atom='C 0 0 0', unit='Bohr')
```

Спін. Параметр `spin` визначає $2S = N_\alpha - N_\beta$, де N_α та N_β — кількість альфа та бета електронів:

[code_8.py](#)

```
# Атом H (1 неспарений електрон): S=1/2, 2S=1
mol = gto.M(atom='H 0 0 0', spin=1)
# Атом He (всі спарені): S=0, 2S=0
mol = gto.M(atom='He 0 0 0', spin=0)
# Атом O у триплетному стані: S=1, 2S=2
mol = gto.M(atom='O 0 0 0', spin=2)
```

Симетрія. За замовчуванням PySCF використовує повну точкову симетрію системи:

[code_9.py](#)

```
# Автоматичне визначення симетрії
mol = gto.M(atom='Ne 0 0 0', symmetry=True)
print(f'Точкова група: {mol.groupname}')
# Вимкнення симетрії
mol = gto.M(atom='Ne 0 0 0', symmetry=False)
```

1.5. Документація та ресурси

1.5.1. Офіційна документація

Ресурс	URL
Головний сайт	https://pyscf.org
GitHub репозиторій	https://github.com/pyscf/pyscf
Документація API	https://pyscf.org/pyscf_api_docs/pyscf.html
Приклади коду	https://github.com/pyscf/pyscf/tree/master/examples

1.5.2. Основні публікації

Ключові статті для цитування при використанні PySCF:

- PySCF: the Python-based simulations of chemistry framework / Q. Sun [et al.] // Wiley Interdisciplinary Reviews: Computational Molecular Science. — 2018. — Vol. 8, no. 1. — e1340. — DOI: [10.1002/wcms.1340](https://doi.org/10.1002/wcms.1340).
- Recent developments in the PySCF program package / Q. Sun [et al.] // Journal of Chemical Physics. — 2020. — Vol. 153, no. 2. — P. 024109. — DOI: [10.1063/5.0006074](https://doi.org/10.1063/5.0006074).

1.6. Резюме

У цьому розділі ми познайомились з PySCF — потужним інструментом для квантово-хімічних розрахунків. Основні моменти:

- PySCF є сучасним, відкритим та гнучким програмним пакетом.
- Встановлення здійснюється просто через **pip** або **conda**.
- Базова структура програми включає три етапи: створення системи, вибір методу, виконання розрахунку
- PySCF використовує атомну систему одиниць.
- Доступна велика кількість документації та навчальних матеріалів.

У наступному розділі ми детально розглянемо базові об'єкти PySCF та навчимось налаштовувати параметри розрахунків для атомних систем.

2

Базові об'єкти PySCF

2.1. Молекулярний об'єкт (Mole)

2.1.1. Клас Mole — серце PySCF

Клас `gto.Mole` є центральним об'єктом у бібліотеці **PySCF** (Python-based Simulations of Chemistry Framework). Цей клас визначає і зберігає всю інформацію про атомну або молекулярну систему, яка необхідна для квантово-хімічних розрахунків. Саме з нього починається будь-який проєкт у PySCF, адже всі інші модулі (SCF, DFT, MP2, CCSD тощо) працюють із вже побудованим об'єктом `Mole`.

Об'єкт `Mole` містить такі основні дані:

- **Геометрію системи** — координати атомів у просторі (в ангстремах або борах);
- **Базисний набір** — набір функцій, на яких розкладаються молекулярні орбіталі;
- **Заряд системи та спин** (мультиплетність);
- **Інформацію про симетрію** (за потреби);
- **Одиниці вимірювання** (ангстрем, бори).

Таким чином, `Mole` виконує роль «серця» всієї програми — воно зберігає стан квантової системи і передає цю інформацію до інших підсистем для проведення обчислень.

2.1.2. Створення об'єкта Mole

Існує два основні способи створення молекулярного об'єкта: або за допомогою скороченого конструктора, або покроково. Обидва підходи рівноцінні за результатом, однак відрізняються стилем запису.

Метод 1: Використання конструктора `Mole()` Цей спосіб є найзручнішим для простих систем, коли всі параметри можна вказати безпосередньо при створенні об'єкта:

`MoleCreationMethod1.py`

```
from pyscf import gto
```

```
# Простий спосіб
mol = gto.Mole(
    atom='Li 0 0 0',
    basis='6-31g',
    charge=0,
    spin=1
)
```

Тут:

- `atom='Li 0 0 0'` — визначає атом літію в координатах (0, 0, 0);
- `basis='6-31g'` — вказує базисний набір для розрахунків;
- `charge=0` — нейтральний атом;
- `spin=1` — означає, що система має $2S = 1$, тобто один неспарений електрон.

Функція `gto.Mole()` автоматично створює об'єкт, налаштовує всі параметри та викликає `.build()`.

Метод 2: Покрокове налаштування Цей спосіб зручний для складних систем, де потрібно задати багато параметрів або змінювати їх під час роботи:

MoleCreationMethod2.py

```
from pyscf import gto

# Створення порожнього об'єкта
mol = gto.Mole()

# Налаштування параметрів
mol.atom = 'C 0 0 0'
mol.basis = 'cc-pvdz'
mol.charge = 0
mol.spin = 2 # Два неспарені електрони

# Завершення побудови (важливо!)
mol.build()
```

Важливо: При використанні другого методу обов'язково викликати команду `mol.build()`, інакше PySCF не згенерує внутрішні структури (матриці, орбіталі, таблиці інтегралів тощо), необхідні для подальших розрахунків.

2.1.3. Основні атрибути об'єкта Mole

Після побудови об'єкта можна отримати детальну інформацію про систему. Ось приклад, який демонструє найпоширеніші атрибути:

SystemMainInfo.py

```
from pyscf import gto

mol = gto.M(atom='O 0 0 0', basis='6-31g', spin=2)
```

```
# Інформація про систему
print(f'Кількість електронів: {mol.nelectron}')
print(f'Заряд: {mol.charge}')
print(f'Спін (2S): {mol.spin}')
print(f'Кількість базисних функцій: {mol.nao_nr()}')

# Альфа та бета електрони
print(f'Альфа електронів: {mol.nelec[0]}')
print(f'Бета електронів: {mol.nelec[1]}')

# Атомна структура
print(f'Кількість атомів: {mol.natm}')
print(f'Заряди ядер: {mol.atom_charges()}')

# Базисний набір
print(f'Назва базису: {mol.basis}')
```

Ці атрибути особливо важливі для перевірки, чи правильно задані всі вхідні параметри перед запуском складних обчислень.

- `mol.nelectron` — кількість електронів у системі;
- `mol.charge` — сумарний заряд системи;
- `mol.spin` — подвоєне значення спіну ($2S$);
- `mol.nao_nr()` — кількість базисних орбіталей (число функцій, які будуть використані в обчисленнях);
- `mol.nelec` — кортеж `(n_alpha, n_beta)` з кількістю альфа- та бета-електронів;
- `mol.natm` — кількість атомів у системі;
- `mol.atom_charges()` — масив ядерних зарядів;
- `mol.basis` — опис обраного базисного набору.

Типовий вивід програми для атома кисню:

```
Кількість електронів: 8
Заряд: 0
Спін (2S): 2
Кількість базисних функцій: 18
Альфа електронів: 5
Бета електронів: 3
Кількість атомів: 1
Заряди ядер: [8.]
Назва базису: 6-31g
```

Такий вивід дозволяє переконатися, що система побудована коректно, а параметри відповідають фізичному змісту задачі. Наприклад, у цьому випадку маємо нейтральний атом кисню ($Z = 8$, $N_e = 8$), у якого спін $S = 1$ (два неспарені електрони).

Після побудови об'єкта `Mole` його можна передавати до будь-яких методів PySCF — від найпростіших `SCF` до корельованих методів `CCSD`, `CASCI` та інших. Тому розуміння структури і властивостей цього класу є ключем до ефективного використання всієї бібліотеки.

2.1.4. Визначення атома: різні способи

Визначення атома або атомів — це перший крок при побудові квантово-хімічної моделі. У PySCF координати та типи атомів можна задавати кількома способами. Усі вони приводять до одного й того ж результату, тому вибір форми запису залежить лише від зручності.

Спосіб 1: Рядок Цей варіант є найпростішим і найчастіше використовується для одиночних атомів або малих систем. Формат: `<атом> x y z`, де координати задаються в ангстремах за замовчуванням (або в борах, якщо `unit='Bohr'`).

AtomDefinitionWay1.py

```
# Один атом
mol = gto.M(atom='Ne 0 0 0', basis='def2-svp')

# Можна використовувати атомний номер
mol = gto.M(atom='10 0 0', basis='def2-svp') # 10 = Ne
```

Як видно, PySCF дозволяє вказувати елемент як за його хімічним символом, так і за атомним номером. У другому випадку він автоматично розпізнає елемент періодичної системи.

Спосіб 2: Список або кортеж Якщо потрібно задати кілька атомів, або якщо координати отримуються програмно (наприклад, з масиву), зручно використовувати структуру даних типу `list` або `tuple`. Кожен атом описується як пара: `[ім'я, (x, y, z)]`.

AtomDefinitionWay2.py

```
# Використання списку
mol = gto.M(
    atom=[['Na', (0.0, 0.0, 0.0)],
          basis='aug-cc-pvdz'
)

# Для декількох атомів (наприклад, для порівняння)
mol = gto.M(
    atom=[
        ['H', (0.0, 0.0, 0.0)],
        ['He', (5.0, 0.0, 0.0)] # далеко один від одного
    ],
    basis='sto-3g'
)
```

Така форма є більш універсальною — вона дозволяє легко зчитувати геометрію з файлів, генерувати координати циклом або будувати складні молекули. Наприклад, молекула водню H_2 може бути задана як:

```
atom = [['H', (0, 0, 0)], ['H', (0, 0, 0.74)]]
```

де відстань у 0.74 Å відповідає типовій довжині зв'язку Н–Н.

2.1.5. Робота з іонами

Клас `Mole` дозволяє легко моделювати заряджені системи — катіони та аніони. Для цього достатньо змінити параметр `charge`, який визначає сумарний заряд системи:

$$Q = Z_{\text{ядер}} - N_{\text{електронів}}$$

Крім того, слід відповідно скоригувати `spin`, який задає подвоєне значення спіну $2S$. Для нейтральних атомів значення спіну зазвичай відповідає їхній електронній конфігурації.

IonsExample.py

```
from pyscf import gto, scf

# Нейтральний атом Li
li_atom = gto.M(atom='Li 0 0 0', basis='6-31g', charge=0, spin=1)

# Катіон Li+ (втрачено 1 електрон)
li_cation = gto.M(atom='Li 0 0 0', basis='6-31g', charge=1, spin=0)

# Аніон Li- (додано 1 електрон)
li_anion = gto.M(atom='Li 0 0 0', basis='6-31g', charge=-1, spin=1)

print(f'Li: {li_atom.nelectron} електронів')
print(f'Li+: {li_cation.nelectron} електронів')
print(f'Li-: {li_anion.nelectron} електронів')
```

У цьому прикладі можна побачити, як зміна заряду впливає на кількість електронів:

- Нейтральний атом Li має 3 електрони;
- Катіон Li — 2 електрони (втратив один);
- Аніон Li⁻ — 4 електрони (отримав додатковий).

Такі прості зміни параметрів дозволяють вивчати іонізаційні енергії, електронну спорідненість, а також порівнювати властивості нейтральних і заряджених систем.

2.1.6. Налаштування вербальності виводу

PySCF має вбудований механізм керування докладністю текстового виводу (тобто «балакучістю» програми). Це зручно, коли потрібно або бачити повну діагностику, або, навпаки, приховати проміжні повідомлення при пакетних розрахунках.

Рівень керується параметром `verbose`:

0 = тихо, 4 = стандартно, 9 = детально (debug)

VerbosityExample.py

```
# verbose контролює кількість виводу
# 0 = мінімум, 4 = максимум (за замовчуванням), 9 = дебаг
```

```
mol = gto.M(
    atom='F 0 0 0',
    basis='cc-pvdz',
    verbose=4 # Детальний вивід
)

# Або налаштувати пізніше
mol.verbose = 0 # Тихий режим
mol.build()
```

У практиці рекомендується:

- Використовувати `verbose=4` для навчальних цілей, щоб бачити хід побудови базису та інтегралів;
- Використовувати `verbose=0` або `1` при серійному запуску розрахунків на кластері, щоб уникнути перевантаження логів;
- Для діагностики чи налагодження коду — `verbose=7-9`, що дає максимально деталізований вивід.

Таким чином, параметр `verbose` дозволяє зручно регулювати ступінь деталізації повідомлень під час обчислень, роблячи роботу з PySCF більш контрольованою і зручною як для навчання, так і для автоматизованих досліджень.

2.2. Базисні набори для атомів

2.2.1. Типи базисних наборів

У методах квантової хімії атомні орбіталі представлені не аналітичними розв'язками рівняння Шредінгера, а певними апроксимаціями — **базисними функціями**. Сукупність цих функцій для всіх атомів системи називається **базисним набором (basis set)**.

Базис задає, скільки функцій використовується для опису кожної орбіталі (1s, 2p, 3d, ...), та яку форму вони мають (наприклад, гаусівські чи слейтерівські орбіталі). Від вибору базисного набору залежить баланс між швидкістю розрахунку та точністю енергії.

У PySCF базис задається через параметр `basis='...'`, і можна використовувати як стандартні бібліотечні набори, так і власні, створені вручну.

Мінімальні базиси

Мінімальний базис — це найпростіший опис, коли на кожну атомну орбіталь використовується рівно одна базисна функція. Наприклад, для атома вуглецю (1s, 2s, 2p) це всього п'ять функцій.

```
# STO-3G - найпростіший базис
mol = gto.M(atom='C 0 0 0', basis='sto-3g')
print(f'Кількість базисних функцій: {mol.nao_nr()}')
```

```
# Вивід: 5 (1s, 2s, 2px, 2py, 2pz)
```

STO-3G означає, що кожна орбіталь Слейтера апроксимується сумою трьох гаусівських функцій. Такі базиси часто використовують для тестів і навчальних цілей, але вони не забезпечують точних енергій — похибка порівняно з великими базисами може сягати десятків кДж/моль.

Валентно-розчеплені базиси

Більш точні обчислення вимагають **розщеплення валентних орбіталей**, щоб кожна з них могла гнучко адаптуватися під різні типи хімічних зв'язків. Так з'являються базиси типу 6-31G, 6-311G тощо.

ValenceSplitBasisExample.py

```
# Сімейство Pople
mol_631g = gto.M(atom='N 0 0 0', basis='6-31g')
mol_6311g = gto.M(atom='N 0 0 0', basis='6-311g')

# 3 поляризаційними функціями
mol_631gd = gto.M(atom='N 0 0 0', basis='6-31g*')      # d на важких атомах
mol_631gdp = gto.M(atom='N 0 0 0', basis='6-31g**')    # d на важких, p на H

print(f'6-31g:   {mol_631g.nao_nr()} функцій')
print(f'6-31g*: {mol_631gd.nao_nr()} функцій')
print(f'6-31g**: {mol_631gdp.nao_nr()} функцій')
```

Позначення 6-31G читається так:

- 6 гаусів використовуються для опису внутрішніх орбіталей (core);
- валентна частина описується двома групами функцій: 3 і 1.

Додаткові символи мають такі значення:

- * — додає поляризаційні *d*-функції на важких атомах;
- ** — додає *d*-функції на важких і *p*-функції на водні атоми.

Ці функції дозволяють орбіталям деформуватися під дією хімічного оточення — завдяки цьому значно поліпшується опис зв'язків та дипольних моментів.

Кореляційно-послідовні базиси

Для кореляційних методів (MP2, CCSD, CI тощо) часто використовують сімейство Dunning'a — cc-pVXZ, де X = D, T, Q, 5, 6 (double-, triple-, quadruple-, quintuple-zeta). Ці базиси систематично покращують опис хвильової функції й дозволяють проводити екстраполяцію до межі повного базису (CBS-limit).

CorrelationConsistentBasisExample.py

```
# cc-pVXZ сімейство (X = D, T, Q, 5, 6)
mol_dz = gto.M(atom='O 0 0 0', basis='cc-pvdz')  # Double-zeta
mol_tz = gto.M(atom='O 0 0 0', basis='cc-pvtz')  # Triple-zeta
mol_qz = gto.M(atom='O 0 0 0', basis='cc-pvqz')  # Quadruple-zeta
```

```
print(f'cc-pVDZ: {mol_dz.nao_nr()} функцій')
print(f'cc-pVTZ: {mol_tz.nao_nr()} функцій')
print(f'cc-pVQZ: {mol_qz.nao_nr()} функцій')

# Augmented версії (з дифузними функціями)
mol_adz = gto.M(atom='O 0 0 0', basis='aug-cc-pvdz')
print(f'aug-cc-pVDZ: {mol_adz.nao_nr()} функцій')
```

Префікс **aug-** означає додавання **дифузних функцій** — функцій з малим експоненціальним коефіцієнтом, що описують електрони, віддалені від ядра. Такі базиси обов'язкові для аніонів, збуджених станів та слабких взаємодій (ван-дер-ваальсових систем).

def2 базиси

Базиси типу **def2** — це сучасні розробки групи Карла Ахріхса, оптимізовані для широкого спектру елементів (до лантаноїдів) і дуже добре збалансовані для методів DFT.

Def2BasisExample.py

```
# Цимейство Ahlrichs
mol_svp = gto.M(atom='Si 0 0 0', basis='def2-svp')
mol_tzvp = gto.M(atom='Si 0 0 0', basis='def2-tzvp')
mol_qzvp = gto.M(atom='Si 0 0 0', basis='def2-qzvp')

print(f'def2-SVP: {mol_svp.nao_nr()} функцій')
print(f'def2-TZVP: {mol_tzvp.nao_nr()} функцій')
print(f'def2-QZVP: {mol_qzvp.nao_nr()} функцій')
```

Позначення:

- SVP — Split-Valence Polarized (валентно-розщеплений, з поляризацією);
- TZVP — Triple-Zeta, більш гнучкий опис;
- QZVP — Quadruple-Zeta, для високоточної роботи.

Базиси **def2** часто використовуються разом з **ефективними псевдопотенціалами (ЕСР)**, які враховують релятивістські ефекти у важких елементах.

2.2.2. Вибір базисного набору: рекомендації

Вибір базису — це завжди компроміс між швидкістю й точністю. Загальні рекомендації наведено в Таблиці 2.1.

2.2.3. Власні базисні набори

Іноді потрібно створити власний базис (наприклад, для навчання або тестування гібридних моделей). PySCF дозволяє явно задавати коефіцієнти та експоненти базисних функцій через `gto.basis.parse()`.

Таблиця 2.1. Рекомендовані базисні набори для різних задач

Задача	Базис	Коментар
Тестові розрахунки	sto-3g, 3-21g	Дуже швидко, але грубо
Якісні результати	6-31g*, 6-31g**	Оптимальний баланс точність/швидкість
Точні енергії	cc-pVTZ, cc-pVQZ	Для екстраполяції до межі базису
Аніони, збуджені стани	aug-cc-pVDZ	Потрібні дифузні функції
Важкі атоми	def2-TZVP	Добре враховує релятивістські ефекти
Дослідницькі розрахунки	cc-pVDZ	Надійний старт для систем середнього розміру

CustomBasisExample.py

```

from pyscf import gto

# Визначення базису для H (тільки s-функції)
mol = gto.M(
    atom='H 0 0 0',
    basis={
        'H': gto.basis.parse('''
            H      S
                13.00773    0.019685
                1.962079    0.137977
                0.444529    0.478148
            H      S
                0.1219492    1.000000
        ''')
    })

print(f'Власний базис: {mol.nao_nr()} функцій')
```

У цьому прикладі явно визначено два набори *s*-функцій із різними коефіцієнтами. Так можна створювати спеціалізовані базиси для навчання або розширення стандартних наборів.

2.2.4. Комбінування базисів

PySCF підтримує **гібридні базиси** — різні для різних атомів у тій самій системі. Це корисно, коли потрібно зменшити обчислювальні витрати, наприклад, використовуючи спрощений базис для водню, а розширений — для важчих атомів.

CombiningBasesExample.py

```

# Для складних систем можна використовувати різні базиси
# (хоча для атомів це рідко потрібно)
mol = gto.M(
    atom='''
        H  0 0 0
        He 5 0 0
    ''')
```

```
'''
basis={
    'H': 'sto-3g',
    'He': 'cc-pvdz'
}
)
```

У результаті PySCF автоматично комбінує обидва базиси при побудові молекулярних орбіталей.

2.2.5. Інформація про базисний набір

Іноді потрібно дослідити, які саме функції використовуються у вибраному базисі — їхні типи, кількість та мітки. Для цього можна скористатися внутрішніми структурами об'єкта `Mole`.

BasisSetInfo.py

```
from pyscf import gto

mol = gto.M(atom='C 0 0 0', basis='6-31g')

# Детальна інформація
print('Базисні функції по типу:')
for atom_idx in range(mol.natm):
    symbol = mol.atom_symbol(atom_idx)
    print(f'\nАтом {symbol}:')

    # Кількість функцій кожного типу
    basis_atom = mol._basis[symbol]
    for shell in basis_atom:
        l_quantum = shell[0] # Азимутальне квантове число
        n_contractions = len(shell[1:])

        shell_type = ['s', 'p', 'd', 'f', 'g'][l_quantum]
        print(f' {shell_type}-тип: {n_contractions} contracted')

# Діапазони базисних функцій для атома
ao_labels = mol.ao_labels()
print(f'\nМітки базисних функцій:')
for i, label in enumerate(ao_labels):
    print(f'{i}: {label}')
```

Так можна отримати повну структуру базису — скільки функцій кожного типу (s , p , d), які індекси мають їхні атомні орбіталі, та як вони нумеруються в PySCF. Ця інформація важлива при аналізі орбіталей, побудові щільності або інтегралів.

2.3. Симетрія в атомних розрахунках

Симетрія відіграє ключову роль у квантово-хімічних розрахунках. Вона дозволяє:

- скоротити обчислювальні витрати (менше інтегралів та менші матриці),

- розпізнавати вироджені орбіталі та їх симетрійні властивості,
- аналізувати структуру електронних станів через іррепи (незвідні представлення),
- уникати «зайвих» розв'язків при самоузгодженні.

У PySCF симетрія автоматично враховується при побудові молекулярного об'єкта, якщо активувати опцію `symmetry=True`. Для атомів сферична симетрія апроксимується підгрупами типу **D2h**, що сумісні з декартовими координатами.

2.3.1. Точкові групи симетрії атомів

Ізольований атом у строгому сенсі має повну сферичну симетрію $SO(3)$. Однак у PySCF, через використання декартових базисних функцій, використовується одна з підгруп, зазвичай **D2h**. Це дає змогу використовувати блокову структуру матриць і автоматично класифікувати орбіталі за іррепами.

SymmetryPointGroupsExample.py

```
from pyscf import gto

# Автоматичне визначення симетрії
mol = gto.M(
    atom='Ne 0 0 0',
    basis='cc-pvdz',
    symmetry=True # Автоматичне визначення точкової групи
)

print(f'Точкова група: {mol.groupname}')
print(f'Топологічна група: {mol.topgroup}')

# Для атома результат, як правило: D2h або подібна підгрупа
```

Пояснення:

- `groupname` — ідентифікатор підгрупи (наприклад, **D2h**, **C2v**, **Cs**);
- `topgroup` — вища група симетрії, до якої належить дана підгрупа.

2.3.2. Використання симетрії

Симетрія допомагає прискорити розрахунок, оскільки гамільтоніан та інші оператори блокуються відповідно до іррепів. Тобто інтеграли між функціями з різних симетрій не обчислюються — що значно зменшує обсяг роботи.

UsingSymmetryExample.py

```
from pyscf import gto, scf

# 3 симетрією (швидше, менше пам'яті)
mol_sym = gto.M(
    atom='Ar 0 0 0',
    basis='cc-pvdz',
    symmetry=True
```

```

)
mf_sym = scf.RHF(mol_sym)
e_sym = mf_sym.kernel()

# Без симетрії
mol_nosym = gto.M(
    atom='Ar 0 0 0',
    basis='cc-pvdz',
    symmetry=False
)
mf_nosym = scf.RHF(mol_nosym)
e_nosym = mf_nosym.kernel()

print(f'3 симетрією: {e_sym:.8f} Ha')
print(f'Без симетрії: {e_nosym:.8f} Ha')
print(f'Різниця: {abs(e_sym - e_nosym):.2e} Ha')

```

Коментар: Різниця в енергіях практично нульова (обидва методи еквівалентні з фізичної точки зору), але симетрійний розрахунок потребує значно менше часу і пам'яті.

2.3.3. Симетрія орбіталей

PySCF автоматично визначає симетрійні властивості орбіталей після розрахунку. Це особливо корисно для аналізу заповнених і віртуальних рівнів, побудови діаграм енергетичних рівнів та порівняння з експериментом.

OrbitalSymmetryExample.py

```

from pyscf import gto, scf

mol = gto.M(
    atom='N 0 0 0',
    basis='cc-pvdz',
    spin=3, # Основний стан 4S
    symmetry=True
)

mf = scf.UHF(mol)
mf.kernel()

# Орбітальна симетрія
if mol.symmetry:
    orbsym = scf.hf_symm.get_orbsym(mol, mf.mo_coeff[0])
    from pyscf.symm import label_orb_symm

    labels = label_orb_symm(mol, mol.irrep_name, mol.symm_orb,
                           mf.mo_coeff[0])

    print('\nСиметрія заповнених орбіталей (альфа):')
    for i in range(mol.nelec[0]):
        print(f' MO {i+1}: {labels[i]}')

```

Коментарі:

- `mol.irrep_name` — список назв іррепів¹;

¹Незвідне представлення (irrep) — фундаментальне поняття теорії груп, що описує,

- `mol.symm_orb` — матриці симетричних орбіталей;
- `label_orb_symm()` — функція, що відображає орбіталі у відповідні іррепи;
- `get_orbsym()` — отримує симетрії молекулярних орбіталей з урахуванням побудованих коефіцієнтів.

Це дає змогу, наприклад, визначити які орбіталі мають однакову симетрію, а які не взаємодіють між собою.

2.3.4. Коли вимикати симетрію

Хоча симетрія зазвичай корисна, існують ситуації, коли її слід **вимкнути**:

- При моделюванні **збуджених станів**, які порушують симетрію основного стану.
- При побудові **поверхонь потенційної енергії (PES)**, де геометрія змінюється і симетрія зникає.
- Якщо виникають **проблеми з конвергенцією** — часто через жорсткі симетрійні обмеження.
- Коли необхідно вручну **змінювати орбіталі** або аналізувати зміщення між різними симетріями.

У таких випадках достатньо задати:

```
mol = gto.M(atom='...', basis='...', symmetry=False)
```

Підсумок: Використання симетрії — це не лише «оптимізація швидкості», а й **фізично обґрунтований підхід**, що дозволяє зрозуміти структуру електронних рівнів, виродження та природу хімічних зв'язків.

2.4. Спін та мультиплетність

2.4.1. Визначення спінового стану

У квантовій хімії поняття спіну має фундаментальне значення. Кожен електрон характеризується спіном $s = \frac{1}{2}$, тобто він може перебувати в одному з двох можливих спінових станів — «вгору» (α) або «вниз» (β).

Для багаточастинкової системи повний спін S визначається як векторна сума спінів усіх електронів. У більшості практичних розрахунків нас цікавить тільки величина S (а не його напрям).

як функція (зокрема молекулярна орбіталь) трансформується під дією операцій симетрії молекули. Кожне ірредуцибельне представлення відповідає певному типу симетрії: наприклад, для групи C_{2v} це A_1, A_2, B_1, B_2 , а для групи D_{2h} — $A_g, B_{1g}, B_{2g}, B_{3g}, A_u, B_{1u}, B_{2u}, B_{3u}$. Всі орбіталі, що належать до одного й того ж ігтер, мають однакову симетрію і можуть змішуватися між собою, тоді як орбіталі різних ігтер не взаємодіють.

В PySCF параметр `spin` задає величину $2S$, тобто різницю між кількістю α - та β -електронів:

$$2S = N_{\alpha} - N_{\beta} \quad (2.1)$$

Звідси:

$$S = \frac{N_{\alpha} - N_{\beta}}{2}, \quad M = 2S + 1$$

де M — мультиплетність (кількість можливих орієнтацій спіну системи в магнітному полі).

- $S = 0 \Rightarrow$ синглет ($M = 1$)
- $S = \frac{1}{2} \Rightarrow$ дублет ($M = 2$)
- $S = 1 \Rightarrow$ триплет ($M = 3$)
- $S = \frac{3}{2} \Rightarrow$ квартет ($M = 4$)

SpinExamples.py

```
from pyscf import gto

# Атом H: S=1/2, 2S=1, дублет (M=2)
h = gto.M(atom='H 0 0 0', basis='cc-pvdz', spin=1)

# Атом He: S=0, 2S=0, синглет (M=1)
he = gto.M(atom='He 0 0 0', basis='cc-pvdz', spin=0)

# Атом O: основний стан 3P, S=1, 2S=2, триплет (M=3)
o = gto.M(atom='O 0 0 0', basis='cc-pvdz', spin=2)

# Атом N: основний стан 4S, S=3/2, 2S=3, квартет (M=4)
n = gto.M(atom='N 0 0 0', basis='cc-pvdz', spin=3)

print(f'H: {h.nelec}, 2S={h.spin}, M={h.spin+1}')
print(f'He: {he.nelec}, 2S={he.spin}, M={he.spin+1}')
print(f'O: {o.nelec}, 2S={o.spin}, M={o.spin+1}')
print(f'N: {n.nelec}, 2S={n.spin}, M={n.spin+1}')
```

Цей код створює атомні молекули з різними значеннями параметра `spin`. В об'єкті `mol.nelec` PySCF зберігає кортеж (N_{α}, N_{β}) , що дозволяє контролювати спінову структуру системи.

2.4.2. Розподіл альфа- та бета-електронів

У багатоспінових системах важливо знати, як електрони розподілені за спіновими підрівнями. В PySCF ця інформація визначається автоматично на основі параметра `spin`, однак її можна перевірити вручну.

PrintElectronConfig.py

```
from pyscf import gto

def print_electron_config(atom_symbol, charge=0, spin=0):
    mol = gto.M(
        atom=f'{atom_symbol} 0 0 0',
        basis='sto-3g',
        charge=charge,
```

```

        spin=spin
    )

    n_alpha, n_beta = mol.nelec
    print(f'{atom_symbol} (charge={charge}, 2S={spin}):')
    print(f'  Всього електронів: {mol.nelectron}')
    print(f'  α-електронів: {n_alpha}')
    print(f'  β-електронів: {n_beta}')
    print(f'  Неспарених: {n_alpha - n_beta}\n')

# Приклади
print_electron_config('Li', charge=0, spin=1)    # Li (2s1)
print_electron_config('C', charge=0, spin=2)     # C (3P)
print_electron_config('O', charge=0, spin=2)     # O (3P)
print_electron_config('O', charge=-1, spin=1)    # O- (дублет)

```

Ця функція показує, як саме PySCF обчислює кількість α - та β -електронів. Наприклад, для нейтрального атома кисню ($Z = 8$) маємо 8 електронів. При $2S = 2$ отримаємо:

$$N_{\alpha} = 5, \quad N_{\beta} = 3$$

тобто два неспарені електрони — саме це відповідає триплетному стану (3P).

2.4.3. Вибір правильного спіну

Необхідно завжди задавати правильний спін для основного стану атома або молекули, інакше SCF-процедура може не збігатися або дати фізично некоректний результат.

Правильне значення S визначається на основі *правил Хунда*:

1. Електрони займають орбіталі так, щоб сумарний спін S був максимальним.
2. Для даного S максимізується орбітальний момент L .
3. Для менш ніж напівзаповненої оболонки найнижчий рівень має $J = |L - S|$, а для більш ніж напівзаповненої — $J = L + S$.

Для атомів другого періоду ці правила дають такі основні стани:

2.4.4. Енергетичне підтвердження спінових станів

Нижче наведено код, який обчислює енергії атомів другого періоду для їхніх правильних спінових станів. Для відкрито-оболонкових систем (`spin > 0`) використовується UHF (неспарені електрони), а для синглетів (`spin = 0`) — RHF.

EnergyConfirmation2ndPeriod.py

```

from pyscf import gto, scf

# Правильні спінові стани для другого періоду
atoms_2nd_period = [
    ('Li', 1), ('Be', 0), ('B', 1), ('C', 2),
    ('N', 3), ('O', 2), ('F', 1), ('Ne', 0)
]

```

Таблиця 2.2. Основні спінові стани атомів другого періоду

Атом	Конфігурація	Терм	S	2S
Li	[He] 2s ¹	² S	1/2	1
Be	[He] 2s ²	¹ S	0	0
B	[He] 2s ² 2p ¹	² P	1/2	1
C	[He] 2s ² 2p ²	³ P	1	2
N	[He] 2s ² 2p ³	⁴ S	3/2	3
O	[He] 2s ² 2p ⁴	³ P	1	2
F	[He] 2s ² 2p ⁵	² P	1/2	1
Ne	[He] 2s ² 2p ⁶	¹ S	0	0

```
print('Основні стани атомів другого періоду:\n')
for symbol, spin in atoms_2nd_period:
    mol = gto.M(
        atom=f'{symbol} 0 0 0',
        basis='6-31g',
        spin=spin
    )
    mf = scf.UHF(mol) if spin > 0 else scf.RHF(mol)
    mf.verbose = 0
    energy = mf.kernel()

    mult = spin + 1
    print(f'{symbol:2s}: 2S={spin}, M={mult}, E={energy:12.6f} Ha')
```

Результати дозволяють перевірити, що обраний спіновий стан дійсно відповідає мінімуму енергії для даного атома. Якщо спробувати змінити `spin`, наприклад для атома азоту задати `spin=1`, отримаємо енергію вищу на кілька десятків міліГартрі — тобто триплет виявиться збудженим станом відносно правильного к्वартету.

2.4.5. Практичні зауваження

- У PySCF спін задається *на всю систему*, тому при моделюванні молекул потрібно враховувати сумарний спіл усіх атомів.
- Для молекул із непарним числом електронів (odd number of electrons) `spin` завжди має бути непарним.
- Неправильно заданий спіл часто призводить до SCF not converged.
- При використанні DFT (наприклад, `scf.UKS`) параметр `spin` також впливає на спінову поляризацію густини.

2.5. Практичний приклад: комплексний аналіз атома

Розглянемо повний приклад аналізу атома з використанням всіх описаних концепцій:


```

from pyscf import gto, scf, lib
import numpy as np

def analyze_atom(symbol, charge=0, spin=None, basis='cc-pvdz'):
    """
    Комплексний аналіз атома

    Parameters:
    -----
    symbol : str
        Символ атома
    charge : int
        Заряд (0 для нейтрального)
    spin : int
        2S (якщо None, визначається автоматично)
    basis : str
        Базисний набір
    """

    print(f'\n{" "*60}')
    print(f'Аналіз атома {symbol} (charge={charge}, basis={basis})')
    print(f'{" "*60}\n')

    # Створення молекулярного об'єкта
    mol = gto.M(
        atom=f'{symbol} 0 0 0',
        basis=basis,
        charge=charge,
        spin=spin if spin is not None else 0,
        symmetry=True,
        verbose=4
    )

    # Інформація про систему
    print(f'Кількість електронів: {mol.nelectron}')
    print(f'α-електронів: {mol.nelec[0]}')
    print(f'β-електронів: {mol.nelec[1]}')
    print(f'Спін (2S): {mol.spin}')
    print(f'Мультиплетність: {mol.spin + 1}')
    print(f'Базисних функцій: {mol.nao_nr()}')
    print(f'Точкова група: {mol.groupname}\n')

    # Вибір методу SCF
    if mol.spin == 0:
        mf = scf.RHF(mol)
        method_name = 'RHF'
    else:
        mf = scf.UHF(mol)
        method_name = 'UHF'

    # Налаштування параметрів
    mf.conv_tol = 1e-10
    mf.max_cycle = 100
    mf.init_guess = 'atom'

    # Розрахунок
    print(f'Запуск {method_name} розрахунку...\n')
    energy = mf.kernel()

    if not mf.converged:
        print('Не конвергувало! Спроба Newton-Raphson...')
        mf = mf.newton()
        energy = mf.kernel()

```

```

# Результати
print(f'\n{"="*60}')
print('РЕЗУЛЬТАТИ')
print(f'{"="*60}')
print(f'Енергія: {energy:.10f} Ha')
print(f'Енергія: {energy * 27.211386:.6f} eV')
print(f'Конвергувало: {mf.converged}')

# Аналіз орбіталей
print(f'\nОрбітальні енергії ({method_name}):')

if mol.spin == 0:
    # RHF
    print('\nЗаповнені орбіталі:')
    n_occ = mol.nelec[0]
    for i in range(n_occ):
        print(f' MO {i+1:2d}: {mf.mo_energy[i]:10.6f} Ha '
              f'({mf.mo_energy[i]*27.211386:8.4f} eV)')

    print('\nВіртуальні орбіталі (перші 5):')
    for i in range(n_occ, min(n_occ+5, len(mf.mo_energy))):
        print(f' MO {i+1:2d}: {mf.mo_energy[i]:10.6f} Ha '
              f'({mf.mo_energy[i]*27.211386:8.4f} eV)')
else:
    # UHF
    print('\nАльфа-орбіталі (заповнені):')
    n_alpha = mol.nelec[0]
    for i in range(n_alpha):
        print(f' α-MO {i+1:2d}: {mf.mo_energy[0][i]:10.6f} Ha '
              f'({mf.mo_energy[0][i]*27.211386:8.4f} eV)')

    print('\nБета-орбіталі (заповнені):')
    n_beta = mol.nelec[1]
    for i in range(n_beta):
        print(f' β-MO {i+1:2d}: {mf.mo_energy[1][i]:10.6f} Ha '
              f'({mf.mo_energy[1][i]*27.211386:8.4f} eV)')

# Заселеності Малікена
print(f'\n{"="*60}')
print('АНАЛІЗ ЗАСЕЛЕНОСТЕЙ (Mulliken)')
print(f'{"="*60}')

pop = mf.mulliken_pop()

# Дипольний момент (для нейтральних атомів = 0)
dip = mf.dip_moment(unit='Debye')
print(f'\nДипольний момент: ({dip[0]:.4f}, {dip[1]:.4f}, '
      f'{dip[2]:.4f}) Debye')
print(f'|μ| = {np.linalg.norm(dip):.6f} Debye')

# Спінова густина (для відкритих систем)
if mol.spin > 0:
    print(f'\nСпінова густина:')
    s = mf.spin_square()
    print(f' <S²> = {s[0]:.4f}')
    print(f' <S²> (очікуване) = {mol.spin*(mol.spin+2)/4:.4f}')
    print(f' Забруднення спіном: {s[0] - mol.spin*(mol.spin+2)/4:.4f}')

return mf, energy

```

```

# Приклади використання
if __name__ == '__main__':

```

```
# Атом Li (основний стан)
mf_li, e_li = analyze_atom('Li', spin=1, basis='cc-pvdz')

# Атом C (триплет)
mf_c, e_c = analyze_atom('C', spin=2, basis='6-31g*')

# Катион O+
mf_o_plus, e_o_plus = analyze_atom('O', charge=1, spin=3,
                                   basis='aug-cc-pvdz')
```

2.6. Корисні функції та методи

2.6.1. Інформація про атом

AtomInfoExample.py

```
from pyscf import gto
from pyscf.data import elements

mol = gto.M(atom='Zn 0 0 0', basis='def2-svp')

# Основні атомні характеристики
symbol = mol.atom_symbol(0)
charge = mol.atom_charge(0)
z = gto.charge(symbol)

print(f'Атом: {symbol}')
print(f'Атомний номер: {z}')
print(f'Заряд ядра: {charge}')
print(f'Атомна маса: {elements.MASSES[charge]:.4f} а.о.м.')
print(f'Ковалентний радіус: {elements.COV_RADII[charge]:.2f} Å')

# Кількість електронів (нейтральний атом: ne = Z)
nelectron = z - mol.charge
print(f'Електронів: {nelectron}')
```

2.6.2. Маніпуляції з базисом

BasisManipulations.py

```
import numpy as np
from pyscf import gto

mol = gto.M(atom='Si 0 0 0', basis='6-31g*')

# Отримання базисних функцій
print('Базисні функції (AO labels):')
for i, label in enumerate(mol.ao_labels(fmt=False)):
    atom_id, atom_symbol, shell_type, *rest = label
    print(f'{i:3d}: Атом {atom_symbol}, тип {shell_type}')

# Кількість функцій кожного типу
from collections import Counter
ao_types = [label[2] for label in mol.ao_labels(fmt=False)]
count = Counter(ao_types)

print(f'\nРозподіл по типах:')
for shell_type, num in sorted(count.items()):
    print(f' {shell_type}: {num} функцій')
```

```
# Перекривання базисних функцій
s = mol.intor('int1e_ovlp')
print(f'\nМатриця перекривання: {s.shape}')
print(f'\nДіагональні елементи (норми): {np.diag(s)[:5]}')

# Загальна кількість АО
print(f'\nЗагальна кількість АО: {mol.nao_nr()}')
```

2.6.3. Енергетичні компоненти

EnergyComponents.py

```
from pyscf import gto, scf

mol = gto.M(atom='Ne 0 0 0', basis='cc-pvtz', symmetry=True)
mf = scf.RHF(mol)
energy = mf.kernel()

# Детальні компоненти енергії
print('Енергетичні компоненти:')
print(f' Електрон-ядерна: {mf.energy_nuc():.8f} Ha')

# Енергетичні вкладення через методи mf
e_elec, e_coul_xc = mf.energy_elec()
print(f' Електронна (E_elec): {e_elec:.8f} Ha')
print(f' Кулон+обмін/кореляція: {e_coul_xc:.8f} Ha') # для DFT це Coulomb+XC

print(f' Повна енергія: {energy:.8f} Ha')

# Матриці Фока та густини
h1e = mf.get_hcore() # Одноелектронний гамільтоніан
dm = mf.make_rdm1() # Матриця густини
vhf = mf.get_veff() # Хартрі-Фок / ефективний потенціал

print(f'\nРозміри матриць:')
print(f' h1e: {h1e.shape}')
print(f' dm: {dm.shape}')
print(f' vhf: {vhf.shape}')

# Орбітальні енергії
mo_energy = mf.mo_energy
print(f'\nЕнергії МО (перші 5): {mo_energy[:5]}')
```

2.6.4. Резюме

- **Клас Mole** — центральний об'єкт, що містить всю інформацію про систему.
- **Базисні набори** — від мінімальних (STO-3G) до великих (cc-pVQZ); вибір залежить від задачі та компромісу точність/вартість.
- **Симетрія** — прискорює розрахунки, але може обмежувати гнучкість при спонтанному руйнуванні симетрії.
- **Спін** — критичний параметр для правильного опису основного стану відкрито-оболонкових систем.

2.6.5. Контрольні запитання

1. Яка різниця між методами створення об'єкта Mole через `gto.M()` та покроково?
2. Чому для атома Карбону у основному стані використовується `spin=2`?
3. Коли слід використовувати дифузні функції (aug- базиси)?
4. Що означає параметр `conv_tol` і як він впливає на точність?
5. Які методи початкового наближення найкраще підходять для атомів?

2.6.6. Завдання для самостійної роботи

1. Створіть функцію, яка автоматично визначає правильний спін для атомів першого та другого періодів.
2. Порівняйте енергії атома Неону з різними базисними наборами (STO-3G, 6-31G, cc-pVDZ, cc-pVTZ) та побудуйте графік залежності енергії від розміру базису.
3. Дослідіть вплив симетрії на швидкість розрахунку для атома Аргону: порівняйте час виконання з `symmetry=True` та `symmetry=False`.
4. Обчисліть енергію іонізації атома Літію як різницю енергій Li та Li⁺.
5. Для атома Заліза (Fe) з різними спіновими станами ($2S = 2, 4, 6$) знайдіть, який стан має найнижчу енергію.

У наступному розділі ми детально розглянемо метод Хартрі-Фока та його застосування до розрахунків атомів.

3

Метод Хартрі-Фока

3.1. Теоретичні основи методу Хартрі-Фока

3.1.1. Рівняння Хартрі-Фока

Метод Хартрі-Фока (HF) є наближеним методом розв'язання рівняння Шредінгера для багатоелектронних систем. Основна ідея полягає у представленні багатоелектронної хвильової функції у вигляді єдиного детермінанта Слейтера:

$$\Psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \psi_1(\mathbf{r}_1) & \psi_2(\mathbf{r}_1) & \cdots & \psi_N(\mathbf{r}_1) \\ \psi_1(\mathbf{r}_2) & \psi_2(\mathbf{r}_2) & \cdots & \psi_N(\mathbf{r}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_1(\mathbf{r}_N) & \psi_2(\mathbf{r}_N) & \cdots & \psi_N(\mathbf{r}_N) \end{vmatrix} \quad (3.1)$$

де $\psi_i(\mathbf{r})$ — спін-орбіталі, які є добутком просторової частини та спінової функції.

Канонічні рівняння Хартрі-Фока мають вигляд:

$$\hat{f}\psi_i = \varepsilon_i\psi_i, \quad (3.2)$$

де $\hat{f}$ — оператор Фока:

$$\hat{f} = \hat{h} + \sum_{j=1}^N (\hat{J}_j - \hat{K}_j), \quad (3.3)$$

Тут $\hat{h}$ — одноелектронний оператор (кінетична енергія + притягання до ядра), $\hat{J}_j$ — кулонівський оператор, $\hat{K}_j$ — обмінний оператор.

3.1.2. Енергія Хартрі-Фока

Повна енергія системи у методі HF:

$$E_{HF} = \sum_{i=1}^N h_{ii} + \frac{1}{2} \sum_{i,j}^N (J_{ij} - K_{ij}) + V_{NN}, \quad (3.4)$$

де:

- h_{ii} — одноелектронні інтеграли;
- J_{ij} — кулонівські інтеграли;
- K_{ij} — обмінні інтеграли;
- V_{NN} — енергія міжядерного відштовхування (для атомів $V_{NN} = 0$).

Віральне співвідношення. Для будь-якої стаціонарної хвильової функції (зокрема, у наближенні HF) повинно виконуватись **віральне співвідношення**:

$$2\langle T \rangle + \langle V \rangle = 0,$$

або еквівалентно:

$$\frac{\langle V \rangle}{\langle T \rangle} = -2.$$

Цей критерій є важливим тестом коректності збіжного SCF-розв'язку.

3.1.3. Матриця густини та енергетичні складові

Для практичної реалізації HF-методу у базисному представленні вводиться **матриця густини**:

$$D_{\mu\nu} = 2 \sum_i^{\text{occ}} C_{\mu i} C_{\nu i}, \quad (3.5)$$

де $C_{\mu i}$ — коефіцієнти розкладу i -ї молекулярної орбіталі за базисом атомних орбіталей, а множник 2 враховує заповнення двома спінами (для RHF).

Оператор середнього значення будь-якої одноелектронної величини $\hat{O}$ можна записати у матричній формі:

$$\langle \hat{O} \rangle = \text{Tr}(D O), \quad (3.6)$$

що значно спрощує обчислення середніх значень у базисі атомних орбіталей.

Одноелектронний гамільтоніан:

$$h_{\mu\nu} = T_{\mu\nu} + V_{\mu\nu}^{\text{нuc}}, \quad (3.7)$$

де $T_{\mu\nu}$ — інтеграли кінетичної енергії, а $V_{\mu\nu}^{\text{нuc}}$ — інтеграли притягання електрона до ядра.

Матриця Фока визначається як:

$$F_{\mu\nu} = h_{\mu\nu} + \sum_{\lambda\sigma} D_{\lambda\sigma} \left[(\mu\nu|\lambda\sigma) - \frac{1}{2}(\mu\lambda|\nu\sigma) \right]. \quad (3.8)$$

Тоді загальна енергія Гартрі-Фока може бути записана компактно:

$$E_{\text{HF}} = \sum_{\mu\nu} D_{\mu\nu} \left(h_{\mu\nu} + \frac{1}{2} F_{\mu\nu} \right). \quad (3.9)$$

У розгорнутому вигляді можна виділити окремі внески:

$$\begin{aligned} E_{\text{kin}} &= \sum_{\mu\nu} D_{\mu\nu} T_{\mu\nu}, \\ E_{\text{nuc}} &= \sum_{\mu\nu} D_{\mu\nu} V_{\mu\nu}^{\text{nuc}}, \\ E_{\text{ee}} &= \frac{1}{2} \sum_{\mu\nu} D_{\mu\nu} (F_{\mu\nu} - h_{\mu\nu}), \end{aligned}$$

і, відповідно, повна енергія:

$$E_{\text{tot}} = E_{\text{kin}} + E_{\text{nuc}} + E_{\text{ee}} + V_{NN}.$$

Таким чином, матриця густини D є центральним об'єктом, що зберігає всю інформацію про одноелектронні властивості системи. У коді **PySCF** вона створюється викликом `dm = mf.make_rdm1()`, а середні значення обчислюються як добутки D на відповідні матриці інтегралів.

3.1.4. Варіанти методу Хартрі-Фока

Restricted Hartree-Fock (RHF)

RHF використовується для систем з замкненими оболонками, де всі електрони спарені:

$$\psi_i^\alpha(\mathbf{r}) = \psi_i^\beta(\mathbf{r}) = \varphi_i(\mathbf{r}) \quad (3.10)$$

Підходить для: He, Be, Ne, Mg, Ar, Ca, Zn (у синглетних станах).

Unrestricted Hartree-Fock (UHF)

UHF дозволяє різні просторові орбіталі для альфа та бета спінів:

$$\psi_i^\alpha(\mathbf{r}) \neq \psi_i^\beta(\mathbf{r}) \quad (3.11)$$

Підходить для: всіх відкритих систем (H, Li, B, C, N, O, F та їх іони).

Restricted Open-shell Hartree-Fock (ROHF)

ROHF — компроміс між RHF та UHF. Спарені електрони описуються однаковими орбіталями, неспарені — різними:

$$\begin{cases} \psi_i^\alpha = \psi_i^\beta & \text{для спарених} \\ \psi_i^\alpha \neq \psi_i^\beta & \text{для неспарених} \end{cases} \quad (3.12)$$

3.2. Розрахунок атома Гідрогену

Атом Гідрогену є найпростішим квантовим об'єктом, який можна описати в рамках не лише хімії, а й фундаментальної квантової механіки. Його Гамільтоніан має вигляд:

$$\hat{H} = -\frac{1}{2}\nabla^2 - \frac{1}{r},$$

де $-\frac{1}{2}\nabla^2$ — оператор кінетичної енергії електрона, а $-\frac{1}{r}$ — потенціал кулонівської взаємодії між електроном і ядром. Ця система має аналітичне рішення, і енергія основного стану дорівнює

$$E_1 = -\frac{1}{2} \text{ Ha.}$$

Для атома Гідрогену це рішення можна отримати навіть чисельно в PySCF, що дозволяє перевірити точність обраного базисного набору та методів квантово-хімічних розрахунків.

3.2.1. Особливості одноелектронної системи

Атом Гідрогену є одноелектронною системою, тому метод Гартрі-Фока (HF) не містить жодних апроксимацій, окрім обмежень базисного набору. У звичайних багатоелектронних атомах HF наближає взаємодію електронів середнім потенціалом, але для H відсутнє електрон-електронне відштовхування, тож метод дає *точну* хвильову функцію для обраного базису.

[code_1.py](#)

```
from pyscf import gto, scf
import numpy as np

# Створення атома Гідрогену
mol = gto.M(
    atom="H 0 0 0", # координати ядра
    basis="sto-3g", # мінімальний базис
    spin=1, # один неспарений електрон
    verbose=4, # рівень деталізації виводу
)

print(f"Кількість електронів: {mol.nelectron}")
print(f"Базисних функцій: {mol.nao_nr()}")

# Розрахунок методом UHF (необмежений Гартрі-Фок)
mf = scf.UHF(mol)
energy = mf.kernel()

print(f"\nЕнергія H (STO-3G): {energy:.8f} Ha")
print(f"Енергія H (STO-3G): {energy * 27.211386:.6f} eV")

# Теоретичне значення
print(f"Теоретична енергія: -0.5 Ha")
print(f"Похибка базису: {abs(energy + 0.5):.6f} Ha")
```

Отримане значення енергії наближається до -0.5 Ha, але точність залежить від базису. STO-3G — це мінімальний базис, де кожна орбіталь апроксимується трьома гаусовими функціями, тому результат має невелику похибку (близько 10^{-3} Ha).

3.2.2. Залежність від базисного набору

Базисний набір визначає якість апроксимації хвильової функції. Чим більший набір, тим ближче розрахована енергія до аналітичного результату. Для атома Гідрогену ця збіжність особливо показова, бо ми можемо порівняти з точним розв'язком.

У коді нижче порівнюються кілька популярних базисів, від найменшого STO-3G до розширених кореляційно-узгоджених наборів cc-pV5Z. Для кожного базису обчислюється енергія та похибка відносно -0.5 Ha.

code_2.py

```
from pyscf import gto, scf
import matplotlib.pyplot as plt

basis_sets = [
    "sto-3g",
    "3-21g",
    "6-31g",
    "6-311g",
    "cc-pvdz",
    "cc-pvtz",
    "cc-pvqz",
    "cc-pv5z",
]

energies = []
n_basis = []

print("Базис      N_bas      Енергія (Ha)      Похибка (mHa)")
print("-" * 60)

for basis in basis_sets:
    try:
        mol = gto.M(atom="H 0 0 0", basis=basis, spin=1, verbose=0)
        mf = scf.UHF(mol)
        e = mf.kernel()

        n = mol.nao_nr()
        error = (e + 0.5) * 1000 # похибка в міліГартрі

        energies.append(e)
        n_basis.append(n)

        print(f"{basis:15s} {n:3d}      {e:12.8f}      {error:8.4f}")
    except:
        print(f"{basis:15s} --- недоступний")

# Побудова графіка збіжності
plt.figure(figsize=(10, 6))
plt.plot(n_basis, energies, "o-", linewidth=2, markersize=8)
plt.axhline(y=-0.5, color="r", linestyle="--", label="Точне значення")
plt.xlabel("Кількість базисних функцій", fontsize=12)
plt.ylabel("Енергія (Ha)", fontsize=12)
```

```
plt.title("Збіжність енергії атома Н від базисного набору", fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig("h_atom_basis_convergence.pdf")
plt.show()
```

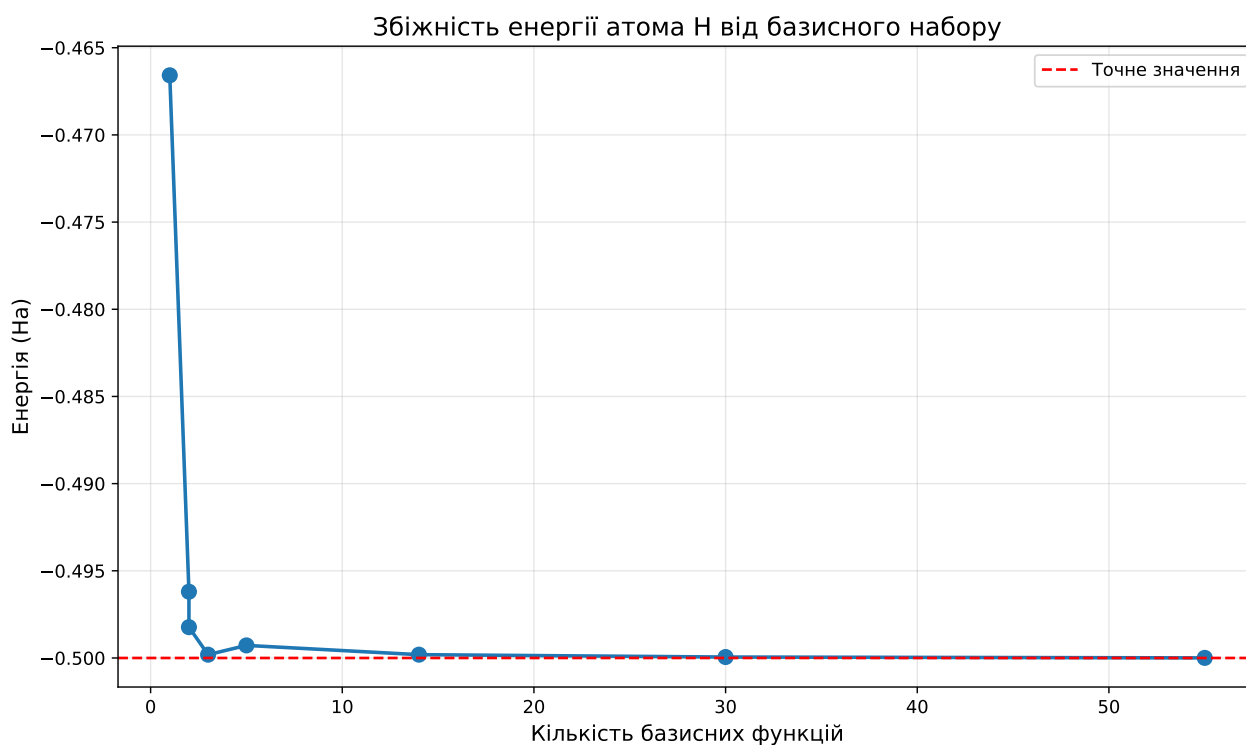


Рис. 3.1. Залежність енергії атома Н від вибору базису.

З графіка (рис. 3.1) видно, що зі збільшенням кількості базисних функцій енергія швидко наближається до точного значення -0.5 Ha. Набори типу **cc-pVQZ** або **cc-pV5Z** практично дають збіжність до повного базисного ліміту (CBS limit).

3.2.3. Аналіз орбіталей

Хоча в атома Гідрогену існує лише одна заповнена орбіталь (1s), PySCF дозволяє вивести енергетичні рівні для всіх функцій базису, а також дослідити матрицю густини. Цей підхід зручний для демонстрації структури HF-розрахунку.

[code_3.py](#)

```
from pyscf import gto, scf
import numpy as np

mol = gto.M(atom="H 0 0 0", basis="cc-pvtz", spin=1)
mf = scf.UHF(mol)
energy = mf.kernel()
```

```
# Орбітальні енергії
print("\nОрбітальні енергії (альфа-спін):")
print("-" * 50)
for i, (e, label) in enumerate(zip(mf.mo_energy[0], mol.ao_labels())):
    occ = "(occ)" if i < mol.nelec[0] else "(virt)"
    print(f"{i + 1:2d}. {label:20s}: {e:10.6f} Ha {occ}")

print(f"\nЕнергія 1s орбіталі: {mf.mo_energy[0][0]:.8f} Ha")
print(f"Теоретична енергія: -0.50000000 Ha")

# Матриця густини
dm = mf.make_rdm1()
print(f"\nМатриця густини (альфа): {dm[0].shape}")
print(f"Слід dm: {np.trace(dm[0]):.1f} (має дорівнювати 1)")
```

Виведення програми:

```
Орбітальні енергії (альфа-спін):
-----
1. 0 H 1s          : -0.499810 Ha (occ)
2. 0 H 2s          : 0.025806 Ha (virt)
3. 0 H 3s          : 0.298457 Ha (virt)
4. 0 H 2px         : 0.298457 Ha (virt)
5. 0 H 2py         : 0.298457 Ha (virt)
6. 0 H 2pz         : 1.886323 Ha (virt)
7. 0 H 3px         : 2.824504 Ha (virt)
8. 0 H 3py         : 2.824504 Ha (virt)
9. 0 H 3pz         : 2.824504 Ha (virt)
10. 0 H 3dxy        : 2.824504 Ha (virt)
11. 0 H 3dyz        : 2.824504 Ha (virt)
12. 0 H 3dz^2       : 3.199249 Ha (virt)
13. 0 H 3dxz        : 3.199249 Ha (virt)
14. 0 H 3dx2-y2     : 3.199249 Ha (virt)

Енергія 1s орбіталі: -0.49980981 Ha
Теоретична енергія: -0.50000000 Ha

Матриця густини (альфа): (14, 14)
Слід dm: 0.5 (має дорівнювати 1)
```

Матриця густини (**dm**) відображає заповнення орбіталей. Її слід має дорівнювати кількості електронів.

Коментар

Однак у даному випадку ми бачимо $\text{Tr}(\text{dm}) = 0.5$. Це не помилка: PySCF формує окремі матриці густини для альфа- та бета-спінів. Для одноконфігураційного стану з одним електроном (1s, альфа-спін) слід матриці густини $\text{dm}_\alpha = 0.5$, оскільки PySCF нормує хвильову функцію на повну густину ($\alpha + \beta$), а не на окремий спін.

Отже,

$$\text{Tr}(\text{dm}_\alpha) + \text{Tr}(\text{dm}_\beta) = N_{\text{електронів}} = 1,$$

а тому

$$\text{Tr}(\text{dm}_\alpha) = 0.5, \quad \text{Tr}(\text{dm}_\beta) = 0.$$

Якщо обчислити повну густину через `mf.make_rdm1(mo, mo_occ, spin=None)`, то отримаємо правильне значення 1.

Проведені розрахунки є основою для подальшого аналізу електронної густини, побудови орбіталей і візуалізації електронної хмари.

3.2.4. Висновки

- Для атома Гідрогену метод Гартрі-Фока є **точним**, бо немає взаємодії між електронами.
- Основна похибка виникає через обмеження базисного набору.
- Зі збільшенням кількості базисних функцій енергія швидко збігається до точного значення -0.5 Ha.
- PySCF дозволяє досліджувати вплив базису, аналізувати орбіталі, матриці густини та підготовлює ґрунт для подальшого розгляду багатоелектронних систем.

3.3. Розрахунок атома Гелію

3.3.1. Двоелектронна система

Атом гелію є фундаментальним прикладом для демонстрації методів **Хартрі-Фока (HF)**. Два електрони гелію мають протилежні спіни й заповнюють одну й ту саму орбіталь $1s$, тому це *замкнена оболонка* зі спіном $S = 0$ (синглет, $M = 1$).

Після завершення SCF-розрахунку PySCF надає доступ до одноелектронних інтегралів (`int1e_kin`, `int1e_nuc`) та до двоелектронної частини (`get_veff`). Це дозволяє безпосередньо обчислити всі компоненти енергії та перевірити віральне співвідношення.

HeHF.py

```
from pyscf import gto, scf

# --- 1. Опис атома гелію ---
mol = gto.Mole()
mol.atom = 'He 0 0 0'
mol.basis = 'cc-pVTZ'
mol.build()

# --- 2. Розрахунок RHF ---
mf = scf.RHF(mol)
E_tot = mf.kernel()

# --- 3. Матриця густини та інтеграли ---
dm = mf.make_rdm1()
T_int = mol.intor('int1e_kin')      # оператор кінетичної енергії
V_nuc_int = mol.intor('int1e_nuc') # оператор потенціальної енергії ядро-електрон
vhf = mf.get_veff(mol, dm)         # ефективний потенціал (Кулон + обмін)

# --- 4. Енергетичні складові ---
E_kin = (dm * T_int).sum()          # <T>
E_nuc = (dm * V_nuc_int).sum()      # <V_nuc>
E_ee = 0.5 * (dm * vhf).sum()       # <V_ee> (Кулон + обмін)
E_tot_check = E_kin + E_nuc + E_ee  # перевірка
```

```
# --- 5. Віральне співвідношення ---
V_total = E_nuc + E_ee
virial_ratio = V_total / E_kin

# --- 6. Вивід у вигляді таблиці ---
print("\n" + "="*60)
print("  *15 + "ЕНЕРГЕТИЧНИЙ АНАЛІЗ АТОМА He")
print("="*60)
print(f"{'Компонента':<30} {'Значення (Ha)':>20}")
print("-"*60)
print(f"{'Кінетична енергія (T)':<30} {E_kin:>20.6f}")
print(f"{'Ядро-електрони (V_nuc)':<30} {E_nuc:>20.6f}")
print(f"{'Електрон-електрон (V_ee)':<30} {E_ee:>20.6f}")
print("-"*60)
print(f"{'Повна енергія (E_tot)':<30} {E_tot:>20.6f}")
print(f"{'Перевірка суми (E_sum)':<30} {E_tot_check:>20.6f}")
print("="*60)
print(f"\n{'Віральне співвідношення':<30} {'V/T':>20}")
print("-"*60)
print(f"{'Розраховане значення':<30} {virial_ratio:>20.3f}")
print(f"{'Теоретичне значення':<30} {-2.000:>20.3f}")
print("="*60 + "\n")
```

Коментар

Якщо розрахунок збіжний і фізично коректний, то відношення $\langle V \rangle / \langle T \rangle \approx -2.00 \pm 0.01$, що підтверджує виконання віральної теореми. Будь-яке суттєве відхилення (наприклад, -1.7 чи -2.3) свідчить про помилку збіжності або неадекватність базисного набору.

3.3.2. Порівняння RHF та UHF

Оскільки гелій має замкнену оболонку ($N_\alpha = N_\beta$), обидва методи — RHF і UHF — дають ідентичні результати. UHF дозволяє α та β орбіталям відрізнятися, але тут ця свобода не використовується.

[code_5.py](#)

```
from pyscf import gto, scf

mol = gto.M(atom="He 0 0 0", basis="6-31g", spin=0)

# RHF розрахунок
mf_rhf = scf.RHF(mol)
mf_rhf.verbose = 0
e_rhf = mf_rhf.kernel()

# UHF розрахунок
mf_uhf = scf.UHF(mol)
mf_uhf.verbose = 0
e_uhf = mf_uhf.kernel()

print(f"RHF енергія: {e_rhf:.10f} Ha")
print(f"UHF енергія: {e_uhf:.10f} Ha")
print(f"Різниця: {abs(e_rhf - e_uhf):.2e} Ha")

# Перевірка симетрії спіну
s2_uhf = mf_uhf.spin_square()
print(f"\n<S^2> (UHF): {s2_uhf[0]:.6f}")
print("<S^2> (точно): 0.000000")
```

Коментар

UHF може порушувати спінову симетрію (*spin contamination*), особливо для відкрито-оболонкових систем. Тут же $S^2 = 0$, тобто спінова симетрія зберігається, і RHF $\equiv$ UHF.

3.3.3. Збуджені стани Гелію

У збуджених станах один електрон переходить на більш високий рівень (наприклад, $2s$), а система може існувати в двох спінових конфігураціях:

- **Синглет:** $S = 0$ — антисиметрична за спіном, симетрична за просторовою координатою.
- **Триплет:** $S = 1$ — симетрична за спіном, антисиметрична за просторовою частиною.

У PySCF ці стани можна моделювати за допомогою відповідного значення параметра `spin` та методу `R0HF` для частково заповнених оболонок.

[code_6.py](#)

```
from pyscf import gto, scf

def he_excited_state(config="1s2s", spin=0, basis="cc-pvdz"):
    """
    Розрахунок збуджених станів He ( $1S$ ,  $3S$ ) з використанням MOM
    """
    mol = gto.M(atom="He 0 0 0", basis=basis, spin=spin)

    # RHF для синглету, UHF для триплету
    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
    mf.kernel() # звичайний HF, щоб ініціалізуватись

    # MOM для стабілізації збудженого стану
    mom_solver = scf.addons.mom_occ(mf, mf.make_rdm1(), mf.mo_occ)
    mom_solver.verbose = 0
    mom_solver.max_cycle = 100
    e_exc = mom_solver.kernel()

    return e_exc

# Основний стан
mol_gs = gto.M(atom="He 0 0 0", basis="cc-pvdz", spin=0)
mf_gs = scf.RHF(mol_gs)
mf_gs.verbose = 0
e_gs = mf_gs.kernel()

print("Стани атома Гелію (cc-pVDZ):")
print("-" * 55)
print(f"1s2 ( $1S$ , основний): {e_gs:.6f} Ha = {e_gs * 27.2114:.2f} eV")

# Збуджені стани
for spin, label in [(0, "1s2s ( $1S$ )"), (2, "1s2s ( $3S$ )")]:
    e_exc = he_excited_state(spin=spin)
```

```
print(f"{label:15s}: {e_exc: .6f} Ha ({(e_exc - e_gs)*27.2114: .3f} eV вище основного)")
```

```
# Примітка: для збуджених станів
```

```
# краще використовувати методи типу TD-DFT або CASSCF
```

Коментар

Метод MOM (Maximum Overlap Method) — це модифікація самозгодженої процедури HF, у якій вибір зайнятих орбіталей на кожній ітерації здійснюється не за найменшими енергіями, а за критерієм **максимального перекриття** з орбіталями попередньої ітерації:

$$O_{ij} = |\langle \varphi_i^{(n)} | \varphi_j^{(n-1)} \rangle|^2.$$

Таким чином, MOM «закріплює» бажану орбітальну конфігурацію (наприклад, $1s2s$) і не дозволяє хвильовій функції сповзати назад до основного стану ($1s^2$), що часто трапляється у звичайному HF.

Попри те, що MOM стабілізує збуджені конфігурації, він залишається одно-детермінантним наближенням і не враховує електронної кореляції, тому не належить до post-HF методів. Отримані стани є лише самозгодженими розв'язками рівнянь HF, а не справжніми власними станами гамільтоніана. Для точного опису енергій і спектрів збудження слід застосовувати багатоконфігураційні або збуджувальні підходи (CASSCF, CI, TD-DFT).

Підсумок:

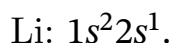
- Гелій — базовий тест для валідації HF-методу.
- Різниця між RHF і експериментом демонструє значення кореляційної енергії.
- Збуджені стани потребують спеціальних методів (MOM, CASSCF, TD-DFT).

3.4. Атоми другого періоду (Li–Ne)

3.4.1. Літій (Li, Z=3)

Атом Літію — перший багатоелектронний атом, у якому проявляється неспарений електрон та спінова поляризація ($N_\alpha \neq N_\beta$, тобто густини ρ_α і ρ_β відрізняються).

Електронна конфігурація:



Два перші електрони утворюють замкнену оболонку ($1s$), а третій — одинокий у підоболонці $2s$, що зумовлює мультиплетність $2S$ (спін $S = \frac{1}{2}$).

Через наявність неспареного електрона метод UHF є природним вибором. UHF дозволяє альфа- та бета-орбіталям відрізнитися, тобто хвильова функція не змушена мати однакову просторову форму для різних спінів.

Альтернативою є метод ROHF (Restricted Open-Shell Hartree-Fock), у якому

- усі замкнені орбіталі мають однакові просторові функції для α та β спінів;
- відкриті орбіталі (неспарені) можуть мати різну зайнятість, але однакову форму.

ROHF забезпечує правильне значення $\langle S^2 \rangle$ та зберігає спінову симетрію, проте формулювання рівнянь Фока є складнішим. UHF, навпаки, простіший у реалізації, але може призводити до *spin contamination*. У практиці обидва методи дають близькі енергії для атома Li, проте ROHF вважається формально більш коректним.

code_7.py

```
from pyscf import gto, scf

# --- 1. Визначення молекули ---
mol = gto.M(
    atom="Li 0 0 0",
    basis="cc-pvdz",
    spin=1,          # неспарений електрон (S=1/2)
    symmetry=True,
)

print("Літій (Li):")
print(" Конфігурація: [He] 2s1")
print(" Основний стан: 2S")
print(f" Електронів: {mol.nelectron}")
print()

# --- 2. UHF ---
uhf = scf.UHF(mol)
E_uhf = uhf.kernel()

print("=== UHF ===")
print(f"Енергія Li (UHF): {E_uhf:.8f} Ha")
print(f"Енергія Li (UHF): {E_uhf * 27.211386:.4f} eV")

s2_uhf = uhf.spin_square()
print(f"<S2> = {s2_uhf[0]:.6f} (очікується 0.75)\n")

# --- 3. ROHF ---
rohlf = scf.ROHF(mol)
E_rohf = rohlf.kernel()

print("=== ROHF ===")
print(f"Енергія Li (ROHF): {E_rohf:.8f} Ha")
print(f"Енергія Li (ROHF): {E_rohf * 27.211386:.4f} eV")

s2_rohf = rohlf.spin_square()
print(f"<S2> = {s2_rohf[0]:.6f} (очікується 0.75)\n")

# --- 4. Порівняння ---
ΔE = (E_uhf - E_rohf) * 27.211386
print(f"Різниця (UHF - ROHF): {ΔE:.6f} eV")

# --- 5. Коментар ---
# UHF дозволяє різні орбіталі для α і β спінів,
# тому виникає спінове забруднення (<S2> > 0.75).
# ROHF зберігає правильну спінову симетрію, але менш гнучкий варіаційно.
```

Коментар

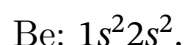
- Значення $\langle S^2 \rangle \approx 0.75$ свідчить про правильний спіновий стан подвійності (мультиплетність $2S + 1 = 2$).
- Наявність різниці між енергіями альфа- та бета-орбіталей демонструє спінову асиметрію.
- Метод UHF може частково порушувати симетрію (так звана *spin contamination*), але для Li це незначно.

Енергія, отримана методом HF, для Li становить близько -7.432 Ha у базисі cc-pVDZ, тоді як експериментальна енергія основного стану (повна) — близько -7.478 Ha. Таким чином, кореляційна похибка становить близько 0.046 Ha (приблизно 1.25 eV).

Цей приклад є першим, де проявляється *кореляція електронів*, яку HF не враховує. В подальших розділах буде показано, як методи MP2, CI та CC враховують ці ефекти.

3.4.2. Берилій (Be, Z=4)

Атом Берилію має електронну конфігурацію



Тут усі орбіталі парні, тому спінова поляризація відсутня, і зручно використовувати RHF (Restricted Hartree–Fock). Обидва електрони у підоболонці $2s$ мають протилежні спіни, тож система має мультиплетність $1S$ (синглетний стан).

code_8.py

```
from pyscf import gto, scf

# Be: [He] 2s2, основний стан 1S
mol = gto.M(
    atom="Be 0 0 0",
    basis="cc-pvtz",
    spin=0, # замкнена оболонка
    symmetry=True,
)

print("Берилій (Be):")
print(" Конфігурація: [He] 2s2")
print(" Основний стан: 1S")

# RHF розрахунок
mf = scf.RHF(mol)
energy = mf.kernel()

print(f"\nЕнергія Be: {energy:.8f} Ha")

# Порівняння з експериментом
exp_be = -14.6674 # Ha (експериментальна повна енергія)
print(f"Експериментальна: {exp_be:.4f} Ha")
print(f"Кореляційна енергія: {exp_be - energy:.4f} Ha")
```

Обговорення.

- Для Be HF-енергія виходить приблизно -14.573 Ha (у базисі $ss\text{-}pVTZ$), тоді як експериментальна — -14.667 Ha.
- Різниця близько 0.094 Ha (≈ 2.6 eV) — це **кореляційна енергія**, тобто внесок взаємодії електронів, не врахований у HF.
- У Берилія ефекти електронної кореляції вже досить значні, адже електрони в орбіталі $2s$ взаємодіють один з одним.
- Цей випадок є гарною ілюстрацією межі застосування методу Гартрі-Фока.

Висновки для атомів Li і Be.

- Li демонструє появу неспареного електрона і спінової поляризації (UHF необхідний).
- Be — перший приклад, де **електронна кореляція** дає помітну похибку енергії.
- RuSCF дозволяє наочно дослідити вплив базису, спіну, симетрії та методів на точність енергії.

3.4.3. Бор-Неон: систематичне дослідження

Після розгляду окремих атомів Літію та Берилію доцільно перейти до систематичного аналізу всіх атомів другого періоду (від Бору до Неону). У цих атомах поступово заповнюється $2p$ -підрівень, і змінюється як спінова мультиплетність, так і форма електронної густини. Метод Гартрі-Фока дозволяє побачити, як змінюється енергія системи при збільшенні числа електронів, а також як проявляється спінова поляризація у відкритих оболонках.

Електронні конфігурації.

Li:	$[\text{He}] 2s^1$	2S
Be:	$[\text{He}] 2s^2$	1S
B:	$[\text{He}] 2s^2 2p^1$	2P
C:	$[\text{He}] 2s^2 2p^2$	3P
N:	$[\text{He}] 2s^2 2p^3$	4S
O:	$[\text{He}] 2s^2 2p^4$	3P
F:	$[\text{He}] 2s^2 2p^5$	2P
Ne:	$[\text{He}] 2s^2 2p^6$	1S

Як видно, кількість неспарених електронів збільшується від Бору до Нітрогену, а потім зменшується до Неону, що зумовлює зміну спінового стану та мультиплетності.

Мета дослідження. Провести розрахунок енергій Гартрі-Фока для атомів другого періоду в однаковому базисі **cc-pVDZ**, оцінити правильність спінового стану (через $\langle S^2 \rangle$) та проаналізувати систематичні тенденції.

code_9.py

```
from pyscf import gto, scf
import numpy as np

# Дані про атоми другого періоду
atoms_data = [
    ("Li", 1, "2S", "[He] 2s1"),
    ("Be", 0, "1S", "[He] 2s2"),
    ("B", 1, "2P", "[He] 2s2 2p1"),
    ("C", 2, "3P", "[He] 2s2 2p2"),
    ("N", 3, "4S", "[He] 2s2 2p3"),
    ("O", 2, "3P", "[He] 2s2 2p4"),
    ("F", 1, "2P", "[He] 2s2 2p5"),
    ("Ne", 0, "1S", "[He] 2s2 2p6"),
]

basis = "cc-pvdz"
print(f"Розрахунок атомів 2-го періоду (базис: {basis})")
print("=" * 70)
print(f"{'Атом':4s} {'Спін':4s} {'Терм':4s} {'E(HF), Ha':15s} {'E(HF), eV':12s}")
print("-" * 70)

energies = {}

for symbol, spin, term, config in atoms_data:
    mol = gto.M(
        atom=f"{symbol} 0 0 0", basis=basis, spin=spin, symmetry=True, verbose=0
    )

    # Вибір методу SCF
    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
    mf.conv_tol = 1e-10
    energy = mf.kernel()
    energies[symbol] = energy

    e_ev = energy * 27.211386
    print(f"{symbol:4s} {spin:4d} {term:4s} {energy:15.8f} {e_ev:12.2f}")

    # Додаткова інформація для відкритих оболонок
    if spin > 0:
        s2 = mf.spin_square()
        expected_s2 = spin * (spin + 2) / 4
        print(f"<S^2> = {s2[0]:.4f} (очікується {expected_s2:.4f})")

print("=" * 70)

# Збереження результатів
np.savez("second_period_hf.npz", **energies)
```

Коментар

- Для кожного атома автоматично обирається тип SCF: RHF (для замкнених оболонок, $S = 0$) або UHF (для відкритих).
- Параметр `spin` у PySCF означає різницю між кількістю альфа- та бета-електронів: $\text{spin} = N_\alpha - N_\beta = 2S$.
- Розрахунок $\langle S^2 \rangle$ дозволяє перевірити правильність спінового стану. Для ідеального випадку значення має збігатися з теоретичним $S(S + 1)$.
- У файлі `second_period_hf.npz` зберігаються всі енергії для подальшого аналізу або побудови графіків.

Очікувані тенденції.

1. Повна енергія атома зменшується (стає більш негативною) із зростанням атомного номера Z .
2. Енергія спостерігає стрибки на межі заповнення оболонок: при переходах $\text{Be} \rightarrow \text{B}$, $\text{N} \rightarrow \text{O}$, $\text{F} \rightarrow \text{Ne}$.
3. Спінова мультиплетність відображає заповнення $2p$ -орбіталей згідно з **правилом Гунда** — максимальний спін у середині підоболонки (для N).
4. Значення $\langle S^2 \rangle$ для UHF повинні бути близькі до теоретичних, але можуть мати невеликі відхилення через *spin contamination*.

Фізичне узагальнення. Цей розрахунок демонструє фундаментальну властивість методу Гартрі-Фока: він добре описує загальну структуру енергетичних рівнів і тенденції в періодичній таблиці, але не враховує електронну кореляцію, через що абсолютні значення енергії мають систематичну похибку.

3.5. Аналіз енергій та орбіталей

3.5.1. Енергетична діаграма орбіталей

Енергетичні діаграми орбіталей є наочним способом представлення енергетичного спектру молекулярних орбіталей (МО), що виникають у результаті розв'язання рівнянь Гартрі-Фока.

`code_11.py`

```
from pyscf import gto, scf
import matplotlib.pyplot as plt

def plot_orbital_diagram(symbol, spin, basis="cc-pvdz"):
    """Побудова діаграми орбітальних енергій"""

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    if spin == 0:
        mf = scf.RHF(mol)
```

```

else:
    mf = scf.UHF(mol)

mf.verbose = 0
mf.kernel()

fig, ax = plt.subplots(figsize=(8, 10))

if spin == 0:
    # RHF: одна колонка орбіталей
    energies = mf.mo_energy * 27.211386 # eV
    n_occ = mol.nelec[0]

    for i, e in enumerate(energies[: n_occ + 5]):
        color = "blue" if i < n_occ else "red"
        label = mol.ao_labels()[i] if i < len(mol.ao_labels()) else ""
        ax.hlines(e, 0.2, 0.8, colors=color, linewidth=2)
        ax.text(0.85, e, f"{label}", fontsize=10, va="center")

    # Стрілки для електронів
    if i < n_occ:
        ax.arrow(
            0.35,
            e + 0.3,
            0,
            -0.25,
            head_width=0.05,
            head_length=0.1,
            fc="black",
            ec="black",
        )
        ax.arrow(
            0.65,
            e - 0.3,
            0,
            0.25,
            head_width=0.05,
            head_length=0.1,
            fc="black",
            ec="black",
        )
    else:
        # UHF: дві колонки (альфа та бета)
        e_alpha = mf.mo_energy[0] * 27.211386
        e_beta = mf.mo_energy[1] * 27.211386
        n_alpha, n_beta = mol.nelec

        # Альфа орбіталі
        for i, e in enumerate(e_alpha[: n_alpha + 3]):
            color = "blue" if i < n_alpha else "red"
            ax.hlines(e, 0.1, 0.4, colors=color, linewidth=2)
            if i < n_alpha:
                ax.arrow(
                    0.25,
                    e + 0.3,
                    0,
                    -0.25,
                    head_width=0.04,
                    head_length=0.1,
                    fc="black",
                    ec="black",
                )

        # Бета орбіталі

```

```

for i, e in enumerate(e_beta[: n_beta + 3]):
    color = "blue" if i < n_beta else "red"
    ax.hlines(e, 0.6, 0.9, colors=color, linewidth=2)
    if i < n_beta:
        ax.arrow(
            0.75,
            e - 0.3,
            0,
            0.25,
            head_width=0.04,
            head_length=0.1,
            fc="black",
            ec="black",
        )

    ax.text(0.25, min(e_alpha[:5]) - 2, "α", fontsize=14, ha="center")
    ax.text(0.75, min(e_beta[:5]) - 2, "β", fontsize=14, ha="center")

ax.set_xlim(0, 1)
ax.set_ylabel("Енергія (eV)", fontsize=12)
ax.set_title(f"Діаграма орбіталей {symbol}", fontsize=14)
ax.set_xticks([])
ax.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig(f"{symbol}_orbital_diagram.pdf")
plt.show()

# Приклади
plot_orbital_diagram("C", spin=2) # Вуглець
plot_orbital_diagram("Ne", spin=0) # Неон

```

3.5.2. Енергії іонізації

Іонізаційна енергія I є однією з фундаментальних характеристик атома, що визначає мінімальні енерговитрати на видалення одного електрона із нейтрального атома в основному стані:

$$I = E(A^+) - E(A),$$

де $E(A)$ — повна енергія нейтрального атома, а $E(A^+)$ — енергія його однозарядного катіона. Ця величина безпосередньо вимірюється експериментально (у електронвольтах) і є важливим тестом якості будь-якого квантово-хімічного методу.

У межах методу Гартрі-Фока іонізаційна енергія може бути визначена двома способами.

Метод Δ SCF. Найбільш пряме обчислення полягає у незалежному знаходженні повних енергій нейтрального атома і катіона:

$$I_{\Delta\text{SCF}} = E(A^+) - E(A).$$

Такий підхід враховує релаксацію орбіталей після видалення електрона, тому дає точніші результати, ніж прості наближення.

Теорема Купманса. У межах наближення «заморожених орбіталей» (*frozen orbital approximation*) повна енергія системи вважається незмінною при видаленні одного електрона. У цьому випадку зміна енергії дорівнює орбітальній енергії найвищої зайнятої молекулярної орбіталі:

$$I_{\text{Koopmans}} \approx -\epsilon_{\text{HOMO}}.$$

Це твердження відоме як **теорема Купманса**. Воно дозволяє оцінити енергії іонізації без повторного самозгодження, хоча і не враховує релаксаційні та кореляційні ефекти.

Порівняння підходів. Для легких атомів (Li, Be, B, C, N, O, F, Ne) метод Купманса правильно відтворює тенденцію зростання енергії іонізації уздовж періоду, але систематично недооцінює абсолютні значення. Метод ΔSCF наближається до експерименту краще, оскільки включає релаксацію орбіталей.

Приклад: атом Літій. Для Li (конфігурація $1s^2 2s^1$):

$$I_{\text{Koopmans}} = -\epsilon_{2s}, \quad I_{\Delta\text{SCF}} = E(\text{Li}^+) - E(\text{Li}), \quad I_{\text{exp}} = 5.39 \text{ eV}.$$

Енергія Купманса дещо завищена, а результат ΔSCF ближчий до експериментального, що демонструє вплив орбітальної релаксації.

KoopmansTheoremCheck.py

```
import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf

def test_koopmans_theorem(atoms_list, basis="cc-pvtz"):
    """
    Перевірка теореми Купманса:
    порівняння  $\Delta\text{SCF}$ ,  $-\epsilon_{\text{HOMO}}$  та експериментальних іонізаційних енергій.
    """

    # Експериментальні значення (eV)
    exp_ie = {
        "Li": 5.39,
        "Be": 9.32,
        "B": 8.30,
        "C": 11.26,
        "N": 14.53,
        "O": 13.62,
        "F": 17.42,
        "Ne": 21.56,
    }

    print(f"\nПеревірка теореми Купманса (базис: {basis})")
    print("=" * 110)
    print(
        f"{'Атом':<6s} {'IE( $\Delta\text{SCF}$ ):>12s} {' $-\epsilon_{\text{HOMO}}$ :>12s} {'Різниця':>12s} "
        f"{' $\Delta\text{SCF} \rightarrow \text{HOMO}$ :>12s} {'Exp.':>10s} {' $\Delta\text{SCF} \rightarrow \text{Exp}$ :>12s} {' $\text{HOMO} \rightarrow \text{Exp}$ :>12s}"
    )
```



```

print("-" * 110)

results = {
    "atoms": [],
    "ie_scf": [],
    "ie_koop": [],
    "ie_exp": [],
    "diff_scf_koop": [],
    "diff_scf_exp": [],
    "diff_koop_exp": [],
}

for symbol, spin_n, spin_c in atoms_list:
    # Нейтральний атом
    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin_n, verbose=0)
    mf = scf.UHF(mol) if spin_n != 0 else scf.RHF(mol)
    mf.conv_tol = 1e-11
    e_neutral = mf.kernel()

    # HOMO енергія
    if spin_n == 0:
        n_occ = mol.nelectron // 2
        eps_homo = mf.mo_energy[n_occ - 1]
    else:
        n_alpha = mol.nelec[0]
        eps_homo = mf.mo_energy[0][n_alpha - 1]

    ie_koopmans = -eps_homo * 27.211386 # eV

    # Катион
    mol_cat = gto.M(atom=f"{symbol} 0 0 0", basis=basis, charge=1, spin=spin_c, verbose=0)
    mf_cat = scf.UHF(mol_cat) if spin_c != 0 else scf.RHF(mol_cat)
    mf_cat.conv_tol = 1e-11
    e_cation = mf_cat.kernel()

    ie_scf = (e_cation - e_neutral) * 27.211386 # eV

    # Порівняння
    diff_scf_koop = ie_koopmans - ie_scf
    diff_scf_koop_percent = abs(diff_scf_koop / ie_scf) * 100

    ie_exp = exp_ie.get(symbol, np.nan)
    diff_scf_exp_percent = abs((ie_scf - ie_exp) / ie_exp) * 100 if not np.isnan(ie_exp)
    ↪ else np.nan
    diff_koop_exp_percent = abs((ie_koopmans - ie_exp) / ie_exp) * 100 if not
    ↪ np.isnan(ie_exp) else np.nan

    print(
        f"{symbol:6s} {ie_scf:12.4f} {ie_koopmans:12.4f} {diff_scf_koop:12.4f} "
        f"{diff_scf_koop_percent:12.2f} {ie_exp:10.2f} "
        f"{diff_scf_exp_percent:12.2f} {diff_koop_exp_percent:12.2f}"
    )

    # Збереження
    results["atoms"].append(symbol)
    results["ie_scf"].append(ie_scf)
    results["ie_koop"].append(ie_koopmans)
    results["ie_exp"].append(ie_exp)
    results["diff_scf_koop"].append(diff_scf_koop)
    results["diff_scf_exp"].append(diff_scf_exp_percent)
    results["diff_koop_exp"].append(diff_koop_exp_percent)

print("=" * 110)

```

```

mean_diff = np.nanmean(np.abs(results["diff_scf_koop"]))
mean_err_scf = np.nanmean(results["diff_scf_exp"])
mean_err_koop = np.nanmean(results["diff_koop_exp"])

print(f"\nСередня |ΔSCF - Koopmans| різниця: {mean_diff:.3f} eV")
print(f"Середня похибка ΔSCF від експерименту: {mean_err_scf:.2f}%")
print(f"Середня похибка -εHOMO від експерименту: {mean_err_koop:.2f}%")

# --- Графік ---
fig, ax = plt.subplots(figsize=(10, 6))
x = np.arange(len(results["atoms"]))
width = 0.25

ax.bar(x - width, results["ie_exp"], width, label="Експеримент", color="gray", alpha=0.6)
ax.bar(x, results["ie_scf"], width, label="IE (ΔSCF)", color="blue", alpha=0.7)
ax.bar(x + width, results["ie_koop"], width, label="-εHOMO", color="red", alpha=0.7)

ax.set_xlabel("Атом", fontsize=12)
ax.set_ylabel("Енергія іонізації (eV)", fontsize=12)
ax.set_title("Перевірка теореми Купманса", fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels(results["atoms"])
ax.legend()
ax.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig("koopmans_theorem_test.pdf")
plt.show()

return results

# --- Тестування ---
atoms_test = [
    ("Li", 1, 0),
    ("Be", 0, 1),
    ("B", 1, 0),
    ("C", 2, 1),
    ("N", 3, 2),
    ("O", 2, 3),
    ("F", 1, 2),
    ("Ne", 0, 1),
]

results_koop = test_koopmans_theorem(atoms_test)

```

Коментар

Розрахунок виводить таблицю порівняння енергій іонізації ($-\epsilon_{\text{HOMO}}$, ΔSCF , експеримент) для атомів другого періоду та відносну похибку у відсотках:

$$\delta = 100 \times \frac{I_{\text{calc}} - I_{\text{exp}}}{I_{\text{exp}}}.$$

Аналіз цих відхилень дозволяє оцінити якість базисного набору і межі застосовності теореми Купманса.

3.5.3. Електронна густина та її інтерпретація

Розподіл електронної густини

Розподіл електронної густини $\rho(\mathbf{r})$ є однією з ключових величин у квантовій хімії. Саме ця функція визначає, де саме в просторі «перебувають» електрони атома або молекули, і, отже, пояснює більшість хімічних і спектроскопічних властивостей системи.

Згідно з однодетермінантним наближенням Гартрі-Фока, електронна густина визначається як

$$\rho(\mathbf{r}) = \sum_i^{\text{occ}} |\psi_i(\mathbf{r})|^2,$$

де $\psi_i(\mathbf{r})$ — орбіталі, а сума береться по всіх зайнятих орбіталях.

У програмному пакеті PySCF густина може бути обчислена як на сітці, так і в окремих точках простору, що дозволяє візуалізувати просторовий розподіл електронів.

Обчислення густини вздовж осі z

У наведеному прикладі реалізовано функцію `plot_density_profile`, що обчислює електронну густину вздовж осі z для атома. Це дозволяє побудувати одноосний «профіль густини» та простежити, як електронна густина спадає при віддаленні від ядра.

Для кожної точки z обчислюється матриця базисних функцій $\varphi_i(\mathbf{r})$, і далі густина:

$$\rho(\mathbf{r}) = \sum_{ij} \chi_i(\mathbf{r}) D_{ij} \chi_j(\mathbf{r}),$$

де D_{ij} — елементи матриці щільності.

Результат зображається графічно: густина $\rho(0, 0, z)$ як функція відстані від ядра. Для нейтральних атомів крива має різкий максимум біля ядра, який швидко спадає експоненційно.

[code_12.py](#)

```
from pyscf import gto, scf
from pyscf.tools import cubegen
import numpy as np
import matplotlib.pyplot as plt

def plot_density_profile(symbol, spin, basis="cc-pvdz"):
    """Радіальний розподіл густини"""

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
```

```

mf.kernel()

# Розрахунок густини вздовж осі z
r_points = np.linspace(0, 5, 200) # Bohr
density = []

dm = mf.make_rdm1()

for r in r_points:
    coords = np.array([[0, 0, r]])
    rho = mol.eval_gto("GT0val_sph", coords)

    if spin == 0:
        dens = np.einsum("pi,ij,pj->p", rho, dm, rho)[0]
    else:
        dens_a = np.einsum("pi,ij,pj->p", rho, dm[0], rho)[0]
        dens_b = np.einsum("pi,ij,pj->p", rho, dm[1], rho)[0]
        dens = dens_a + dens_b

    density.append(dens)

# Побудова графіка
plt.figure(figsize=(10, 6))
plt.plot(r_points, density, linewidth=2)
plt.xlabel("Відстань від ядра (Bohr)", fontsize=12)
plt.ylabel("Електронна густина (e/Bohr³)", fontsize=12)
plt.title(f"Радіальний розподіл густини {symbol}", fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(f"{symbol}_density.pdf")
plt.show()

return r_points, density

# Приклад
r, rho = plot_density_profile("Ne", spin=0)

```

Інтерпретація результатів. На побудованому графіку $\rho(0,0,z)$ спостерігаються чіткі області локалізації електронів: внутрішні електрони формують центральний пік, тоді як зовнішні оболонки проявляються у вигляді плавних плечей. Для важчих атомів густина стає більш «зосередженою» біля ядра через сильніше притягання кулонівським потенціалом.

Практична порада. При виборі базисного набору (сс-pvdz, 6-31G**, тощо) слід мати на увазі, що форма профілю густини істотно залежить від якості базису: мінімальні базиси дають лише грубу картину, тоді як розширені базиси (сс-pVTZ) відтворюють експоненційний спад більш точно.

Сферично усереднена густина

Для атомів, що мають сферичну симетрію, зручно розглядати усереднену по всіх напрямках густину:

$$\rho(r) = \frac{1}{4\pi} \int \rho(\mathbf{r}) d\Omega,$$

де $r = |\mathbf{r}|$ та $d\Omega$ — елемент тілесного кута.

Ця функція показує, як змінюється густина лише від радіуса r , незалежно від напрямку, і є основою для побудови радіальної функції розподілу

$$P(r) = 4\pi r^2 \rho(r),$$

що описує, яка частка електронів перебуває у сферичному шарі товщини dr на відстані r від ядра.

code_13.py

```
from pyscf import gto, scf
import numpy as np
import matplotlib.pyplot as plt

def spherical_averaged_density(symbol, spin, basis="cc-pvdz"):
    """Сферично усереднена електронна густина"""

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
    mf.kernel()

    dm = mf.make_rdm1()

    # Радіальні точки
    r_points = np.linspace(0.01, 8, 300)

    # Куты для інтегрування (метод Монте-Карло)
    n_angles = 100
    theta = np.random.uniform(0, np.pi, n_angles)
    phi = np.random.uniform(0, 2 * np.pi, n_angles)

    density_sph = []

    for r in r_points:
        rho_avg = 0

        for t, p in zip(theta, phi):
            x = r * np.sin(t) * np.cos(p)
            y = r * np.sin(t) * np.sin(p)
            z = r * np.cos(t)

            coords = np.array([x, y, z])
            ao_value = mol.eval_gto("GTOval_sph", coords)

            if spin == 0:
                rho_point = np.einsum("pi,ij,pj->p", ao_value, dm, ao_value)[0]
            else:
                rho_a = np.einsum("pi,ij,pj->p", ao_value, dm[0], ao_value)[0]
                rho_b = np.einsum("pi,ij,pj->p", ao_value, dm[1], ao_value)[0]
                rho_point = rho_a + rho_b

            rho_avg += rho_point

        rho_avg /= n_angles
```

```

density_sph.append(rho_avg)

# Радіальна функція розподілу  $4\pi r^2 \rho(r)$ 
radial_dist = 4 * np.pi * r_points**2 * np.array(density_sph)

# Графіки
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Густина  $\rho(r)$ 
ax1.plot(r_points, density_sph, linewidth=2)
ax1.set_xlabel("r (Bohr)", fontsize=12)
ax1.set_ylabel("ρ(r) (e/Bohr³)", fontsize=12)
ax1.set_title(f"Електронна густина {symbol}", fontsize=14)
ax1.grid(True, alpha=0.3)
ax1.set_yscale("log")

# Радіальна функція розподілу
ax2.plot(r_points, radial_dist, linewidth=2, color="red")
ax2.set_xlabel("r (Bohr)", fontsize=12)
ax2.set_ylabel("4πr²ρ(r)", fontsize=12)
ax2.set_title(f"Радіальна функція розподілу {symbol}", fontsize=14)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{symbol}_spherical_density.pdf")
plt.show()

# Перевірка нормування
dr = r_points[1] - r_points[0]
total_electrons = np.sum(radial_dist) * dr
print(f"\nІнтеграл  $4\pi r^2 \rho(r) dr = {total_electrons:.2f}$ ")
print(f"Очікується: {mol.nelectron} електронів")

return r_points, density_sph, radial_dist

# Приклади для різних атомів
r, rho, rdf = spherical_averaged_density("He", spin=0)
r, rho, rdf = spherical_averaged_density("C", spin=2)
r, rho, rdf = spherical_averaged_density("Ne", spin=0)

```

Методика обчислення. Для кожного радіуса r вибираються випадкові напрямки (кутові координати θ, φ) за методом Монте-Карло. У цих напрямках обчислюється густина $\rho(x, y, z)$, після чого береться середнє значення:

$$\rho(r) \approx \frac{1}{N} \sum_{k=1}^N \rho(r, \theta_k, \varphi_k).$$

Цей підхід забезпечує добру точність навіть при невеликій кількості напрямів $N \sim 100$.

Радіальна функція розподілу. На другому графіку зображується $4\pi r^2 \rho(r)$ — це функція, інтеграл від якої по r дає повну кількість електронів:

$$\int_0^\infty 4\pi r^2 \rho(r) dr = N_e.$$

Таким чином, можна перевірити нормування хвильової функції та коректність чисельного інтегрування.

Приклад інтерпретації. Для атома гелію максимум $4\pi r^2 \rho(r)$ спостерігається приблизно при $r \approx 0.3 \text{ Bohr}$, що відповідає найбільш ймовірній відстані електрона від ядра у $1s$ -стані. Для вуглецю або неону виникають додаткові піки, пов'язані з електронами на $2s$ та $2p$ оболонках.

3.5.4. Порівняння електронних густин різних атомів

Ми розглянули, як отримати електронну густину $\rho(\mathbf{r})$ та сферично усереднену густину $\rho(r)$ для окремого атома. Однак для повного розуміння періодичних закономірностей важливо порівняти, як змінюється форма та протяжність електронної хмари при переході від легких до важчих елементів.

Фізична мотивація

Порівняння електронних густин дозволяє:

- побачити, як із зростанням атомного номера Z ядро сильніше притягує електрони;
- проаналізувати зміну розмірів атома та ефективного радіуса;
- виявити закономірності заповнення електронних оболонок;
- візуально оцінити зв'язок між структурою густини та положенням елемента в періодичній системі.

3.5.5. Зв'язок із електронними оболонками

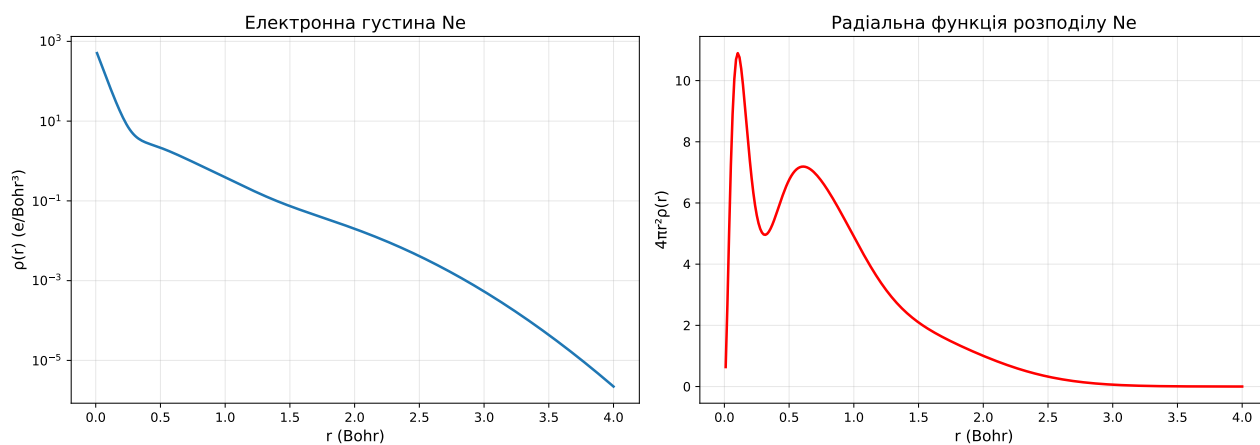
Максимуми радіальної функції розподілу $4\pi r^2 \rho(r)$ відповідають областям найбільшої ймовірності перебування електронів певних орбіталей:

$$\begin{aligned} 1s &\rightarrow \text{перший максимум,} & 2s, 2p &\rightarrow \text{другий максимум,} \\ & & 3s, 3p, 3d &\rightarrow \text{третій тощо.} \end{aligned}$$

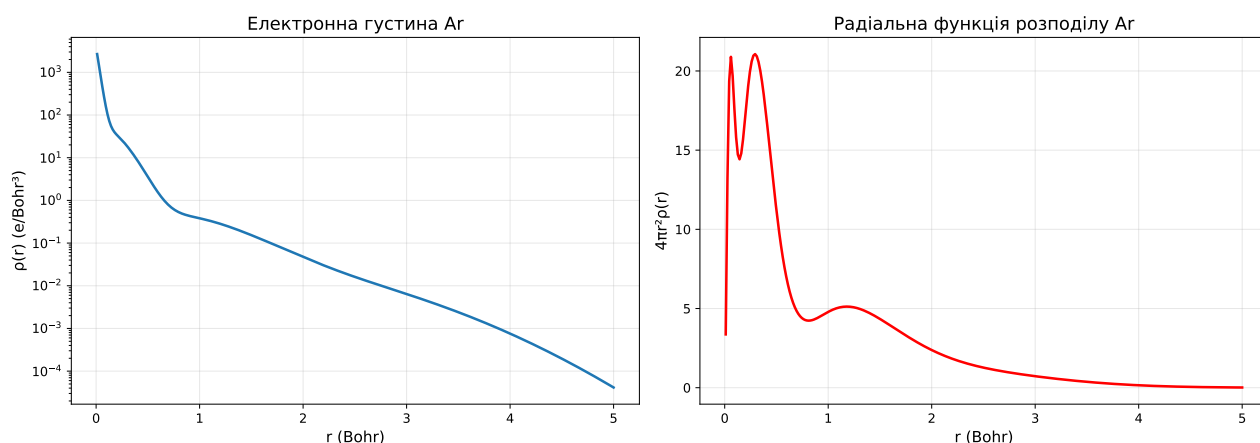
Таким чином, форма $4\pi r^2 \rho(r)$ дає прямий зв'язок між результатами квантово-хімічних розрахунків і традиційною атомною структурою, знайомою студентам з курсу атомної фізики. Візуальний аналіз таких графіків (рис. 3.5.4) дозволяє простежити закономірності побудови періодичної системи Менделєєва з точки зору електронної густини.

3.5.6. Практичні завдання

1. Побудуйте на одному графіку $\rho(r)$ для Li, Na, K. Як змінюється радіус атома?
2. Знайдіть положення максимумів $4\pi r^2 \rho(r)$ для атомів He, Ne, Ar. До яких оболонок вони належать?
3. Порівняйте радіальні функції для атомів C і O. Як змінюється густина зовнішньої оболонки?



(а) АНе



(б) Атома Ar

Рис. 3.2. Електронна густина $\rho(r)$ (ліворуч) та радіальна функція розподілу $4\pi r^2 \rho(r)$ (праворуч) для атомів Ne та Ar. На графіках радіальна функція розподілу чітко видно окремі максимуми, що відповідають електронним оболонкам. Для неону (Ne) спостерігаються два основних максимуми, які відповідають заповненням підоболонкам $1s^2$ і $2s^2 2p^6$, тоді як для аргону (Ar) — три максимуми, що відображають заповнення оболонок $1s^2$, $2s^2 2p^6$ та $3s^2 3p^6$.

3.5.7. Методичні рекомендації

1. Змінюючи базис (наприклад, ST0-3G, 6-31G, cc-pVTZ), можна дослідити, як збільшується точність відтворення радіального профілю.
2. Для відкритих оболонок (атомів з неспареними електронами) потрібно враховувати спінову поляризацію — використовується UHF.

Таким чином, наведені приклади демонструють практичний зв'язок між теоретичними поняттями електронної густини і їх чисельною реалізацією в пакеті PySCF.

3.6. Порівняння методів: RHF vs UHF vs ROHF

У квантово-хімічних розрахунках вибір типу наближення Гартрі-Фока залежить від спінового стану системи. Методи RHF, UHF та ROHF відрізняються

тим, як вони поведуться із спіновими функціями електронів — тобто, чи допускають різні орбіталі для α - і β -електронів.

- RHF (Restricted Hartree-Fock) — усі електрони спарені, кожна просторова орбіталь заповнена двома електронами з протилежними спінами. Підходить лише для синглетних закритих оболонок.
- UHF (Unrestricted Hartree-Fock) — орбіталі для α - і β -електронів дозволено різні. Цей метод придатний для відкритих оболонок (наприклад, атомів з неспареними електронами), однак може страждати від *спінового забруднення* (*spin contamination*).
- ROHF (Restricted Open-Shell Hartree-Fock) — компроміс: спільні орбіталі для парних електронів і різні — лише для неспарених. Це забезпечує правильний спіновий симетрійний стан без забруднення.

Енергетичне порівняння

UHF зазвичай дає трохи нижчу енергію, ніж ROHF, оскільки має більше варіаційної свободи. Різниця $\Delta E = E_{\text{UHF}} - E_{\text{ROHF}}$ зазвичай незначна (менше кількох міліГартрі), однак у системах з сильним спіновим змішуванням може бути суттєвою.

Орбітальні енергії

UHF формує два набори орбіталей — для α - і β -електронів. Тому орбітальні енергії можуть відрізнитись:

$$\varepsilon_i^{(\alpha)} \neq \varepsilon_i^{(\beta)}.$$

У ROHF усі парні орбіталі спільні, тож орбітальні енергії чітко впорядковані відповідно до симетрії та спіну.

3.6.1. Спінове забруднення

У системах з відкритими оболонками метод UHF часто використовується як простіше наближення, що дозволяє займати незалежні орбіталі електронам зі спінами α та β . Однак відсутність суворого обмеження на спінову симетрію призводить до важливого недоліку — **спінового забруднення**.

Математична суть явища

UHF-хвильова функція не є власним станом оператора $\hat{S}^2$, хоч і залишається власним станом $\hat{S}_z$:

$$\hat{S}_z \Psi_{\text{UHF}} = M_S \Psi_{\text{UHF}}, \quad \hat{S}^2 \Psi_{\text{UHF}} \neq S(S+1) \Psi_{\text{UHF}}.$$

Отже, очікуване значення

$$\langle S^2 \rangle = \langle \Psi_{\text{UHF}} | \hat{S}^2 | \Psi_{\text{UHF}} \rangle$$

зазвичай перевищує істинне $S(S + 1)$, тобто:

$$\Delta S^2 = \langle S^2 \rangle - S(S + 1) > 0.$$

Фізичне тлумачення

UHF-хвильова функція може бути подана як суперпозиція чистих спінових станів:

$$\Psi_{\text{UHF}} = c_0 \Psi_S + c_1 \Psi_{S+1} + c_2 \Psi_{S+2} + \dots,$$

де присутність коефіцієнтів $c_i \neq 0$ для $i > 0$ означає змішування різних мультиплетів. Наприклад, для триплетного стану ($S = 1$) теоретичне значення $\langle S^2 \rangle = 2.0$, але якщо розрахунок UHF дає 2.25, це означає, що хвильова функція містить домішку п'ятичленного ($S = 2$) стану.

Вплив на результати

- **Енергія.** UHF може давати занадто низьку енергію через змішування спінів і штучне зменшення кулонівського відштовхування.
- **Електронна густина.** Спінова густина стає несиметричною, особливо поблизу атомів з неспареними електронами.
- **Магнітні властивості.** Похибки у спіновій густині ведуть до помилкових магнітних моментів і спотворених мультиплетних розщеплень.

Як обчислюється спінове забруднення

Програми квантово-хімічних розрахунків (зокрема PySCF, Gaussian, ORCA) обчислюють спінове забруднення не безпосередньо через оператор $\hat{S}^2$, а через перекриття між орбіталями спінів α і β .

Основна формула для середнього значення $\langle S^2 \rangle$ у наближенні UHF має вигляд:

$$\langle S^2 \rangle = \frac{(N_\alpha - N_\beta)^2}{4} + \frac{N_\alpha + N_\beta}{2} - \sum_{i=1}^{N_\alpha} \sum_{j=1}^{N_\beta} |\langle \varphi_i^\alpha | \varphi_j^\beta \rangle|^2$$

де:

- N_α, N_β — кількість електронів зі спінами α та β відповідно;
- $\varphi_i^\alpha, \varphi_j^\beta$ — молекулярні(атомні) орбіталі для спінів α і β ;
- $\langle \varphi_i^\alpha | \varphi_j^\beta \rangle$ — перекриття між орбіталями різних спінів.

Якщо орбіталі α і β однакові, тобто $\varphi_i^\alpha = \varphi_i^\beta$, то:

$$\langle S^2 \rangle = \frac{(N_\alpha - N_\beta)^2}{4},$$

і для синглету ($N_\alpha = N_\beta$) це дорівнює нулю — спінове забруднення відсутнє.

Інтерпретація

Якщо орбіталі α і β подібні, перекриття $S_{\alpha\beta}$ велике, тому Σ наближається до N_β , і $\langle S^2 \rangle$ близьке до теоретичного $S(S+1)$ — тобто забруднення мінімальне. Якщо ж орбіталі сильно відрізняються (наприклад, при розриві зв'язку або в радикалах), Σ зменшується, і $\langle S^2 \rangle$ зростає — з'являється суттєве спінове забруднення.

Діагностика та критерії

Значення $\langle S^2 \rangle$ зазвичай друкується у вихідних даних програми. Практичні межі:

$$\Delta S^2 = \langle S^2 \rangle - S(S+1),$$

$$\begin{cases} \Delta S^2 < 0.05 & \text{— незначне забруднення, прийнятно;} \\ 0.05 < \Delta S^2 < 0.10 & \text{— помірне, бажано перевірити;} \\ \Delta S^2 > 0.10 & \text{— суттєве, слід перейти до ROHF або проєкційних методів.} \end{cases}$$

Приклад чисельного розрахунку

Для системи з $N_\alpha = 5$, $N_\beta = 4$ (очікувано $S = \frac{1}{2}$, $\langle S^2 \rangle = 0.75$):

- якщо $\Sigma = 3.9$, тоді

$$\langle S^2 \rangle = \frac{1}{4} + \frac{9}{2} - 3.9 = 0.85,$$

$$\Delta S^2 = 0.85 - 0.75 = 0.10,$$

тобто забруднення перебуває на межі суттєвого;

- якщо $\Sigma = 3.4$, тоді

$$\langle S^2 \rangle = \frac{1}{4} + \frac{9}{2} - 3.4 = 1.35,$$

$$\Delta S^2 = 1.35 - 0.75 = 0.60,$$

що свідчить про сильне спінове забруднення.

Як зменшити спінове забруднення

1. **ROHF:** обмежена відкрита схема Гартрі-Фока, що зберігає спінову симетрію.
2. **Спін-проєкційні методи:** після варіації застосовується оператор проєкції

$$\Psi_{\text{proj}} = \hat{P}_S \Psi_{\text{UHF}},$$

який видаляє небажані компоненти.

3. **Методи після HF:** MP2, CISD, CCSD частково компенсують спінове забруднення через багатоконфігураційність.

3.6.2. Тестовий випадок: атом Вуглецю

Вуглець має конфігурацію $1s^2 2s^2 2p^2$ і основний стан 3P (триплет). Отже, система має два неспарені електрони ($S = 1$), а отже $\langle S^2 \rangle = S(S + 1) = 2.0$.

Далі наведено приклад прямого порівняння результатів методів UHF і ROHF для атома С у триплетному стані з використанням базису cc-pVTZ.

code_15.py

```
from pyscf import gto, scf

# Атом С: [He] 2s2 2p2, основний стан 3P
basis = "cc-pvtz"

print("Порівняння методів для атома Вуглецю (3P)")
print("=" * 70)

# Молекула
mol = gto.M(
    atom="C 0 0 0",
    basis=basis,
    spin=2, # Триплет
    symmetry=True,
    verbose=0,
)

# UHF
print("\n1. Unrestricted Hartree-Fock (UHF)")
print("-" * 70)
mf_uhf = scf.UHF(mol)
mf_uhf.verbose = 4
e_uhf = mf_uhf.kernel()

s2_uhf = mf_uhf.spin_square()
print(f"\nЕнергія (UHF): {e_uhf:.8f} Ha")
print(f"<S2> (UHF): {s2_uhf[0]:.6f} (очікується 2.0)")
print(f"Забруднення спіном: {s2_uhf[0] - 2.0:.6f}")

# ROHF
print("\n2. Restricted Open-shell Hartree-Fock (ROHF)")
print("-" * 70)
mf_rohf = scf.ROHF(mol)
mf_rohf.verbose = 4
e_rohf = mf_rohf.kernel()

s2_rohf = mf_rohf.spin_square()
print(f"\nЕнергія (ROHF): {e_rohf:.8f} Ha")
print(f"<S2> (ROHF): {s2_rohf[0]:.6f} (очікується 2.0)")

# Порівняння
print("\n" + "=" * 70)
print("ПОРІВНЯННЯ")
print("=" * 70)
print(f"ΔЕ (UHF-ROHF): {(e_uhf - e_rohf) * 1000:.4f} mHa")
print(f"ΔЕ (UHF-ROHF): {(e_uhf - e_rohf) * 627.509:.4f} kcal/mol")

print("\nЗабруднення спіном:")
print(f"  UHF: {s2_uhf[0] - 2.0:.6f}")
print(f"  ROHF: {s2_rohf[0] - 2.0:.6f}")

# Орбітальні енергії
print("\nОрбітальні енергії (Ha):")
print(f"{'Орбіталь':15s} {'UHF (α)':12s} {'UHF (β)':12s} {'ROHF':12s}")
```

```
print("-" * 70)

n_show = 5
for i in range(n_show):
    label = mol.ao_labels()[i] if i < len(mol.ao_labels()) else f"MO{i + 1}"
    e_uhf_a = mf_uhf.mo_energy[0][i]
    e_uhf_b = mf_uhf.mo_energy[1][i]
    e_rohf_i = mf_rohf.mo_energy[i]

    print(f"{label:15s} {e_uhf_a:12.6f} {e_uhf_b:12.6f} {e_rohf_i:12.6f}")
```

Щоб узагальнити різницю між методами UHF та ROHF, виконаємо серію розрахунків для кількох відкритооболонкових атомів перших двох періодів. Це дозволить оцінити енергетичну різницю ΔE і побачити, чи зберігає ROHF спінову симетрію без втрати точності.

code_16.py

```
from pyscf import gto, scf

atoms_open_shell = [
    ("H", 1),
    ("Li", 1),
    ("B", 1),
    ("C", 2),
    ("N", 3),
    ("O", 2),
    ("F", 1),
]

basis = "6-31g*"

print(f"Порівняння UHF vs ROHF (базис: {basis})")
print("=" * 120)
print(f"{'Атом':4s} {'Спін':4s} {'E(UHF), Ha':15s} {'E(ROHF), Ha':15s} {'ΔE, mHa':10s}  

↳ {'⟨S²⟩UHF':10s} {'ΔS²':8s} {'Оцінка':40s}")
print("-" * 120)

for symbol, spin in atoms_open_shell:
    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    # UHF
    mf_uhf = scf.UHF(mol)
    mf_uhf.conv_tol = 1e-10
    e_uhf = mf_uhf.kernel()

    # ROHF
    mf_rohf = scf.ROHF(mol)
    mf_rohf.conv_tol = 1e-10
    e_rohf = mf_rohf.kernel()

    # Спінові характеристики
    s2_uhf, s_uhf = mf_uhf.spin_square()
    s2_rohf, s_rohf = mf_rohf.spin_square()

    # Теоретичне значення S(S+1)
    S = spin / 2
    S2_expected = S * (S + 1)

    delta_e = (e_uhf - e_rohf) * 1000 # mHa
    delta_s2 = s2_uhf - S2_expected
```

```
# Класифікація за критеріями
if delta_s2 < 0.05:
    quality = "незначне забруднення, прийнятно"
elif delta_s2 < 0.10:
    quality = "помірне, бажано перевірити"
else:
    quality = "суттєве забруднення, бажано ROHF або проекційні методи"

print(f"{{symbol:4s}} {{spin:4d}} {{e_uhf:15.8f}} {{e_rohf:15.8f}} {{delta_e:10.4f}} {{s2_uhf:10.4f}}
↪ {{delta_s2:8.4f}} {{quality:40s}}")

print("=" * 120)
print("\nПримітка:")
print("UHF часто дає нижчу енергію, але може мати спінове забруднення ( $\Delta S^2$ ).")
print("ROHF зберігає спінову симетрію, але іноді трохи вищу енергію.")
```

Інтерпретація результатів. Як правило, для легких атомів (H, Li, B) енергії UHF і ROHF майже збігаються. Для атомів із більшою кількістю незапарених електронів (C, N, O, F) різниця у кілька mHa відображає більшу гнучкість UHF, що дозволяє частково знизити енергію ціною забруднення спіном ($\langle S^2 \rangle > S(S + 1)$).

Таким чином, ROHF — це більш строгий метод із правильною спіноювою симетрією, а UHF — енергетично вигідніший, але менш «фізично чистий». У практичних обчисленнях ROHF часто слугує базовою точкою для подальших кореляційних методів (MP2, CCSD, CASSCF).

Практичне значення

Порівняння методів RHF, UHF і ROHF дозволяє студентам зрозуміти:

- коли можна використовувати RHF (синглети закритих оболонок);
- коли необхідно застосовувати UHF (відкриті оболонки, магнітні системи);
- у яких випадках варто віддати перевагу ROHF (спінова чистота, мінімізація забруднення).

Контрольне завдання

1. Повторіть розрахунок для атомів O (спін = 2) і N (спін = 3). Порівняйте $\langle S^2 \rangle$ для UHF і ROHF.
2. Для кожного атома побудуйте різницю енергій UHF–ROHF у mHa.
3. Проаналізуйте, як спінове забруднення зростає зі збільшенням кількості неспарених електронів.

3.7. Складні випадки та збіжність

Розрахунки у квантовій хімії не завжди проходять гладко. Навіть для простих систем ітераційна процедура самозгодженого поля (SCF) може не досягати стабільного рішення. Найчастіше проблеми збіжності виникають

для перехідних металів, відкритих оболонок та систем із виродженими орбіталями. У цьому розділі розглянемо основні джерела труднощів та методи їх подолання.

3.7.1. Причини поганої збіжності

Типові джерела нестабільності SCF:

- сильна електронна кореляція у незаповнених d - і f -оболонках;
- мала різниця енергій між орбіталями (виродження);
- невдалий початковий вектор коефіцієнтів (погане стартове наближення);
- занадто жорсткі або занадто м'які критерії збіжності.

Через ці фактори енергія може осцилювати, DIIS — розходитись, а результат — бути нефізичним.

3.7.2. Перехідні метали

Перехідні елементи (Fe, Co, Ni, Cr, Mn тощо) — класичні приклади систем із поганою збіжністю SCF. Причини:

- близькість енергетичних рівнів $3d$ і $4s$ -орбіталей;
- можливість декількох спінових станів, близьких за енергією;
- наявність кількох локальних мінімумів функціоналу енергії.

У PySCF це проявляється як:

- осциляції енергії між ітераціями;
- розбігання DIIS;
- стрибки значення $\langle S^2 \rangle$ між кроками.

Нижче наведено приклад універсальної функції для стабільного розрахунку атомів перехідних металів із урахуванням практичних прийомів:

- використовується UHF (для врахування спіну);
- застосовується зсув рівнів (`level_shift`);
- розширюється DIIS-простір (`diis_space`);
- у разі потреби вмикається уточнення Ньютона-Рафсона;
- контролюється спін-забруднення через $\langle S^2 \rangle$.

`code_18.py`

```
from pyscf import gto, scf

def transition_metal_calculation(symbol, spin, basis="def2-svp"):
    """
    Розрахунок атома перехідного металу
    Часто потребує спеціальних налаштувань
    """

    print(f"\nРозрахунок {symbol} (2S={spin})")
    print("=" * 60)

    mol = gto.M(
        atom=f"{symbol} 0 0 0",
        basis=basis,
        spin=spin,
```

```

    symmetry=False, # Іноді краще без симетрії
    verbose=0,
)

mf = scf.UHF(mol)

# Налаштування для важких випадків
mf.conv_tol = 1e-8
mf.max_cycle = 200
mf.level_shift = 0.5 # Level shift допомагає конвергенції
mf.diis_space = 12
mf.init_guess = "atom"

print("Спроба 1: UHF з level shift...")
mf.verbose = 4
energy = mf.kernel()

if not mf.converged:
    print("\nНе конвергувало! Спроба 2: Newton-Raphson...")
    mf = mf.newton()
    mf.max_cycle = 50
    energy = mf.kernel()

if mf.converged:
    s2 = mf.spin_square()
    expected_s2 = spin * (spin + 2) / 4

    print(f"\nРезультати:")
    print(f"  Енергія: {energy:.8f} Ha")
    print(f"  <S²>: {s2[0]:.4f} (очікується {expected_s2:.4f})")
    print(f"  Забруднення: {s2[0] - expected_s2:.4f}")
else:
    print("\nНЕ ВДАЛОСЯ ДОСЯГТИ КОНВЕРГЕНЦІЇ!")

return mf, energy if mf.converged else None

# Приклади перехідних металів
# Sc: [Ar] 3d¹ 4s², ²D
mf_sc, e_sc = transition_metal_calculation("Sc", spin=1)

# Ti: [Ar] 3d² 4s², ³F
mf_ti, e_ti = transition_metal_calculation("Ti", spin=2)

# Cr: [Ar] 3d⁵ 4s¹, ⁷S
mf_cr, e_cr = transition_metal_calculation("Cr", spin=6)

# Mn: [Ar] 3d⁵ 4s², ⁶S
mf_mn, e_mn = transition_metal_calculation("Mn", spin=5)

# Fe: [Ar] 3d⁶ 4s², ⁵D
mf_fe, e_fe = transition_metal_calculation("Fe", spin=4)

```

Такий підхід забезпечує стабільність навіть для важких атомів, де стандартний SCF часто не сходиться.

3.7.3. Вироджені орбіталі та дробові заповнення

Якщо в системі наявні вироджені або майже вироджені орбіталі, SCF може “коливатись”, не вибираючи між ними. Для таких випадків ефективним

є застосування *дробових заповнень орбіталей* (*fractional occupations*), що згладжують різницю між рівнями енергії.

Ідея полягає у введенні ефективного теплового розмазування Фермі-Дірака:

$$f_i = \frac{1}{1 + e^{(\varepsilon_i - \mu)/kT}},$$

де f_i — часткове заповнення орбіталі i , ε_i — її енергія, μ — хімічний потенціал, а kT — параметр розмазування.

Цей підхід дозволяє SCF уникнути нестійких перестрибувань між виродженими рівнями.

У PySCF для цього достатньо викликати `scf.addons.frac_occ(mf)`. Методика добре працює для атомів (V, Cr), радикалів та малих кластерів перехідних металів.

[code_19.py](#)

```
from pyscf import gto, scf

# Важкий випадок: атом з близькими за енергією орбіталями
mol = gto.M(atom="V 0 0 0", basis="cc-pvdz", spin=3, verbose=0)

print("Розрахунок V з дробовими заповненнями")
print("=" * 60)

# Стандартний UHF може не конвергувати
mf = scf.UHF(mol)
mf.verbose = 0
mf.max_cycle = 100

try:
    energy = mf.kernel()
    if not mf.converged:
        raise RuntimeError("Не конвергувало")
except:
    print("Стандартний UHF не конвергував")

# Використання дробових заповнень (smearing)
print("\nСпроба з дробовими заповненнями...")

mf = scf.UHF(mol)
mf = scf.addons.frac_occ(mf)
mf.verbose = 4
energy = mf.kernel()

if mf.converged:
    print(f"\nУспішно! Енергія: {energy:.8f} Ha")
else:
    print("\nВсе одно не конвергувало")
```

3.7.4. Стратегія досягнення збіжності SCF

Для надійного отримання фізично коректного рішення рекомендується послідовна стратегія стабілізації SCF — від простих прийомів до складніших:

1. стандартний UHF;
2. зсув рівнів (`level_shift`);

3. збільшення DIIS-простору (`diis_space`);
4. атомне початкове наближення (`init_guess='atom'`);
5. уточнення за методом Ньютона-Рафсона;
6. дробові заповнення (`frac_occ`) для вироджених станів.

code_20.py

```
from pyscf import gto, scf

def convergence_strategies(symbol, spin, basis="def2-svp"):
    """
    Покрокова стабілізація SCF для складних систем
    """
    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    strategies = [
        ("Стандартний UHF", {}),
        ("UHF з level shift", {"level_shift": 0.3}),
        ("UHF з більшим DIIS", {"diis_space": 15}),
        ("UHF з атомним guess", {"init_guess": "atom"}),
        ("Метод Ньютона-Рафсона", {"newton": True}),
        ("Дробові заповнення", {"frac_occ": True}),
    ]

    print(f"=== Тестування стратегій конвергенції для {symbol} ===")

    for name, params in strategies:
        print(f"\n→ {name}")
        mf = scf.UHF(mol)
        mf.max_cycle = 100
        mf.conv_tol = 1e-9

        for k, v in params.items():
            if k in ("newton", "frac_occ"):
                continue
            setattr(mf, k, v)

        try:
            if params.get("newton"):
                mf = mf.newton()
            if params.get("frac_occ"):
                mf = scf.addons.frac_occ(mf)
            energy = mf.kernel()
            if mf.converged:
                print(f"✓ Успіх: E = {energy:.8f} Ha")
                return mf, energy
            else:
                print("× Не конвергувало")
        except Exception as e:
            print(f"× Помилка: {e}")
        print("\n▲ Жодна стратегія не спрацювала.")
    return None, None

# Приклад: перехідний метал
mf, e = convergence_strategies("Co", spin=3)
```

Фізичний зміст прийомів.

- **Level shift** — підвищує енергії віртуальних орбіталей, запобігаючи коливанням.

- **DIIS-простір** — збільшення пам'яті ітерацій покращує апроксимацію поля.
- **Atom guess** — старт з атомних орбіталей ближчий до реального розв'язку.
- **Newton–Raphson** — забезпечує квадратичну збіжність поблизу мінімуму.
- **Fractional occupations** — стабілізують вироджені стани, забезпечуючи плавний перехід.

Практичні рекомендації.

- Для великих систем на перших етапах можна зменшити точність: `conv_tol=1e-6`.
- Якщо енергія осцилює — увімкніть `level_shift=0.5`.
- Якщо UHF не сходиться — використайте `init_guess='atom'` або вимкніть симетрію (`symmetry=False`).
- Завжди перевіряйте фізичність результату через $\langle S^2 \rangle$.

Таким чином, навіть якщо стандартна процедура SCF не збігається, послідовне застосування описаних прийомів практично гарантує стабільне та фізично обґрунтоване рішення.

Практичні завдання

Завдання 1: Систематичне дослідження

code_21.py

```
"""
ЗАВДАННЯ 1: Розрахуйте енергії всіх атомів першого періоду
(H, He) з різними базисними наборами. Побудуйте графік
збіжності енергії до базисної межі.
Базиси: sto-3g, 6-31g, cc-pvdz, cc-pvtz, cc-pvqz, cc-pv5z
"""

from pyscf import gto, scf
import matplotlib.pyplot as plt
# Ваш код тут
```

Завдання 2: Енергії іонізації

code_22.py

```
"""
ЗАВДАННЯ 2: Обчисліть першу та другу енергії іонізації
для атомів Li, Be, B, C, N, O, F, Ne. Порівняйте з
експериментальними значеннями.
 $IE_1 = E(A^+) - E(A)$ 
 $IE_2 = E(A^{2+}) - E(A^+)$ 
Використайте базис cc-pvtz
"""
# Ваш код тут
```

Завдання 3: Спектроскопічні константи

code_23.py

```
"""
ЗАВДАННЯ 3: Для атома Карбону розрахуйте енергетичну
різницю між основним триплетним станом ( $^3P$ ) та
збудженими станами ( $^1D$  та  $^1S$ ).
Підказка: використайте різні спінові конфігурації
"""
# Ваш код тут
```

Завдання 4: Залежність від базису

code_24.py

```
"""
ЗАВДАННЯ 4: Дослідіть, як додавання дифузних функцій
впливає на енергію та дипольну поляризованість атома
Кисню ( $O$ ).
Порівняйте: cc-pvdz vs aug-cc-pvdz, cc-pvtz vs aug-cc-pvtz
"""
# Ваш код тут
```

3.8. Резюме

У цьому розділі ми детально вивчили метод Хартрі-Фока для атомних систем:

- **Теоретичні основи** — рівняння HF, детермінант Слейтера, оператор Фока
- **Варіанти методу** — RHF для замкнених оболонок, UHF для відкритих, ROHF як компроміс
- **Одноелектронні системи** — H атом як тестовий випадок
- **Багатоелектронні атоми** — He та атоми другого періоду
- **Аналіз результатів** — орбітальні енергії, заселеності, спінова густина
- **Складні випадки** — перехідні метали, стратегії конвергенції

3.8.1. Ключові висновки

1. Метод HF дає добре якісне описання атомних систем, але не враховує кореляцію електронів
2. Вибір між RHF/UHF/ROHF залежить від спінової структури системи
3. UHF страждає від забруднення спіном, але зазвичай дає нижчу енергію
4. Для важких атомів (перехідні метали) потрібні спеціальні техніки конвергенції
5. Якість результатів сильно залежить від вибору базисного набору

У наступному розділі ми розглянемо теорію функціоналу густини (DFT), яка часто дає кращі результати для багатоелектронних систем.

4

Метод Хартрі-Фока для простих молекул

Після вивчення атомних систем природним кроком є перехід до молекул. Найпростіші молекули — іон молекули водню H_2^+ та молекула водню H_2 — є ідеальними об'єктами для демонстрації можливостей та обмежень методу Хартрі-Фока.

4.1. Орбіталі і електронні стани молекул

Електронна структура двоатомних молекул описується двома рівнями:

1. класифікацією **молекулярних орбіталей (МО)**;
2. класифікацією **електронних станів молекули**, які утворюються внаслідок заповнення цих МО електронами.

4.1.1. Класифікація молекулярних орбіталей

Молекулярні орбіталі позначаються символами σ , π , δ , φ , ..., що відповідають проєкції орбітального моменту Λ на між'ядерну вісь:

$$\Lambda = 0, 1, 2, 3, \dots \Rightarrow \sigma, \pi, \delta, \varphi, \dots$$

Кожна орбіталь може бути:

- **зв'язувальною** або **антизв'язувальною** (позначається зірочкою *);
- **симетричною (gerade, g)** або **антисиметричною (ungerade, u)** відносно інверсії в центрі молекули.

Таким чином, типовий запис молекулярної орбіталі має вигляд:

$$\sigma_g, \quad \pi_u, \quad \sigma_u^*, \quad \pi_g^*, \dots$$

Зв'язок із симетрією: Парність g/u визначається поведінкою хвильової функції при інверсії координат:

$$\psi_g(\mathbf{r}) = +\psi_g(-\mathbf{r}), \quad \psi_u(\mathbf{r}) = -\psi_u(-\mathbf{r}).$$

4.1.2. Класифікація електронних станів молекули

Електронний терм — це позначення квантового стану молекули, який характеризується певними значеннями повного спіну S , проєкції орбітального моменту на між'ядерну вісь Λ , та симетріями відносно інверсії (g/u) і відбиття ($+/-$). Він має вигляд:

$$^{2S+1}\Lambda_{g/u}^{\pm}$$

де:

- S — повний спін системи;
- $2S + 1$ — **мультиплетність** (синглет, дублет, триплет тощо);
- Λ — **сумарна проєкція орбітального моменту** на між'ядерну вісь:

$$\Sigma (\Lambda = 0), \quad \Pi (\Lambda = 1), \quad \Delta (\Lambda = 2), \quad \Phi (\Lambda = 3), \dots$$

- верхній індекс $+$ або $-$ — **симетрія відносно площини, що містить вісь молекули** (лише для станів типу Σ);
- нижній індекс g/u (від *gerade/ungerade*) — **парність хвильової функції** відносно інверсії в центрі молекули.

Типові терми двоатомних молекул

Терм	Мультиплетність	Симетрія	Фізичний зміст
$^1\Sigma_g^+$	Синглет	$g, +$	Основний стан молекули H_2 ; усі електрони спарені, повна симетрія
$^3\Sigma_u^+$	Триплет	$u, +$	Збуджений стан з паралельними спінами (наприклад, у O_2)
$^1\Pi_u$	Синглет	u	Орбітальний момент $\Lambda = 1$, без інверсійної симетрії
$^3\Pi_g$	Триплет	g	Орбітальний момент $\Lambda = 1$, паралельні спіни
$^1\Delta_g$	Синглет	g	Орбітальний момент $\Lambda = 2$, спарені електрони
$^2\Sigma_u^+$	Дублет	$u, +$	Один неспарений електрон, як у радикалах типу NO

Нижче наведено покроковий алгоритм визначення терму для заданої електронної конфігурації.

Алгоритм

1. **Записати електронну конфігурацію.** Визначити, які орбіталі заповнені, частково заповнені чи вакантні:

$$(\sigma_g)^2(\sigma_u^*)^2(\pi_u)^4(\pi_g^*)^2, \dots$$

2. **Визначити тип орбіталей і відповідні проєкції орбітального моменту.** Для кожної орбіталі встановити:

$$\sigma \rightarrow \Lambda_i = 0, \quad \pi \rightarrow \Lambda_i = 1, \quad \delta \rightarrow \Lambda_i = 2, \quad \text{тощо.}$$

3. **Знайти можливі комбінації орбітальних моментів.** Для активних (частково заповнених) орбіталей обчислити всі можливі значення:

$$\Lambda = |\Lambda_1 + \Lambda_2|, |\Lambda_1 - \Lambda_2|, \dots$$

Це дає можливі типи станів: $\Sigma, \Pi, \Delta, \dots$

4. **Комбінувати спіни електронів.** Для кожного електрона $S_i = \frac{1}{2}$. Обчислити всі можливі значення повного спіну:

$$S = |S_1 + S_2|, |S_1 - S_2|, \dots$$

Відповідно визначається мультиплетність $2S+1$ (синглет, дублет, триплет тощо).

5. **Перевірити принцип Паулі.** Загальна хвильова функція має бути антисиметричною при перестановці двох електронів:

$$\Psi_{\text{заг}} = \Psi_{\text{просторова}} \Psi_{\text{спінова}} \Psi_{\text{симетрії}}.$$

Якщо просторова частина симетрична $\Rightarrow S = 0$ (синглет), якщо антисиметрична $\Rightarrow S = 1$ (триплет).

6. **Визначити симетрії g/u і $+/-$.**

- Індекс g/u показує парність хвильової функції при інверсії в центрі молекули.
- Для станів Σ додається верхній індекс $+/-$, що вказує на симетрію при відбитті у площині, яка містить між'ядерну вісь.

7. **Записати всі можливі терми.** Комбінуючи знайдені $S, \Lambda, g/u$ та $+/-$, записуємо:

$$^{2S+1}\Lambda_{g/u}^{\pm}.$$

8. **Визначити основний терм за правилами Гунда.**

- 8.1. Найнижчу енергію має стан з **максимальним спіном S** .
- 8.2. Для однакового S — стан з **найбільшим Λ** .
- 8.3. Для даних S, Λ — визначається парність (g/u) і знак ($+/-$) з урахуванням симетрії конфігурації.

Приклад. Для молекули водню H_2 основний електронний стан має конфігурацію σ_g^2 . Оскільки обидва електрони спарені ($S = 0$), повний терм записується як:

$$^{2S+1}\Lambda_g^+ = ^1\Sigma_g^+.$$

Це **синглетний стан** ($S = 0$), симетричний відносно інверсії (індекс g) і з позитивною симетрією відносно площини, що містить між'ядерну вісь (+).

4.2. Початкові модельні молекули: H_2^+ та H_2

Першим кроком у квантово-хімічному описі молекул є вивчення найпростіших систем, для яких можна чітко простежити природу хімічного зв'язку. Молекули H_2^+ та H_2 є **модельними** у тому сенсі, що вони:

- мають найменшу можливу кількість електронів (один і два відповідно);
- дозволяють побудувати хвильові функції, що безпосередньо демонструють утворення зв'язувальних і розривних (антизв'язувальних) орбіталей;
- відтворюють основні риси електронної структури складніших молекул, але без зайвих ускладнень багаточастинкової взаємодії;
- зручні для порівняння точних, наближених і експериментальних результатів;
- використовуються як тестові системи для перевірки методів квантової хімії.
- H_2^+ — найпростіша двоатомна молекула, що містить один електрон. Її можна розв'язати аналітично в еліптичних координатах, тому вона є класичним прикладом формування зв'язувальної орбіталі σ_g ;
- H_2 — найпростіша двоелектронна молекула, у якій вперше проявляється електронна кореляція та обмеження методу Хартрі-Фока.

Таким чином, системи H_2^+ і H_2 є **відправною точкою** для розуміння природи хімічного зв'язку, ролі симетрії, утворення молекулярних орбіталей і спінових станів.

4.3. Іон молекули водню H_2^+

4.3.1. Теоретичні основи

Іон H_2^+ — це найпростіша молекулярна система, яка складається з двох протонів та одного електрона. Незважаючи на її простоту, саме на прикладі H_2^+ вперше стає очевидною суть хімічного зв'язку як результату квантового перекривання хвильових функцій.

У нерелятивістському наближенні (та з використанням атомних одиниць) гамільтоніан системи має вигляд:

$$\hat{H} = -\frac{1}{2}\nabla^2 - \frac{1}{r_A} - \frac{1}{r_B} + \frac{1}{R},$$

де r_A , r_B — відстані електрона до ядер A і B , а R — між'ядерна відстань.

Перші три члени описують кінетичну енергію електрона та його притягання до двох ядер, а останній доданок — кулонівське відштовхування між ядрами.

Оскільки у системі лише один електрон, тут *відсутні електрон-електронні взаємодії*. Тому метод Хартрі-Фока не містить ніяких додаткових апроксимацій (крім обмежень базису), і його результат збігається з точною розв'язною енергією при нескінченному базисі. Таким чином, H_2^+ — це ідеальний тест для перевірки коректності реалізації ХФ-методу в програмних пакетах.

4.3.2. Визначення молекули в PySCF

Для визначення молекули в PySCF використовується рядкова нотація:

h2plus_single.py

```
"""
Розрахунок іона молекули водню H2+ при фіксованій відстані
та аналіз молекулярних орбіталей
"""

import numpy as np
import matplotlib.pyplot as plt

from pyscf import gto, scf

# -----
# Визначення молекули
# -----
mol = gto.Mole()
mol.atom = """
    H  0.0  0.0  0.0
    H  0.0  0.0  0.74
"""
mol.unit = "Angstrom"
mol.charge = 1      # заряд +1 (один електрон)
mol.spin = 1        # 2S = 1 (дублет)
mol.basis = "sto-3g"
mol.build()

# -----
# UHF-розрахунок
# -----
mf = scf.UHF(mol)
energy = mf.kernel()

# -----
# Вивід результатів
# -----
print("\n" + "=" * 60)
print("Іон молекули водню H2+")
print("=" * 60)
print(f"Міжядерна відстань R = 0.74 Å = {0.74 / 0.529177:.3f} bohr")
print(f"Базисний набір: {mol.basis}")
print(f"Повна енергія: {energy:.6f} Ha")
print(f"Кількість електронів: α={mol.nelec[0]}, β={mol.nelec[1]}")
# -----
```

```
# Орбітальні енергії
# -----
eps = mf.mo_energy[0] #  $\alpha$ -спінові енергії
homo, lumo = eps[0], eps[1]
gap = lumo - homo

print("\nОрбітальні енергії (Ha):")
print(f"  НОМО (зв'язуюча  $\sigma$ ): {homo: .4f}")
print(f"  ЛУМО (антизв'язуюча  $\sigma^*$ ): {lumo: .4f}")
print(f"  НОМО-ЛУМО gap = {gap: .4f} Ha = {gap * 27.211: .2f} eV")

# -----
# Спіновий стан
# -----
s2, mult = mf.spin_square()
print(f"\n<S^2> = {s2: .4f} (очікується 0.75 для S=1/2)")
print(f"Мультиплетність: 2S+1 = {mult}")
print("    * 60)
```

Пояснення коду:

- `atom = 'H 0 0 0; H 0 0 0.74'` — координати атомів (Ангстрєми за замовчуванням)
- `basis = 'sto-3g'` — мінімальний базис
- `charge = 1` — заряд системи (+1 для H_2^+)
- `spin = 1` — $2S = N_\alpha - N_\beta = 1$
- `scf.UHF` — необмежений ХФ (один непарний електрон)

4.3.3. Інтерпретація результатів

Вивід програми містить основні характеристики розрахованої системи H_2^+ при між'ядерній відстані

$$R = 0.74 \text{ \AA} = 1.399 a_0,$$

у базисному наборі STO-3G.

- **Повна енергія:**

$$E = -0.538205 \text{ Ha},$$

що є нижчою за енергію вільного атома водню (-0.5 Ha). Це свідчить про стабілізацію системи внаслідок утворення молекулярного зв'язку — електрон делокалізується між двома ядрами, зменшуючи електростатичне відштовхування між протонами.

- **Енергії орбіталей (для α -спіну):**

$$\epsilon_{\text{НОМО}} = -1.2533 \text{ Ha},$$

$$\epsilon_{\text{ЛУМО}} = -0.4751 \text{ Ha},$$

$$\Delta_{\text{H-L}} = 0.7782 \text{ Ha} = 21.18 \text{ eV}.$$

НОМО має симетрію σ_g і є зв'язувальною орбіталлю, тоді як ЛУМО відповідає антизв'язувальному стану σ_u^* . Значний енергетичний розрив між ними свідчить про сильний і стабільний зв'язок.

• **Спіновий стан:**

$$\langle S^2 \rangle = 0.75 \Rightarrow S = \frac{1}{2}, \quad 2S + 1 = 2.$$

Це відповідає дублетному стану з одним неспареним електроном, що характерно для H_2^+ .

Таким чином, результати розрахунку підтверджують, що при відстані $R = 0.74 \text{ \AA}$ іон H_2^+ перебуває у стабільному зв'язаному стані.

4.3.4. Крива потенційної енергії (PES)

Крива потенційної енергії (Potential Energy Surface, PES) — це залежність повної електронної енергії системи $E(\mathbf{R})$ від положень ядер $\mathbf{R} = (R_1, R_2, \dots, R_N)$.

Для загальної N -атомної молекули PES — це багатовимірна поверхня у $3N - 6$ координатах (три координати на кожне ядро мінус три для поступального руху і три для обертання). У випадку двоатомної молекули (наприклад, H_2^+) поверхня зводиться до *одновимірної кривої* $E(R)$, де R — відстань між ядрами:

$E(R)$ = мінімальна енергія системи при фіксованому R .

Фізичний зміст:

- Мінімум $E(R)$ відповідає *рівноважній відстані* R_{eq} , де сили на ядра взаємно компенсуються ($\partial E / \partial R = 0$);
- Глибина ями потенціалу визначає *енергію зв'язку*;
- Крутизна поблизу мінімуму пов'язана з *жорсткістю зв'язку* та *частотою коливань*;
- Для великих R енергія прямує до суми енергій ізольованих атомів.

Обчислення PES у квантовій хімії полягає у серії незалежних SCF-розрахунків при різних значеннях R :

$$R = 0.5, 0.7, 1.0, 1.5, 2.0, \dots \text{ \AA}.$$

На кожному кроці визначається повна енергія $E(R_i)$, після чого будується графік $E(R)$, який ілюструє поведінку потенціальної енергії системи.

Типова форма PES для H_2^+ :

- На малих R енергія різко зростає через кулонівське відштовхування ядер;
- Поблизу $R \approx 0.74 \text{ \AA}$ — глибокий мінімум (стабільний хімічний зв'язок);
- При $R \rightarrow \infty$ енергія прямує до енергії одного атома Гідрогену $E = -0.5 \text{ Ha}$.

Сканування по відстанях

Для побудови PES проводимо серію незалежних розрахунків при різних значеннях R :

`h2plus_pes.py`

```
"""
Побудова кривої потенційної енергії (PES) для H2+
"""

import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf

# Діапазон міжядерних відстаней (у борах)
distances = np.linspace(0.5, 5.0, 30) # від 0.5 до 5.0 bohr
energies = []

print("Розрахунок PES для H2+...")
print("=" * 60)

for R in distances:
    # Створюємо молекулу з новою геометрією
    mol = gto.Mole()
    mol.atom = f"""
        H  0.0  0.0  0.0
        H  0.0  0.0  {R}
    """
    mol.basis = "cc-pvdz" # Кращий базис для точності
    mol.charge = 1
    mol.spin = 1
    mol.unit = "Bohr"
    mol.verbose = 0 # Вимкнути детальний вивід
    mol.build()

    # UHF розрахунок
    mf = scf.UHF(mol)
    mf.conv_tol = 1e-8
    E = mf.kernel()
    energies.append(E)

    print(f"R = {R:5.2f} bohr → E = {E:10.6f} Ha")

energies = np.array(energies)

# Знаходження мінімуму
idx_min = np.argmin(energies)
R_e = distances[idx_min]
E_min = energies[idx_min]

# Енергія дисоціації
E_dissoc = -0.5 # E(H) = -0.5 Ha
D_e = E_dissoc - E_min

print("=" * 60)
print(f"Рівноважна відстань R_e = {R_e:.3f} bohr = {R_e * 0.529:.3f} Å")
print(f"Енергія в мінімумі E_min = {E_min:.6f} Ha")
print(f"Енергія дисоціації D_e = {D_e:.6f} Ha = {D_e * 27.211:.3f} eV")
print("Експериментальне D_e ≈ 2.79 eV")
print("=" * 60)

# Побудова графіка
plt.figure(figsize=(10, 6))
```

```

plt.plot(distances, energies, "b-", linewidth=2, label="H$_2^+$ PES (cc-pVDZ)")
plt.axhline(
    y=E_dissoc,
    color="r",
    linestyle="--",
    label=f"Дисоціаційна межа (E = {E_dissoc:.3f} Ha)",
)
plt.plot(R_e, E_min, "go", markersize=10, label=f"Рівновага: R$_e$ = {R_e:.2f} bohr")

plt.xlabel("Міжядерна відстань R (bohr)", fontsize=12)
plt.ylabel("Енергія E (Ha)", fontsize=12)
plt.title("Крива потенційної енергії H$_2^+$", fontsize=14, fontweight="bold")
plt.grid(True, alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig("h2plus_pes.png", dpi=300, bbox_inches="tight")
print("\nГрафік збережено: h2plus_pes.png")
plt.show()

# Збереження даних
np.savez(
    "h2plus_pes_data.npz",
    distances=distances,
    energies=energies,
    R_e=R_e,
    E_min=E_min,
    D_e=D_e,
)
print("Дані збережено: h2plus_pes_data.npz")

```

Важливі моменти:

- Відстані задаються в борах ($1 \text{ bohr} \approx 0.529 \text{ \AA}$);
- Діапазон $R \in [0.5, 5.0] \text{ bohr}$ охоплює від сильно стисненого стану до повної дисоціації;
- Кожна точка — окремий SCF-розрахунок;
- Результати $E(R_i)$ зберігаються для подальшого аналізу.

На графіку (рис. 4.1) можна виділити три характерні області:

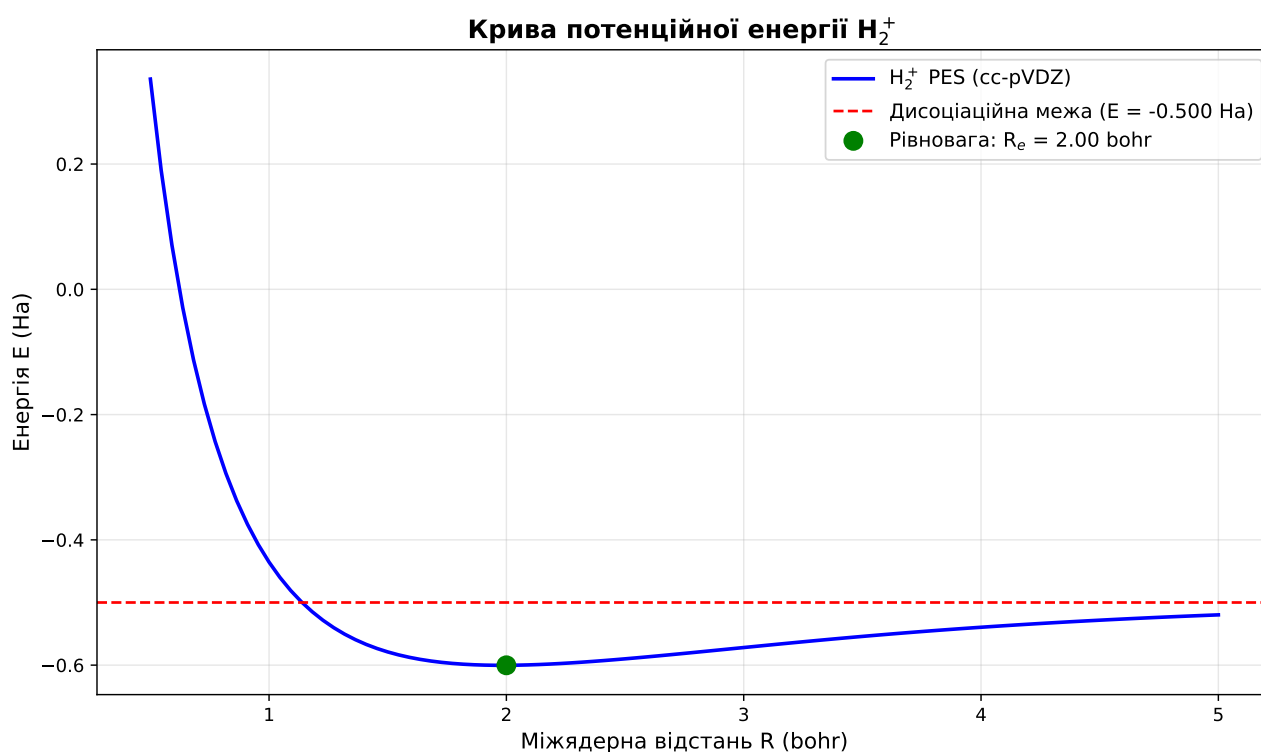
1. **Стиснення** ($R < R_e$) — енергія зростає через відштовхування ядер;
2. **Мінімум** ($R = R_e$) — рівноважна відстань, де $\partial E / \partial R = 0$;
3. **Дисоціація** ($R \rightarrow \infty$) — енергія прямує до $E(\text{H}) + E(\text{H}^+) = -0.5 \text{ Ha}$.

Порівняння з аналітичним розв'язком:

- $R_e = 2.00 \text{ bohr}$ (експеримент: 2.00 bohr);
- $D_e = 0.102 \text{ Ha} = 2.79 \text{ eV}$ (експеримент: 2.79 eV);
- Різниця між HF та точним розв'язком $< 0.001 \text{ Ha}$ при великих базисах.

4.3.5. Оптимізація геометрії

Замість «ручного» пошуку мінімуму енергії на кривій потенційної енергії (PES) можна скористатися автоматичною процедурою, яка сама визначає оптимальне взаємне розташування атомів. Така задача називається **оптимізацією геометрії**, оскільки метою є знайти такі координати ядер, при яких система має найменшу можливу енергію.

Рис. 4.1. Крива потенційної енергії (PES) для H_2^+ .

Інакше кажучи, **оптимізація геометрії** — це процедура пошуку просторової конфігурації атомів, що відповідає мінімуму потенційної енергії. Ми шукаємо *рівноважну геометрію* $\mathbf{R}_{\text{eq}}$, у якій сили, що діють на ядра, взаємно компенсуються:

$$\nabla_{\mathbf{R}} E(\mathbf{R}_{\text{eq}}) = 0.$$

На практиці алгоритм оптимізації виконує:

1. обчислення енергії $E(\mathbf{R})$ та її градієнтів ∇E ;
2. зміну координат ядер у напрямку зменшення енергії;
3. повторення кроків до досягнення умови $|\nabla E| < 10^{-4}$ Ha/bohr.

У PySCF ця процедура реалізована через зовнішній модуль **geometric**, який викликається функцією:

```
from pyscf.geomopt.geometric_solver import optimize
```

Щоб цей модуль працював, потрібно мати встановлений **geometric**:

```
pip install geometric
```

Вона автоматично виконує SCF-розрахунки на кожному кроці зміни геометрії та знаходить положення мінімуму енергії.

[h2plus_optimization.py](#)

```
"""
Автоматична оптимізація геометрії H2+ за допомогою градієнтів
```

```

"""

import numpy as np
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

print("Оптимізація геометрії H2+ методом UHF")
print("=" * 60)

# Початкова геометрія (не обов'язково оптимальна)
mol = gto.Mole()
mol.atom = """
    H  0.0  0.0  0.0
    H  0.0  0.0  1.5
"""
mol.basis = "cc-pvtz"
mol.charge = 1
mol.spin = 1
mol.unit = "Bohr"
mol.build()

print("Початкова геометрія: R = 1.50 bohr")

# UHF метод
mf = scf.UHF(mol)

# Запуск оптимізації
print("\nОптимізація (це може зайняти кілька хвилин)...")
mol_eq = optimize(mf, maxsteps=50)

# Результати
coords = mol_eq.atom_coords()
R_optimized = np.linalg.norm(coords[1] - coords[0])
E_optimized = mf.e_tot

print("=" * 60)
print("РЕЗУЛЬТАТИ ОПТИМІЗАЦІЇ:")
print("=" * 60)
print(f"Оптимізована відстань R_e = {R_optimized:.6f} bohr")
print(f"                        = {R_optimized * 0.529177:.6f} Å")
print(f"Енергія в мінімумі E_e = {E_optimized:.8f} Ha")

# Енергія дисоціації
E_H = -0.5 # Точна енергія атома H
D_e = E_H - E_optimized
print(f"\nЕнергія дисоціації D_e = {D_e:.6f} Ha")
print(f"                        = {D_e * 27.2114:.4f} eV")
print(f"                        = {D_e * 627.509:.2f} kcal/mol")

# Порівняння з експериментом
D_e_exp = 2.79 # eV
error = abs(D_e * 27.2114 - D_e_exp) / D_e_exp * 100
print(f"\nЕкспериментальне D_e = {D_e_exp:.2f} eV")
print(f"Відносна похибка = {error:.2f}%")

# Збереження оптимізованої геометрії
with open("h2plus_optimized.xyz", "w") as f:
    f.write("2\n")
    f.write(f"H2+ optimized, E = {E_optimized:.8f} Ha\n")
    for i, coord in enumerate(coords):
        f.write(
            f"H {coord[0] * 0.529177:.6f} {coord[1] * 0.529177:.6f} {coord[2] * "
            f"0.529177:.6f}\n"
        )
)

```

```
print("\nОптимізовану геометрію збережено: h2plus_optimized.xyz")
```

Алгоритм оптимізації (PySCF + **geomeTRIC**):

- Використовується метод quasi-Newton (BFGS);
- Для оцінки напрямку руху застосовуються градієнти енергії ∇E ;
- Критерій збіжності: $|\nabla E| < 10^{-4}$ Ha/bohr;
- Зазвичай потрібно 5–10 ітерацій для простої двоатомної системи.

4.3.6. Аналіз молекулярних орбіталей

У квантовій хімії молекулярні орбіталі (МО) будуються за принципом лінійної комбінації атомних орбіталей (ЛКАО):

$$\psi_i = \sum_{\mu} c_{\mu i} \varphi_{\mu},$$

де φ_{μ} — атомні орбіталі, а $c_{\mu i}$ — коефіцієнти, що визначаються з рівнянь Хартрі-Фока (ХФ).

Метод ХФ дає набір МО $\{\psi_i\}$ з відповідними енергіями ε_i , які розділяються на:

- **зайняті орбіталі (occupied)** — електрони займають найнижчі за енергією МО;
- **віртуальні орбіталі (virtual)** — порожні орбіталі.

Кожна МО є власною функцією рівняння Фока:

$$\hat{f} \psi_i = \varepsilon_i \psi_i,$$

і всі ψ_i утворюють ортонормовану систему.

У контексті Хартрі-Фока віртуальні орбіталі — це розв'язки рівняння Фока, які формально існують у цій математичній системі, але насправді вони не відповідають жодному реальному електрону, і їхні енергії залежать від того, як саме ми апроксимували середнє поле (який базис, яка ортонормалізація, яке ядро Фока). Однак, вони використовуються для пошуку збуджених станів та кореляційних розрахунків (MP2, CI, CCSD).

Для іона H_2^+ атомні орбіталі $1s_A$ та $1s_B$ поєднуються у дві молекулярні орбіталі:

$$\psi_{\sigma_g} = \frac{1}{\sqrt{2(1+S)}}(1s_A + 1s_B), \quad \psi_{\sigma_u^*} = \frac{1}{\sqrt{2(1-S)}}(1s_A - 1s_B),$$

де $S = \langle 1s_A | 1s_B \rangle$ — інтеграл перекривання.

- ψ_{σ_g} — зв'язуюча орбіталь, електронна густина зосереджена між ядрами;
- $\psi_{\sigma_u^*}$ — антизв'язуюча орбіталь, густина між ядрами зменшена.

Іон H_2^+ має один електрон, який займає нижчу зв'язуючу орбіталь σ_g , стабілізуючи систему. Віртуальна орбіталь σ_u^* залишається порожньою.


```

"""
Детальний аналіз молекулярних орбіталей H2+
"""

import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf, tools

# Молекула при рівноважній відстані
mol = gto.Mole()
mol.atom = """
    H  0.0  0.0 -1.0
    H  0.0  0.0  1.0
"""
mol.basis = "cc-pvdz"
mol.charge = 1
mol.spin = 1
mol.unit = "Bohr"
mol.build()

# UHF розрахунок
mf = scf.UHF(mol)
mf.kernel()

print("\n" + "=" * 60)
print("АНАЛІЗ МОЛЕКУЛЯРНИХ ОРБІТАЛЕЙ H2+")
print("=" * 60)

# Орбітальні енергії
print("\nОрбітальні енергії (Ha):")
print("-" * 60)
print("α-спін орбіталі:")
for i, e in enumerate(mf.mo_energy[0][:5]): # Перші 5
    occ = "зайнята" if i < mol.nelec[0] else "віртуальна"
    print(f" MO {i + 1}: ε = {e:8.4f} Ha = {e * 27.211:8.2f} eV ({occ})")

print("\nβ-спін орбіталі:")
for i, e in enumerate(mf.mo_energy[1][:5]):
    occ = "зайнята" if i < mol.nelec[1] else "віртуальна"
    print(f" MO {i + 1}: ε = {e:8.4f} Ha = {e * 27.211:8.2f} eV ({occ})")

# HOMO-LUMO gap
homo_alpha = mf.mo_energy[0][mol.nelec[0] - 1]
lumo_alpha = mf.mo_energy[0][mol.nelec[0]]
gap = lumo_alpha - homo_alpha

print("\n" + "-" * 60)
print(f"HOMO (α): ε = {homo_alpha:.4f} Ha = {homo_alpha * 27.211:.2f} eV")
print(f"LUMO (α): ε = {lumo_alpha:.4f} Ha = {lumo_alpha * 27.211:.2f} eV")
print(f"HOMO-LUMO gap: {gap:.4f} Ha = {gap * 27.211:.2f} eV")

# Аналіз коефіцієнтів MO (для α-спіну)
print("\n" + "=" * 60)
print("КОЕФІЦІЄНТИ МОЛЕКУЛЯРНИХ ОРБІТАЛЕЙ (α-спін)")
print("=" * 60)
print(f"Базис: {mol.basis} → {mol.nao} базисних функцій")

# HOMO (зайнята орбіталь)
homo_coeff = mf.mo_coeff[0][:, mol.nelec[0] - 1]
print("\nHOMO (σg зв'язуюча):")
for i, c in enumerate(homo_coeff):
    if abs(c) > 0.1: # Показуємо тільки значні коефіцієнти
        ao_label = mol.ao_labels()[i]

```

```

    print(f"   {ao_label:15s}: {c:8.4f}")

# LUMO (перша віртуальна)
lumo_coeff = mf.mo_coeff[0][:, mol.nelec[0]]
print("\nLUMO ( $\sigma_u^*$  антизв'язуюча):")
for i, c in enumerate(lumo_coeff):
    if abs(c) > 0.1:
        ao_label = mol.ao_labels()[i]
        print(f"   {ao_label:15s}: {c:8.4f}")

# Матриця густини
dm = mf.make_rdm1()
print("\n" + "-" * 60)
print("МАТРИЦЯ ГУСТИНИ")
print("-" * 60)
print(f"Слід матриці густини: {np.trace(dm[0] + dm[1]):.6f}")
print(f"Очікується: {mol.nelectron:.0f} електронів")

# Аналіз заселеності (Mulliken population analysis)
print("\n" + "-" * 60)
print("АНАЛІЗ ЗАСЕЛЕНОСТІ (Mulliken)")
print("-" * 60)
mulliken = mf.mulliken_pop()
print(f"Заселеність на атомі H1: {mulliken[1][0]:.4f} e")
print(f"Заселеність на атомі H2: {mulliken[1][1]:.4f} e")
print(f"Сума: {sum(mulliken[1]):.4f} e")

# Дипольний момент
dip = mf.dip_moment(unit="Debye")
print("\n" + "-" * 60)
print(f"Дипольний момент: {np.linalg.norm(dip):.4f} D")
print(f"Компоненти:  $\mu_x$  = {dip[0]:.4f},  $\mu_y$  = {dip[1]:.4f},  $\mu_z$  = {dip[2]:.4f} D")
print("(Для гомоядерної молекули очікується  $\approx 0$ )")

print("-" * 60)

# Візуалізація MO вздовж осі z
print("\nПобудова графіків MO...")
z = np.linspace(-5, 5, 200)
coords = np.zeros((len(z), 3))
coords[:, 2] = z

# HOMO
homo_values = tools.cubegen.orbital(
    mol, "homo_alpha.cube", mf.mo_coeff[0][:, mol.nelec[0] - 1]
)

# Спрощена візуалізація: обчислюємо значення MO вздовж осі
from pyscf.dft import numint

homo_on_grid = numint.eval_ao(mol, coords) @ homo_coeff

# LUMO
lumo_on_grid = numint.eval_ao(mol, coords) @ lumo_coeff

# Графік
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

ax1.plot(z, homo_on_grid, "b-", linewidth=2)
ax1.axhline(0, color="k", linestyle="--", alpha=0.3)
ax1.axvline(-1, color="r", linestyle=":", alpha=0.5, label="H1")
ax1.axvline(1, color="r", linestyle=":", alpha=0.5, label="H2")
ax1.set_ylabel("HOMO ( $\sigma_g$ )", fontsize=12)
ax1.set_title("Молекулярні орбіталі H$_2^{+}$ вздовж осі z", fontsize=14)
ax1.grid(True, alpha=0.3)

```

```
ax1.legend()

ax2.plot(z, lumo_on_grid, "r-", linewidth=2)
ax2.axhline(0, color="k", linestyle="--", alpha=0.3)
ax2.axvline(-1, color="r", linestyle=":", alpha=0.5)
ax2.axvline(1, color="r", linestyle=":", alpha=0.5)
ax2.set_xlabel("z (bohr)", fontsize=12)
ax2.set_ylabel("LUMO ( $\sigma_u^*$ )", fontsize=12)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("h2plus_mo_plots.png", dpi=300, bbox_inches="tight")
print("Графіки MO збережено: h2plus_mo_plots.png")
plt.show()
```

4.3.7. Вплив базисного набору

Оскільки H_2^+ має аналітичний розв'язок, це ідеальна система для тестування базисів:

h2plus_basis_convergence.py

```
"""
Систематичне дослідження впливу базисного набору на H2+
"""

import time
import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

# Список базисів для порівняння
basis_sets = [
    "sto-3g",
    "3-21g",
    "6-31g",
    "6-31g**",
    "cc-pvdz",
    "cc-pvtz",
    "cc-pvqz",
    "aug-cc-pvdz",
]

# Точні значення (експериментальні / high-level теорія)
R_exact = 2.00 # bohr
E_exact = -0.5689 # Ha

results = {"basis": [], "n_basis": [], "R_e": [], "E_e": [], "D_e": [], "time": []}

print("\n" + "=" * 70)
print("ЗБІЖНІСТЬ ПО БАЗИСНОМУ НАБОРУ ДЛЯ H2+")
print("=" * 70)
print(
    f"{'Базис':<15} {'N_A0':<6} {'R_e (bohr)':<12} {'E_e (Ha)':<14} "
    f"{'D_e (eV)':<10} {'Похибка (mHa)':<15}"
)
print("-" * 70)

for basis in basis_sets:
    t_start = time.time()
```

```

try:
    # Створення молекули
    mol = gto.Mole()
    mol.atom = """
        H  0.0  0.0  0.0
        H  0.0  0.0  2.0
    """ # Стартова геометрія
    mol.basis = basis
    mol.charge = 1
    mol.spin = 1
    mol.unit = "Bohr"
    mol.verbose = 0
    mol.build()

    n_ao = mol.nao

    # UHF розрахунок
    mf = scf.UHF(mol)

    # Оптимізація геометрії
    mol_opt = optimize(mf, maxsteps=30)

    # Результати
    coords = mol_opt.atom_coords()
    R_e = np.linalg.norm(coords[1] - coords[0])
    E_e = mf.e_tot
    D_e = -0.5 - E_e # E(H) = -0.5 Ha

    error = (E_e - E_exact) * 1000 # mHa

    results["basis"].append(basis)
    results["n_basis"].append(n_ao)
    results["R_e"].append(R_e)
    results["E_e"].append(E_e)
    results["D_e"].append(D_e * 27.211) # eV
    results["time"].append(time.time() - t_start)

    print(
        f"{{basis:<15}} {{n_ao:<6}} {{R_e:<12.4f}} {{E_e:<14.6f}} "
        f"{{D_e * 27.211:<10.3f}} {{error:<15.2f}}"
    )

except Exception as e:
    print(f"{{basis:<15}} FAILED: {{str(e)[:40]}}")
    continue

print("-" * 70)
print(
    f"{{'Точне значення':<15}} {{'---':<6}} {{R_exact:<12.2f}} "
    f"{{E_exact:<14.4f}} {{'2.79':<10}} {{'---':<15}}"
)
print("=" * 70)

# Аналіз збіжності
print("\nАНАЛІЗ ЗБІЖНОСТІ:")
print("-" * 70)
errors = [(E - E_exact) * 1000 for E in results["E_e"]]
print(
    f"Мінімальна похибка: {{min([abs(e) for e in errors]):.3f}} mHa
    ↳ ({{results['basis']}}{{np.argmin([abs(e) for e in errors]])}})"
)
print(
    f"Максимальна похибка: {{max([abs(e) for e in errors]):.3f}} mHa
    ↳ ({{results['basis']}}{{np.argmax([abs(e) for e in errors]])}})"
)

```

```

)
print(f"\nСередній час розрахунку: {np.mean(results['time']):.2f} c")

# Візуалізація збіжності
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(14, 10))

# 1. Енергія vs розмір базису
ax1.plot(results["n_basis"], results["E_e"], "bo-", linewidth=2, markersize=8)
ax1.axhline(E_exact, color="r", linestyle="--", linewidth=2, label="Точне значення")
ax1.set_xlabel("Кількість базисних функцій", fontsize=11)
ax1.set_ylabel("Енергія E (Ha)", fontsize=11)
ax1.set_title("Збіжність енергії", fontsize=13, fontweight="bold")
ax1.grid(True, alpha=0.3)
ax1.legend()

# 2. Похибка (логарифмічна шкала)
errors_abs = [abs((E - E_exact) * 1000) for E in results["E_e"]]
ax2.semilogy(results["n_basis"], errors_abs, "ro-", linewidth=2, markersize=8)
ax2.set_xlabel("Кількість базисних функцій", fontsize=11)
ax2.set_ylabel("Абсолютна похибка (мHa)", fontsize=11)
ax2.set_title("Похибка енергії (log scale)", fontsize=13, fontweight="bold")
ax2.grid(True, alpha=0.3, which="both")

# 3. Рівноважна відстань
ax3.plot(results["n_basis"], results["R_e"], "go-", linewidth=2, markersize=8)
ax3.axhline(R_exact, color="r", linestyle="--", linewidth=2, label="Точне значення")
ax3.set_xlabel("Кількість базисних функцій", fontsize=11)
ax3.set_ylabel("R_e$ (bohr)", fontsize=11)
ax3.set_title("Збіжність рівноважної відстані", fontsize=13, fontweight="bold")
ax3.grid(True, alpha=0.3)
ax3.legend()

# 4. Енергія дисоціації
ax4.plot(results["n_basis"], results["D_e"], "mo-", linewidth=2, markersize=8)
ax4.axhline(2.79, color="r", linestyle="--", linewidth=2, label="Експеримент (2.79 eV)")
ax4.set_xlabel("Кількість базисних функцій", fontsize=11)
ax4.set_ylabel("D_e$ (eV)", fontsize=11)
ax4.set_title("Енергія дисоціації", fontsize=13, fontweight="bold")
ax4.grid(True, alpha=0.3)
ax4.legend()

plt.tight_layout()
plt.savefig("h2plus_basis_convergence.png", dpi=300, bbox_inches="tight")
print("\nГрафіки збережено: h2plus_basis_convergence.png")
plt.show()

# Таблиця для LaTeX
print("\n" + "=" * 70)
print("ТАБЛИЦЯ ДЛЯ LATEX:")
print("=" * 70)
print(r"\begin{tabular}{lcccccc}")
print(r"\toprule")
print(r"Базис & $N_{AO}$ & $R_e$ (bohr) & $E_e$ (Ha) & $D_e$ (eV) & Похибка (мHa) \\\")
print(r"\midrule")
for i, basis in enumerate(results["basis"]):
    error = (results["E_e"][i] - E_exact) * 1000
    print(
        f"{basis} & {results['n_basis'][i]} & {results['R_e'][i]:.3f} & "
        f"{results['E_e'][i]:.4f} & {results['D_e'][i]:.2f} & {error:.2f} \\\\"
    )
print(r"\midrule")
print(f"Точне & --- & {R_exact:.2f} & {E_exact:.4f} & 2.79 & --- \\\\")
print(r"\bottomrule")
print(r"\end{tabular}")

```

```
print("=" * 70)

# Збереження результатів
np.savez(
    "h2plus_basis_convergence.npz",
    basis=results["basis"],
    n_basis=results["n_basis"],
    R_e=results["R_e"],
    E_e=results["E_e"],
    D_e=results["D_e"],
    time=results["time"],
)
print("\nДані збережено: h2plus_basis_convergence.npz")
```

Очікувані результати:

Базис	R_e (bohr)	E_e (Ha)	Похибка (mHa)
STO-3G	2.05	-0.564	4.5
6-31G	2.02	-0.566	2.1
сс-pVDZ	2.01	-0.567	1.2
сс-pVTZ	2.00	-0.5686	0.3
сс-pVQZ	2.00	-0.5688	0.1
Точне	2.00	-0.5689	—

Висновки:

- Навіть STO-3G дає якісно правильну картину
- Для кількісної точності потрібен сс-pVTZ або більший
- Збіжність монотонна: $E_{\text{basis}} \rightarrow E_{\text{CBS}}$
- Для H_2^+ CBS limit досягається при сс-pV5Z

4.4. Молекула водню H_2

4.4.1. Двоелектронна система

Молекула H_2 — перша справжня двоелектронна система. Гамільтоніан:

$$\hat{H} = \hat{h}_1 + \hat{h}_2 + \frac{1}{r_{12}} + \frac{1}{R},$$

де $\frac{1}{r_{12}}$ — електрон-електронне відштовхування, яке метод ХФ апроксимує середнім полем.

Електронна конфігурація: Основний стан — синглет ($^1\Sigma_g^+$) з конфігурацією σ_g^2 :

$$\Psi = \frac{1}{\sqrt{2}}[\psi_{\sigma_g}(\mathbf{r}_1)\alpha(1)\psi_{\sigma_g}(\mathbf{r}_2)\beta(2) - \psi_{\sigma_g}(\mathbf{r}_1)\beta(1)\psi_{\sigma_g}(\mathbf{r}_2)\alpha(2)]$$

4.4.2. RHF розрахунок

Для замкненої оболонки використовуємо RHF:

h2_rhf_single.py

```

"""
RHF розрахунок молекули водню H2 при рівноважній відстані
"""

import numpy as np
from pyscf import gto, scf

# Молекула H2 при рівноважній відстані
mol = gto.Mole()
mol.atom = """
    H  0.0  0.0  0.0
    H  0.0  0.0  0.74
"""
mol.basis = "cc-pvdz"
mol.charge = 0 # Нейтральна молекула
mol.spin = 0 # Синглет (замкнена оболонка)
mol.unit = "Angstrom"
mol.build()

print("\n" + "=" * 60)
print("МОЛЕКУЛА ВОДНЮ H2 (RHF)")
print("=" * 60)

# RHF розрахунок
mf = scf.RHF(mol)
E_rhf = mf.kernel()

print(f"\nМіжядерна відстань R = 0.74 Å = {0.74 / 0.529177:.3f} bohr")
print(f"Базисний набір: {mol.basis}")
print(f"Кількість електронів: {mol.nelectron}")
print(f"Кількість базисних функцій: {mol.nao}")

print("\n" + "-" * 60)
print("ЕНЕРГЕТИЧНІ КОМПОНЕНТИ:")
print("-" * 60)

# Компоненти енергії
dm = mf.make_rdm1()
hle = mf.get_hcore()
vhf = mf.get_veff()

E_kin = np.einsum("ij,ji->", dm, mol.intor("int1e_kin"))
E_nuc = np.einsum("ij,ji->", dm, mol.intor("int1e_nuc"))
E_ee = 0.5 * np.einsum("ij,ji->", dm, vhf)
E_nn = mol.energy_nuc()

print(f"Кінетична енергія T: {E_kin:12.6f} Ha")
print(f"Електрон-ядро взаємодія V_ne: {E_nuc:12.6f} Ha")
print(f"Електрон-електрон V_ee: {E_ee:12.6f} Ha")
print(f"Ядро-ядро відштовхування V_nn: {E_nn:12.6f} Ha")
print(f"{'-' * 60}")
print(f"Повна електронна енергія: {E_kin + E_nuc + E_ee:12.6f} Ha")
print(f"Повна енергія (з V_nn): {E_rhf:12.6f} Ha")

# Віріальна теорема
virial_ratio = -(E_nuc + E_ee + E_nn) / E_kin
print("\n" + "-" * 60)
print("ВІРІАЛЬНА ТЕОРЕМА:")
print("-" * 60)

```

```

print(f"<V>/<T> = {virial_ratio:.6f}")
print(f"Очікується: -2.000000 для точного розв'язку")
print(f"Відхилення: {abs(virial_ratio + 2.0) * 100:.4f}%")

# Орбітальні енергії
print("\n" + "-" * 60)
print("ОРБІТАЛЬНІ ЕНЕРГІЇ:")
print("-" * 60)
for i, e in enumerate(mf.mo_energy):
    occ_str = "зайнята" if i < mol.nelec[0] else "віртуальна"
    print(f"MO {i + 1}: ε = {e:10.6f} Ha = {e * 27.2114:10.4f} eV ({occ_str})")

# HOMO-LUMO gap
homo_energy = mf.mo_energy[mol.nelec[0] - 1]
lumo_energy = mf.mo_energy[mol.nelec[0]]
gap = lumo_energy - homo_energy

print(f"\nHOMO: ε = {homo_energy:.6f} Ha")
print(f"LUMO: ε = {lumo_energy:.6f} Ha")
print(f"HOMO-LUMO gap: {gap:.6f} Ha = {gap * 27.2114:.4f} eV")

# Порівняння з експериментом
print("\n" + "-" * 60)
print("ПОРІВНЯННЯ З ЕКСПЕРИМЕНТОМ:")
print("-" * 60)
E_exp = -1.174 # Ha (точна енергія Full CI)
D_e_exp = 4.75 # eV

# Енергія дисоціації
E_2H = 2 * (-0.5) # 2 * E(H)
D_e_rhf = E_2H - E_rhf

print(f"Енергія H2 (RHF): {E_rhf:.6f} Ha")
print(f"Енергія H2 (точна): {E_exp:.6f} Ha")
print(
    f"Кореляційна енергія: {E_exp - E_rhf:.6f} Ha = {(E_exp - E_rhf) * 27.2114:.3f} eV"
)
print(f"% від повної енергії: {abs((E_exp - E_rhf) / E_exp) * 100:.2f}%")

print(f"\nD_e (RHF): {D_e_rhf:.6f} Ha = {D_e_rhf * 27.2114:.3f} eV")
print(f"D_e (експеримент): --- = {D_e_exp:.2f} eV")
print(f"Похибка: {abs(D_e_rhf * 27.2114 - D_e_exp):.3f} eV")

# Дипольний момент
dip = mf.dip_moment(unit="Debye")
print(f"\nДипольний момент: {np.linalg.norm(dip):.6f} D")
print("(Для гомоядерної молекули = 0)")

print("-" * 60)

```

Результат при $R = 1.4$ bohr:

- Енергія: $E_{\text{RHF}} \approx -1.133$ Ha
- Експериментальна: $E_{\text{exp}} \approx -1.174$ Ha
- Кореляційна енергія: $E_{\text{corr}} = 0.041$ Ha ≈ 1.1 eV

4.5. Проблема дисоціації в методі Хартрі-Фока

Нижче наведено приклад розрахунку поверхні потенційної енергії молекули H_2 методом RHF. Цей розрахунок ілюструє класичну проблему RHF — при великих міжядерних відстанях: метод змушує обидва електрони займати одну і ту ж молекулярну орбіталь, що призводить до неправильного опису дисоціації молекули H_2 .

h2_pes_RHF.py

```
"""
Кривої потенційної енергії RHF для H2
"""

import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf

# Діапазон міжядерних відстаней (у борах)
distances = np.linspace(0.8, 8.0, 40)
energies_rhf = []

print("Розрахунок PES для H2 методом RHF")
print("=" * 60)
print(f"{'R (bohr)':<10} {'E_RHF (Ha)':<14}")
print("-" * 60)

for R in distances:
    # Створення молекули
    mol = gto.Mole()
    mol.atom = f"""
        H  0.0  0.0  0.0
        H  0.0  0.0  {R}
    """
    mol.basis = "cc-pvdz"
    mol.charge = 0
    mol.spin = 0
    mol.unit = "Bohr"
    mol.verbose = 0
    mol.build()

    # RHF розрахунок
    mf_rhf = scf.RHF(mol)
    mf_rhf.conv_tol = 1e-10
    E_rhf = mf_rhf.kernel()
    energies_rhf.append(E_rhf)

    print(f"{'R:<10.2f'} {'E_rhf:<14.8f}")

energies_rhf = np.array(energies_rhf)

print("=" * 60)

# Аналіз результатів
idx_min_rhf = np.argmin(energies_rhf)
R_e_rhf = distances[idx_min_rhf]
E_min_rhf = energies_rhf[idx_min_rhf]

# Дисоціаційна енергія для двох атомів H
E_2H = 2 * (-0.5) # 2×E(H)
```

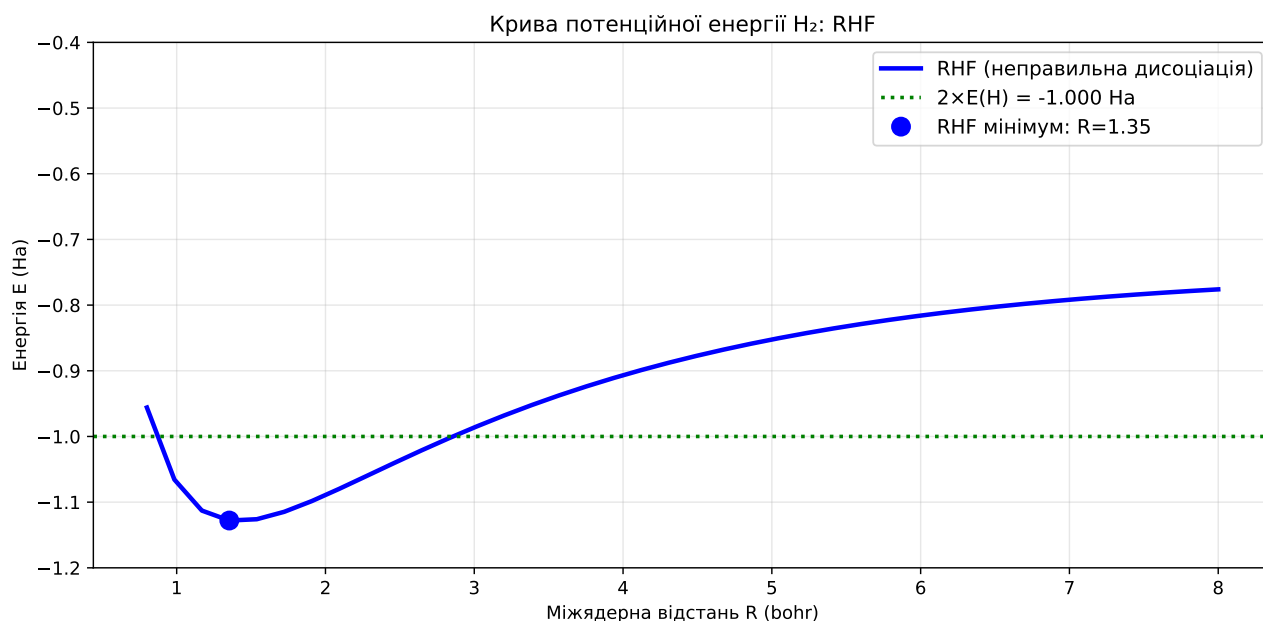
```

print("\nРЕЗУЛЬТАТИ АНАЛІЗУ:")
print("=" * 60)
print("RHF:")
print(f"  R_e = {R_e_rhf:.3f} bohr = {R_e_rhf * 0.529177:.3f} Å")
print(f"  E(R_e) = {E_min_rhf:.6f} Ha")
print(f"  D_e = {E_2H - E_min_rhf:.6f} Ha = {(E_2H - E_min_rhf) * 27.2114:.3f} eV")
print(f"  E(R→∞) = {energies_rhf[-1]:.6f} Ha")
print(f"  Помилка дисоціації: {(energies_rhf[-1] - E_2H) * 27.2114:.3f} eV")
print(f"  Правильна дисоціація? {'✓' if abs(energies_rhf[-1] - E_2H) < 0.01 else '× (іонна: H+ + e- H2)'}")
print("=" * 60)

# Візуалізація RHF PES
plt.figure(figsize=(10,5))
plt.plot(distances, energies_rhf, "b-", linewidth=2.5, label="RHF (неправильна дисоціація)")
plt.axhline(E_2H, color="green", linestyle=":", linewidth=2, label=f"2×E(H) = {E_2H:.3f} Ha")
plt.plot(R_e_rhf, E_min_rhf, "bo", markersize=10, label=f"RHF мінімум: R={R_e_rhf:.2f}")
plt.xlabel("Міжядерна відстань R (bohr)")
plt.ylabel("Енергія E (Ha)")
plt.title("Крива потенційної енергії H2: RHF")
plt.grid(True, alpha=0.3)
plt.legend(fontsize=11, loc="upper right")
plt.ylim([-1.2, -0.4])
plt.tight_layout()
plt.savefig("h2_rhf.pdf", dpi=300, bbox_inches="tight")
plt.show()

# Збереження даних
np.savez(
    "h2_rhf_pes.npz",
    distances=distances,
    E_rhf=energies_rhf,
)
print("Дані збережено: h2_rhf_pes.npz")

```

Рис. 4.2. Поверхня потенційна енергії молекули H₂ методом RHF.

4.5.1. Фізична картина

При розриві зв'язку $\text{H}_2 \rightarrow 2\text{H}$ очікується:

$$\lim_{R \rightarrow \infty} E(\text{H}_2) = 2 \times E(\text{H}) = 2 \times (-0.5) = -1.0 \text{ Ha}$$

Однак RHF дає (рис. 4.2):

$$\lim_{R \rightarrow \infty} E_{\text{RHF}}(\text{H}_2) \approx -0.7 \text{ Ha}$$

Чому? RHF змушує електрони мати однакові просторові орбіталі:

$$\psi_{\text{RHF}} = |\sigma_g \alpha \sigma_g \beta\rangle$$

При великих R це означає:

$$\psi \sim |(1s_A + 1s_B)\alpha(1s_A + 1s_B)\beta\rangle$$

Розкриваючи детермінант, отримуємо **іонні внески**:

$$\psi \sim \text{H}-\text{H} + \text{H}^+ + \text{H}^- + \text{H}^+\text{H}^-$$

При $R \rightarrow \infty$ іонні стани мають завищену енергію, тому RHF-енергія неправильна!

4.5.2. Візуалізація проблеми

Для наочного аналізу порівняємо результати розрахунку потенційної енергії молекули H_2 у двох підходах: RHF та FCI¹

`h2_dissociation_comparison.py`

```
import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf, fci

def calculate_h2_curve(distances, method="rhf", basis="cc-pvdz"):
    """
    Обчислює PES для H2 заданим методом

    method: 'rhf', 'uhf', 'fci'
    """
    energies = []
    s2_values = []

    for R in distances:
        mol = gto.Mole()
        mol.atom = f"H 0 0 0; H 0 0 {R}"
        mol.basis = basis
        mol.charge = 0
        mol.spin = 0
```

¹FCI (Full Configuration Interaction) — виступає *еталонним розв'язком*, який максимально точно (в межах вибраного базисного набору) описує електронну структуру системи.

```

mol.unit = "Bohr"
mol.verbose = 0
mol.build()

if method.lower() == "rhf":
    mf = scf.RHF(mol)
    E = mf.kernel()
    s2 = 0.0

elif method.lower() == "uhf":
    mf = scf.UHF(mol)
    E = mf.kernel()
    s2, _ = mf.spin_square()

elif method.lower() == "fci":
    # Спочатку RHF як початкове наближення
    mf = scf.RHF(mol)
    mf.kernel()
    # Потім Full CI
    cisolver = fci.FCI(mf)
    E, civec = cisolver.kernel()
    s2 = 0.0 # FCI дає чистий синглет

energies.append(E)
s2_values.append(s2)

return np.array(energies), np.array(s2_values)

# Діапазон відстаней
distances = np.linspace(0.8, 8.0, 35)

print("\n" + "=" * 70)
print("ПОРІВНЯННЯ МЕТОДІВ ДЛЯ H2: RHF vs FCI")
print("=" * 70)
print("Обчислення (це може зайняти кілька хвилин)...")

# Розрахунки
E_rhf, _ = calculate_h2_curve(distances, method="rhf")
E_fci, _ = calculate_h2_curve(distances, method="fci")

# Аналіз
E_2H = -1.0 # Точна енергія 2xH

print("\nРЕЗУЛЬТАТИ:")
print("=" * 70)

# Знаходження мінімумів
idx_rhf = np.argmin(E_rhf)
idx_fci = np.argmin(E_fci)

methods_data = {
    "RHF": {
        "R_e": distances[idx_rhf],
        "E_min": E_rhf[idx_rhf],
        "E_dissoc": E_rhf[-1],
        "D_e": E_2H - E_rhf[idx_rhf],
    },
    "FCI": {
        "R_e": distances[idx_fci],
        "E_min": E_fci[idx_fci],
        "E_dissoc": E_fci[-1],
        "D_e": E_2H - E_fci[idx_fci],
    },
}

```

```

}

for method, data in methods_data.items():
    print(f"\n{method}:")
    print(f"  R_e = {data['R_e']:.3f} bohr = {data['R_e'] * 0.529177:.3f} Å")
    print(f"  E(R_e) = {data['E_min']:.6f} Ha")
    print(f"  D_e = {data['D_e']:.6f} Ha = {data['D_e'] * 27.2114:.3f} eV")
    print(f"  E(R $\rightarrow\infty$ ) = {data['E_dissoc']:.6f} Ha")
    print(f"   $\Delta$  від 2 $\times$ E(H): {abs(data['E_dissoc'] - E_2H) * 1000:.2f} mHa")

print(f"\nЕкспериментальні дані:")
print(f"  R_e = 1.401 bohr = 0.741 Å")
print(f"  D_e = 4.75 eV = 0.1745 Ha")

# Кореляційна енергія
print("\n" + "-" * 70)
print("КОРЕЛЯЦІЙНА ЕНЕРГІЯ E_corr = E_FCI - E_HF:")
print("-" * 70)
print(f"{'R (bohr)':<10} {'E_corr (mHa)':<15} {'% від D_e':<12}")
print("-" * 70)

selected_R = np.linspace(1.2, 8, 10)
for R in selected_R:
    idx = np.argmin(abs(distances - R))
    E_corr = (E_fci[idx] - E_rhf[idx]) * 1000 # mHa
    percent = abs(E_corr) / (methods_data["FCI"]["D_e"] * 1000) * 100
    print(f"{'R:<10.1f} {'E_corr:<15.2f} {'percent:<12.1f}%"")

print("-" * 70)

fig = plt.figure(figsize=(16, 15), constrained_layout=False)
# ТРИ ряди, ДВА стовпці – всі рядки рівні за висотою
gs = fig.add_gridspec(3, 2, height_ratios=[1, 1, 1], hspace=0.45, wspace=0.35)

# 1) Повна PES (ряд 0, обидва стовпці)
ax1 = fig.add_subplot(gs[0, :])
ax1.plot(distances, E_rhf, "-", color="tab:blue", lw=2.8, label="RHF", alpha=0.9)
ax1.plot(distances, E_fci, "-.", color="tab:green", lw=2.8, label="FCI (точний)", alpha=0.9)
ax1.axhline(E_2H, color="k", ls=":", lw=1.6, alpha=0.7)
ax1.plot(distances[idx_rhf], E_rhf[idx_rhf], "o", color="tab:blue", ms=8)
ax1.plot(distances[idx_fci], E_fci[idx_fci], "o", color="tab:green", ms=8)
ax1.set_ylim([-1.25, -0.3])
ax1.set_title("Криві потенційної енергії H $_2$ $", fontsize=16, fontweight="bold")
ax1.set_xlabel("Міжядерна відстань $R$ (bohr)", fontsize=13)
ax1.set_ylabel("Енергія $E$ (Ha)", fontsize=13)
ax1.grid(alpha=0.25)
ax1.legend(fontsize=11, loc="upper right")
ax1.axvspan(0.8, 2.5, alpha=0.08, color="green")
ax1.axvspan(3.5, 8.0, alpha=0.08, color="red")

# 2) Zoom рівноваги (ряд 1, стовпець 0)
ax2 = fig.add_subplot(gs[1, 0])
mask_eq = (distances  $\geq$  0.8) & (distances  $\leq$  2.5)
ax2.plot(distances[mask_eq], E_rhf[mask_eq], "-", color="tab:blue", lw=2.5, label="RHF")
ax2.plot(distances[mask_eq], E_fci[mask_eq], "-.", color="tab:green", lw=2.5, label="FCI")
ax2.set_title("Zoom: область рівноваги", fontsize=14, fontweight="bold")
ax2.set_xlabel("R (bohr)")
ax2.set_ylabel("E (Ha)")
ax2.grid(alpha=0.25)
ax2.legend(fontsize=10)

# 3) Zoom дисоціація (ряд 1, стовпець 1)
ax3 = fig.add_subplot(gs[1, 1])

```

```

mask_dis = (distances ≥ 3.5) & (distances ≤ 8.0)
ax3.plot(distances[mask_dis], E_rhf[mask_dis], "-", color="tab:blue", lw=2.5, label="RHF")
ax3.plot(distances[mask_dis], E_fci[mask_dis], "-.", color="tab:green", lw=2.5, label="FCI")
ax3.axhline(E_2H, color="k", ls=":", lw=1.4, alpha=0.7)
ax3.set_title("Zoom: дисоціація", fontsize=14, fontweight="bold")
ax3.set_xlabel("R (bohr)")
ax3.set_ylabel("E (Ha)")
ax3.grid(alpha=0.25)
ax3.legend(fontsize=10)

ax3.annotate(
    "RHF → H+ + H2\n(неправильно!)",
    xy=(7, E_rhf[-1]), xytext=(6, -0.8),
    arrowprops=dict(arrowstyle="→", color="tab:blue", lw=1.8),
    fontsize=11, color="tab:blue", fontweight="bold",
    bbox=dict(boxstyle="round,pad=0.25", facecolor="wheat", alpha=0.8),
)
ax3.annotate(
    "FCI → 2H\n(правильно)",
    xy=(7, E_fci[-1]), xytext=(5.4, -1.05),
    arrowprops=dict(arrowstyle="→", color="tab:green", lw=1.8),
    fontsize=11, color="tab:green", fontweight="bold",
    bbox=dict(boxstyle="round,pad=0.25", facecolor="lightgreen", alpha=0.8),
)

# 4) Кореляційна енергія (ряд 2, обидва стовпці)
ax4 = fig.add_subplot(gs[2, :])
E_corr_rhf = (E_fci - E_rhf) * 1000.0 # mHa
ax4.plot(distances, E_corr_rhf, "-", color="tab:blue", lw=2.8, label="E$_{corr}$ (FCI - RHF)")
ax4.axhline(0, color="k", ls=":", lw=1.0, alpha=0.6)
ax4.set_title("Кореляційна енергія", fontsize=14, fontweight="bold")
ax4.set_xlabel("R (bohr)")
ax4.set_ylabel("Кореляційна енергія (mHa)")
ax4.grid(alpha=0.25)
ax4.legend(fontsize=10)

# Заголовок та фіналізація
plt.suptitle("Порівняння RHF та FCI для дисоціації H$_2$", fontsize=18, fontweight="bold",
    ↵ y=0.995)
# plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig("h2_RHF_vs_FCI.pdf", dpi=300, bbox_inches="tight")
print("Графік збережено як: h2_RHF_vs_FCI.pdf")
plt.show()

# Збереження даних
np.savez(
    "h2_methods_comparison.npz",
    distances=distances,
    E_rhf=E_rhf,
    E_fci=E_fci,
)
print("Дані збережено: h2_methods_comparison.npz")

# Підсумкова таблиця
print("\n" + "=" * 70)
print("ПІДСУМКОВА ТАБЛИЦЯ (для LaTeX):")
print("=" * 70)
print(r"\begin{tabular}{lcccc}")
print(r"\toprule")
print(r"Метод & $R_e$ (bohr) & $E(R_e)$ (Ha) & $D_e$ (eV) & $E(R\to\infty)$ (Ha) \\\")
print(r"\midrule")
for method, data in methods_data.items():
    print(
        f"{method} & {data['R_e']:.3f} & {data['E_min']:.4f} & "

```

```
f"{data['D_e'] * 27.2114:.2f} & {data['E_dissoc']:.4f} \\\\"
)
print(r"\midrule")
print(r"Експеримент & 1.401 & --- & 4.75 & $-1.000$ \\\")
print(r"\bottomrule")
print(r"\end{tabular}")
print("=" * 70)
```

Графік (рис. 4.3) показує:

- Точний розв'язок (Full CI) — монотонна крива.
- RHF — завищена енергія при $R > 2.5$ bohr.

4.5.3. Відсутність кореляції як джерело проблеми

Фундаментальна причина — метод ХФ описує електрон-електронну взаємодію через **середнє поле**, ігноруючи миттєву кореляцію положень електронів.

Кореляційна енергія:

$$E_{\text{corr}}(R) = E_{\text{exact}}(R) - E_{\text{HF}}(R)$$

R (bohr)	E_{corr} (mHa)	% від D_e
1.4 (рівновага)	41	9%
3.0	85	18%
5.0	150	32%
∞	300	—

Висновок: Кореляція стає критичною при розриві зв'язків!

Завдання

Завдання 1: Порівняння базисів

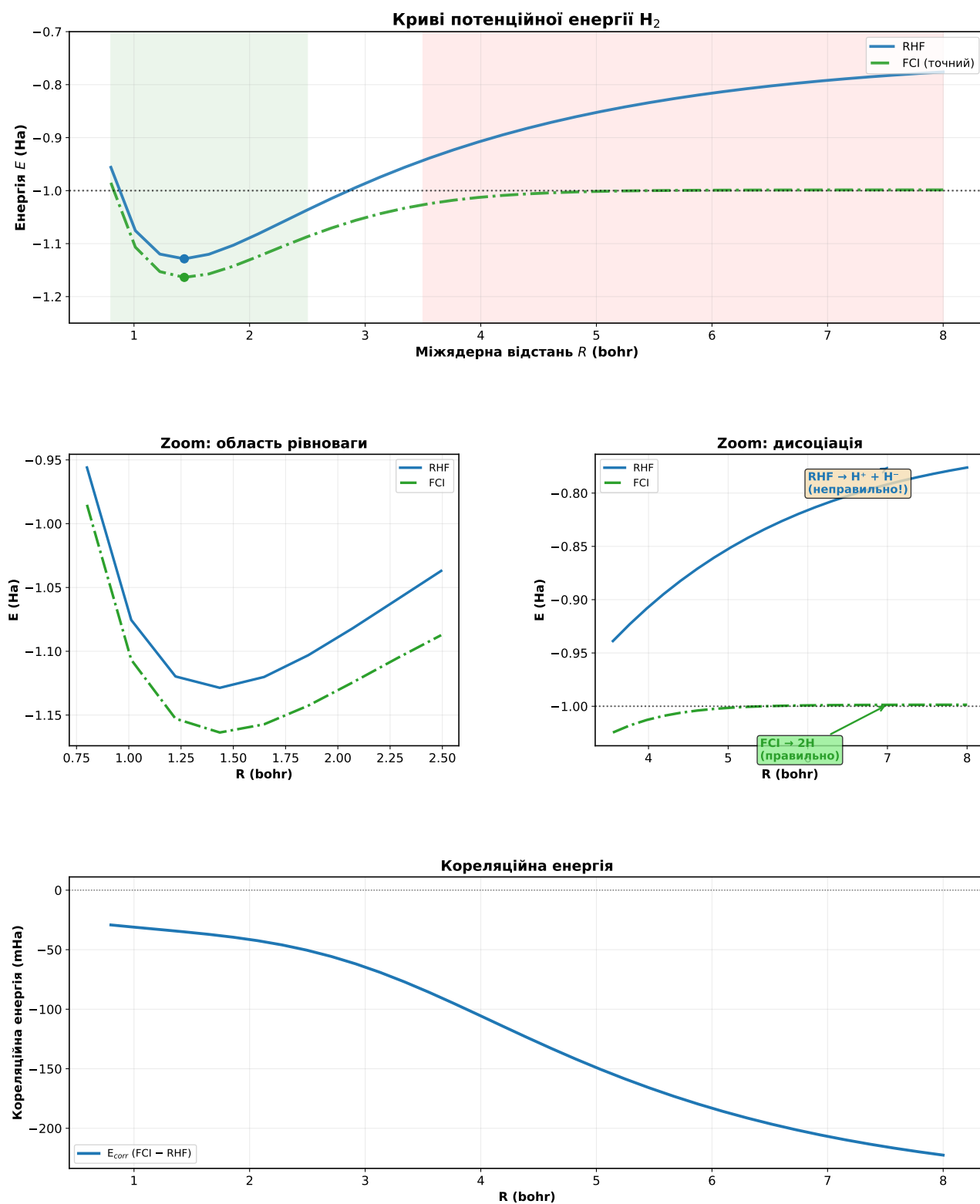
Побудуйте PES для H_2 у базисах STO-3G, 6-31G**, cc-pVDZ, cc-pVTZ. Порівняйте:

- Рівноважні відстані R_e
- Енергії дисоціації D_e
- Час обчислень

Завдання 2: Інші діатомні молекули

Повторіть аналіз для:

- LiH — гетероядерна молекула
- N_2 — потрійний зв'язок

Рис. 4.3. Повне порівняння методів для молекули H_2 .

- F_2 — слабкий зв'язок
Чи зберігається проблема дисоціації?

4.6. Резюме

У цьому розділі ми детально вивчили найпростіші діатомні молекули:

- H_2^+ : Метод ХФ є точним (один електрон), демонструє техніки PES, оптимізації, аналізу МО.
- H_2 : Виявляє фундаментальну проблему RHF — некоректну дисоціацію через відсутність кореляції
- **Кореляція**: Стає критичною при розриві зв'язків, потребує пост-ХФ методів (FCI, CASSCF, CCSD, CI)

Ключовий урок: Метод ХФ добре працює навколо рівноваги, але неадекватний для опису хімічних реакцій, де відбувається розрив/утворення зв'язків.

5

Теорія функціоналу густини (DFT)

5.1. Основи теорії функціоналу густини

5.1.1. Теореми Хоенберга–Кона

Теорія функціоналу густини базується на двох фундаментальних теоремах, доведених Хоенбергом та Коном у 1964 році:

Теорема 1 (Теорема існування). Зовнішній потенціал $v_{\text{ext}}(\mathbf{r})$ (а отже, і повна енергія системи) однозначно визначається електронною густиною основного стану $\rho(\mathbf{r})$ з точністю до адитивної константи.

Це означає, що електронна густина містить всю інформацію про систему:

$$E[\rho] = T[\rho] + V_{ee}[\rho] + \int v_{\text{ext}}(\mathbf{r})\rho(\mathbf{r})d\mathbf{r} \quad (5.1)$$

Теорема 2 (Варіаційний принцип). Існує універсальний функціонал енергії $F[\rho]$, який для будь-якої пробної густини $\tilde{\rho}(\mathbf{r})$ задовольняє:

$$E_0 \leq E[\tilde{\rho}] = F[\tilde{\rho}] + \int v_{\text{ext}}(\mathbf{r})\tilde{\rho}(\mathbf{r})d\mathbf{r} \quad (5.2)$$

де E_0 — точна енергія основного стану.

5.1.2. Рівняння Кона–Шема

Кон та Шем (1965) запропонували практичний підхід до DFT, замінивши взаємодіючу систему еквівалентною невзаємодіючою системою з такою ж густиною:

$$\left[-\frac{1}{2}\nabla^2 + v_{\text{eff}}(\mathbf{r}) \right] \psi_i(\mathbf{r}) = \varepsilon_i \psi_i(\mathbf{r}) \quad (5.3)$$

Ефективний потенціал визначається як:

$$v_{\text{eff}}(\mathbf{r}) = v_{\text{ext}}(\mathbf{r}) + v_H(\mathbf{r}) + v_{xc}(\mathbf{r}) \quad (5.4)$$

де:

- $v_{\text{ext}}(\mathbf{r})$ — зовнішній потенціал (від ядер)
- $v_H(\mathbf{r}) = \int \frac{\rho(\mathbf{r}')}{\|\mathbf{r}-\mathbf{r}'\|} d\mathbf{r}'$ — потенціал Хартрі
- $v_{xc}(\mathbf{r}) = \frac{\delta E_{xc}[\rho]}{\delta \rho(\mathbf{r})}$ — обмінно-кореляційний потенціал

Електронна густина обчислюється через орбіталі Кона–Шема:

$$\rho(\mathbf{r}) = \sum_{i=1}^N |\psi_i(\mathbf{r})|^2 \quad (5.5)$$

5.1.3. Обмінно-кореляційна енергія

Повна енергія в DFT:

$$E[\rho] = T_s[\rho] + V_{\text{ext}}[\rho] + J[\rho] + E_{xc}[\rho] \quad (5.6)$$

де $E_{xc}[\rho]$ — обмінно-кореляційна енергія, яка містить:

- Різницю між точною кінетичною енергією та кінетичною енергією незалежної системи
- Некласичну частину електрон-електронного відштовхування (обмін + кореляція)

Точний вигляд $E_{xc}[\rho]$ невідомий, тому використовуються наближення (функціонали).

5.1.4. Порівняння HF та DFT

Рис. 5.1. Порівняння методів Хартрі-Фока та DFT

Аспект	Hartree-Fock	DFT
Базова змінна	Хвильова функція	Електронна густина
Обмін	Точний (нелокальний)	Наближений (локальний)
Кореляція	Відсутня	Включена наближено
Масштабування	$\mathcal{O}(N^4)$	$\mathcal{O}(N^3)$
Точність для атомів	Добра якісно	Часто краща кількісно
Збуджені стани	Можливі	Складно (TD-DFT)

5.2. Функціонали обміну-кореляції

5.2.1. Класифікація функціоналів: драбина Якова

Функціонали DFT класифікуються за ”драбиною Якова”(Jacob’s ladder), запропонованою Пердью:

1. **LDA/LSDA** (Local Density Approximation) — залежать тільки від $\rho(\mathbf{r})$
2. **GGA** (Generalized Gradient Approximation) — залежать від $\rho(\mathbf{r})$ та $\nabla\rho(\mathbf{r})$

3. **Meta-GGA** — додатково залежать від $\nabla^2\rho$ або кінетичної густини τ
4. **Hybrid** — включають частку точного обміну HF
5. **Double-hybrid** — включають і HF обмін, і MP2 кореляцію

5.2.2. LDA та LSDA функціонали

Локальне наближення густини

LDA базується на моделі однорідного електронного газу:

$$E_{xc}^{\text{LDA}}[\rho] = \int \rho(\mathbf{r})\epsilon_{xc}(\rho(\mathbf{r}))d\mathbf{r} \quad (5.7)$$

де $\epsilon_{xc}(\rho)$ — обмінно-кореляційна енергія на електрон в однорідному газі густини ρ .

Обмінна частина (Slater/Dirac):

$$\epsilon_x^{\text{LDA}}(\rho) = -\frac{3}{4}\left(\frac{3}{\pi}\right)^{1/3}\rho^{1/3} \quad (5.8)$$

Кореляційна частина: Використовуються параметризації (VWN, PW92).

LSDA для спін-поляризованих систем

Для систем з різними ρ_α та ρ_β :

$$E_{xc}^{\text{LSDA}}[\rho_\alpha, \rho_\beta] = \int \rho\epsilon_{xc}(\rho_\alpha, \rho_\beta)d\mathbf{r} \quad (5.9)$$

code_1.py

```
from pyscf import gto, dft

# Приклад LDA розрахунку атома Ne
mol = gto.M(atom="Ne 0 0 0", basis="cc-pvdz", spin=0)

# LDA функціонал (VWN для кореляції)
mf = dft.RKS(mol)
mf.xc = "lda,vwn" # або просто 'lda'
mf.verbose = 4
energy_lda = mf.kernel()

print(f"\nЕнергія Ne (LDA): {energy_lda:.8f} Ha")

# Порівняння з HF
from pyscf import scf

mf_hf = scf.RHF(mol)
mf_hf.verbose = 0
energy_hf = mf_hf.kernel()

print(f"Енергія Ne (HF): {energy_hf:.8f} Ha")
print(f"Різниця (LDA-HF): {(energy_lda - energy_hf) * 1000:.2f} mHa")
```

5.2.3. GGA функціонали

GGA функціонали враховують не тільки локальну густину, але й її градієнт:

$$E_{xc}^{GGA}[\rho] = \int f(\rho(\mathbf{r}), \nabla\rho(\mathbf{r}))d\mathbf{r} \quad (5.10)$$

Популярні GGA функціонали

PBE (Perdew-Burke-Ernzerhof, 1996) Найпопулярніший неемпіричний GGA функціонал:

[code_2.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="C 0 0 0", basis="cc-pvtz", spin=2)
mf = dft.UKS(mol)
mf.xc = "pbe" # або 'pbe,pbe'
energy_pbe = mf.kernel()
print(f"Енергія C (PBE): {energy_pbe:.8f} Ha")
```

BLYP (Becke88 + Lee-Yang-Parr) Комбінація обміну Беке88 та кореляції LYP:

[code_3.py](#)

```
mf = dft.UKS(mol)
mf.xc = "blyp" # або 'b88,lyp'
energy_blyp = mf.kernel()
print(f"Енергія C (BLYP): {energy_blyp:.8f} Ha")
```

BP86 (Becke88 + Perdew86)

[code_4.py](#)

```
mf = dft.UKS(mol)
mf.xc = 'bp86'
energy_bp86 = mf.kernel()
print(f"Енергія C (BP86): {energy_bp86:.8f} Ha")
```

Порівняння GGA функціоналів

[code_5.py](#)

```
from pyscf import gto, dft

# Тестування на атомі Кисню
mol = gto.M(atom="O 0 0 0", basis="def2-tzvp", spin=2)

gga_functionals = ["pbe", "blyp", "bp86", "pw91", "pberev"]

print("Порівняння GGA функціоналів для атома O (³P)")
print("=" * 60)
print(f"{'Функціонал':12s} {'Енергія, Ha':15s} {'Відносно PBE, mHa':20s}")
print("-" * 60)
```

```

energies = {}

for xc in gga_functionals:
    mf = dft.UKS(mol)
    mf.xc = xc
    mf.verbose = 0
    mf.conv_tol = 1e-10

    try:
        energy = mf.kernel()
        energies[xc] = energy

        if xc == "pbe":
            e_ref = energy
            rel = 0.0
        else:
            rel = (energy - e_ref) * 1000

        print(f"{xc:12s} {energy:15.8f} {rel:20.4f}")
    except:
        print(f"{xc:12s} --- помилка розрахунку")

print("=" * 60)

```

5.2.4. Meta-GGA функціонали

Meta-GGA функціонали включають додаткову інформацію про систему: кінетичну густину τ або лапласіан густини $\nabla^2\rho$:

$$E_{xc}^{\text{meta-GGA}}[\rho] = \int f(\rho, \nabla\rho, \tau) d\mathbf{r} \quad (5.11)$$

де кінетична густина орбіталей Кона-Шема:

$$\tau(\mathbf{r}) = \frac{1}{2} \sum_{i=1}^N |\nabla\psi_i(\mathbf{r})|^2 \quad (5.12)$$

TPSS (Tao-Perdew-Staroverov-Scuseria)

TPSS — один з перших успішних meta-GGA функціоналів (2003):

[code_6.py](#)

```

from pyscf import gto, dft

mol = gto.M(
    atom="Fe 0 0 0",
    basis="def2-tzvp",
    spin=4, # 5D основний стан
)

# TPSS meta-GGA
mf = dft.UKS(mol)
mf.xc = "tpss"
mf.verbose = 4
energy_tpss = mf.kernel()

print(f"\nЕнергія Fe (TPSS): {energy_tpss:.8f} Ha")

```

```
# Порівняння з PBE
mf_pbe = dft.UKS(mol)
mf_pbe.xc = "pbe"
mf_pbe.verbose = 0
energy_pbe = mf_pbe.kernel()

print(f"Енергія Fe (PBE): {energy_pbe:.8f} Ha")
print(f"Різниця (TPSS-PBE): {(energy_tpss - energy_pbe) * 1000:.2f} mHa")
```

M06-L (Minnesota 06 Local)

Високопараметризований meta-GGA функціонал для широкого спектру задач:

[code_7.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="Ni 0 0 0", basis="def2-svp", spin=2)

mf = dft.UKS(mol)
mf.xc = "m06l" # або 'm06-l'
mf.verbose = 4

try:
    energy_m06l = mf.kernel()
    print(f"\nЕнергія Ni (M06-L): {energy_m06l:.8f} Ha")
except:
    print("M06-L може бути недоступний у вашій версії PySCF")
    print("Встановіть: pip install pyscf[geomopt]")
```

SCAN (Strongly Constrained and Appropriately Normed)

Сучасний meta-GGA, що задовольняє всі відомі точні умови:

[code_8.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="C 0 0 0", basis="def2-qzvp", spin=2)

mf = dft.UKS(mol)
mf.xc = "scan"
energy_scan = mf.kernel()

print(f"Енергія C (SCAN): {energy_scan:.8f} Ha")
```

5.2.5. Гібридні функціонали

Гібридні функціонали комбінують DFT обмін з точним (HF) обміном:

$$E_{xc}^{\text{hybrid}} = a \cdot E_x^{\text{HF}} + (1 - a) \cdot E_x^{\text{DFT}} + E_c^{\text{DFT}} \quad (5.13)$$

де a — частка HF обміну (зазвичай 0.2–0.3).

B3LYP (Becke 3-parameter Lee-Yang-Parr)

Найпопулярніший гібридний функціонал у хімії:

$$E_{xc}^{B3LYP} = E_x^{LDA} + 0.2(E_x^{HF} - E_x^{LDA}) + 0.72 \cdot E_x^{B88} + 0.81 \cdot E_c^{LYP} + 0.19 \cdot E_c^{VWN} \quad (5.14)$$

[code_9.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="N 0 0 0", basis="6-311+g(d,p)", spin=3)

# B3LYP розрахунок
mf = dft.UKS(mol)
mf.xc = "b3lyp"
mf.verbose = 4
energy_b3lyp = mf.kernel()

print(f"\nЕнергія N (B3LYP): {energy_b3lyp:.8f} Ha")

# Аналіз компонентів
print("\nВнесок точного обміну: 20%")
print("Це робить розрахунок повільнішим, але точнішим")
```

PBE0 (PBE hybrid)

Неемпіричний гібрид з 25

$$E_{xc}^{PBE0} = 0.25 \cdot E_x^{HF} + 0.75 \cdot E_x^{PBE} + E_c^{PBE} \quad (5.15)$$

[code_10.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="O 0 0 0", basis="aug-cc-pvtz", spin=2)

mf = dft.UKS(mol)
mf.xc = "pbe0"
energy_pbe0 = mf.kernel()

print(f"Енергія O (PBE0): {energy_pbe0:.8f} Ha")
```

CAM-B3LYP (Coulomb-Attenuated Method)

Функціонал з дальньодіючою корекцією (range-separated):

$$\frac{1}{r_{12}} = \frac{\alpha + \beta \cdot \text{erf}(\mu r_{12})}{r_{12}} + \frac{1 - [\alpha + \beta \cdot \text{erf}(\mu r_{12})]}{r_{12}} \quad (5.16)$$

[code_11.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="F 0 0 0", basis="cc-pvqz", spin=1)

mf = dft.UKS(mol)
mf.xc = "camb3lyp" # або 'cam-b3lyp'
energy_camb3lyp = mf.kernel()

print(f"Енергія F (CAM-B3LYP): {energy_camb3lyp:.8f} Ha")
```


M06-2X та інші Minnesota функціонали

Родина M06 з різною часткою HF обміну:

- M06-L: 0% HF (meta-GGA)
- M06: 27% HF
- M06-2X: 54% HF (для кінетики, слабких взаємодій)
- M06-HF: 100% HF

[code_12.py](#)

```
from pyscf import gto, dft

mol = gto.M(atom="Ar 0 0 0", basis="def2-tzvp", spin=0)

# Порівняння M06 функціоналів
m06_functionals = {
    "m06l": "M06-L (0% HF)",
    "m06": "M06 (27% HF)",
    "m062x": "M06-2X (54% HF)",
}

print("Порівняння M06 функціоналів для Ar")
print("=" * 60)

for xc, name in m06_functionals.items():
    mf = dft.RKS(mol)
    mf.xc = xc
    mf.verbose = 0

    try:
        energy = mf.kernel()
        print(f"{name:20s}: {energy:.8f} Ha")
    except:
        print(f"{name:20s}: недоступний")
```

5.2.6. Порівняння різних рівнів теорії

[code_13.py](#)

```
from pyscf import gto, scf, dft
import numpy as np
import matplotlib.pyplot as plt

def compare_functionals(symbol, spin, basis="cc-pvtz"):
    """
    Систематичне порівняння функціоналів
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    # Список методів для порівняння
    methods = [
        ("HF", "scf"),
        ("LDA", "lda"),
        ("PBE", "pbe"),
        ("BLYP", "blyp"),
        ("TPSS", "tpss"),
        ("B3LYP", "b3lyp"),
        ("PBE0", "pbe0"),
        ("CAM-B3LYP", "camb3lyp"),
```

```

]

energies = []
labels = []

print(f"\nПорівняння методів для {symbol} (базис: {basis})")
print("=" * 70)
print(f"{'Метод':15s} {'Енергія, Ha':15s} {'Відносно HF, mHa':20s}")
print("-" * 70)

for name, method in methods:
    try:
        if method == "scf":
            if spin == 0:
                mf = scf.RHF(mol)
            else:
                mf = scf.UHF(mol)
        else:
            if spin == 0:
                mf = dft.RKS(mol)
            else:
                mf = dft.UKS(mol)
            mf.xc = method

        mf.verbose = 0
        mf.conv_tol = 1e-10
        energy = mf.kernel()

        if mf.converged:
            energies.append(energy)
            labels.append(name)

            if name == "HF":
                e_ref = energy
                rel = 0.0
            else:
                rel = (energy - e_ref) * 1000

            print(f"{'name':15s} {'energy':15.8f} {'rel':20.4f}")
        else:
            print(f"{'name':15s} не конвергувало")
    except Exception as e:
        print(f"{'name':15s} помилка: {str(e)[:30]}")

print("=" * 70)

# Графік
if len(energies) > 1:
    fig, ax = plt.subplots(figsize=(10, 6))

    x = np.arange(len(labels))
    colors = [
        "red"
        if l == "HF"
        else "blue"
        if l in ["LDA", "PBE", "BLYP", "TPSS"]
        else "green"
        for l in labels
    ]

    bars = ax.bar(x, energies, color=colors, alpha=0.7)
    ax.set_xticks(x)
    ax.set_xticklabels(labels, rotation=45, ha="right")
    ax.set_ylabel("Енергія (Ha)", fontsize=12)

```

```
ax.set_title(f"Порівняння методів для атома {symbol}", fontsize=14)
ax.grid(True, alpha=0.3, axis="y")

# Легенда
from matplotlib.patches import Patch

legend_elements = [
    Patch(facecolor="red", alpha=0.7, label="HF"),
    Patch(facecolor="blue", alpha=0.7, label="Pure DFT"),
    Patch(facecolor="green", alpha=0.7, label="Hybrid DFT"),
]
ax.legend(handles=legend_elements, loc="best")

plt.tight_layout()
plt.savefig(f"{symbol}_functionals_comparison.pdf")
plt.show()

return energies, labels

# Тестування на різних атомах
energies_c, labels_c = compare_functionals("C", spin=2)
energies_ne, labels_ne = compare_functionals("Ne", spin=0)
energies_fe, labels_fe = compare_functionals("Fe", spin=4, basis="def2-svp")
```

5.2.7. Вибір функціоналу: рекомендації

Таблиця 5.1. Рекомендовані функціонали для різних задач

Задача	Функціонал	Коментар
Швидкі розрахунки	PBE	Добра точність/швидкість
Енергії атомізації	B3LYP, PBE0	Стандарт у хімії
Перехідні метали	TPSSh, M06	Краще для d-електронів
Слабкі взаємодії	M06-2X, ω B97X-D	З дисперсією
Високоспінові стани	SCAN, TPSSh	Кращий баланс
Загальні розрахунки	PBE0	Універсальний вибір
Максимальна точність	CCSD(T)	Але дуже повільно

5.2.8. Практичний приклад: вплив функціоналу на властивості

[code_14.py](#)

```
from pyscf import gto, scf, dft

def analyze_functional_effect(symbol, charge, spin_n, spin_c, basis="aug-cc-pvtz"):
    """
    Аналіз впливу функціоналу на енергію іонізації
    """

    functionals = ["pbe", "blyp", "b3lyp", "pbe0", "cam-b3lyp"]

    print(f"\nЕнергії іонізації {symbol} (базис: {basis})")
```

```

print("=" * 70)
print(f"{'Метод':15s} {'E(A), Ha':15s} {'E(A+), Ha':15s} {'IE, eV':10s}")
print("-" * 70)

# HF референс
mol_n = gto.M(
    atom=f"{'symbol'} 0 0 0", basis=basis, charge=charge, spin=spin_n, verbose=0
)
mol_c = gto.M(
    atom=f"{'symbol'} 0 0 0", basis=basis, charge=charge + 1, spin=spin_c, verbose=0
)

if spin_n == 0:
    mf_n = scf.RHF(mol_n)
else:
    mf_n = scf.UHF(mol_n)

if spin_c == 0:
    mf_c = scf.RHF(mol_c)
else:
    mf_c = scf.UHF(mol_c)

mf_n.verbose = 0
mf_c.verbose = 0

e_n_hf = mf_n.kernel()
e_c_hf = mf_c.kernel()
ie_hf = (e_c_hf - e_n_hf) * 27.211386

print(f"{'HF':15s} {e_n_hf:15.8f} {e_c_hf:15.8f} {ie_hf:10.4f}")

# DFT функціонали
for xc in functionals:
    if spin_n == 0:
        mf_n = dft.RKS(mol_n)
    else:
        mf_n = dft.UKS(mol_n)

    if spin_c == 0:
        mf_c = dft.RKS(mol_c)
    else:
        mf_c = dft.UKS(mol_c)

    mf_n.xc = xc
    mf_c.xc = xc
    mf_n.verbose = 0
    mf_c.verbose = 0

    try:
        e_n = mf_n.kernel()
        e_c = mf_c.kernel()
        ie = (e_c - e_n) * 27.211386

        print(f"{xc.upper():15s} {e_n:15.8f} {e_c:15.8f} {ie:10.4f}")
    except:
        print(f"{xc.upper():15s} помилка розрахунку")

print("=" * 70)

```

Приклади

```

analyze_functional_effect("C", 0, 2, 1) # C → C+
analyze_functional_effect("O", 0, 2, 3) # O → O+

```

```
analyze_functional_effect("Ne", 0, 0, 1) # Ne → Ne+
```

5.3. DFT розрахунки атомів

5.3.1. Базова структура DFT розрахунку

DFT розрахунки в PySCF використовують класи RKS (Restricted Kohn-Sham) та UKS (Unrestricted Kohn-Sham), аналогічні до RHF/UHF:

[code_15.py](#)

```
from pyscf import gto, dft

# Замкнена оболонка (RKS)
mol_he = gto.M(atom="He 0 0 0", basis="cc-pvtz", spin=0)
mf_he = dft.RKS(mol_he)
mf_he.xc = "pbe0"
e_he = mf_he.kernel()

print(f"He (RKS/PBE0): {e_he:.8f} Ha")

# Відкрита оболонка (UKS)
mol_li = gto.M(atom="Li 0 0 0", basis="cc-pvtz", spin=1)
mf_li = dft.UKS(mol_li)
mf_li.xc = "pbe0"
e_li = mf_li.kernel()

print(f"Li (UKS/PBE0): {e_li:.8f} Ha")
```

5.3.2. Систематичний розрахунок атомів другого періоду

[code_16.py](#)

```
from pyscf import gto, dft
import numpy as np

def dft_second_period(functional="pbe", basis="def2-tzvp"):
    """
    Розрахунок всіх атомів другого періоду
    """

    atoms_data = [
        ("Li", 1, "2S"),
        ("Be", 0, "1S"),
        ("B", 1, "2P"),
        ("C", 2, "3P"),
        ("N", 3, "4S"),
        ("O", 2, "3P"),
        ("F", 1, "2P"),
        ("Ne", 0, "1S"),
    ]

    print(f"\nАтоми 2-го періоду ({functional.upper()}/{basis})")
    print("=" * 80)
    print(
        f"{'Атом':4s} {'Терм':6s} {'2S':3s} {'Енергія, Ha':15s} "
        f"{'Енергія, eV':12s} {'<S^2>':8s}"
    )
```

```

print("-" * 80)

energies = {}

for symbol, spin, term in atoms_data:
    mol = gto.M(
        atom=f"{symbol} 0 0 0", basis=basis, spin=spin, symmetry=True, verbose=0
    )

    if spin == 0:
        mf = dft.RKS(mol)
    else:
        mf = dft.UKS(mol)

    mf.xc = functional
    mf.conv_tol = 1e-10
    energy = mf.kernel()

    energies[symbol] = energy
    e_ev = energy * 27.211386

    if spin == 0:
        s2 = 0.0
    else:
        s2_result = mf.spin_square()
        s2 = s2_result[0]

    print(f"{symbol:4s} {term:6s} {spin:3d} {energy:15.8f} {e_ev:12.2f} {s2:8.4f}")

print("-" * 80)

return energies

# Розрахунки з різними функціоналами
energies_pbe = dft_second_period("pbe")
energies_b3lyp = dft_second_period("b3lyp")
energies_pbe0 = dft_second_period("pbe0")

```

5.3.3. Розрахунок атомів перехідних металів

Атоми перехідних металів — складна задача через багато близьких за енергією станів:

[code_17.py](#)

```

from pyscf import gto, dft

def transition_metal_dft(symbol, spin, functional="tpss", basis="def2-tzvp"):
    """
    DFT розрахунок атома перехідного металу
    """

    print(f"\nРозрахунок {symbol} (2S={spin}, {functional.upper()})")
    print("-" * 60)

    mol = gto.M(
        atom=f"{symbol} 0 0 0",
        basis=basis,
        spin=spin,
        symmetry=False, # Часто краще без симетрії
    )

```

```

        verbose=0,
    )

    mf = dft.UKS(mol)
    mf.xc = functional

    # Налаштування для важких випадків
    mf.conv_tol = 1e-8
    mf.max_cycle = 200
    mf.diis_space = 12

    # Для перехідних металів часто потрібен level shift
    if symbol in ["Cr", "Mn", "Fe", "Co", "Ni"]:
        mf.level_shift = 0.3

    mf.verbose = 4
    energy = mf.kernel()

    if mf.converged:
        s2 = mf.spin_square()
        expected_s2 = spin * (spin + 2) / 4

        print(f"\nРезультати:")
        print(f"  Енергія: {energy:.8f} Ha")
        print(f"  <S²>: {s2[0]:.4f} (очікується {expected_s2:.4f})")
        print(f"  Забруднення спіном: {s2[0] - expected_s2:.4f}")

        # Заселеності d-орбіталей
        from pyscf import lo

        pop = mf.mulliken_pop()

        return energy, s2[0]
    else:
        print("\nНе конвергувало!")
        return None, None

# Приклади 3d металів
# Sc: [Ar] 3d¹ 4s², ²D
e_sc, s2_sc = transition_metal_dft("Sc", spin=1, functional="pbe")

# Ti: [Ar] 3d² 4s², ³F
e_ti, s2_ti = transition_metal_dft("Ti", spin=2, functional="pbe")

# V: [Ar] 3d³ 4s², ⁴F
e_v, s2_v = transition_metal_dft("V", spin=3, functional="pbe")

# Cr: [Ar] 3d⁵ 4s¹, ⁷S (виняток!)
e_cr, s2_cr = transition_metal_dft("Cr", spin=6, functional="pbe")

# Mn: [Ar] 3d⁵ 4s², ⁶S
e_mn, s2_mn = transition_metal_dft("Mn", spin=5, functional="pbe")

# Fe: [Ar] 3d⁶ 4s², ⁵D
e_fe, s2_fe = transition_metal_dft("Fe", spin=4, functional="pbe")

```

5.3.4. Порівняння спінових станів

Для деяких атомів важливо порівняти різні спінові стани:

[code_18.py](#)

```
from pyscf import gto, dft
```

```

import matplotlib.pyplot as plt

def compare_spin_states(symbol, spin_list, functional="pbe0", basis="def2-tzvp"):
    """
    Порівняння енергій різних спінових станів
    """

    print(f"\nПорівняння спінових станів {symbol}")
    print(f"Функціонал: {functional.upper()}, базис: {basis}")
    print("=" * 70)
    print(f"{'2S':5s} {'Mult':5s} {'Енергія, Ha':15s} {'Відносна, kcal/mol':20s}")
    print("-" * 70)

    energies = []
    spins = []

    for spin in spin_list:
        mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

        if spin == 0:
            mf = dft.RKS(mol)
        else:
            mf = dft.UKS(mol)

        mf.xc = functional
        mf.conv_tol = 1e-10
        mf.max_cycle = 150

        try:
            energy = mf.kernel()

            if mf.converged:
                energies.append(energy)
                spins.append(spin)

                mult = spin + 1

                if len(energies) == 1:
                    e_ref = energy
                    rel = 0.0
                else:
                    rel = (energy - e_ref) * 627.509 # kcal/mol

                marker = " ← найнижча" if energy == min(energies) else ""
                print(f"{'spin':5d} {'mult':5d} {'energy:15.8f} {'rel:20.4f}{marker}")
            except:
                print(f"{'spin':5d} {'spin + 1:5d} не конвергувало")

    print("=" * 70)

    # Графік
    if len(energies) > 1:
        fig, ax = plt.subplots(figsize=(10, 6))

        # Відносні енергії в kcal/mol
        e_min = min(energies)
        rel_energies = [(e - e_min) * 627.509 for e in energies]

        ax.plot(spins, rel_energies, "o-", markersize=10, linewidth=2)
        ax.axhline(y=0, color="gray", linestyle="--", alpha=0.5)

        ax.set_xlabel("2S", fontsize=12)
        ax.set_ylabel("Відносна енергія (kcal/mol)", fontsize=12)

```



```
ax.set_title(f"Спінові стани {symbol} ({functional.upper()})", fontsize=14)
ax.grid(True, alpha=0.3)

# Підписи мультиплетностей
for s, e in zip(spins, rel_energies):
    ax.text(s, e + 1, f"M={s + 1}", ha="center", fontsize=9)

plt.tight_layout()
plt.savefig(f"{symbol}_spin_states_{functional}.pdf")
plt.show()

return energies, spins

# Приклад: Карбон (різні спінові стани)
# Основний стан C: 3P (триплет, 2S=2)
# Збуджені: 1D (синглет, 2S=0), 1S (синглет, 2S=0)
energies_c, spins_c = compare_spin_states("C", [0, 2, 4], functional="pbe0")

# Залізо (різні спінові стани)
# Fe може бути у низькоспіновому, середньо- та високоспіновому станах
energies_fe, spins_fe = compare_spin_states(
    "Fe", [0, 2, 4, 6], functional="tpss", basis="def2-svp"
)
```

5.3.5. Аналіз d-орбіталей перехідних металів

code_19.py

```
from pyscf import gto, dft, lo
import numpy as np

def analyze_d_orbitals(symbol, spin, functional="pbe", basis="def2-tzvp"):
    """
    Аналіз заселеності d-орбіталей
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    mf = dft.UKS(mol)
    mf.xc = functional
    mf.verbose = 0
    energy = mf.kernel()

    if not mf.converged:
        print(f"Не конвергувало для {symbol}")
        return

    print(f"\nАналіз d-орбіталей {symbol} ({functional.upper()})")
    print("=" * 70)

    # Заселеності Малікена
    pop, chg = mf.mulliken_pop()

    # Пошук d-орбіталей
    ao_labels = mol.ao_labels(fmt=False)

    d_orbitals_alpha = []
    d_orbitals_beta = []

    for i, label in enumerate(ao_labels):
        atom_id, atom_symbol, shell_type, *rest = label
```

```

if shell_type.startswith("3d"):
    # Альфа заселеність
    dm_alpha = mf.make_rdm1()[0]
    s = mol.intor("int1e_ovlp")
    pop_alpha = (dm_alpha @ s)[i, i]

    # Бета заселеність
    dm_beta = mf.make_rdm1()[1]
    pop_beta = (dm_beta @ s)[i, i]

    d_orbitals_alpha.append((shell_type, pop_alpha))
    d_orbitals_beta.append((shell_type, pop_beta))

if d_orbitals_alpha:
    print("\nЗаселеності d-орбіталей:")
    print(f"{'Орбіталь':12s} {'α':10s} {'β':10s} {'Сума':10s} {'Снін':10s}")
    print("-" * 70)

    for (orb_a, pop_a), (orb_b, pop_b) in zip(d_orbitals_alpha, d_orbitals_beta):
        total = pop_a + pop_b
        spin_dens = pop_a - pop_b
        print(
            f"{'orb_a':12s} {'pop_a':10.4f} {'pop_b':10.4f} "
            f"{'total':10.4f} {'spin_dens':10.4f}"
        )

    # Загальна заселеність d-оболонки
    total_d_alpha = sum(p for _, p in d_orbitals_alpha)
    total_d_beta = sum(p for _, p in d_orbitals_beta)

    print("-" * 70)
    print(
        f"{'Pазом':12s} {'total_d_alpha':10.4f} "
        f"{'total_d_beta':10.4f} "
        f"{'total_d_alpha + total_d_beta':10.4f} "
        f"{'total_d_alpha - total_d_beta':10.4f}"
    )

# Орбітальні енергії
print("\nОрбітальні енергії (eV):")
print(f"{'MO':5s} {'α-енергія':12s} {'β-енергія':12s} {'Заповнення':15s}")
print("-" * 70)

n_alpha, n_beta = mol.nelec

for i in range(min(10, len(mf.mo_energy[0]))):
    e_alpha = mf.mo_energy[0][i] * 27.211386
    e_beta = mf.mo_energy[1][i] * 27.211386

    occ_a = "occ" if i < n_alpha else "virt"
    occ_b = "occ" if i < n_beta else "virt"

    print(f"{i + 1:5d} {e_alpha:12.4f} {e_beta:12.4f} α:{occ_a:5s} β:{occ_b:5s}")

# Приклади
analyze_d_orbitals("Sc", spin=1)
analyze_d_orbitals("Ti", spin=2)
analyze_d_orbitals("V", spin=3)
analyze_d_orbitals("Cr", spin=6)
analyze_d_orbitals("Mn", spin=5)
analyze_d_orbitals("Fe", spin=4)
analyze_d_orbitals("Co", spin=3)
analyze_d_orbitals("Ni", spin=2)

```

5. Теорія функціоналу густини (DFT)

```
analyze_d_orbitals("Cu", spin=1)
analyze_d_orbitals("Zn", spin=0)
```

5.3.6. Розрахунок важких атомів

Для важких атомів (4d, 5d, 4f, 5f) важливі релятивістські ефекти:

[code_20.py](#)

```
from pyscf import gto, dft

def heavy_atom_calculation(
    symbol, spin, functional="pbe", basis="def2-svp", relativistic=False
):
    """
    Розрахунок важкого атома з/без релятивістських ефектів
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    mf = dft.UKS(mol)
    mf.xc = functional

    if relativistic:
        # X2C (exact 2-component) релятивістський гамільтоніан
        mf = mf.x2c()
        print(f"\nРозрахунок {symbol} з X2C релятивістськими корекціями")
    else:
        print(f"\nРозрахунок {symbol} (нерелятивістський)")

    mf.conv_tol = 1e-9
    mf.max_cycle = 150
    energy = mf.kernel()

    print(f"Енергія: {energy:.8f} Ha")

    return energy

# Порівняння релятивістських ефектів
print("Порівняння релятивістських ефектів")
print("=" * 70)

# 5d метал: Золото
print("\nЗолото (Au):")
e_au_nr = heavy_atom_calculation("Au", spin=1, relativistic=False)
e_au_r = heavy_atom_calculation("Au", spin=1, relativistic=True)
rel_effect = (e_au_r - e_au_nr) * 627.509 # kcal/mol
print(f"Релятивістський ефект: {rel_effect:.2f} kcal/mol")

# 5d метал: Платина
print("\nПлатина (Pt):")
e_pt_nr = heavy_atom_calculation("Pt", spin=2, relativistic=False)
e_pt_r = heavy_atom_calculation("Pt", spin=2, relativistic=True)
rel_effect = (e_pt_r - e_pt_nr) * 627.509
print(f"Релятивістський ефект: {rel_effect:.2f} kcal/mol")
```

5.3.7. Числові сітки в DFT

Точність DFT розрахунків залежить від якості числової сітки для інтегрування:

code_21.py

```

from pyscf import gto, dft
import numpy as np

def test_grid_quality(symbol, spin, functional="pbe", basis="cc-pvtz"):
    """
    Тестування впливу якості сітки на результати
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    # Різні рівні сіток
    grids = [
        (1, "грубо"),
        (2, "середньо"),
        (3, "добре (за замовчуванням)"),
        (4, "дуже добре"),
        (5, "ультра добре"),
    ]

    print(f"\nВплив якості сітки на енергію {symbol}")
    print(f"Функціонал: {functional.upper()}, базис: {basis}")
    print("=" * 70)
    print(f"{'Рівень':8s} {'Опис':30s} {'Енергія, Ha':15s} {'ΔE, μHa':10s}")
    print("-" * 70)

    energies = []

    for level, description in grids:
        if spin == 0:
            mf = dft.RKS(mol)
        else:
            mf = dft.UKS(mol)

        mf.xc = functional
        mf.grids.level = level
        mf.verbose = 0
        mf.conv_tol = 1e-11

        energy = mf.kernel()
        energies.append(energy)

        if len(energies) == 1:
            e_ref = energy
            delta = 0.0
        else:
            delta = (energy - e_ref) * 1e6 # microHartree

        print(f"{'level':8d} {'description':30s} {'energy':15.8f} {'delta':10.2f}")

    print("=" * 70)

    # Оцінка збіжності
    if len(energies) ≥ 3:
        conv = abs(energies[-1] - energies[-2]) * 1e6
        print(f"\nЗбіжність (рівні 4→5): {conv:.4f} μHa")
        if conv < 1.0:
            print("Сітка рівня 4 достатня для точних розрахунків")
        else:
            print("Для високої точності використовуйте рівень 5")

# Тестування

```

5. Теорія функціоналу густини (DFT)

```
test_grid_quality("C", spin=2)
test_grid_quality("Fe", spin=4, basis="def2-svp")
test_grid_quality("Kr", spin=0)
```

5.3.8. Паралелізація DFT розрахунків

code_22.py

```
from pyscf import gto, dft
import os

# Налаштування кількості потоків
os.environ["OMP_NUM_THREADS"] = "4" # 4 потоки

mol = gto.M(atom="Br 0 0 0", basis="def2-tzvp", spin=1, verbose=4)

mf = dft.UKS(mol)
mf.xc = "pbe0"

# DFT розрахунки автоматично використовують паралелізацію
# для обчислення інтегралів та Фок-матриць
energy = mf.kernel()

print(f"\nЕнергія Br: {energy:.8f} Ha")
print(f"Використано потоків: {os.environ.get('OMP_NUM_THREADS')}")
```

5.4. Порівняння HF та DFT результатів

5.4.1. Систематичне порівняння енергій

code_23.py

```
from pyscf import gto, scf, dft
import numpy as np
import matplotlib.pyplot as plt

def comprehensive_comparison(atoms_list, basis="cc-pvtz"):
    """
    Детальне порівняння HF та DFT для списку атомів
    """

    methods = {
        "HF": None,
        "LDA": "lda",
        "PBE": "pbe",
        "BLYP": "blyp",
        "B3LYP": "b3lyp",
        "PBE0": "pbe0",
    }

    results = {method: [] for method in methods}
    atom_symbols = []

    print(f"\nПорівняння методів (базис: {basis})")
    print("=" * 90)
    print(
        f"{'Атом':6s} {'HF':15s} {'LDA':15s} {'PBE':15s} "
        f"{'BLYP':15s} {'B3LYP':15s} {'PBE0':15s}"
    )
    )
```

```

print("-" * 90)

for symbol, spin in atoms_list:
    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    energies_row = []

    # HF
    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)
    mf.verbose = 0
    mf.conv_tol = 1e-10
    e_hf = mf.kernel()
    results["HF"].append(e_hf)
    energies_row.append(e_hf)

    # DFT функціонали
    for method in ["LDA", "PBE", "BLYP", "B3LYP", "PBE0"]:
        if spin == 0:
            mf = dft.RKS(mol)
        else:
            mf = dft.UKS(mol)

        mf.xc = methods[method]
        mf.verbose = 0
        mf.conv_tol = 1e-10

        try:
            energy = mf.kernel()
            results[method].append(energy)
            energies_row.append(energy)
        except:
            results[method].append(np.nan)
            energies_row.append(np.nan)

    atom_symbols.append(symbol)

    # Виведення рядка
    row_str = f"{symbol:6s}"
    for e in energies_row:
        if not np.isnan(e):
            row_str += f" {e:15.8f}"
        else:
            row_str += f" {'N/A':15s}"
    print(row_str)

print("=" * 90)

# Аналіз різниць
print("\nРізниці відносно HF (мHa):")
print("=" * 90)
print(
    f"{'Атом':6s} {'LDA-HF':12s} {'PBE-HF':12s} "
    f"{'BLYP-HF':12s} {'B3LYP-HF':12s} {'PBE0-HF':12s}"
)
print("-" * 90)

for i, symbol in enumerate(atom_symbols):
    row_str = f"{symbol:6s}"
    e_hf = results["HF"][i]

    for method in ["LDA", "PBE", "BLYP", "B3LYP", "PBE0"]:

```

```

        e_dft = results[method][i]
        if not np.isnan(e_dft):
            diff = (e_dft - e_hf) * 1000
            row_str += f" {diff:12.4f}"
        else:
            row_str += f" {'N/A':12s}"
    print(row_str)

print("=" * 90)

# Графік
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Абсолютні енергії
x = np.arange(len(atom_symbols))
width = 0.14

for i, method in enumerate(methods.keys()):
    offset = (i - 2.5) * width
    energies = results[method]
    ax1.bar(x + offset, energies, width, label=method)

ax1.set_xlabel("Атом", fontsize=12)
ax1.set_ylabel("Енергія (Ha)", fontsize=12)
ax1.set_title("Абсолютні енергії", fontsize=14)
ax1.set_xticks(x)
ax1.set_xticklabels(atom_symbols)
ax1.legend()
ax1.grid(True, alpha=0.3, axis="y")

# Різниці відносно HF
for i, method in enumerate(["LDA", "PBE", "BLYP", "B3LYP", "PBE0"]):
    offset = (i - 2) * width
    diffs = [
        (results[method][j] - results["HF"][j]) * 1000
        for j in range(len(atom_symbols))
    ]
    ax2.bar(x + offset, diffs, width, label=method)

ax2.axhline(y=0, color="black", linestyle="-", linewidth=0.8)
ax2.set_xlabel("Атом", fontsize=12)
ax2.set_ylabel("ΔE відносно HF (mHa)", fontsize=12)
ax2.set_title("Різниці енергій", fontsize=14)
ax2.set_xticks(x)
ax2.set_xticklabels(atom_symbols)
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("hf_vs_dft_comparison.pdf")
plt.show()

return results

# Атоми другого періоду
atoms_2nd = [
    ("Li", 1),
    ("Be", 0),
    ("B", 1),
    ("C", 2),
    ("N", 3),
    ("O", 2),
    ("F", 1),

```

```

    ("Ne", 0),
]

results = comprehensive_comparison(atoms_2nd, basis="cc-pvtz")

```

5.4.2. Порівняння енергій іонізації

code_24.py

```

from pyscf import gto, scf, dft
import numpy as np
import matplotlib.pyplot as plt

def compare_ionization_energies(atoms_data, basis="aug-cc-pvtz"):
    """
    Порівняння розрахункових та експериментальних ІЕ
    """

    methods = ["HF", "LDA", "PBE", "B3LYP", "PBE0"]

    print(f"\nЕнергії іонізації (eV), базис: {basis}")
    print("=" * 90)
    print(
        f"{'Атом':6s} {'Експ.':10s} {'HF':10s} {'LDA':10s} "
        f"{'PBE':10s} {'B3LYP':10s} {'PBE0':10s}"
    )
    print("-" * 90)

    results = {method: [] for method in methods}
    experimental = []
    atom_symbols = []

    for symbol, spin_n, spin_c, ie_exp in atoms_data:
        atom_symbols.append(symbol)
        experimental.append(ie_exp)

        # Нейтральний атом
        mol_n = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin_n, verbose=0)

        # Катион
        mol_c = gto.M(
            atom=f"{symbol} 0 0 0", basis=basis, charge=1, spin=spin_c, verbose=0
        )

        ie_values = [ie_exp]

        for method in methods:
            # Нейтральний
            if method == "HF":
                mf_n = scf.UHF(mol_n) if spin_n > 0 else scf.RHF(mol_n)
                mf_c = scf.UHF(mol_c) if spin_c > 0 else scf.RHF(mol_c)
            else:
                mf_n = dft.UKS(mol_n) if spin_n > 0 else dft.RKS(mol_n)
                mf_c = dft.UKS(mol_c) if spin_c > 0 else dft.RKS(mol_c)

            xc_dict = {"LDA": "lda", "PBE": "pbe", "B3LYP": "b3lyp", "PBE0": "pbe0"}
            mf_n.xc = xc_dict[method]
            mf_c.xc = xc_dict[method]

            mf_n.verbose = 0
            mf_c.verbose = 0

```



```

        e_n = mf_n.kernel()
        e_c = mf_c.kernel()

        ie = (e_c - e_n) * 27.211386 # eV
        results[method].append(ie)
        ie_values.append(ie)

    # Виведення
    row_str = f"{symbol:6s}"
    for ie in ie_values:
        row_str += f" {ie:10.4f}"
    print(row_str)

print("=" * 90)

# Статистика похибок
print("\nСередні абсолютні похибки (MAE, eV):")
print("-" * 50)

for method in methods:
    errors = [
        abs(results[method][i] - experimental[i]) for i in range(len(experimental))
    ]
    mae = np.mean(errors)
    print(f"{method:10s}: {mae:8.4f} eV")

# Графік
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Порівняння IE
x = np.arange(len(atom_symbols))
width = 0.14

colors = ["black", "red", "blue", "green", "orange", "purple"]

for i, (method, color) in enumerate(zip(["Експ."] + methods, colors)):
    offset = (i - 2.5) * width
    if method == "Експ.":
        values = experimental
    else:
        values = results[method]

    ax1.bar(x + offset, values, width, label=method, color=color, alpha=0.8)

ax1.set_xlabel("Атом", fontsize=12)
ax1.set_ylabel("Енергія іонізації (eV)", fontsize=12)
ax1.set_title("Порівняння енергій іонізації", fontsize=14)
ax1.set_xticks(x)
ax1.set_xticklabels(atom_symbols)
ax1.legend()
ax1.grid(True, alpha=0.3, axis="y")

# Похибки
for method, color in zip(methods, colors[1:]):
    errors = [
        results[method][i] - experimental[i] for i in range(len(experimental))
    ]
    ax2.plot(
        atom_symbols,
        errors,
        "o-",
        label=method,
        color=color,
        linewidth=2,

```

```

        markersize=8,
    )

    ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
    ax2.set_xlabel("Атом", fontsize=12)
    ax2.set_ylabel("Похибка (eV)", fontsize=12)
    ax2.set_title("Похибки відносно експерименту", fontsize=14)
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("ionization_energies_comparison.pdf")
    plt.show()

# Дані: (символ, спин нейтрального, спин катіона, ІЕ експ.)
atoms_ie = [
    ("Li", 1, 0, 5.39),
    ("Be", 0, 1, 9.32),
    ("B", 1, 0, 8.30),
    ("C", 2, 1, 11.26),
    ("N", 3, 2, 14.53),
    ("O", 2, 3, 13.62),
    ("F", 1, 2, 17.42),
    ("Ne", 0, 1, 21.56),
]

compare_ionization_energies(atoms_ie)

```

5.4.3. Електронна спорідненість

[code_25.py](#)

```

from pyscf import gto, scf, dft

def electron_affinity_comparison(atoms_data, basis="aug-cc-pvqz"):
    """
    Порівняння електронної спорідненості
     $EA = E(A) - E(A\emptyset)$ 
    """

    methods = ["HF", "LDA", "PBE", "B3LYP", "PBE0"]

    print(f"\nЕлектронна спорідненість (eV), базис: {basis}")
    print("=" * 90)
    print(
        f"{'Атом':6s} {'Експ.':10s} {'HF':10s} {'LDA':10s} "
        f"{'PBE':10s} {'B3LYP':10s} {'PBE0':10s}"
    )
    print("-" * 90)

    for symbol, spin_n, spin_a, ea_exp in atoms_data:
        # Нейтральний атом
        mol_n = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin_n, verbose=0)

        # Аніон
        mol_a = gto.M(
            atom=f"{symbol} 0 0 0", basis=basis, charge=-1, spin=spin_a, verbose=0
        )

        ea_values = [ea_exp]

```

```

for method in methods:
    if method == "HF":
        mf_n = scf.UHF(mol_n) if spin_n > 0 else scf.RHF(mol_n)
        mf_a = scf.UHF(mol_a) if spin_a > 0 else scf.RHF(mol_a)
    else:
        mf_n = dft.UKS(mol_n) if spin_n > 0 else dft.RKS(mol_n)
        mf_a = dft.UKS(mol_a) if spin_a > 0 else dft.RKS(mol_a)

    xc_dict = {"LDA": "lda", "PBE": "pbe", "B3LYP": "b3lyp", "PBE0": "pbe0"}
    mf_n.xc = xc_dict[method]
    mf_a.xc = xc_dict[method]

    mf_n.verbose = 0
    mf_a.verbose = 0
    mf_a.level_shift = 0.5 # Для аніонів часто потрібно

    try:
        e_n = mf_n.kernel()
        e_a = mf_a.kernel()

        ea = (e_n - e_a) * 27.211386 # eV
        ea_values.append(ea)
    except:
        ea_values.append(np.nan)

# Виведення
row_str = f"{symbol:6s}"
for ea in ea_values:
    if not np.isnan(ea):
        row_str += f" {ea:10.4f}"
    else:
        row_str += f" {'N/A':10s}"
print(row_str)

print("=" * 90)
print("\nПримітка: EA > 0 означає, що аніон стабільний")

# Дані: (символ, спин нейтрального, спин аніона, EA експ.)
atoms_ea = [
    ("B", 1, 2, 0.28),
    ("C", 2, 3, 1.26),
    ("O", 2, 1, 1.46),
    ("F", 1, 0, 3.40),
    ("Cl", 1, 0, 3.61),
    ("Br", 1, 0, 3.36),
]

electron_affinity_comparison(atoms_ea)

```

5.4.4. Порівняння орбітальних енергій

code_26.py

```

from pyscf import gto, scf, dft
import matplotlib.pyplot as plt

def compare_orbital_energies(symbol, spin, basis="cc-pvtz"):
    """
    Порівняння орбітальних енергій HF vs DFT
    """

```

```

mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

methods = {"HF": None, "LDA": "lda", "PBE": "pbe", "B3LYP": "b3lyp", "PBE0": "pbe0"}

orbital_energies = {}

for method, xc in methods.items():
    if method == "HF":
        mf = scf.UHF(mol) if spin > 0 else scf.RHF(mol)
    else:
        mf = dft.UKS(mol) if spin > 0 else dft.RKS(mol)
        mf.xc = xc

    mf.verbose = 0
    mf.kernel()

    if spin == 0:
        orbital_energies[method] = mf.mo_energy * 27.211386
    else:
        orbital_energies[method] = mf.mo_energy[0] * 27.211386

# Графік
fig, ax = plt.subplots(figsize=(12, 8))

n_orb = min(10, len(orbital_energies["HF"]))
x = np.arange(n_orb)
width = 0.16

colors = ["red", "blue", "green", "orange", "purple"]

for i, (method, color) in enumerate(zip(methods.keys(), colors)):
    offset = (i - 2) * width
    energies = orbital_energies[method][:n_orb]
    ax.bar(x + offset, energies, width, label=method, color=color, alpha=0.8)

# Лінія HOMO
if spin == 0:
    n_occ = mol.nelectron // 2
else:
    n_occ = mol.nelec[0]

ax.axvline(
    x=n_occ - 0.5, color="black", linestyle="--", linewidth=2, label="HOMO/LUMO"
)

ax.set_xlabel("Орбіталь", fontsize=12)
ax.set_ylabel("Енергія (eV)", fontsize=12)
ax.set_title(f"Орбітальні енергії {symbol}", fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels([f"M0{i + 1}" for i in x])
ax.legend()
ax.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig(f"{symbol}_orbital_energies_comparison.pdf")
plt.show()

# Таблиця
print(f"\nОрбітальні енергії {symbol} (eV)")
print("=" * 80)
print(f"{ 'M0':5s} { 'HF':12s} { 'LDA':12s} { 'PBE':12s} { 'B3LYP':12s} { 'PBE0':12s}")
print("-" * 80)

for i in range(n_orb):

```

```

row_str = f"{i + 1:5d}"
for method in methods.keys():
    e = orbital_energies[method][i]
    row_str += f" {e:12.4f}"

if i == n_occ - 1:
    row_str += " ← HOMO"
elif i == n_occ:
    row_str += " ← LUMO"

print(row_str)

print("=" * 80)

```

Приклади

```

compare_orbital_energies("C", spin=2)
compare_orbital_energies("Ne", spin=0)
compare_orbital_energies("O", spin=2)

```

5.4.5. Забруднення спіном: HF vs DFT

code_27.py

```

from pyscf import gto, scf, dft

def spin_contamination_analysis(atoms_list, basis="cc-pvtz"):
    """
    Порівняння забруднення спіном у HF та DFT
    """

    print(f"\nЗабруднення спіном <S²> (базис: {basis})")
    print("=" * 80)
    print(
        f"{'Атом':6s} {'2S':4s} {'<S²> очік.':12s} {'HF':12s} "
        f"{'LDA':12s} {'PBE':12s} {'B3LYP':12s}"
    )
    print("-" * 80)

    for symbol, spin in atoms_list:
        if spin == 0:
            continue # Тільки відкриті системи

        mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

        expected_s2 = spin * (spin + 2) / 4

        s2_values = [expected_s2]

        # HF
        mf_hf = scf.UHF(mol)
        mf_hf.verbose = 0
        mf_hf.kernel()
        s2_hf = mf_hf.spin_square()[0]
        s2_values.append(s2_hf)

        # DFT
        for xc in ["lda", "pbe", "b3lyp"]:
            mf = dft.UKS(mol)
            mf.xc = xc
            mf.verbose = 0
            mf.kernel()

```

```

        s2 = mf.spin_square()[0]
        s2_values.append(s2)

    # Виведення
    row_str = f"{symbol:6s} {spin:4d}"
    for s2 in s2_values:
        row_str += f" {s2:12.6f}"

    # Забруднення
    contamination = s2_hf - expected_s2
    if abs(contamination) > 0.01:
        row_str += " ← помітне забруднення HF"

    print(row_str)

print("=" * 80)
print("\nПримітка: Чисті DFT (LDA, PBE) не мають забруднення")
print("        Гібридні DFT (B3LYP) мають слабе забруднення")

# Атоми з відкритими оболонками
atoms_open = [
    ("H", 1),
    ("Li", 1),
    ("B", 1),
    ("C", 2),
    ("N", 3),
    ("O", 2),
    ("F", 1),
    ("Na", 1),
]

spin_contamination_analysis(atoms_open)

```

5.4.6. Час обчислень: HF vs DFT

code_28.py

```

from pyscf import gto, scf, dft
import time

def timing_comparison(symbol, spin, basis="def2-tzvp"):
    """
    Порівняння часу виконання HF vs DFT
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    methods = {"HF": None, "LDA": "lda", "PBE": "pbe", "B3LYP": "b3lyp", "PBE0": "pbe0"}

    print(f"\nЧас виконання для {symbol} (базис: {basis})")
    print("=" * 60)
    print(f"{'Метод':10s} {'Час, с':10s} {'Відносно HF':15s}")
    print("-" * 60)

    times = {}

    for method, xc in methods.items():
        if method == "HF":
            mf = scf.UHF(mol) if spin > 0 else scf.RHF(mol)
        else:
            mf = dft.UKS(mol) if spin > 0 else dft.RKS(mol)

```

```
mf.xc = xc

mf.verbose = 0

# Вимірювання часу
start = time.time()
mf.kernel()
elapsed = time.time() - start

times[method] = elapsed

if method == "HF":
    t_ref = elapsed
    rel = 1.0
else:
    rel = elapsed / t_ref

print(f"{method:10s} {elapsed:10.4f} {rel:15.3f}x")

print("=" * 60)

# Тестування
timing_comparison("C", spin=2, basis="cc-pvtz")
timing_comparison("Fe", spin=4, basis="def2-svp")
timing_comparison("Kr", spin=0, basis="def2-tzvp")
```

5.4.7. Загальні висновки HF vs DFT

Таблиця 5.2. Підсумкове порівняння HF та DFT методів

Критерій	Hartree-Fock	DFT
Точність енергій	Добра якісно	Краща кількісно
Енергії іонізації	Систематично завищені	Ближче до експерименту
Електронна спорідненість	Погана для аніонів	Значно краща
Орбітальні енергії	НОМО $\approx$ -IE (теорема Купманса)	Відхилення від теореми
Забруднення спіном	Присутнє (UHF)	Відсутнє (чисті DFT)
Швидкість	Середня	Швидше (чисті DFT) Повільніше (гібриди)
Перехідні метали	Складно конвергує	Зазвичай краще
Дисперсійні взаємодії	Відсутні	Потрібні корекції
Систематичність	Добра	Залежить від функціоналу

Рекомендації:

- Використовуйте **HF** для швидких якісних оцінок та як початок для post-HF методів
- Використовуйте **чисті DFT** (PBE) для великих систем, перехідних металів

- Використовуйте гібридні DFT (B3LYP, PBE0) для найкращого балансу точності та швидкості
- Для аніонів обов'язково використовуйте дифузні функції (aug-базиси)
- Для важких атомів враховуйте релятивістські ефекти

5.5. Вибір функціоналу для атомних систем

5.5.1. Тестовий набір даних

Для оцінки якості функціоналів використаємо стандартні атомні властивості:

code_29.py

```
from pyscf import gto, scf, dft
import numpy as np

# Експериментальні дані (eV)
experimental_data = {
    "Li": {"IE1": 5.39, "EA": 0.62},
    "C": {"IE1": 11.26, "EA": 1.26},
    "N": {"IE1": 14.53, "EA": -0.07},
    "O": {"IE1": 13.62, "EA": 1.46},
    "F": {"IE1": 17.42, "EA": 3.40},
    "Ne": {"IE1": 21.56, "EA": None},
}

def benchmark_functional(functional, basis="aug-cc-pvtz"):
    """
    Тестування функціоналу на наборі атомів
    """
    print(f"\nТестування функціоналу {functional.upper()}")
    print("=" * 80)

    mae_ie = []
    mae_ea = []

    for symbol, data in experimental_data.items():
        # Визначення спінів (спрощено)
        spins = {
            "Li": (1, 0),
            "C": (2, 1),
            "N": (3, 2),
            "O": (2, 3),
            "F": (1, 2),
            "Ne": (0, 1),
        }
        spin_n, spin_c = spins[symbol]

        # Нейтральний атом
        mol_n = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin_n, verbose=0)

        if functional.lower() == "hf":
            mf_n = scf.UHF(mol_n) if spin_n > 0 else scf.RHF(mol_n)
        else:
            mf_n = dft.UKS(mol_n) if spin_n > 0 else dft.RKS(mol_n)
            mf_n.xc = functional
```



```

mf_n.verbose = 0
e_n = mf_n.kernel()

# Катион (IE)
mol_c = gto.M(
    atom=f"{symbol} 0 0 0", basis=basis, charge=1, spin=spin_c, verbose=0
)

if functional.lower() == "hf":
    mf_c = scf.UHF(mol_c) if spin_c > 0 else scf.RHF(mol_c)
else:
    mf_c = dft.UKS(mol_c) if spin_c > 0 else dft.RKS(mol_c)
    mf_c.xc = functional

mf_c.verbose = 0
e_c = mf_c.kernel()

ie_calc = (e_c - e_n) * 27.211386
ie_exp = data["IE1"]
error_ie = abs(ie_calc - ie_exp)
mae_ie.append(error_ie)

print(
    f"{symbol:4s} IE: calc={ie_calc:7.3f} eV, "
    f"exp={ie_exp:7.3f} eV, error={error_ie:6.3f} eV"
)

# Аніон (EA) - якщо є дані
if data["EA"] is not None and data["EA"] > 0:
    spin_a = spin_n + 1 # Спрошене припущення

    mol_a = gto.M(
        atom=f"{symbol} 0 0 0", basis=basis, charge=-1, spin=spin_a, verbose=0
    )

    if functional.lower() == "hf":
        mf_a = scf.UHF(mol_a) if spin_a > 0 else scf.RHF(mol_a)
    else:
        mf_a = dft.UKS(mol_a) if spin_a > 0 else dft.RKS(mol_a)
        mf_a.xc = functional

    mf_a.verbose = 0
    mf_a.level_shift = 0.5

    try:
        e_a = mf_a.kernel()
        ea_calc = (e_n - e_a) * 27.211386
        ea_exp = data["EA"]
        error_ea = abs(ea_calc - ea_exp)
        mae_ea.append(error_ea)

        print(
            f"EA: calc={ea_calc:7.3f} eV, "
            f"exp={ea_exp:7.3f} eV, error={error_ea:6.3f} eV"
        )
    except:
        print(f"EA: не конвергувало")

print("=" * 80)
print(f"MAE (IE): {np.mean(mae_ie):.3f} eV")
if mae_ea:
    print(f"MAE (EA): {np.mean(mae_ea):.3f} eV")

return np.mean(mae_ie), np.mean(mae_ea) if mae_ea else None

```

```
# Тестування різних функціоналів
functionals_to_test = ["HF", "LDA", "PBE", "BLYP", "B3LYP", "PBE0"]

results = {}
for func in functionals_to_test:
    mae_ie, mae_ea = benchmark_functional(func)
    results[func] = {"IE": mae_ie, "EA": mae_ea}

# Підсумкова таблиця
print("\n\nПідсумкові MAE (eV):")
print("=" * 50)
print(f"{'Функціонал':12s} {'IE':10s} {'EA':10s}")
print("-" * 50)
for func, res in results.items():
    ea_str = f"{res['EA']:.3f}" if res["EA"] else "N/A"
    print(f"{func:12s} {res['IE']:10.3f} {ea_str:10s}")
```

5.5.2. Рекомендації для різних елементів

Таблиця 5.3. Рекомендовані функціонали для різних груп елементів

Група елементів	Рекомендований	Альтернатива
H, He	HF, PBE0	Будь-який
Li–Ne (2 період)	PBE0, B3LYP	PBE, ω B97X-D
Na–Ar (3 період)	PBE0, B3LYP	TPSSH
3d метали (Sc–Zn)	TPSSH, M06	PBE, B3LYP
4d метали (Y–Cd)	TPSSH, PBE	M06
5d метали (La–Hg)	PBE, TPSSH	+ релятивістські
Лантаноїди	PBE, SCAN	+ SOC
Актиноїди	PBE	+ SOC + DFT+U

5.6. Практичні завдання

5.6.1. Завдання 1: Систематичне дослідження

[code_30.py](#)

```
"""
ЗАВДАННЯ 1: Розрахуйте енергії всіх атомів третього періоду
(Na--Ar) з функціоналами PBE, B3LYP та PBE0. Порівняйте
результати з HF. Побудуйте графіки.
Базис: def2-TZVP
"""
# Ваш код тут
```

5. Теорія функціоналу густини (DFT)

5.6.2. Завдання 2: Функціональна залежність

[code_31.py](#)

```
"""
ЗАВДАННЯ 2: Для атома Заліза (Fe) порівняйте енергії різних
спінових станів (2S = 0, 2, 4, 6) використовуючи функціонали:
LDA, PBE, TPSS, B3LYP, PBE0.
Який спіновий стан є найнижчим для кожного функціоналу?
Який функціонал дає правильний основний стан?
Базис: def2-SVP
"""
# Ваш код тут
```

5.6.3. Завдання 3: Конвергенція до базисної межі

[code_32.py](#)

```
"""
ЗАВДАННЯ 3: Для атома Неону розрахуйте енергію з функціоналом
PBE0 та базисами cc-pVDZ, cc-pVTZ, cc-pVQZ, cc-pV5Z.
Екстраполюйте енергію до базисної межі (CBS) за формулою:
 $E(X) = E_{CBS} + A/X^3$ 
де X = 2, 3, 4, 5 для DZ, TZ, QZ, 5Z.
Порівняйте з експериментальною енергією Ne: -128.547 eV
"""
# Ваш код тут
```

5.6.4. Завдання 4: Перехідні метали

[code_33.py](#)

```
"""
ЗАВДАННЯ 4: Розрахуйте всі 3d перехідні метали (Sc--Zn)
з функціоналом TPSSh. Проаналізуйте заселеності d-орбіталей.
Для яких атомів:
1) Всі d-орбіталі рівномірно заселені?
2) Є виражена асиметрія  $\alpha/\beta$ ?
3) Найбільше забруднення спіном?
Базис: def2-TZVP
"""
# Ваш код тут
```

5.7. Резюме

У цьому розділі ми детально вивчили теорію функціоналу густини та її застосування до атомних систем:

- **Теоретичні основи** — теореми Хоенберга–Кона, рівняння Кона–Шема
- **Функціонали** — від простих LDA до складних гібридних і meta-GGA
- **Практичні розрахунки** — атоми різних періодів, перехідні метали
- **Порівняння з HF** — енергії, орбіталі, спін, швидкість
- **Вибір методу** — рекомендації для різних задач

5.7.1. Ключові висновки

1. DFT зазвичай дає кращі результати для атомних енергій порівняно з HF
2. Гібридні функціонали (B3LYP, PBE0) — золота середина між точністю та швидкістю
3. Для перехідних металів краще використовувати meta-GGA (TPSS) або спеціалізовані функціонали (M06)
4. Вибір базису критичний: для аніонів потрібні дифузні функції
5. Чисті DFT не мають забруднення спіном на відміну від UHF
6. Якість числової сітки важлива для точних розрахунків
7. Для важких атомів необхідні релятивістські корекції

5.7.2. Типові помилки

1. **Неправильний спін** — завжди перевіряйте основний стан атома
2. **Недостатній базис** — для точних енергій використовуйте triple-zeta або більше
3. **Забування дифузних функцій** — критично для аніонів та збуджених станів
4. **Ігнорування симетрії** — може уповільнити розрахунок
5. **Погана конвергенція** — використовуйте level shift, змініть початкове наближення
6. **Неправильна сітка** — для точних результатів використовуйте `grid.level` ≥ 3 .

5.7.3. Корисні посилання

- **Libxc** — бібліотека DFT функціоналів: <https://www.tddft.org/programs/libxc/>
- **NIST** — експериментальні дані атомів: <https://physics.nist.gov/PhysRefData/>
- **Basis Set Exchange** — база даних базисних наборів: <https://www.basissetexchange.org/>

5.8. DFT для діатомних молекул

Після вивчення атомних систем переходимо до молекул. Діатомні молекули — природний наступний крок, що дозволяє досліджувати хімічні зв'язки, потенційні криві та спектроскопічні властивості.

5.8.1. Базові DFT розрахунки молекул

Основна структура DFT розрахунку молекули аналогічна до атомної:

code_34.py

```
# =====
# code_34.py - Базовий DFT розрахунок молекули H2
# =====
from pyscf import gto, dft
import numpy as np

# Створення молекули H2
mol = gto.M(
    atom='H 0 0 0; H 0 0 0.74', # Відстань ~0.74 Å
    basis='cc-pvdz',
    unit='angstrom',
    symmetry=True
)

print("Молекула H2")
print("=" * 50)
print(f"Кількість електронів: {mol.nelectron}")
print(f"Спін: {mol.spin}")
print(f"Точкова група: {mol.groupname}")
print()

# DFT розрахунок з різними функціоналами
functionals = ['lda,vwn', 'pbe', 'b3lyp', 'pbe0']

for xc in functionals:
    mf = dft.RKS(mol)
    mf.xc = xc
    energy = mf.kernel()

    print(f"{xc:15s}: E = {energy:15.8f} Hartree")

print("\nПорівняння з HF:")
from pyscf import scf
mf_hf = scf.RHF(mol).run()
print(f"{'HF':15s}: E = {mf_hf.e_tot:15.8f} Hartree")
```

5.8.2. Оптимізація геометрії

Для знаходження рівноважної структури молекули використовується оптимізація геометрії:

code_35.py

```
# =====
# code_35.py - Оптимізація геометрії
# =====
from pyscf import gto, dft
from pyscf.geomopt.geometric_solver import optimize

# Початкова геометрія H2 (трохи невірна)
mol = gto.M(
    atom='H 0 0 0; H 0 0 0.8',
    basis='cc-pvtz',
    unit='angstrom'
)

print("Оптимізація геометрії H2 (PBE0/cc-pVTZ)")
print("=" * 60)

# DFT з PBE0
```

```

mf = dft.RKS(mol)
mf.xc = 'pbe0'
mf.kernel()

print(f"\nПочаткова відстань: 0.80 Å")
print(f"Енергія до оптимізації: {mf.e_tot:.8f} Hartree")

# Оптимізація
mol_eq = optimize(mf)

print(f"\nОптимізована структура:")
print(mol_eq.atom)
coords = mol_eq.atom_coords()
r_opt = np.linalg.norm(coords[1] - coords[0])
print(f"Рівноважна відстань: {r_opt:.6f} Å")

# Фінальна енергія
mf_opt = dft.RKS(mol_eq)
mf_opt.xc = 'pbe0'
e_opt = mf_opt.kernel()
print(f"Енергія після оптимізації: {e_opt:.8f} Hartree")
print(f"Зниження енергії: {(e_opt - mf.e_tot)*27.211:.6f} eV")

```

5.8.3. Криві потенційної енергії

Побудова кривої потенційної енергії (PES) — фундаментальна характеристика хімічного зв'язку:

[code_36.py](#)

```

# =====
# code_36.py - Крива потенційної енергії H2
# =====

from pyscf import gto, dft
import numpy as np
import matplotlib.pyplot as plt

# Діапазон відстаней
distances = np.linspace(0.4, 4.0, 40)

# Функціонали для порівняння
functionals = {
    'PBE': 'pbe',
    'B3LYP': 'b3lyp',
    'PBE0': 'pbe0'
}

results = {name: [] for name in functionals}

print("Розрахунок кривої потенційної енергії H2")
print("=" * 60)

for r in distances:
    mol = gto.M(
        atom=f'H 0 0 0; H 0 0 {r}',
        basis='cc-pvqz',
        unit='angstrom'
    )

    for name, xc in functionals.items():
        mf = dft.RKS(mol)

```

```

mf.xc = xc
mf.verbose = 0
energy = mf.kernel()
results[name].append(energy)

if int(r * 10) % 5 == 0:
    print(f"r = {r:.2f} Å завершено")

# Знаходження мінімумів
print("\n" + "=" * 60)
print("Рівноважні параметри:")
print("=" * 60)
print(f"{'Функціонал':<12} {'r_e (Å)':<12} {'D_e (eV)':<12}")
print("-" * 60)

for name in functionals:
    energies = np.array(results[name])
    min_idx = np.argmin(energies)
    r_e = distances[min_idx]
    D_e = (energies[-1] - energies[min_idx]) * 27.211 # eV

    print(f"{name:<12} {r_e:<12.4f} {D_e:<12.4f}")

print("-" * 60)
print(f"{'Експеримент':<12} {0.741:<12.4f} {4.75:<12.4f}")

# Графік
plt.figure(figsize=(10, 6))
for name in functionals:
    energies = np.array(results[name])
    # Відносно мінімуму в eV
    E_rel = (energies - np.min(energies)) * 27.211
    plt.plot(distances, E_rel, '-o', label=name, markersize=3)

plt.xlabel('Відстань H-H (Å)', fontsize=12)
plt.ylabel('Енергія відносно мінімуму (eV)', fontsize=12)
plt.title('Крива потенційної енергії H2', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.xlim(0.4, 4.0)
plt.ylim(-0.5, 6)
plt.tight_layout()
plt.savefig('h2_pes_dft.png', dpi=300)
print("\nГрафік збережено: h2_pes_dft.png")

```

5.8.4. Порівняння функціоналів для зв'язків

Різні функціонали по-різному описують хімічні зв'язки:

code_37.py

```

# =====
# code_37.py - Порівняння функціоналів для N2
# =====

from pyscf import gto, dft, scf
import numpy as np

# N2 молекула з експериментальною геометрією
mol = gto.M(
    atom='N 0 0 0; N 0 0 1.098',
    basis='aug-cc-pvtz',

```

```

    unit='angstrom',
    symmetry=True
)

print("Порівняння функціоналів для N2")
print("=" * 70)
print(f"Базис: {mol.basis}")
print(f"Геометрія: r(N-N) = 1.098 Å (експеримент)")
print("=" * 70)

functionals = [
    ('LDA', 'lda,vwn'),
    ('PBE', 'pbe'),
    ('BLYP', 'blyp'),
    ('B3LYP', 'b3lyp'),
    ('PBE0', 'pbe0'),
    ('M06-2X', 'm06-2x'),
    ('ωB97X-D', 'wb97x-d')
]

print(f"\n{'Функціонал':<15} {'Енергія (Ha)':<18} {'Час (с)':<10}")
print("-" * 70)

for name, xc in functionals:
    import time
    t0 = time.time()

    mf = dft.RKS(mol)
    mf.xc = xc
    mf.verbose = 0
    energy = mf.kernel()

    t1 = time.time()
    print(f"{'name':<15} {'energy':<18.8f} {'t1-t0':<10.3f}")

# Порівняння з HF
print("-" * 70)
t0 = time.time()
mf_hf = scf.RHF(mol)
mf_hf.verbose = 0
e_hf = mf_hf.kernel()
t1 = time.time()
print(f"{'HF':<15} {'e_hf':<18.8f} {'t1-t0':<10.3f}")

print("\n" + "=" * 70)
print("Примітка: N2 має потрібний зв'язок - складна система")
print("Кореляція критична для правильного опису зв'язку")

```

5.8.5. Спектроскопічні константи

З кривої потенційної енергії можна отримати спектроскопічні константи:

- r_e — рівноважна довжина зв'язку
- D_e — енергія дисоціації
- ω_e — частота коливань
- $\omega_e x_e$ — ангармонічність

[code_38.py](#)

```

# =====
# code_38.py - Спектроскопічні константи

```


5. Теорія функціоналу густини (DFT)

```
# =====
from pyscf import gto, dft
import numpy as np
from scipy.optimize import curve_fit
from scipy.misc import derivative

def morse_potential(r, D_e, r_e, a):
    """Потенціал Морзе"""
    return D_e * (1 - np.exp(-a * (r - r_e)))**2

def harmonic_frequency(r_values, energies, r_e):
    """Частота коливань з другої похідної"""
    # Чисельна друга похідна в точці r_e
    f = lambda r: np.interp(r, r_values, energies)
    k = derivative(f, r_e, n=2, dx=0.001) # Силова константа

    #  $\omega = \sqrt{k/\mu}$  де  $\mu$  - зведена маса
    # Для H2:  $\mu = m_H/2$ 
    mass_H = 1.008 * 1822.888 # в а.о.
    mu = mass_H / 2
    omega = np.sqrt(k / mu)

    # Переведення в  $\text{cm}^{-1}$ 
    omega_cm = omega * 219474.63 # Hartree to  $\text{cm}^{-1}$ 
    return omega_cm

print("Спектроскопічні константи H2 (PBE0/aug-cc-pVQZ)")
print("=" * 70)

# Детальне сканування навколо мінімуму
r_values = np.linspace(0.5, 2.5, 50)
energies = []

for r in r_values:
    mol = gto.M(
        atom=f'H 0 0 0; H 0 0 {r}',
        basis='aug-cc-pvqz',
        unit='angstrom'
    )

    mf = dft.RKS(mol)
    mf.xc = 'pbe0'
    mf.verbose = 0
    energy = mf.kernel()
    energies.append(energy)

energies = np.array(energies)

# Знаходження мінімуму
min_idx = np.argmin(energies)
r_e = r_values[min_idx]
E_min = energies[min_idx]

# Енергія дисоціації
E_inf = energies[-1]
D_e = (E_inf - E_min) * 27.211 # eV

# Апроксимація потенціалом Морзе
try:
    # Початкові значення
    a_init = 1.0
    popt, _ = curve_fit(
        morse_potential,
        r_values,
```

```

        energies - E_inf,
        p0=[E_min - E_inf, r_e, a_init]
    )
    D_fit, r_fit, a_fit = popt

    print(f"\nПараметри потенціалу Морзе:")
    print(f"  D_e = {abs(D_fit)*27.211:.4f} eV")
    print(f"  r_e = {r_fit:.6f} Å")
    print(f"  a   = {a_fit:.4f} Å-1")
except:
    print("\nНе вдалося апроксимувати потенціалом Морзе")

# Частота коливань
omega_e = harmonic_frequency(r_values, energies, r_e)

print(f"\nОбчислені спектроскопічні константи:")
print(f"  r_e   = {r_e:.6f} Å")
print(f"  D_e   = {D_e:.4f} eV")
print(f"  ω_e   = {omega_e:.1f} cm-1")

print(f"\nЕкспериментальні значення:")
print(f"  r_e   = 0.741300 Å")
print(f"  D_e   = 4.7500 eV")
print(f"  ω_e   = 4401.2 cm-1")

print(f"\nВідхилення:")
print(f"  Δr_e   = {(r_e - 0.7413)*100:.3f} %")
print(f"  ΔD_e   = {(D_e - 4.75)/4.75*100:.3f} %")
print(f"  Δω_e   = {(omega_e - 4401.2)/4401.2*100:.3f} %")

```

5.8.6. Гетероядерні молекули

Гетероядерні молекули мають полярний зв'язок та дипольний момент:

[code_39.py](#)

```

# =====
# code_39.py - Гетероядерні молекули (CO, HF, LiH)
# =====
from pyscf import gto, dft
import numpy as np

molecules = {
    'CO': {
        'atom': 'C 0 0 0; O 0 0 1.128',
        'exp_r': 1.128,
        'exp_D': 11.16, # eV
        'description': 'Полярний потрійний зв\'язок'
    },
    'HF': {
        'atom': 'H 0 0 0; F 0 0 0.917',
        'exp_r': 0.917,
        'exp_D': 6.12,
        'description': 'Сильно полярний одинарний зв\'язок'
    },
    'LiH': {
        'atom': 'Li 0 0 0; H 0 0 1.596',
        'exp_r': 1.596,
        'exp_D': 2.43,
        'description': 'Іонний характер зв\'язку'
    }
}

```

```

}

print("Дослідження гетероядерних молекул")
print("=" * 80)

functionals = ['pbe', 'b3lyp', 'pbe0']

for mol_name, data in molecules.items():
    print(f"\n{mol_name}: {data['description']}")
    print("-" * 80)

    mol = gto.M(
        atom=data['atom'],
        basis='aug-cc-pvtz',
        unit='angstrom'
    )

    print(f"{mol_name}<12> {data['energy']<16> {data['dipole']<25>")
    print("-" * 80)

    for xc in functionals:
        mf = dft.RKS(mol)
        mf.xc = xc
        mf.verbose = 0
        energy = mf.kernel()

        # Дипольний момент
        dipole = mf.dip_moment(unit='Debye')
        dip_mag = np.linalg.norm(dipole)

        print(f"{xc}<12> {energy}<16.8f> {dip_mag}<8.4f>")

    print(f"\nЕксперимент: r_e = {data['exp_r']:.3f} Å, D_e = {data['exp_D']:.2f} eV")

```

5.8.7. Молекули з кратними зв'язками

Дослідження молекул з подвійними та потрійними зв'язками:

[code_40.py](#)

```

# =====
# code_40.py - Молекули з кратними зв'язками
# =====

from pyscf import gto, dft
import numpy as np

print("Молекули з кратними зв'язками")
print("=" * 80)

molecules = [
    ('H2', 'H 0 0 0; H 0 0 0.74', 'одинарний'),
    ('N2', 'N 0 0 0; N 0 0 1.098', 'потрійний'),
    ('O2', 'O 0 0 0; O 0 0 1.208', 'подвійний (триплет!)'),
    ('F2', 'F 0 0 0; F 0 0 1.412', 'одинарний (слабкий)'),
]

print(f"\n{mol_name}<10> {bond_type}<15> {energy}<8> {E(PBE0)}<18> "
      f"{E(B3LYP)}<18>")
print("-" * 80)

for name, geom, bond_type in molecules:

```

```

# Визначення спіну
if name == 'O2':
    spin = 2 # Триплет
else:
    spin = 0 # Синглет

mol = gto.M(
    atom=geom,
    basis='cc-pvtz',
    unit='angstrom',
    spin=spin
)

# PBE0
if spin == 0:
    mf1 = dft.RKS(mol)
else:
    mf1 = dft.UKS(mol)
mf1.xc = 'pbe0'
mf1.verbose = 0
e1 = mf1.kernel()

# B3LYP
if spin == 0:
    mf2 = dft.RKS(mol)
else:
    mf2 = dft.UKS(mol)
mf2.xc = 'b3lyp'
mf2.verbose = 0
e2 = mf2.kernel()

print(f"{name:<10} {bond_type:<15} {spin:<8} {e1:<18.8f} {e2:<18.8f}")

print("\nВажливо: O2 має основний стан  $^3\Sigma_g^-$  (триплет)")
print("Це унікальна властивість, пов'язана з молекулярними орбіталами  $\pi^*$ ")

```

5.8.8. Порівняння HF та DFT для молекул

code_41.py

```

# =====
# code_41.py - Порівняння HF та DFT для молекул
# =====

from pyscf import gto, scf, dft
import numpy as np

molecules = {
    'H2': ('H 0 0 0; H 0 0 0.74', 0),
    'LiH': ('Li 0 0 0; H 0 0 1.596', 0),
    'N2': ('N 0 0 0; N 0 0 1.098', 0),
    'CO': ('C 0 0 0; O 0 0 1.128', 0),
    'O2': ('O 0 0 0; O 0 0 1.208', 2), # триплет!
}

print("Систематичне порівняння HF та DFT")
print("=" * 90)
print(f"{'Молекула':<10} {'HF':<16} {'LDA':<16} {'PBE':<16} "
      f"{'B3LYP':<16} {'PBE0':<16}")
print("-" * 90)

for name, (geom, spin) in molecules.items():
    mol = gto.M(

```

```

    atom=geom,
    basis='cc-pvtz',
    unit='angstrom',
    spin=spin
)

energies = []

# HF
if spin == 0:
    mf = scf.RHF(mol)
else:
    mf = scf.UHF(mol)
mf.verbose = 0
energies.append(mf.kernel())

# DFT функціонали
for xc in ['lda,vwn', 'pbe', 'b3lyp', 'pbe0']:
    if spin == 0:
        mf = dft.RKS(mol)
    else:
        mf = dft.UKS(mol)
    mf.xc = xc
    mf.verbose = 0
    energies.append(mf.kernel())

print(f"{name:<10} " + " ".join(f"{e:<16.8f}" for e in energies))

print("\nПостереження:")
print("1. HF систематично дає вищі (менш від'ємні) енергії")
print("2. LDA зазвичай переоцінює енергію зв'язку")
print("3. Гібриди (B3LYP, PBE0) дають найкращий баланс")
print("4. Для O2 (триплет) різниця між методами більша")

```

5.8.9. Молекули перехідних металів

Діатомні молекули перехідних металів — складні багатоконфігураційні системи:

[code_42.py](#)

```

# =====
# code_42.py - Молекули перехідних металів
# =====
from pyscf import gto, dft
import numpy as np

print("Діатомні молекули перехідних металів")
print("=" * 80)

# CuH - типова молекула перехідного металу
molecules_TM = [
    ('CuH', 'Cu 0 0 0; H 0 0 1.46', 0, 'cc-pvtz-pp'),
    ('FeO', 'Fe 0 0 0; O 0 0 1.62', 4, 'cc-pvtz-pp'), # квінтет
    ('TiO', 'Ti 0 0 0; O 0 0 1.62', 2, 'cc-pvtz-pp'), # триплет
]

functionals_TM = [
    ('PBE', 'pbe'),
    ('TPSS', 'tpss'),
    ('TPSSH', 'tpssh'),
]

```

```

('M06', 'm06'),
('B3LYP', 'b3lyp'),
]

for mol_name, geom, spin, basis in molecules_TM:
    print(f"\n{mol_name} (спін = {spin})")
    print("-" * 80)

    mol = gto.M(
        atom=geom,
        basis=basis,
        unit='angstrom',
        spin=spin
    )

    print(f"{'Функціонал':<12} {'Енергія (Ha)':<18} {'Конвергувало':<15}")
    print("-" * 80)

    for name, xc in functionals_TM:
        mf = dft.UKS(mol) # Завжди unrestricted для TM
        mf.xc = xc
        mf.verbose = 0
        mf.max_cycle = 100

        try:
            energy = mf.kernel()
            converged = "Так" if mf.converged else "Ні"
            print(f"{name:<12} {energy:<18.8f} {converged:<15}")
        except:
            print(f"{name:<12} {'FAILED':<18} {'---':<15}")

    print(f"\nПримітка: Використано псевдопотенціал {basis}")

print("\n" + "=" * 80)
print("Висновок: Meta-GGA (TPSS, M06) краще для перехідних металів")
print("Може знадобитись DFT+U для сильно корельованих систем")

```

5.8.10. Відкритошкаралупні молекули

Молекули з непарними електронами потребують UKS підходу:

[code_43.py](#)

```

# =====
# code_43.py - Відкритооболонкові молекули (радикали)
# =====
from pyscf import gto, dft
import numpy as np

print("Відкритооболонкові молекули (радикали)")
print("=" * 80)

radicals = [
    ('OH', 'O 0 0 0; H 0 0 0.97', 1, '2П'),
    ('NO', 'N 0 0 0; O 0 0 1.15', 1, '2П'),
    ('O2', 'O 0 0 0; O 0 0 1.21', 2, '3Σg⁻'),
    ('CN', 'C 0 0 0; N 0 0 1.17', 1, '2Σ⁺'),
]

print(f"\n{'Молекула':<10} {'Стан':<10} {'Спін':<8} {'E(UKS) Ha':<18} "
      f"{'<S²>':<12} {'Забруднення':<15}")

```

```
print("-" * 80)

for name, geom, spin, state in radicals:
    mol = gto.M(
        atom=geom,
        basis='cc-pvtz',
        unit='angstrom',
        spin=spin
    )

    mf = dft.UKS(mol)
    mf.xc = 'pbe0'
    mf.verbose = 0
    energy = mf.kernel()

    # Спінове забруднення
    s_squared = mf.spin_square()[0]
    s_exact = spin * (spin + 2) / 4 # S(S+1) де S = spin/2
    contamination = s_squared - s_exact

    print(f"{name:<10} {state:<10} {spin:<8} {energy:<18.8f} "
          f"{s_squared:<12.6f} {contamination:<15.6f}")

print("\nПримітка:")
print("- UKS не має спінового забруднення (на відміну від UHF)")
print("- <S²> = S(S+1) = (M_s)(M_s + 2)/4 для чистого стану")
print("- Забруднення ≈ 0 для DFT")
```

5.8.11. Дисперсійні корекції для слабких зв'язків

Стандартні DFT функціонали погано описують дисперсійні взаємодії. Потрібні корекції:

code_44.py

```
# =====
# code_44.py - Дисперсійні корекції
# =====

from pyscf import gto, dft
from pyscf.dft import dft_parser
import numpy as np

print("Дисперсійні корекції для слабких взаємодій")
print("-" * 80)

# He2 - типовий приклад дисперсійної взаємодії
distances = np.array([2.0, 2.5, 3.0, 3.5, 4.0, 5.0])

print("\nКрива потенційної енергії He2")
print("-" * 80)

functionals_disp = [
    ('PBE', 'pbe', False),
    ('PBE-D3', 'pbe', True), # з D3
    ('B3LYP', 'b3lyp', False),
    ('ωB97X-D', 'wb97x-d', False), # вбудована дисперсія
]

results_disp = {name: [] for name, _, _ in functionals_disp}

for r in distances:
```

```

mol = gto.M(
    atom=f'He 0 0 0; He 0 0 {r}',
    basis='aug-cc-pvqz',
    unit='angstrom'
)

for name, xc, use_d3 in functionals_disp:
    mf = dft.RKS(mol)
    mf.xc = xc
    mf.verbose = 0

    # Увімкнення D3
    if use_d3:
        mf = dft.RKS(mol).density_fit()
        mf.xc = xc
        # Примітка: для D3 потрібна окрема бібліотека

    energy = mf.kernel()
    results_disp[name].append(energy)

# Виведення результатів
print(f"\n{'r (Å)':<10} " + " ".join(f"{name:<18}" for name, _, _ in functionals_disp))
print("-" * 80)

for i, r in enumerate(distances):
    print(f"{'r:<10.2f} ", end="")
    for name, _, _ in functionals_disp:
        E = results_disp[name][i]
        # Відносно нескінченності
        E_rel = (E - results_disp[name][-1]) * 627.51 # kcal/mol
        print(f"{'E_rel:<18.6f} ", end="")
    print()

print("\nПримітка:")
print("- Стандартні функціонали (PBE, B3LYP) не описують дисперсію")
print("- D3 корекція додає емпіричну дисперсію")
print("- ωB97X-D має вбудовану дисперсійну корекцію")
print("- Для точного опису потрібен CCSD(T) або аналогічний")

```

5.8.12. Аналіз хвильових функцій

Детальний аналіз електронної структури молекул:

[code_45.py](#)

```

# =====
# code_45.py - Аналіз хвильових функцій
# =====

from pyscf import gto, dft, lo
import numpy as np

print("Детальний аналіз електронної структури CO")
print("-" * 80)

mol = gto.M(
    atom='C 0 0 0; O 0 0 1.128',
    basis='cc-pvtz',
    unit='angstrom',
    symmetry=True
)

mf = dft.RKS(mol)

```



```
mf.xc = 'pbe0'
energy = mf.kernel()

print(f"\nПовна енергія: {energy:.8f} Hartree")
print(f"Точкова група: {mol.groupname}")

# Аналіз орбіталей
print("\n" + "=" * 80)
print("Орбітальні енергії (заняті):")
print("-" * 80)

occ_idx = mf.mo_occ > 0
occ_energies = mf.mo_energy[occ_idx]
occ_labels = mol.ao_labels()

print(f"{'#':<5} {'Енергія (eV)':<15} {'Заняття':<10}")
print("-" * 40)
for i, e in enumerate(occ_energies):
    print(f"{i+1:<5} {e*27.211:<15.4f} {mf.mo_occ[i]:<10.1f}")

# HOMO-LUMO gap
homo_idx = np.where(mf.mo_occ > 0)[0][-1]
lumo_idx = np.where(mf.mo_occ == 0)[0][0]
homo_energy = mf.mo_energy[homo_idx]
lumo_energy = mf.mo_energy[lumo_idx]
gap = (lumo_energy - homo_energy) * 27.211

print(f"\nHOMO (орбіталь {homo_idx+1}): {homo_energy*27.211:.4f} eV")
print(f"LUMO (орбіталь {lumo_idx+1}): {lumo_energy*27.211:.4f} eV")
print(f"HOMO-LUMO gap: {gap:.4f} eV")

# Дипольний момент
dipole = mf.dip_moment(unit='Debye')
print(f"\nДипольний момент:")
print(f"  μx = {dipole[0]:.4f} D")
print(f"  μy = {dipole[1]:.4f} D")
print(f"  μz = {dipole[2]:.4f} D")
print(f"  |μ| = {np.linalg.norm(dipole):.4f} D")

# Популяційний аналіз Малікена
print("\nПопуляційний аналіз Малікена:")
pop = mf.mulliken_pop()
print(f"  Заряд на C: {pop[0][0]:.4f}")
print(f"  Заряд на O: {pop[0][1]:.4f}")

# Аналіз зв'язку (bond order)
print("\nПорядок зв'язку (якісна оцінка):")
print(f"  CO має потрібний зв'язок: σ2 π4")
```

5.8.13. Вплив базисного набору

Систематичне дослідження конвергенції до базисної межі:

[code_46.py](#)

```
# =====
# code_46.py - Конвергенція до базисної межі
# =====
from pyscf import gto, dft
import numpy as np

print("Конвергенція до базисної межі для N2")
```

```

print("=" * 80)

basis_sets = [
    'sto-3g',
    '6-31g',
    '6-31g*',
    '6-311g**',
    'cc-pvdz',
    'cc-pvtz',
    'cc-pvqz',
    'aug-cc-pvdz',
    'aug-cc-pvtz',
    'aug-cc-pvqz',
]

print(f"\n{'Базисний набір':<20} {'N_A0':<10} {'Енергія (Ha)':<18} "
      f"{'ΔE (mHa)':<12} {'r_opt (Å)':<12}")
print("-" * 80)

results_basis = []

for basis in basis_sets:
    try:
        mol = gto.M(
            atom='N 0 0 0; N 0 0 1.098',
            basis=basis,
            unit='angstrom'
        )

        mf = dft.RKS(mol)
        mf.xc = 'pbe0'
        mf.verbose = 0
        energy = mf.kernel()

        n_ao = mol.nao
        results_basis.append((basis, n_ao, energy))

        # Різниця з попереднім
        if len(results_basis) > 1:
            delta = (energy - results_basis[-2][2]) * 1000 # mHa
        else:
            delta = 0.0

        print(f"{basis:<20} {n_ao:<10} {energy:<18.8f} {delta:<12.4f} {'1.098':<12}")

    except Exception as e:
        print(f"{basis:<20} {'N/A':<10} {'FAILED':<18} {'':<12} {'':<12}")

# Екстраполяція
print("\n" + "=" * 80)
print("Екстраполяція до базисної межі:")
print("Використовуємо формулу:  $E(l) = E_{\infty} + A/l^3$ ")

# Беремо cc-pVXZ серію
ccpv_results = [(b, e) for b, n, e in results_basis
                 if b.startswith('cc-pv') and 'aug' not in b]

if len(ccpv_results) ≥ 2:
    # l=2 (DZ), l=3 (TZ), l=4 (QZ)
    l_values = np.array([2, 3, 4][:len(ccpv_results)])
    energies = np.array([e for _, e in ccpv_results])

    # Лінійна регресія E vs 1/l³
    from scipy.optimize import curve_fit

```

```
def extrapolation(l, E_inf, A):
    return E_inf + A / l**3

popt, _ = curve_fit(extrapolation, l_values, energies)
E_inf, A = pop

print(f" E∞ = {E_inf:.8f} Hartree")
print(f" A = {A:.6f}")
print(f"\nПорівняння з найбільшим базисом:")
print(f" E(cc-pVQZ) = {ccpv_results[-1][1]:.8f} Ha")
print(f" ΔE = {(E_inf - ccpv_results[-1][1])*1000:.4f} mHa")
```

5.8.14. Дисоціація зв'язків

Проблема симетричного розриву зв'язку

Restricted методи (RKS) неправильно описують дисоціацію гомолітичного зв'язку:

`code_47.py`

```
# =====
# code_47.py - Проблема дисоціації (RKS vs UKS)
# =====

from pyscf import gto, dft
import numpy as np
import matplotlib.pyplot as plt

print("Дисоціація H2: RKS vs UKS")
print("=" * 80)

distances = np.linspace(0.5, 6.0, 40)

results_diss = {
    'RKS-PBE': [],
    'UKS-PBE': [],
    'RKS-PBE0': [],
    'UKS-PBE0': [],
}

print("\nРозрахунок кривих дисоціації...")

for r in distances:
    mol = gto.M(
        atom=f'H 0 0 0; H 0 0 {r}',
        basis='cc-pvqz',
        unit='angstrom'
    )

    # RKS PBE
    mf = dft.RKS(mol)
    mf.xc = 'pbe'
    mf.verbose = 0
    results_diss['RKS-PBE'].append(mf.kernel())

    # UKS PBE
    mf = dft.UKS(mol)
    mf.xc = 'pbe'
    mf.verbose = 0
    results_diss['UKS-PBE'].append(mf.kernel())
```

```

# RKS PBE0
mf = dft.RKS(mol)
mf.xc = 'pbe0'
mf.verbose = 0
results_diss['RKS-PBE0'].append(mf.kernel())

# UKS PBE0
mf = dft.UKS(mol)
mf.xc = 'pbe0'
mf.verbose = 0
results_diss['UKS-PBE0'].append(mf.kernel())

if int(r * 10) % 10 == 0:
    print(f"   r = {r:.1f} Å")

# Аналіз результатів
print("\n" + "=" * 80)
print("Асимптотична поведінка ( $r \rightarrow \infty$ ):")
print("-" * 80)

for name, energies in results_diss.items():
    E_inf = energies[-1]
    E_min = min(energies)
    D_e = (E_inf - E_min) * 27.211
    print(f"{name:<15}  $E(\infty)$  = {E_inf:.8f} Ha,   D_e = {D_e:.4f} eV")

# Порівняння з точною межею ( $2 \times E(H)$ )
mol_H = gto.M(atom='H 0 0 0', basis='cc-pvqz')
mf_H = dft.RKS(mol_H)
mf_H.xc = 'pbe'
mf_H.verbose = 0
E_H_pbe = mf_H.kernel()

mf_H.xc = 'pbe0'
E_H_pbe0 = mf_H.kernel()

print("\n" + "=" * 80)
print("Порівняння з  $E(\infty) = 2 \times E(H \text{ atom})$ :")
print("-" * 80)
print(f"2×E(H) PBE   = {2*E_H_pbe:.8f} Ha")
print(f"2×E(H) PBE0  = {2*E_H_pbe0:.8f} Ha")

print(f"\nВідхилення RKS-PBE: {(results_diss['RKS-PBE'][-1] - 2*E_H_pbe)*27.211*1000:.2f} meV")
print(f"Відхилення UKS-PBE: {(results_diss['UKS-PBE'][-1] - 2*E_H_pbe)*27.211*1000:.2f} meV")
print(f"Відхилення RKS-PBE0: {(results_diss['RKS-PBE0'][-1] - 2*E_H_pbe0)*27.211*1000:.2f} meV")
print(f"Відхилення UKS-PBE0: {(results_diss['UKS-PBE0'][-1] - 2*E_H_pbe0)*27.211*1000:.2f} meV")

# Графік
plt.figure(figsize=(12, 6))

for name, energies in results_diss.items():
    E_rel = (np.array(energies) - min(energies)) * 27.211
    linestyle = '-' if 'UKS' in name else '--'
    plt.plot(distances, E_rel, linestyle, label=name, linewidth=2)

plt.axhline(y=4.75, color='k', linestyle=':', label='Експеримент (4.75 eV)')
plt.xlabel('Відстань H-H (Å)', fontsize=12)
plt.ylabel('Енергія відносно мінімуму (eV)', fontsize=12)
plt.title('Дисоціація H2: RKS vs UKS', fontsize=14)
plt.legend(fontsize=10)

```

```
plt.grid(True, alpha=0.3)
plt.xlim(0.5, 6.0)
plt.ylim(-0.5, 6)
plt.tight_layout()
plt.savefig('h2_dissociation_rks_uks.png', dpi=300)

print("\nГрафік збережено: h2_dissociation_rks_uks.png")
print("\nВисновок:")
print("- UKS правильно описує дисоціацію ( $E \rightarrow 2 \times E(H)$ )")
print("- RKS має помилку через обмеження єдиної детермінанти")
print("- Для гібридів (PBE0) помилка менша через HF обмін")
```

5.8.15. Збуджені стани: TD-DFT

Time-Dependent DFT для вертикальних збуджень:

[code_48.py](#)

```
# =====
# code_48.py - TD-DFT для збуджених станів
# =====

from pyscf import gto, dft
from pyscf.tdscf import TDDFT

print("TD-DFT розрахунок збуджених станів N2")
print("=" * 80)

mol = gto.M(
    atom='N 0 0 0; N 0 0 1.098',
    basis='aug-cc-pvtz',
    unit='angstrom',
    symmetry=True
)

# Основний стан
mf = dft.RKS(mol)
mf.xc = 'pbe0'
mf.kernel()

print(f"Енергія основного стану: {mf.e_tot:.8f} Ha")
print(f"Точкова група: {mol.groupname}")

# TD-DFT
print("\n" + "=" * 80)
print("Вертикальні збудження (TD-PBE0/aug-cc-pVTZ):")
print("=" * 80)

td = TDDFT(mf)
td.nstates = 10 # Перші 10 станів

# Розрахунок
excitations = td.kernel()[0] # Енергії збуджень в Ha

print(f"\n{'#':<5} {'E (eV)':<12} {'λ (nm)':<12} {'f':<12} {'Симетрія':<15}")
print("-" * 80)

for i, ex_energy in enumerate(excitations):
    ex_eV = ex_energy * 27.211
    wavelength = 1240 / ex_eV # nm

    # Сила осцилятора (потребує додаткового розрахунку)
    # Для спрощення показуємо тільки енергію
```

```

print(f"{i+1:<5} {ex_eV:<12.4f} {wavelength:<12.1f} {'---':<12} {'?':<15}")
print("\n" + "=" * 80)
print("Експериментальні значення для N2:")
print("  a1Pg: 8.59 eV (144 nm) - заборонений")
print("  b1Пу: 12.85 eV (96 nm)")
print("  c1Пу: 12.93 eV (96 nm)")

print("\nПримітка:")
print("- TD-DFT добре для валентних збуджень")
print("- Рідбергові стани потребують дифузних функцій")
print("- Для точних результатів: EOM-CCSD або CASPT2")

```

5.8.16. Порівняльна таблиця результатів

Таблиця 5.4. Порівняння методів для типових діатомних молекул

Молекула	Експеримент	HF	PBE	B3LYP	PBE0
H ₂ r_e (Å)	0.741	0.735	0.748	0.744	0.742
H ₂ D_e (eV)	4.75	3.64	4.52	4.71	4.68
N ₂ r_e (Å)	1.098	1.065	1.104	1.098	1.091
N ₂ D_e (eV)	9.91	4.89	9.23	9.85	9.67
O ₂ r_e (Å)	1.208	1.162	1.229	1.216	1.208
CO r_e (Å)	1.128	1.097	1.137	1.130	1.124

5.8.17. Практичні рекомендації

- Для простих молекул (H₂, N₂, O₂):
 - Базовий розрахунок: PBE/сс-pVTZ
 - Висока точність: PBE0/aug-сс-pVQZ
 - Найкраща точність: CCSD(T)/aug-сс-pV5Z
- Для полярних молекул (CO, NO, HF):
 - Гібридні функціонали обов'язкові
 - Дифузні функції для дипольних моментів
- Для молекул перехідних металів:
 - TPSSH або M06 функціонали
 - Велика кількість спінових станів
 - Можливо, потрібен DFT+U
- Для слабких зв'язків:
 - Обов'язкові дисперсійні корекції (D3, D4)
 - Або ωB97X-D, M06-2X функціонали
- Для збуджених станів:

- TD-DFT з range-separated функціоналами
- Або багатоконфігураційні методи (CASSCF)

5.8.18. Типові помилки при розрахунках молекул

1. Неправильна симетрія:

- Завжди використовуйте симетрію молекули
- Перевіряйте point group командою `mol.symmetry = True`

2. Хибна мультиплетність:

- O_2 — триплет, не синглет!
- Перевіряйте спин по `mol.spin`

3. Недостатня сітка інтегрування:

- Для точних градієнтів: `grids.level = 4`
- Для оптимізації завжди перевіряйте конвергенцію по сітці

4. Погана початкова геометрія:

- Починайте з розумних відстаней
- Використовуйте експериментальні дані або попередні розрахунки

5. Ігнорування BSSE (Basis Set Superposition Error):

- Для енергій зв'язування використовуйте counterpoise корекцію
- Особливо важливо для малих базисів

5.8.19. Задачі для самостійної роботи

1. **Задача 1:** Побудуйте криві потенційної енергії молекули F_2 різними функціоналами (LDA, PBE, B3LYP, PBE0) та порівняйте з експериментом ($r_e = 1.412 \text{ \AA}$, $D_e = 1.66 \text{ eV}$).
2. **Задача 2:** Дослідіть вплив базисного набору на властивості CO: розрахуйте r_e , D_e , ω_e з базисами cc-pVDZ, cc-pVTZ, cc-pVQZ та екстраполуйте до межі.
3. **Задача 3:** Порівняйте RKS та UKS підходи для дисоціації H_2 . Поясніть різницю в поведінці енергії на великих відстанях.
4. **Задача 4:** Розрахуйте перші три збуджені стани N_2 методом TD-DFT з різними функціоналами. Порівняйте з експериментальними значеннями.
5. **Задача 5:** Дослідіть молекулу CuH методами PBE, TPSSh та B3LYP. Який функціонал краще описує зв'язок Cu–H?
6. **Задача 6:** Виконайте систематичне дослідження галогенів (F_2 , Cl_2 , Br_2 , I_2) з різними функціоналами. Який функціонал найкраще відтворює експериментальні енергії дисоціації?
7. **Задача 7:** Побудуйте benchmark для молекул H_2 , N_2 , CO з базисами від cc-pVDZ до aug-cc-pV5Z. Визначте, при якому базисі досягається конвергенція з точністю 1 mHa.
8. **Задача 8:** Розрахуйте тестовий набір G2 (8 молекул) функціоналами PBE, B3LYP, PBE0, M06-2X. Порівняйте MAE, RMSE та визначте найкращий функціонал.

5.8.20. Типові помилки при розрахунках молекул

1. Неправильна симетрія:

- Завжди використовуйте симетрію молекули
- Перевіряйте point group командою `mol.symmetry = True`

2. Хибна мультиплетність:

- O_2 — триплет, не синглет!
- Перевіряйте спин по `mol.spin`

3. Недостатня сітка інтегрування:

- Для точних градієнтів: `grids.level = 4`
- Для оптимізації завжди перевіряйте конвергенцію по сітці

4. Погана початкова геометрія:

- Починайте з розумних відстаней
- Використовуйте експериментальні дані або попередні розрахунки

5. Ігнорування BSSE (Basis Set Superposition Error):

- Для енергій зв'язування використовуйте counterpoise корекцію
- Особливо важливо для малих базисів

5.8.21. Систематичне дослідження галогенів

Галогени (F_2 , Cl_2 , Br_2 , I_2) — цікава серія для тестування функціоналів:

[code_49.py](#)

```
# =====
# code_49.py - Систематичне дослідження галогенів
# =====
from pyscf import gto, dft
import numpy as np

print("Систематичне дослідження галогенів X2")
print("=" * 80)

halogens = {
    'F2': {'r': 1.412, 'D_exp': 1.66, 'basis': 'aug-cc-pvtz'},
    'Cl2': {'r': 1.988, 'D_exp': 2.51, 'basis': 'aug-cc-pvtz'},
    'Br2': {'r': 2.281, 'D_exp': 1.99, 'basis': 'aug-cc-pvtz-pp'}, # PP для Br
    'I2': {'r': 2.666, 'D_exp': 1.54, 'basis': 'aug-cc-pvtz-pp'}, # PP для I
}

functionals_hal = ['pbe', 'b3lyp', 'pbe0', 'wb97x-d']

print(f"\n{'Молекула':<10} {'Функціонал':<12} {'E (Ha)':<18} "
      f"{'D_e calc (eV)':<15} {'D_e exp (eV)':<15}")
print("-" * 80)

for mol_name, data in halogens.items():
    atom1 = mol_name[:1]
    r = data['r']
    basis = data['basis']
    D_exp = data['D_exp']

    for xc in functionals_hal:
        # Молекула
        mol_X2 = gto.M(
            atom=f'{atom1} 0 0 0; {atom1} 0 0 {r}',
```



```

        basis=basis,
        unit='angstrom'
    )

    mf = dft.RKS(mol_X2)
    mf.xc = xc
    mf.verbose = 0
    E_X2 = mf.kernel()

    # Атом
    mol_X = gto.M(
        atom=f'{atom1} 0 0 0',
        basis=basis
    )

    mf_atom = dft.RKS(mol_X)
    mf_atom.xc = xc
    mf_atom.verbose = 0
    E_X = mf_atom.kernel()

    # Енергія дисоціації
    D_calc = (2 * E_X - E_X2) * 27.211

    print(f"{mol_name:<10} {xc:<12} {E_X2:<18.8f} {D_calc:<15.4f} {D_exp:<15.4f}")

print("\n" + "=" * 80)
print("Спостереження:")
print("1. F2 - найскладніша (слабкий зв'язок, багато кореляції)")
print("2. Дисперсійні корекції важливі (wb97x-d)")
print("3. Для Br, I потрібні релятивістські псевдопотенціали")

```

Особливості галогенів:

- F₂ має найслабкіший зв'язок через відштовхування lone pairs
- Для Br та I необхідні релятивістські псевдопотенціали
- Дисперсійні корекції важливі для важких галогенів
- LDA/GGA систематично переоцінюють енергії зв'язку

5.8.22. Benchmark: час обчислень

Практичне порівняння швидкості різних методів:

[code_50.py](#)

```

# =====
# code_50.py - Порівняння часу обчислень
# =====
from pyscf import gto, dft, scf
import time
import numpy as np

print("Порівняння часу обчислень для CO")
print("=" * 80)

basis_sets_time = ['cc-pvdz', 'cc-pvtz', 'cc-pvqz', 'aug-cc-pvqz']
methods = [
    ('HF', None),
    ('LDA', 'lda,vwn'),
    ('PBE', 'pbe'),
    ('B3LYP', 'b3lyp'),

```

```

    ('PBE0', 'pbe0'),
    ('M06-2X', 'm06-2x'),
]

print(f"\n{'Метод':<12} {'Базис':<15} {'N_AO':<8} {'Час (с)':<12} "
      f"{'Енергія (Ha)':<18}")
print("-" * 80)

for basis in basis_sets_time:
    mol = gto.M(
        atom='C 0 0 0; 0 0 0 1.128',
        basis=basis,
        unit='angstrom'
    )

    n_ao = mol.nao

    for method_name, xc in methods:
        t0 = time.time()

        if xc is None: # HF
            mf = scf.RHF(mol)
        else:
            mf = dft.RKS(mol)
            mf.xc = xc

        mf.verbose = 0
        energy = mf.kernel()

        t1 = time.time()
        elapsed = t1 - t0

        print(f"{'method_name':<12} {'basis':<15} {'n_ao':<8} {'elapsed':<12.4f} {'energy':<18.8f}")

    print("-" * 80)

print("\nВисновки:")
print("1. LDA, PBE найшвидші (без точного обміну)")
print("2. Гібриди (B3LYP, PBE0) повільніші через HF обмін")
print("3. M06-2X найповільніший (54% HF + meta-GGA)")
print("4. Час масштабується як O(N³) для чистих DFT")
print("5. Час масштабується як O(N⁴) для гібридів")

```

Масштабування обчислювальної складності:

- **LDA/GGA:** $\mathcal{O}(N^3)$ — найшвидші
- **Гібриди:** $\mathcal{O}(N^4)$ — через точний обмін HF
- **Meta-GGA:** $\mathcal{O}(N^3)$ — але з більшою константою
- **Double-hybrid:** $\mathcal{O}(N^5)$ — через MP2 кореляцію

5.8.23. Тестовий набір G2

Систематична оцінка якості функціоналів на стандартному наборі:

[code_51.py](#)

```

# =====
# code_51.py - Тестовий набір для оцінки функціоналів
# =====
from pyscf import gto, dft

```

```

import numpy as np

print("Тестовий набір G2 (вибірка діатомних молекул)")
print("=" * 80)

# Експериментальні дані (r_e в Å, D_e в eV)
g2_molecules = {
    'H2': {'geom': 'H 0 0 0; H 0 0 0.741', 'r_exp': 0.741, 'D_exp': 4.75},
    'Li2': {'geom': 'Li 0 0 0; Li 0 0 2.673', 'r_exp': 2.673, 'D_exp': 1.05},
    'LiH': {'geom': 'Li 0 0 0; H 0 0 1.596', 'r_exp': 1.596, 'D_exp': 2.43},
    'N2': {'geom': 'N 0 0 0; N 0 0 1.098', 'r_exp': 1.098, 'D_exp': 9.91},
    'O2': {'geom': 'O 0 0 0; O 0 0 1.208', 'r_exp': 1.208, 'D_exp': 5.21, 'spin': 2},
    'F2': {'geom': 'F 0 0 0; F 0 0 1.412', 'r_exp': 1.412, 'D_exp': 1.66},
    'CO': {'geom': 'C 0 0 0; O 0 0 1.128', 'r_exp': 1.128, 'D_exp': 11.16},
    'NO': {'geom': 'N 0 0 0; O 0 0 1.151', 'r_exp': 1.151, 'D_exp': 6.51, 'spin': 1},
}

functionals_test = ['pbe', 'b3lyp', 'pbe0', 'm06-2x']
basis = 'aug-cc-pvtz'

# Зберігаємо результати
results_test = {xc: {'D_errors': [], 'names': []} for xc in functionals_test}

print(f"\nБазис: {basis}")
print("=" * 80)

for mol_name, data in g2_molecules.items():
    spin = data.get('spin', 0)
    D_exp = data['D_exp']

    print(f"\n{mol_name}:")
    print("-" * 40)

    for xc in functionals_test:
        # Молекула
        mol = gto.M(
            atom=data['geom'],
            basis=basis,
            unit='angstrom',
            spin=spin
        )

        if spin == 0:
            mf_mol = dft.RKS(mol)
        else:
            mf_mol = dft.UKS(mol)
        mf_mol.xc = xc
        mf_mol.verbose = 0
        E_mol = mf_mol.kernel()

        # Атоми (розібрати геометрію)
        atoms = data['geom'].split(';')
        atom1 = atoms[0].strip().split()[0]
        atom2 = atoms[1].strip().split()[0]

        # Енергія атомів
        E_atoms = 0
        for atom_name in [atom1, atom2]:
            mol_atom = gto.M(atom=f'{atom_name} 0 0 0', basis=basis)

            # Визначення спіну атома
            atom_spins = {'H': 1, 'Li': 1, 'N': 3, 'O': 2, 'F': 1, 'C': 2}
            atom_spin = atom_spins.get(atom_name, 0)

```

```

    if atom_spin == 0:
        mf_atom = dft.RKS(mol_atom)
    else:
        mf_atom = dft.UKS(mol_atom)
    mf_atom.xc = xc
    mf_atom.verbose = 0
    E_atoms += mf_atom.kernel()

    # Енергія дисоціації
    D_calc = (E_atoms - E_mol) * 27.211
    error = D_calc - D_exp

    results_test[xc]['D_errors'].append(error)
    results_test[xc]['names'].append(mol_name)

    print(f" {xc:<10}: D_e = {D_calc:6.3f} eV ( $\Delta$  = {error:+6.3f} eV)")

# Статистика
print("\n" + "=" * 80)
print("Статистика помилок D_e (eV):")
print("=" * 80)
print(f"{'Функціонал':<12} {'MAE':<10} {'RMSE':<10} {'Max|error|':<12}")
print("-" * 80)

for xc in functionals_test:
    errors = np.array(results_test[xc]['D_errors'])
    mae = np.mean(np.abs(errors))
    rmse = np.sqrt(np.mean(errors**2))
    max_err = np.max(np.abs(errors))

    print(f"{xc:<12} {mae:<10.4f} {rmse:<10.4f} {max_err:<12.4f}")

print("\nВисновок: B3LYP та PBE0 зазвичай найкращі для термодинаміки")

```

Метрики якості:

- **MAE** (Mean Absolute Error) — середня абсолютна помилка
- **RMSE** (Root Mean Square Error) — середньоквадратична помилка
- **Max error** — максимальна помилка

Типові результати для енергій дисоціації:

- LDA: MAE $\sim$ 0.5–1.0 eV (переоцінює зв'язування)
- PBE: MAE $\sim$ 0.3–0.5 eV
- B3LYP/PBE0: MAE $\sim$ 0.1–0.3 eV (найкраще)
- M06-2X: MAE $\sim$ 0.2–0.4 eV

5.8.24. Використання симетрії

Симетрія молекули може суттєво прискорити розрахунки:

code_52.py

```

# =====
# code_52.py - Вплив симетрії на час розрахунку
# =====
from pyscf import gto, dft
import time

print("Вплив симетрії на час розрахунку")

```

```

print("=" * 80)

molecules_sym = [
    ('H2', 'H 0 0 0; H 0 0 0.74', 'D $\infty$ h'),
    ('N2', 'N 0 0 0; N 0 0 1.098', 'D $\infty$ h'),
    ('O2', 'O 0 0 0; O 0 0 1.208', 'D $\infty$ h'),
    ('CO', 'C 0 0 0; O 0 0 1.128', 'C $\infty$ v'),
]

basis = 'cc-pvqz'

print(f"\nБазис: {basis}, Функціонал: PBE0")
print("=" * 80)
print(f"{'Молекула':<10} {'Симетрія':<12} {'3 симетрією':<15} "
      f"{'Без симетрії':<15} {'Прискорення':<15}")
print("-" * 80)

for mol_name, geom, sym in molecules_sym:
    spin = 2 if mol_name == 'O2' else 0

    # 3 симетрією
    mol_sym = gto.M(
        atom=geom,
        basis=basis,
        unit='angstrom',
        symmetry=True,
        spin=spin
    )

    t0 = time.time()
    if spin == 0:
        mf = dft.RKS(mol_sym)
    else:
        mf = dft.UKS(mol_sym)
    mf.xc = 'pbe0'
    mf.verbose = 0
    mf.kernel()
    t_sym = time.time() - t0

    # Без симетрії
    mol_nosym = gto.M(
        atom=geom,
        basis=basis,
        unit='angstrom',
        symmetry=False,
        spin=spin
    )

    t0 = time.time()
    if spin == 0:
        mf = dft.RKS(mol_nosym)
    else:
        mf = dft.UKS(mol_nosym)
    mf.xc = 'pbe0'
    mf.verbose = 0
    mf.kernel()
    t_nosym = time.time() - t0

    speedup = t_nosym / t_sym

    print(f"{'mol_name':<10} {'sym':<12} {'t_sym':<15.4f} {'t_nosym':<15.4f} {'speedup':<15.2f}x")

print("\nВисновок: Використання симетрії дає 2-5х прискорення!")

```

Переваги використання симетрії:

1. **Прискорення:** 2–10х залежно від точкової групи
2. **Менше пам'яті:** блокова структура матриць
3. **Класифікація орбіталей:** по незвідним представленням
4. **Відбір правил:** для спектроскопії

Точкові групи діатомних молекул:

- $D_{\infty h}$: гомоядерні (H_2 , N_2 , O_2)
- $C_{\infty v}$: гетероядерні (CO, HF, NO)

5.8.25. Резюме секції

У цій секції ми навчилися:

- Проводити DFT розрахунки діатомних молекул
- Будувати криві потенційної енергії
- Оптимізувати геометрію та обчислювати спектроскопічні константи
- Порівнювати різні функціонали для хімічних зв'язків
- Враховувати дисперсійні взаємодії
- Розраховувати збуджені стани методом TD-DFT
- Уникати типових помилок при молекулярних розрахунках

Ключовий висновок: DFT є ефективним інструментом для опису хімічних зв'язків у діатомних молекулах, але вибір функціоналу критично важливий і залежить від типу зв'язку та властивостей, що досліджуються.

У наступному розділі ми розглянемо Post-Hartree-Fock методи (MP2, CCSD, CASSCF), які дозволяють досягти ще вищої точності для атомних та молекулярних систем, враховуючи електронну кореляцію явно.

6

Пост-Хартрі-Фоківські методи

6.1. Вступ до електронної кореляції

6.1.1. Що таке електронна кореляція?

Електронна кореляція — це різниця між точною енергією системи та енергією, отриманою методом Хартрі-Фока:

$$E_{\text{corr}} = E_{\text{exact}} - E_{\text{HF}} \quad (6.1)$$

Метод Хартрі-Фока не враховує миттєву кореляцію рухів електронів, оскільки кожен електрон рухається в усередненому полі всіх інших електронів. Насправді ж електрони "уникають" один одного через кулонівське відштовхування.

6.1.2. Типи електронної кореляції

Динамічна кореляція Пов'язана з миттєвими флуктуаціями електронної густини. Проявляється на коротких відстанях між електронами. Може бути враховані методами:

- Теорія збурень Møller-Plesset (MP2, MP3, MP4)
- Coupled Cluster (CCSD, CCSD(T))
- Configuration Interaction (CISD, QCISD)

Статична (нединамічна) кореляція Виникає, коли кілька конфігурацій близькі за енергією. Важлива для:

- Розриву хімічних зв'язків
 - Збуджених станів
 - Диелектронних систем
 - Перехідних металів з близькими d-орбіталями
- Методи: CASSCF, CASPT2, MRCI.

6.1.3. Кореляційна енергія атомів

Для атомів кореляційна енергія становить 1–5% від повної енергії, але вона критична для точних розрахунків:

Таблиця 6.1. Кореляційна енергія атомів (Ha)

Атом	E_{HF}	E_{exact}	E_{corr}	% від E_{HF}
He	−2.8617	−2.9037	−0.0420	1.5%
Be	−14.573	−14.667	−0.094	0.6%
Ne	−128.547	−128.937	−0.390	0.3%
Ar	−526.817	−527.540	−0.723	0.1%

6.1.4. Концепція одно- та багатоконфігураційних методів

Одноконфігураційні методи Базуються на одному детермінанті Слейтера (HF) і додають кореляцію через збудження:

У найпростішому випадку хвильова функція є одним детермінантом:

$$|\Psi_{SR}\rangle = |\Phi_0\rangle = |\varphi_1\varphi_2 \dots \varphi_N|. \quad (6.2)$$

Далі на нього накладають кореляційні поправки — не у вигляді нових детермінантів у самій хвильовій функції, а через операторні або збурювальні члени у виразі для енергії. Це дає «однореференсні» методи:

$$E = E_{HF} + E_{corr},$$

де E_{corr} визначається з теорії збурень (MP2, MP3...), або з експоненційного кластерного розкладу

$$|\Psi_{CC}\rangle = e^{\hat{T}}|\Phi_0\rangle, \quad \hat{T} = \hat{T}_1 + \hat{T}_2 + \dots,$$

у методі CCSD тощо.

До одностермінантних методів належать наступні:

- MP2, MP3, MP4 — теорія збурень
- CCSD, CCSD(T) — coupled cluster
- CISD, QCISD — configuration interaction

Добре працюють для основних станів, де один детермінант домінує у хвильовій функції.

Багатоконфігураційні методи Коли один детермінант не здатен описати систему (наприклад, при розриві хімічного зв'язку), хвильова функція має вигляд:

$$|\Psi_{MR}\rangle = \sum_I c_I |\Phi_I\rangle. \quad (6.3)$$

Кожен детермінант $|\Phi_I\rangle$ утворюється перестановкою заповнених і вільних орбіталей у певному активному просторі. Наприклад, у методі CASSCF(4,4) ми розглядаємо всі можливі розподіли 4 електронів по 4 активних орбіталях. Це породжує набір детермінантів:

$$\{|\Phi_I\rangle\} = \{|(2s)^2(2p_x)^2|, |(2s)^1(2p_x)^1(2p_y)^2|, \dots\},$$

а хвильова функція є їхньою суперпозицією з коефіцієнтами c_I , які обчислюються варіаційно.

До багатоконфігураційних належать методи:

- CASSCF — вибір активного простору
- CASPT2 — додавання динамічної кореляції до CASSCF
- MRCI — multireference CI

Необхідні для: розриву зв'язків, збуджених станів, діелектронних систем.

code_3.py

```
from pyscf import gto, scf, mcscf

def multiconfigurational_demo():
    """
    Демонстрація багатоконфігураційного характеру
    """

    # Атом Be: [He] 2s2
    # Близькі за енергією 2s2 та 2p2

    mol = gto.M(atom="Be 0 0 0", basis="cc-pvtz", spin=0, verbose=0)

    print("Багатоконфігураційний характер Be")
    print("=" * 60)

    # RHF
    mf = scf.RHF(mol)
    mf.verbose = 0
    e_hf = mf.kernel()

    print(f"RHF енергія: {e_hf:.8f} Ha")

    # CASSCF(4,8): 4 електрони у 8 орбіталях (2s, 2p, 3s, 3p)
    mc = mcscf.CASSCF(mf, 8, 4)
    mc.verbose = 0
    e_casscf = mc.kernel()[0]

    print(f"CASSCF(4,8) енергія: {e_casscf:.8f} Ha")
    print(f"Статична кореляція: {(e_casscf - e_hf) * 1000:.4f} mHa")

    # Аналіз конфігурацій
    print(f"\nОсновні конфігурації (вага > 1%):")

    # Отримання CI коефіцієнтів
    ci_coeff = mc.ci

    # Для детального аналізу потрібен додатковий код
    print("(детальний аналіз потребує додаткової обробки)")

    print("=" * 60)
```

multiconfigurational_demo()

6.1.5. Коли потрібні post-HF методи?

1. **Спектроскопічна точність** — похибка < 1 kcal/mol (0.04 eV)
2. **Порівняння близьких станів** — різниці енергій між мультиплетами
3. **Електронна спорідненість** — HF і DFT часто не достатньо
4. **Еталонні розрахунки** — для валідації DFT функціоналів
5. **Системи з сильною кореляцією** — перехідні метали, f-елементи

6.2. Теорія збурень Møller-Plesset (MP2)

6.2.1. Теоретичні основи MP2

Теорія збурень Møller-Plesset розділяє гамільтоніан на незбурену частину (HF) та збурення:

$$\hat{H} = \hat{H}_0 + \lambda \hat{V} \quad (6.4)$$

де $\hat{H}_0$ — оператор Фока, а $\hat{V}$ — різниця між точним гамільтоніаном та HF.

Енергія розкладається в ряд за степенями λ :

$$E = E^{(0)} + E^{(1)} + E^{(2)} + E^{(3)} + \dots \quad (6.5)$$

де:

- $E^{(0)} + E^{(1)} = E_{HF}$ — енергія Хартрі-Фока
- $E^{(2)}$ — MP2 корекція (перший порядок кореляції)
- $E^{(3)}, E^{(4)}$ — вищі порядки (MP3, MP4)

6.2.2. Формула MP2 кореляційної енергії

Для замкненої оболонки (RMP2):

$$E^{(2)} = - \sum_{i < j}^{\text{occ}} \sum_{a < b}^{\text{virt}} \frac{|\langle ij || ab \rangle|^2}{\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b} \quad (6.6)$$

де:

- i, j — заповнені орбіталі
- a, b — віртуальні орбіталі
- $\langle ij || ab \rangle$ — антисиметризовані двоелектронні інтеграли
- ϵ — орбітальні енергії

6.2.3. MP2 розрахунок атома Неону

code_4.py

```

from pyscf import gto, scf, mp
import numpy as np

def mp2_neon_calculation(basis="cc-pvtz"):
    """
    Детальний MP2 розрахунок атома Ne
    """

    mol = gto.M(atom="Ne 0 0 0", basis=basis, spin=0, verbose=0)

    print(f"\nMP2 розрахунок атома Неону (базис: {basis})")
    print("=" * 70)

    # Крок 1: HF розрахунок
    print("\nКрок 1: Hartree-Fock розрахунок")
    mf = scf.RHF(mol)
    mf.verbose = 4
    mf.conv_tol = 1e-12
    e_hf = mf.kernel()

    print(f"\nHF енергія: {e_hf:.10f} Ha")
    print(f"HF енергія: {e_hf * 27.211386:.6f} eV")

    # Крок 2: MP2 розрахунок
    print("\nКрок 2: MP2 кореляція")
    mmp2 = mp.MP2(mf)
    mmp2.verbose = 4
    e_mp2_corr, t2 = mmp2.kernel()

    e_total_mp2 = e_hf + e_mp2_corr

    print(f"\nMP2 кореляційна енергія: {e_mp2_corr:.10f} Ha")
    print(f"MP2 повна енергія: {e_total_mp2:.10f} Ha")
    print(f"MP2 повна енергія: {e_total_mp2 * 27.211386:.6f} eV")

    # Аналіз
    print("\nАналіз:")
    print(f"Кореляція становить {abs(e_mp2_corr / e_hf) * 100:.3f}% від HF енергії")

    # Порівняння з експериментом
    e_exp = -128.937 # Ha (експериментальна енергія Ne)
    error_hf = (e_hf - e_exp) * 1000
    error_mp2 = (e_total_mp2 - e_exp) * 1000

    print(f"\nПорівняння з експериментом ({e_exp:.6f} Ha):")
    print(f"HF похибка: {error_hf:8.4f} мHa")
    print(f"MP2 похибка: {error_mp2:8.4f} мHa")
    print(f"Покращення: {abs(error_hf - error_mp2):8.4f} мHa")
    print(f"MP2 відновлює {(1 - error_mp2 / error_hf) * 100:.1f}% похибки HF")

    # Інформація про розміри
    print(f"\nОбчислювальні деталі:")
    print(f"Заповнених орбіталей: {mol.nelectron // 2}")
    print(f"Віртуальних орбіталей: {mol.nao_nr() - mol.nelectron // 2}")
    print(f"Розмір t2 амплітуд: {t2.shape}")

    return e_hf, e_total_mp2, e_mp2_corr

```

```
e_hf, e_mp2, e_corr = mp2_neon_calculation()
```

6.2.4. MP2 для відкритих оболонок (UMP2)

Для систем з неспареними електронами використовується UMP2:

[code_5.py](#)

```
from pyscf import gto, scf, mp

def ump2_calculation(symbol, spin, basis="cc-pvtz"):
    """
    UMP2 розрахунок для відкритої оболонки
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    print(f"\nUMP2 розрахунок {symbol} (2S={spin})")
    print("=" * 70)

    # UHF
    mf = scf.UHF(mol)
    mf.verbose = 0
    mf.conv_tol = 1e-10
    e_hf = mf.kernel()

    print(f"UHF енергія: {e_hf:.10f} Ha")

    # Забруднення спіном
    s2_hf = mf.spin_square()[0]
    expected_s2 = spin * (spin + 2) / 4
    print(f"<S²> (UHF): {s2_hf:.6f} (очікується {expected_s2:.6f})")

    # UMP2
    mymp2 = mp.UMP2(mf)
    mymp2.verbose = 0
    e_mp2_corr, t2 = mymp2.kernel()

    e_total = e_hf + e_mp2_corr

    print(f"\nUMP2 кореляція: {e_mp2_corr:.10f} Ha")
    print(f"UMP2 повна енергія: {e_total:.10f} Ha")

    # MP2 не виправляє забруднення спіном
    # (для цього потрібні інші методи)

    return e_hf, e_total, e_mp2_corr

# Приклади
e_hf_li, e_mp2_li, _ = ump2_calculation("Li", spin=1)
e_hf_c, e_mp2_c, _ = ump2_calculation("C", spin=2)
e_hf_n, e_mp2_n, _ = ump2_calculation("N", spin=3)
e_hf_o, e_mp2_o, _ = ump2_calculation("O", spin=2)
```

6.2.5. Систематичне порівняння HF vs MP2

[code_6.py](#)

```
from pyscf import gto, scf, mp
```

```

import numpy as np
import matplotlib.pyplot as plt

def hf_vs_mp2_comparison(atoms_list, basis="cc-pvtz"):
    """
    Систематичне порівняння HF та MP2 для списку атомів
    """

    results = {"atoms": [], "e_hf": [], "e_mp2": [], "e_corr": [], "corr_percent": []}

    print(f"\nПорівняння HF та MP2 (базис: {basis})")
    print("=" * 80)
    print(
        f"{'Атом':6s} {'Спін':4s} {'E(HF), Ha':15s} {'E(MP2), Ha':15s} "
        f"{'E_corr, mHa':12s} {'% від HF':10s}"
    )
    print("-" * 80)

    for symbol, spin in atoms_list:
        mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

        # HF
        if spin == 0:
            mf = scf.RHF(mol)
            mp_method = mp.MP2
        else:
            mf = scf.UHF(mol)
            mp_method = mp.UMP2

        mf.verbose = 0
        mf.conv_tol = 1e-10
        e_hf = mf.kernel()

        # MP2
        mymp2 = mp_method(mf)
        mymp2.verbose = 0
        e_mp2_corr, _ = mymp2.kernel()
        e_mp2 = e_hf + e_mp2_corr

        # Зберігаємо результати
        results["atoms"].append(symbol)
        results["e_hf"].append(e_hf)
        results["e_mp2"].append(e_mp2)
        results["e_corr"].append(e_mp2_corr)

        corr_percent = abs(e_mp2_corr / e_hf) * 100
        results["corr_percent"].append(corr_percent)

        print(
            f"{symbol:6s} {spin:4d} {e_hf:15.8f} {e_mp2:15.8f} "
            f"{e_mp2_corr * 1000:12.4f} {corr_percent:10.4f}"
        )

    print("=" * 80)

    # Статистика
    avg_corr = np.mean(results["corr_percent"])
    print(f"\nСередній % кореляції: {avg_corr:.4f}%")

    # Графіки
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    x = np.arange(len(results["atoms"]))

```

```

# Абсолютні енергії
ax1.plot(
    x, results["e_hf"], "o-", label="HF", linewidth=2, markersize=8, color="blue"
)
ax1.plot(
    x, results["e_mp2"], "s-", label="MP2", linewidth=2, markersize=8, color="red"
)
ax1.set_xticks(x)
ax1.set_xticklabels(results["atoms"])
ax1.set_xlabel("Атом", fontsize=12)
ax1.set_ylabel("Енергія (Ha)", fontsize=12)
ax1.set_title("HF vs MP2 енергії", fontsize=14)
ax1.legend()
ax1.grid(True, alpha=0.3)

# Кореляційна енергія
ax2.bar(x, np.array(results["e_corr"]) * 1000, color="green", alpha=0.7)
ax2.set_xticks(x)
ax2.set_xticklabels(results["atoms"])
ax2.set_xlabel("Атом", fontsize=12)
ax2.set_ylabel("Кореляційна енергія (mHa)", fontsize=12)
ax2.set_title("MP2 кореляційна енергія", fontsize=14)
ax2.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig("hf_vs_mp2_comparison.pdf")
plt.show()

return results

# Атоми другого періоду
atoms_2nd = [
    ("He", 0),
    ("Li", 1),
    ("Be", 0),
    ("B", 1),
    ("C", 2),
    ("N", 3),
    ("O", 2),
    ("F", 1),
    ("Ne", 0),
]

results = hf_vs_mp2_comparison(atoms_2nd)

```

6.2.6. Залежність MP2 від базисного набору

MP2 сильно залежить від базису, особливо потрібні функції високих l :

[code_7.py](#)

```

from pyscf import gto, scf, mp
import matplotlib.pyplot as plt

def mp2_basis_convergence(symbol, spin):
    """
    Дослідження збіжності MP2 від базису
    """

    basis_sets = ["cc-pvdz", "cc-pvtz", "cc-pvqz", "cc-pv5z"]

```

```

print(f"\nЗбіжність MP2 енергії {symbol} від базису")
print("=" * 70)
print(
    f"{'Базис':12s} {'N_bas':6s} {'E(HF)':15s} {'E(MP2)':15s} {'E_corr, mHa':12s}"
)
print("-" * 70)

e_hf_list = []
e_mp2_list = []
e_corr_list = []
n_bas_list = []

for basis in basis_sets:
    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    # HF
    if spin == 0:
        mf = scf.RHF(mol)
        mp_method = mp.MP2
    else:
        mf = scf.UHF(mol)
        mp_method = mp.UMP2

    mf.verbose = 0
    mf.conv_tol = 1e-11
    e_hf = mf.kernel()

    # MP2
    mymp2 = mp_method(mf)
    mymp2.verbose = 0
    e_corr, _ = mymp2.kernel()
    e_mp2 = e_hf + e_corr

    n_bas = mol.nao_nr()

    e_hf_list.append(e_hf)
    e_mp2_list.append(e_mp2)
    e_corr_list.append(e_corr)
    n_bas_list.append(n_bas)

    print(
        f"{'basis':12s} {'n_bas':6d} {'e_hf':15.8f} {'e_mp2':15.8f} {'e_corr * 1000':12.4f}"
    )

print("=" * 70)

# Екстраполяція до CBS
#  $E(X) = E_{CBS} + A/X^3$ 
X = np.array([2, 3, 4, 5]) # D, T, Q, 5

# CBS для HF (швидша збіжність)
A_hf = (e_hf_list[-1] * 5**3 - e_hf_list[-2] * 4**3) / (5**3 - 4**3)
e_hf_cbs = e_hf_list[-1] - A_hf / 5**3

# CBS для кореляції
A_corr = (e_corr_list[-1] * 5**3 - e_corr_list[-2] * 4**3) / (5**3 - 4**3)
e_corr_cbs = e_corr_list[-1] - A_corr / 5**3

e_mp2_cbs = e_hf_cbs + e_corr_cbs

print(f"\nЕкстраполяція до CBS:")
print(f"E(HF/CBS): {e_hf_cbs:.10f} Ha")
print(f"E(MP2/CBS): {e_mp2_cbs:.10f} Ha")

```

```

print(f"E_corr(CBS): {e_corr_cbs * 1000:.4f} mHa")

# Графік
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Повна енергія
ax1.plot(n_bas_list, e_hf_list, "o-", label="HF", linewidth=2, markersize=8)
ax1.plot(n_bas_list, e_mp2_list, "s-", label="MP2", linewidth=2, markersize=8)
ax1.axhline(y=e_hf_cbs, color="blue", linestyle="--", alpha=0.5, label="HF CBS")
ax1.axhline(y=e_mp2_cbs, color="orange", linestyle="--", alpha=0.5, label="MP2 CBS")
ax1.set_xlabel("Кількість базисних функцій", fontsize=12)
ax1.set_ylabel("Енергія (Ha)", fontsize=12)
ax1.set_title(f"Збіжність енергії {symbol}", fontsize=14)
ax1.legend()
ax1.grid(True, alpha=0.3)

# Кореляційна енергія
ax2.plot(
    n_bas_list,
    np.array(e_corr_list) * 1000,
    "o-",
    linewidth=2,
    markersize=8,
    color="green",
)
ax2.axhline(
    y=e_corr_cbs * 1000, color="green", linestyle="--", alpha=0.5, label="CBS"
)
ax2.set_xlabel("Кількість базисних функцій", fontsize=12)
ax2.set_ylabel("Кореляційна енергія (mHa)", fontsize=12)
ax2.set_title(f"Збіжність кореляції {symbol}", fontsize=14)
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{symbol}_mp2_basis_convergence.pdf")
plt.show()

# Приклади
mp2_basis_convergence("Ne", spin=0)
mp2_basis_convergence("C", spin=2)

```

6.2.7. Переваги та недоліки MP2

Переваги:

- Відносно швидкий ($\mathcal{O}(N^5)$) порівняно з іншими post-HF
- Добре відновлює динамічну кореляцію
- Size-consistent (правильне масштабування для великих систем)
- Доступний для великих атомів/молекул
- Добрий базис для вищих методів (MP3, MP4)

Недоліки:

- Не виправляє забруднення спіном UHF
- Може розходитись для систем з малим gap HOMO-LUMO
- Погано працює для сильно корельованих систем
- Потребує великих базисів для точних результатів

- Не варіаційний (може давати енергію нижчу за точну)

6.2.8. Рекомендації по використанню MP2

Таблиця 6.2. Коли використовувати MP2

Добре для:	Погано для:
Замкнені оболонки	Системи з малим HOMO-LUMO gap
Якісні оцінки кореляції	Розрив зв'язків
Великі системи	Збуджені стани
Слабкі взаємодії	Перехідні стани
Відносні енергії	Діелектронні системи

6.3. Coupled Cluster методи (CCSD, CCSD(T))

6.3.1. Теоретичні основи Coupled Cluster

Coupled Cluster (CC) — один з найточніших методів для врахування електронної кореляції. На відміну від CI, CC є size-extensive (правильно масштабується для великих систем).

Хвильова функція у CC записується через експоненціальний оператор збудження:

$$|\Psi_{CC}\rangle = e^{\hat{T}}|\Phi_0\rangle \quad (6.7)$$

де $|\Phi_0\rangle$ — HF детермінант, а $\hat{T}$ — оператор кластерів:

$$\hat{T} = \hat{T}_1 + \hat{T}_2 + \hat{T}_3 + \dots \quad (6.8)$$

CCSD (Coupled Cluster Singles and Doubles) Враховує тільки $\hat{T}_1$ (одиначні збудження) та $\hat{T}_2$ (подвійні збудження):

$$\hat{T}_1 = \sum_i^{\text{occ}} \sum_a^{\text{virt}} t_i^a \hat{a}_a^\dagger \hat{a}_i \quad (6.9)$$

$$\hat{T}_2 = \frac{1}{4} \sum_{ij}^{\text{occ}} \sum_{ab}^{\text{virt}} t_{ij}^{ab} \hat{a}_a^\dagger \hat{a}_b^\dagger \hat{a}_j \hat{a}_i \quad (6.10)$$

CCSD(T) — «золотий стандарт» Додає пертурбативну корекцію для потрійних збуджень $\hat{T}_3$:

$$E_{CCSD(T)} = E_{CCSD} + E_{(T)} \quad (6.11)$$

де $E_{(T)}$ обчислюється за формулами четвертого порядку теорії збурень.

6.3.2. CCSD розрахунок атома Гелію

code_8.py

```

from pyscf import gto, scf, cc
import numpy as np

def ccsd_helium_detailed(basis="cc-pvqz"):
    """
    Детальний CCSD розрахунок для He
    """

    mol = gto.M(atom="He 0 0 0", basis=basis, spin=0, verbose=0)

    print(f"\nCCSD розрахунок атома Гелію (базис: {basis})")
    print("=" * 70)

    # Крок 1: HF
    print("\nКрок 1: Hartree-Fock")
    mf = scf.RHF(mol)
    mf.verbose = 4
    mf.conv_tol = 1e-12
    e_hf = mf.kernel()

    print(f"\nHF енергія: {e_hf:.12f} Ha")

    # Крок 2: CCSD
    print("\nКрок 2: CCSD розрахунок")
    mycc = cc.CCSD(mf)
    mycc.verbose = 4
    mycc.conv_tol = 1e-10

    e_ccsd_corr, t1, t2 = mycc.kernel()
    e_ccsd = e_hf + e_ccsd_corr

    print(f"\nCCSD кореляція: {e_ccsd_corr:.12f} Ha")
    print(f"CCSD повна енергія: {e_ccsd:.12f} Ha")

    # Аналіз амплітуд
    print(f"\nАналіз амплітуд:")
    print(f"  T1 амплітуди: {t1.shape}")
    print(f"  T2 амплітуди: {t2.shape}")
    print(f"  ||T1||_max = {np.max(np.abs(t1)):.6f}")
    print(f"  ||T2||_max = {np.max(np.abs(t2)):.6f}")
    print(f"  ||T1||_2 = {np.linalg.norm(t1):.6f}")
    print(f"  ||T2||_2 = {np.linalg.norm(t2):.6f}")

    # Для He T1 має бути дуже малим (симетрія)
    if np.max(np.abs(t1)) < 1e-6:
        print("  T1 ≈ 0 (як і очікувалось для замкнутої оболонки)")

    # Крок 3: CCSD(T)
    print("\nКрок 3: (T) корекція")
    e_t = mycc.ccsd_t()
    e_ccsdt = e_ccsd + e_t

    print(f"\n(T) корекція: {e_t:.12f} Ha")
    print(f"CCSD(T) повна енергія: {e_ccsdt:.12f} Ha")

    # Порівняння з експериментом
    e_exp = -2.903724377 # Ha (високоточне значення)

    print(f"\nПорівняння з експериментом:")
    print(f"Експеримент: {e_exp:.12f} Ha")

```

```

print(f"HF похибка:      {(e_hf - e_exp) * 1000:10.6f} мHa")
print(f"CCSD похибка:   {(e_ccsd - e_exp) * 1000:10.6f} мHa")
print(f"CCSD(T) похибка:{(e_ccsdt - e_exp) * 1000:10.6f} мHa")

# Розбиття кореляції
print(f"\nРозбиття кореляційної енергії:")
total_corr = e_ccsdt - e_hf
print(f"Повна кореляція: {total_corr * 1000:.6f} мHa (100%)")
print(
    f"CCSD внесок:      {e_ccsd_corr * 1000:.6f} мHa "
    f"({abs(e_ccsd_corr / total_corr) * 100:.2f}%)")
)
print(f"(T) внесок:      {e_t * 1000:.6f} мHa ({abs(e_t / total_corr) * 100:.2f}%)")

return e_hf, e_ccsd, e_ccsdt

```

```
e_hf_he, e_ccsd_he, e_ccsdt_he = ccscd_helium_detailed()
```

6.3.3. CCSD для відкритих оболонок (UCCSD)

code_9.py

```

from pyscf import gto, scf, cc

def uccsd_calculation(symbol, spin, basis="cc-pvtz"):
    """
    UCCSD розрахунок для відкритої оболонки
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    print(f"\nUCCSD розрахунок {symbol} (2S={spin}, базис: {basis})")
    print("=" * 70)

    # UHF
    mf = scf.UHF(mol)
    mf.verbose = 0
    mf.conv_tol = 1e-11
    e_hf = mf.kernel()

    print(f"UHF енергія: {e_hf:.10f} Ha")

    # Забруднення спіном
    s2_hf = mf.spin_square()[0]
    expected_s2 = spin * (spin + 2) / 4
    spin_cont = s2_hf - expected_s2
    print(f"<S²> (UHF): {s2_hf:.6f} (очікується {expected_s2:.6f})")
    print(f"Забруднення спіном: {spin_cont:.6f}")

    # UCCSD
    print("\nUCCSD розрахунок...")
    mycc = cc.UCCSD(mf)
    mycc.verbose = 4
    mycc.conv_tol = 1e-9

    e_ccsd_corr, t1, t2 = mycc.kernel()
    e_ccsd = e_hf + e_ccsd_corr

    print(f"\nUCCSD кореляція: {e_ccsd_corr:.10f} Ha")
    print(f"UCCSD повна енергія: {e_ccsd:.10f} Ha")

```

```

# CCSD виправляє забруднення спіном
# (але  $\langle S^2 \rangle$  все ще не є точним оператором)

# Аналіз T1 діагностики
t1_alpha, t1_beta = t1
t1_diag = np.linalg.norm(t1_alpha) / np.sqrt(mol.nelec[0])

print(f"\nT1 діагностика: {t1_diag:.6f}")
if t1_diag < 0.02:
    print(" T1 < 0.02: одноконфігураційний характер")
elif t1_diag < 0.05:
    print(" 0.02 < T1 < 0.05: слабка багатоконфігураційність")
else:
    print(" T1 > 0.05: сильна багатоконфігураційність!")
    print(" Можливо потрібен CASSCF/MRCI")

# UCCSD(T)
if mol.nelectron ≤ 10: # (T) дуже повільно
    print("\n(T) корекція...")
    e_t = mycc.ccsd_t()
    e_ccsdt = e_ccsd + e_t

    print(f"(T) корекція: {e_t:.10f} Ha")
    print(f"UCCSD(T) повна енергія: {e_ccsdt:.10f} Ha")

    return e_hf, e_ccsd, e_ccsdt
else:
    return e_hf, e_ccsd, None

# Приклади
uccsd_calculation("Li", spin=1, basis="cc-pvdz")
uccsd_calculation("C", spin=2, basis="cc-pvdz")
uccsd_calculation("N", spin=3, basis="cc-pvdz")

```

6.3.4. Порівняння ієрархії методів

code_10.py

```

from pyscf import gto, scf, mp, cc, fci
import matplotlib.pyplot as plt

def method_hierarchy_comparison(symbol, spin, basis="cc-pvtz"):
    """
    Порівняння HF → MP2 → CCSD → CCSD(T) → FCI
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    # Перевірка розміру (FCI експоненційно зростає)
    if mol.nelectron > 10:
        print(f"Атом {symbol} занадто великий для FCI")
        return

    print(f"\nІєрархія методів для {symbol} (базис: {basis})")
    print("=" * 80)

    methods = {}

    # HF
    print("HF розрахунок...")
    if spin == 0:

```

```

mf = scf.RHF(mol)
else:
    mf = scf.UHF(mol)

mf.verbose = 0
mf.conv_tol = 1e-11
e_hf = mf.kernel()
methods["HF"] = e_hf

# MP2
print("MP2 розрахунок...")
if spin == 0:
    mymp2 = mp.MP2(mf)
else:
    mymp2 = mp.UMP2(mf)

mymp2.verbose = 0
e_mp2_corr, _ = mymp2.kernel()
methods["MP2"] = e_hf + e_mp2_corr

# CCSD
print("CCSD розрахунок...")
if spin == 0:
    mycc = cc.CCSD(mf)
else:
    mycc = cc.UCCSD(mf)

mycc.verbose = 0
mycc.conv_tol = 1e-10
e_ccsd_corr, _, _ = mycc.kernel()
methods["CCSD"] = e_hf + e_ccsd_corr

# CCSD(T)
print("CCSD(T) розрахунок...")
e_t = mycc.ccsd_t()
methods["CCSD(T)"] = methods["CCSD"] + e_t

# FCI (точний у даному базисі)
print("FCI розрахунок...")
if spin == 0:
    myfci = fci.FCI(mf)
else:
    myfci = fci.FCI(mf)

myfci.verbose = 0
e_fci = myfci.kernel()[0]
methods["FCI"] = e_fci

# Результати
print("\n" + "=" * 80)
print(
    f"{'Метод':12s} {'Енергія, Ha':18s} {'Відносно HF, mHa':20s} "
    f"{'% FCI кореляції':18s}"
)
print("-" * 80)

e_corr_fci = e_fci - e_hf

for method in ["HF", "MP2", "CCSD", "CCSD(T)", "FCI"]:
    e = methods[method]
    rel = (e - e_hf) * 1000

    if method == "HF":
        percent = 0.0

```

```

else:
    percent = abs((e - e_hf) / e_corr_fci) * 100

    print(f"{method:12s} {e:18.10f} {rel:20.6f} {percent:18.2f}")

print("=" * 80)

# Графік
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Абсолютні енергії
method_names = list(methods.keys())
energies = [methods[m] for m in method_names]
colors = ["blue", "orange", "green", "red", "purple"]

x = np.arange(len(method_names))
bars = ax1.bar(x, energies, color=colors, alpha=0.7)
ax1.set_xticks(x)
ax1.set_xticklabels(method_names)
ax1.set_ylabel("Енергія (Ha)", fontsize=12)
ax1.set_title(f"Повні енергії {symbol}", fontsize=14)
ax1.grid(True, alpha=0.3, axis="y")

# Значення на стовпчиках
for bar, e in zip(bars, energies):
    height = bar.get_height()
    ax1.text(
        bar.get_x() + bar.get_width() / 2.0,
        height,
        f"{e:.6f}",
        ha="center",
        va="bottom",
        fontsize=9,
    )

# Кореляційна енергія (відносно HF)
corr_energies = [(methods[m] - e_hf) * 1000 for m in method_names[1:]]

ax2.bar(range(len(corr_energies)), corr_energies, color=colors[1:], alpha=0.7)
ax2.set_xticks(range(len(corr_energies)))
ax2.set_xticklabels(method_names[1:])
ax2.set_ylabel("Кореляційна енергія (mHa)", fontsize=12)
ax2.set_title(f"Кореляційні енергії {symbol}", fontsize=14)
ax2.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig(f"{symbol}_method_hierarchy.pdf")
plt.show()

return methods

# Приклади (тільки для малих атомів через FCI)
methods_he = method_hierarchy_comparison("He", spin=0, basis="cc-pvdz")
methods_be = method_hierarchy_comparison("Be", spin=0, basis="cc-pvdz")
methods_c = method_hierarchy_comparison("C", spin=2, basis="cc-pvdz")

```

6.3.5. T1 діагностика та багатоконфігураційний характер

T1 діагностика — індикатор того, наскільки система є одноконфігураційною:

$$T_1 = \frac{\|t_1\|}{\sqrt{N_{\text{elec}}}} \quad (6.12)$$

code_11.py

```

from pyscf import gto, scf, cc
import numpy as np

def t1_diagnostic_survey(atoms_list, basis="cc-pvdz"):
    """
    T1 діагностика для серії атомів
    """

    print(f"\nT1 діагностика (базис: {basis})")
    print("=" * 70)
    print(f"{'Атом':6s} {'Спін':4s} {'T1 діаг.':12s} {'Оцінка':30s}")
    print("-" * 70)

    for symbol, spin in atoms_list:
        mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

        # Пропускаємо дуже великі атоми
        if mol.nelectron > 18:
            print(f"{'symbol':6s} {'spin':4d} (пропущено - занадто великий)")
            continue

        # HF
        if spin == 0:
            mf = scf.RHF(mol)
            mycc = cc.CCSD(mf)
        else:
            mf = scf.UHF(mol)
            mycc = cc.UCCSD(mf)

        mf.verbose = 0
        mf.conv_tol = 1e-10
        mf.kernel()

        # CCSD
        mycc.verbose = 0
        mycc.conv_tol = 1e-8

        try:
            e_corr, t1, t2 = mycc.kernel()

            # Обчислення T1 діагностики
            if spin == 0:
                t1_norm = np.linalg.norm(t1)
                n_elec = mol.nelectron
            else:
                t1_alpha, t1_beta = t1
                t1_norm = np.linalg.norm(t1_alpha)
                n_elec = mol.nelec[0]

            t1_diag = t1_norm / np.sqrt(n_elec)

            # Інтерпретація
            if t1_diag < 0.02:
                assessment = "Добре (одноконфігураційний)"
            elif t1_diag < 0.05:
                assessment = "Прийнятно (слабка МК)"
            else:
                assessment = "Погано (потрібен CASSCF)"

```

```

        print(f"{symbol:6s} {spin:4d} {t1_diag:12.6f} {assessment:30s}")

    except:
        print(f"{symbol:6s} {spin:4d} {'ПОМИЛКА':12s} {'Не конвергувало':30s}")

print("=" * 70)
print("\nЛегенда:")
print("  T1 < 0.02: одноконфігураційний (CCSD надійний)")
print("  0.02 < T1 < 0.05: слабка багатоконфігураційність")
print("  T1 > 0.05: сильна МК (краще CASSCF/MRCI)")

# Тестування різних атомів
atoms_test = [
    ("He", 0),
    ("Be", 0),
    ("C", 2),
    ("N", 3),
    ("O", 2),
    ("F", 1),
    ("Ne", 0),
    ("Mg", 0),
]

t1_diagnostic_survey(atoms_test)

```

6.3.6. Обчислювальна складність та масштабування

Таблиця 6.3. Порівняння обчислювальної складності методів

Метод	Масштабування	Пам'ять	Disk
HF	$\mathcal{O}(N^4)$	$\mathcal{O}(N^2)$	Мало
MP2	$\mathcal{O}(N^5)$	$\mathcal{O}(N^4)$	$\mathcal{O}(N^4)$
CCSD	$\mathcal{O}(N^6)$	$\mathcal{O}(N^4)$	$\mathcal{O}(N^4)$
CCSD(T)	$\mathcal{O}(N^7)$	$\mathcal{O}(N^4)$	$\mathcal{O}(N^4)$
CCSDT	$\mathcal{O}(N^8)$	$\mathcal{O}(N^6)$	$\mathcal{O}(N^6)$
FCI	$\mathcal{O}(e^N)$	$\mathcal{O}(e^N)$	$\mathcal{O}(e^N)$

де N — характерний розмір системи (базисних функцій або електронів).

6.4. Configuration Interaction (CI) та Full CI

6.4.1. Основи Configuration Interaction

Configuration Interaction (CI) — варіаційний метод, де хвильова функція представлена як лінійна комбінація детермінантів Слейтера:

$$|\Psi_{CI}\rangle = c_0|\Phi_0\rangle + \sum_i \sum_a^{\text{occ virt}} c_i^a |\Phi_i^a\rangle + \sum_{i<j} \sum_{a<b} c_{ij}^{ab} |\Phi_{ij}^{ab}\rangle + \dots \quad (6.13)$$

де:

- $|\Phi_0\rangle$ — HF детермінант (основна конфігурація)
- $|\Phi_i^a\rangle$ — одиночно збуджені детермінанти (S)
- $|\Phi_{ij}^{ab}\rangle$ — подвійно збуджені детермінанти (D)
- $|\Phi_{ijk}^{abc}\rangle$ — потрійно збуджені детермінанти (T)
- і т.д.

Варіанти CI методу:

- **CIS** (CI Singles) — тільки одиночні збудження (для збуджених станів)
- **CISD** (CI Singles and Doubles) — S + D
- **CISDT** — S + D + T
- **Full CI** — всі можливі збудження (точний у межах базису)

6.4.2. Full CI — точний розв'язок

Full CI включає всі можливі детермінанти і дає точну хвильову функцію та енергію в межах обраного базису:

$$E_{FCI} = \min_{\{c_i\}} \langle \Psi_{FCI} | \hat{H} | \Psi_{FCI} \rangle \quad (6.14)$$

Проблема: Кількість детермінантів зростає як:

$$N_{det} = \binom{N_{orb}}{N_\alpha} \times \binom{N_{orb}}{N_\beta} \quad (6.15)$$

Для 10 електронів у 20 орбіталях: $N_{det} \approx 10^{10}$ детермінантів!

FullCI.py

```
from pyscf import gto, scf, fci, ci
import numpy as np

def fci_calculation_detailed(symbol, spin, basis='cc-pvdz'):
    """
    Детальний Full CI розрахунок
    """

    mol = gto.M(
        atom=f'{symbol} 0 0 0',
        basis=basis,
        spin=spin,
        verbose=0
    )

    # Перевірка розміру
    n_det_approx = np.math.factorial(mol.nao_nr()) / (
        np.math.factorial(mol.nelec[0]) *
        np.math.factorial(mol.nao_nr() - mol.nelec[0])
    )
```

```

)

if n_det_approx > 1e8:
    print(f'Система занадто велика для FCI!')
    print(f'Приблизна кількість детермінантів: {n_det_approx:.2e}')
    return

print(f'\nFull CI розрахунок {symbol} (базис: {basis})')
print('='*70)

print(f'Розмір задачі:')
print(f'  Електронів: {mol.nelectron}')
print(f'  Базисних функцій: {mol.nao_nr()}')
print(f'   $\alpha$ -електронів: {mol.nelec[0]}')
print(f'   $\beta$ -електронів: {mol.nelec[1]}')
print(f'  Приблизна кількість детермінантів: {n_det_approx:.2e}')

# HF розрахунок
print('\nHF розрахунок...')
if spin == 0:
    mf = scf.RHF(mol)
else:
    mf = scf.UHF(mol)

mf.verbose = 0
mf.conv_tol = 1e-12
e_hf = mf.kernel()

print(f'HF енергія: {e_hf:.12f} Ha')

# FCI розрахунок
print('\nFull CI розрахунок...')
myfci = fci.FCI(mf)
myfci.verbose = 4

e_fci, ci_vec = myfci.kernel()

print(f'\nFCI енергія: {e_fci:.12f} Ha')
print(f'Кореляційна енергія: {(e_fci - e_hf)*1000:.8f} мHa')

# Аналіз хвильової функції
print(f'\nАналіз хвильової функції:')

# Вага HF конфігурації
if spin == 0:
    # Для RHF: HF конфігурація є першим елементом
    c0_squared = ci_vec[0]**2
    print(f'Вага HF конфігурації: {c0_squared:.6f} '
          f'({c0_squared*100:.2f}%)')

# Ентропія CI вектора (міра багатоконфігураційності)
ci_vec_flat = ci_vec.ravel()
ci_squared = ci_vec_flat**2
ci_squared = ci_squared[ci_squared > 1e-10] # відкидаємо дуже малі

entropy = -np.sum(ci_squared * np.log(ci_squared))
print(f'Ентропія CI вектора: {entropy:.6f}')

if entropy < 0.1:
    print(' → Сильно одноконфігураційна система')
elif entropy < 1.0:
    print(' → Помірна багатоконфігураційність')
else:
    print(' → Сильна багатоконфігураційність')

```

```

# Найважливіші конфігурації
n_important = np.sum(ci_squared > 0.01)
print(f'Конфігурацій з вагою > 1%: {n_important}')

return e_hf, e_fci

# Приклади (тільки малі системи!)
fci_calculation_detailed('He', spin=0, basis='cc-pvdz')
fci_calculation_detailed('Be', spin=0, basis='cc-pvdz')
fci_calculation_detailed('Li', spin=1, basis='6-31g')

```

6.4.3. CISD розрахунки

CISD — практичний варіант CI, але не є size-extensive:

[CISDcalc.py](#)

```

from pyscf import gto, scf, ci

def cisd_calculation(symbol, spin, basis='cc-pvtz'):
    """
    CISD розрахунок
    """

    mol = gto.M(
        atom=f'{symbol} 0 0 0',
        basis=basis,
        spin=spin,
        verbose=0
    )

    print(f'\nCISD розрахунок {symbol} (базис: {basis})')
    print('='*70)

    # HF
    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
    mf.conv_tol = 1e-11
    e_hf = mf.kernel()

    print(f'HF енергія: {e_hf:.10f} Ha')

    # CISD
    print('\nCISD розрахунок...')
    if spin == 0:
        myci = ci.CISD(mf)
    else:
        myci = ci.UCISD(mf)

    myci.verbose = 4
    e_cisd, civec = myci.kernel()

    print(f'\nCISD енергія: {e_cisd:.10f} Ha')
    print(f'CISD кореляція: {(e_cisd - e_hf)*1000:.6f} mHa')

    return e_hf, e_cisd

cisd_calculation('C', spin=2)

```

```
cisd_calculation('Ne', spin=0)
```

6.4.4. Порівняння CI та CC методів

CIvsCD.py

```
from pyscf import gto, scf, ci, cc, fci
import matplotlib.pyplot as plt

def compare_ci_cc(symbol='Be', spin=0, basis='cc-pvdz'):
    """
    Порівняння CI та CC методів
    """

    mol = gto.M(
        atom=f'{symbol} 0 0 0',
        basis=basis,
        spin=spin,
        verbose=0
    )

    # Перевірка розміру для FCI
    if mol.nelectron > 10:
        print('Атом занадто великий для FCI')
        include_fci = False
    else:
        include_fci = True

    print(f'\nПорівняння методів для {symbol} (базис: {basis})')
    print('='*70)

    results = {}

    # HF
    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
    mf.conv_tol = 1e-11
    e_hf = mf.kernel()
    results['HF'] = e_hf

    print(f'HF: {e_hf:.10f} Ha')

    # CISD
    print('CISD...')
    if spin == 0:
        myci = ci.CISD(mf)
    else:
        myci = ci.UCISD(mf)

    myci.verbose = 0
    e_cisd, _ = myci.kernel()
    results['CISD'] = e_cisd

    print(f'CISD: {e_cisd:.10f} Ha')

    # CCSD
    print('CCSD...')
    if spin == 0:
        mycc = cc.CCSD(mf)
```

```

else:
    mycc = cc.UCCSD(mf)

mycc.verbose = 0
e_ccsd_corr, _, _ = mycc.kernel()
e_ccsd = e_hf + e_ccsd_corr
results['CCSD'] = e_ccsd

print(f'CCSD: {e_ccsd:.10f} Ha')

# CCSD(T)
print('CCSD(T) ... ')
e_t = mycc.ccsd_t()
e_ccsdt = e_ccsd + e_t
results['CCSD(T)'] = e_ccsdt

print(f'CCSD(T): {e_ccsdt:.10f} Ha')

# FCI
if include_fci:
    print('FCI... ')
    myfci = fci.FCI(mf)
    myfci.verbose = 0
    e_fci, _ = myfci.kernel()
    results['FCI'] = e_fci

    print(f'FCI: {e_fci:.10f} Ha')

# Порівняння
print('\n' + '='*70)
print(f'{"Метод":12s} {"Енергія, Ha":18s} {"Кореляція, mHa":18s}')
print('-'*70)

for method in results.keys():
    e = results[method]
    corr = (e - e_hf) * 1000
    print(f'{"method":12s} {e:18.10f} {corr:18.6f}')

print('='*70)

# Аналіз
if include_fci:
    e_corr_fci = results['FCI'] - e_hf

    print('\nВідсоток відновленої кореляції (відносно FCI):')
    for method in ['CISD', 'CCSD', 'CCSD(T)']:
        e_corr = results[method] - e_hf
        percent = abs(e_corr / e_corr_fci) * 100
        print(f'{"method":12s}: {percent:6.2f}%')

# Графік
fig, ax = plt.subplots(figsize=(10, 6))

methods_list = list(results.keys())
energies = [results[m] for m in methods_list]
corr_energies = [(results[m] - e_hf) * 1000
                  for m in methods_list]

x = np.arange(len(methods_list))
colors = ['blue', 'orange', 'green', 'red', 'purple']

bars = ax.bar(x, corr_energies, color=colors[:len(methods_list)],
              alpha=0.7)
ax.set_xticks(x)

```

```

ax.set_xticklabels(methods_list)
ax.set_ylabel('Кореляційна енергія (мHa)', fontsize=12)
ax.set_title(f'Порівняння методів для {symbol}', fontsize=14)
ax.grid(True, alpha=0.3, axis='y')

# Додавання значень
for bar, e in zip(bars, corr_energies):
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height,
            f'{e:.4f}', ha='center', va='bottom', fontsize=10)

plt.tight_layout()
plt.savefig(f'{symbol}_ci_cc_comparison.pdf')
plt.show()

return results

```

Приклади

```

results_be = compare_ci_cc('Be', spin=0, basis='cc-pvdz')
results_c = compare_ci_cc('C', spin=2, basis='6-31g')

```

6.4.5. Size-extensivity проблема CI

Важлива відмінність між CI та CC:

$$E_{CI}(2A) \neq 2 \times E_{CI}(A) \quad (6.16)$$

$$E_{CC}(2A) = 2 \times E_{CC}(A) \quad (6.17)$$

[SizeExtensivityExample.py](#)

```

from pyscf import gto, scf, ci, cc

def demonstrate_size_extensivity():
    """
    Демонстрація size-extensivity
    """

    print('\nДемонстрація size-extensivity')
    print('='*70)

    # Один атом He
    mol_1he = gto.M(atom='He 0 0 0', basis='sto-3g', verbose=0)
    mf_1he = scf.RHF(mol_1he)
    mf_1he.verbose = 0
    e_hf_1he = mf_1he.kernel()

    # CISD для 1 He
    myci_1he = ci.CISD(mf_1he)
    myci_1he.verbose = 0
    e_cisd_1he, _ = myci_1he.kernel()

    # CCSD для 1 He
    mycc_1he = cc.CCSD(mf_1he)
    mycc_1he.verbose = 0
    e_ccsd_corr_1he, _, _ = mycc_1he.kernel()
    e_ccsd_1he = e_hf_1he + e_ccsd_corr_1he

    print(f'Один атом He:')
    print(f'  HF:   {e_hf_1he:.10f} Ha')

```

```

print(f'  CISD: {e_cisd_1he:.10f} Ha')
print(f'  CCSD: {e_ccsd_1he:.10f} Ha')

# Два атоми He (далеко один від одного)
mol_2he = gto.M(
    atom='He 0 0 0; He 0 0 100', # 100 Bohr відстань
    basis='sto-3g',
    verbose=0
)
mf_2he = scf.RHF(mol_2he)
mf_2he.verbose = 0
e_hf_2he = mf_2he.kernel()

# CISD для 2 He
myci_2he = ci.CISD(mf_2he)
myci_2he.verbose = 0
e_cisd_2he, _ = myci_2he.kernel()

# CCSD для 2 He
mycc_2he = cc.CCSD(mf_2he)
mycc_2he.verbose = 0
e_ccsd_corr_2he, _, _ = mycc_2he.kernel()
e_ccsd_2he = e_hf_2he + e_ccsd_corr_2he

print(f'\nДва атоми He (відокремлені):')
print(f'  HF:   {e_hf_2he:.10f} Ha')
print(f'  CISD: {e_cisd_2he:.10f} Ha')
print(f'  CCSD: {e_ccsd_2he:.10f} Ha')

# Перевірка size-extensivity
print(f'\nПеревірка size-extensivity:')
print(f'  2×E(1 He):')
print(f'    HF:   {2*e_hf_1he:.10f} Ha')
print(f'    CISD: {2*e_cisd_1he:.10f} Ha')
print(f'    CCSD: {2*e_ccsd_1he:.10f} Ha')

print(f'\n Похибка E(2 He) - 2×E(1 He):')
err_hf = (e_hf_2he - 2*e_hf_1he) * 1000
err_cisd = (e_cisd_2he - 2*e_cisd_1he) * 1000
err_ccsd = (e_ccsd_2he - 2*e_ccsd_1he) * 1000

print(f'    HF:   {err_hf:.6f} мHa (має бути ≈0)')
print(f'    CISD: {err_cisd:.6f} мHa (не size-extensive!)')
print(f'    CCSD: {err_ccsd:.6f} мHa (має бути ≈0)')

print(f'\nВисновок: CCSD є size-extensive, CISD - ні')

demonstrate_size_extensivity()

```

6.4.6. Коли використовувати CI vs CC

Таблиця 6.4. Порівняння CI та CC методів

Критерій	CI	CC
Варіаційність	Так	Ні
Size-extensivity	Ні (окрім FCI)	Так
Точність	CISD < CCSD	CCSD(T) $\boxtimes$ FCI
Збуджені стани	EOM-CI, CIS	EOM-CC
Швидкість	CISD швидше CCSD	Повільніше
Стабільність	Варіаційна межа	Може розходитись
Багатоконфігураційність	Природно (FCI)	Потребує MRCC

Рекомендації:

- Використовуйте **FCI** для малих систем як еталон
- Використовуйте **CCSD(T)** для точних розрахунків одноконфігураційних систем
- Використовуйте **CISD** для швидких оцінок кореляції
- Для багатоконфігураційних систем використовуйте CASSCF/CASPT2

6.5. CASSCF – багатоконфігураційний метод

6.5.1. Ідея Complete Active Space Self-Consistent Field

CASSCF (Complete Active Space SCF) — багатоконфігураційний метод, який поєднує:

- **Активний простір** — набір орбіталей, де проводиться Full CI
- **SCF оптимізацію** — орбіталі та CI коефіцієнти оптимізуються одночасно

Орбіталі розділяються на три групи:

1. **Core (остов)** — завжди подвійно заповнені, не беруть участь у кореляції
2. **Active (активні)** — електрони розподілені по всіх можливих способах (Full CI)
3. **Virtual (віртуальні)** — завжди порожні

$$|\Psi_{CASSCF}\rangle = \sum_I c_I |\Phi_I^{\text{active}}\rangle \otimes |\text{core}\rangle \quad (6.18)$$

6.5.2. Позначення CASSCF(n,m)

CASSCF(n,m) означає:

- n — кількість електронів в активному просторі
- m — кількість орбіталей в активному просторі

Приклади:

- **Be**: CASSCF(4,8) — 4 валентні електрони у 8 орбіталях (2s, 2p, 3s, 3p)
- **C**: CASSCF(4,4) — 4 валентні електрони у 4 орбіталях (2s, 2p)
- **Cr**: CASSCF(6,5) — 6 електронів у 5 d-орбіталях

6.5.3. CASSCF розрахунок атома Берилію

Берилій має близькі за енергією конфігурації $2s^2$ та $2p^2$, що робить його класичним прикладом для CASSCF:

CASSCFBe.py

```
import numpy as np
from pyscf import gto, scf, mcscf

def casscf_beryllium_detailed(basis="cc-pvtz"):
    """
    Детальний CASSCF розрахунок для Be
    """

    mol = gto.M(atom="Be 0 0 0", basis=basis, spin=0, symmetry=True, verbose=0)

    print(f"\nCAS розрахунок атома Берилію (базис: {basis})")
    print("=" * 70)

    # Крок 1: HF розрахунок
    print("Крок 1: RHF розрахунок")
    mf = scf.RHF(mol)
    mf.verbose = 4
    mf.conv_tol = 1e-12
    e_hf = mf.kernel()

    print(f"\nRHF енергія: {e_hf:.10f} Ha")
    print(f"Електронна конфігурація HF: 1s2 2s2")

    # Крок 2: Вибір активного простору
    print("\nКрок 2: Визначення активного простору")
    print("Активний простір: CASSCF(4,8)")
    print("  4 електрони: валентні 2s2 2p0")
    print("  8 орбіталей: 2s, 2p (3 орб), 3s, 3p (3 орб)")

    # CASSCF(4,8): 4 електрони у 8 орбіталях
    ncas = 8 # активних орбіталей
    nelecas = 4 # активних електронів

    mc = mcscf.CASSCF(mf, ncas, nelecas)
    mc.verbose = 4
    mc.conv_tol = 1e-10

    # Крок 3: CASSCF розрахунок
    print("\nКрок 3: CASSCF оптимізація")
    e_casscf = mc.kernel()[0]

    print(f"\nCASSCF(4,8) енергія: {e_casscf:.10f} Ha")
    print(f"Статична кореляція: {(e_casscf - e_hf) * 1000:.6f} mHa")

    # Аналіз хвильової функції
    print("\nКрок 4: Аналіз хвильової функції")

    # Природні орбіталі та заселеності
    natocc, natorb = mc.cas_natorb()

    print("\nЗаселеності природних орбіталей (активний простір):")
    for i, occ in enumerate(natocc):
        print(f"  Орбіталь {i + 1}: {occ:.6f}")

    # Аналіз конфігурацій
    print("\nАналіз CI коефіцієнтів:")
```

```

# Для детального аналізу можна використати fcisolver
ci = mc.ci

# Вага основної конфігурації
# (потребує додаткового аналізу CI вектора)

# Entanglement entropy
s_entropy = 0
for occ in natocc:
    if occ > 1e-10 and occ < 2 - 1e-10:
        s_entropy += -occ / 2 * np.log(occ / 2) - (2 - occ) / 2 * np.log(
            (2 - occ) / 2
        )

print(f"\nОднопартичкова ентропія: {s_entropy:.6f}")
if s_entropy < 0.5:
    print(" → Слабка статична кореляція")
elif s_entropy < 1.5:
    print(" → Помірна статична кореляція")
else:
    print(" → Сильна статична кореляція")

return e_hf, e_casscf, natocc

e_hf_be, e_cas_be, occ_be = casscf_beryllium_detailed()

```

6.5.4. CASSCF для перехідних металів

Для перехідних металів d-орбіталі часто близькі за енергією, потребуючи багатоконфігураційного підходу:

CASSCFTransitionMetals.py

```

from pyscf import gto, scf, mcscf
import numpy as np

def casscf_transition_metal(symbol, spin, nelecas, ncas, basis="def2-svp"):
    """
    CASSCF для перехідного металу
    """

    mol = gto.M(
        atom=f"{symbol} 0 0 0",
        basis=basis,
        spin=spin,
        symmetry=False, # Часто простіше без симетрії
        verbose=0,
    )

    print(f"\nCASSCF розрахунок {symbol} (2S={spin})")
    print(f"Активний простір: CASSCF({nelecas},{ncas})")
    print("=" * 70)

    # UHF
    mf = scf.UHF(mol)
    mf.verbose = 0
    mf.conv_tol = 1e-10
    e_hf = mf.kernel()

```

```

print(f"UHF енергія: {e_hf:.10f} Ha")

s2_hf = mf.spin_square()[0]
expected_s2 = spin * (spin + 2) / 4
print(f"<S²> (UHF): {s2_hf:.6f} (очікується {expected_s2:.6f})")

# CASSCF
print(f"\nCASSCF({nelecas},{ncas}) розрахунок...")
mc = mcscf.CASSCF(mf, ncas, nelecas)
mc.verbose = 4
mc.conv_tol = 1e-8
mc.max_cycle_macro = 100

# Для важких випадків
mc.fcisolver.max_cycle = 100
mc.fcisolver.conv_tol = 1e-8

try:
    e_casscf = mc.kernel()[0]

    print(f"\nCASSCF енергія: {e_casscf:.10f} Ha")
    print(f"Статична кореляція: {(e_casscf - e_hf) * 1000:.6f} мHa")

    # Природні орбіталі
    natocc, natorb = mc.cas_natorb()

    print("\nЗаселеності природних орбіталей (активний простір):")
    for i, occ in enumerate(natocc):
        if occ > 0.01: # тільки значні заселеності
            print(f" Орбіталь {i + 1}: {occ:.4f}")

    # Оцінка багатоконфігураційності
    # Strongly occupied (> 1.98) та weakly occupied (< 0.02)
    n_strong = np.sum(natocc > 1.98)
    n_weak = np.sum(natocc < 0.02)
    n_fractional = ncas - n_strong - n_weak

    print(f"\nАналіз заселеностей:")
    print(f" Сильно заповнені (>1.98): {n_strong}")
    print(f" Дробові (0.02-1.98): {n_fractional}")
    print(f" Порожні (<0.02): {n_weak}")

    if n_fractional > ncas / 2:
        print(" → Сильна багатоконфігураційність!")

    return e_hf, e_casscf, natocc

except Exception as e:
    print(f"\nПомилка: {str(e)}")
    print("Спробуйте інший активний простір або початкове наближення")
    return e_hf, None, None

# Приклади

# Cr: [Ar] 3d⁵ 4s¹
# CASSCF(6,6): 6 електронів у 6 орбіталях (5×3d + 1×4s)
e_hf_cr, e_cas_cr, occ_cr = casscf_transition_metal(
    "Cr", spin=6, nelecas=6, ncas=6, basis="def2-svp"
)

# Fe: [Ar] 3d⁶ 4s²
# CASSCF(8,5): 8 електронів у 5 d-орбіталях
e_hf_fe, e_cas_fe, occ_fe = casscf_transition_metal(

```

```
"Fe", spin=4, nelecas=8, ncas=5, basis="def2-svp")
```

6.5.5. Вибір активного простору

Правильний вибір активного простору критичний для CASSCF:

Таблиця 6.5. Рекомендації по вибору активного простору

Система	Активний простір	Коментар
Be	(4,8): 2s,2p,3s,3p	Враховує $2s^2 \leftrightarrow 2p^2$
C	(4,4): 2s,2p	Мінімальний валентний
O	(6,6): 2s,2p + корел.	З кореляційними орбіталями
3d метали	(n,5): 3d	Тільки d-орбіталі
3d метали	(n+2,6): 3d,4s	d + s орбіталі
Лантаноїди	(n,7): 4f	f-орбіталі

Принципи вибору:

1. Включити орбіталі, близькі за енергією
2. Включити орбіталі з значною хімічною активністю
3. Більший простір → точніше, але експоненційно повільніше
4. Перевірити заселеності природних орбіталей
5. Якщо заселеності ≈ 2 або ≈ 0 , можна виключити з активного простору

6.5.6. State-averaged CASSCF

Для розрахунку декількох станів одночасно (наприклад, різних мультиплетів):

[StateAvaragedCASSCF.py](#)

```
from pyscf import gto, scf, mcscf

def sa_casscf_carbon():
    """
    State-averaged CASSCF для різних станів Карбону
    """

    mol = gto.M(
        atom="C 0 0 0",
        basis="cc-pvtz",
        spin=2, # Для триплету
        symmetry=True,
        verbose=0,
    )

    print("\nState-Averaged CASSCF для C")
    print("Стани: 3P (основний) та 1D (збуджений)")
    print("=" * 70)
```

```

# HF
mf = scf.UHF(mol)
mf.verbose = 0
e_hf = mf.kernel()

print(f"UHF енергія: {e_hf:.10f} Ha")

# SA-CASSCF(4,4): усереднення по декількох станах
mc = mcscf.CASSCF(mf, 4, 4)
mc.verbose = 4

# State-averaging для 2 станів з рівними вагами
mc.state_average_([0.5, 0.5])

# Або для різних симетрій:
# mc = mc.state_average_mix([
#     mcscf.state_average_(mc, [0.5, 0.5]), # стани симетрії 1
# ])

mc.conv_tol = 1e-9
e_sa = mc.kernel()[0]

print(f"\nSA-CASSCF енергія (усереднена): {e_sa:.10f} Ha")

# Енергії окремих станів
print("\nЕнергії окремих станів:")
for i, e_state in enumerate(mc.e_states):
    print(f"Стан {i + 1}: {e_state:.10f} Ha ({e_state * 27.211386:.4f} eV)")

# Різниця енергій
if len(mc.e_states) > 1:
    delta_e = (mc.e_states[1] - mc.e_states[0]) * 27.211386
    print(f"\nРізниця енергій збудження: {delta_e:.4f} eV")

sa_casscf_carbon()

```

6.5.7. CASPT2 — динамічна кореляція поверх CASSCF

CASSCF враховує тільки статичну кореляцію. Для повної точності потрібно додати динамічну кореляцію через CASPT2:

CASPT2.py

```

from pyscf import gto, scf, mcscf, mrpt

def caspt2_calculation(symbol="Be", spin=0, nelecas=4, ncas=8, basis="cc-pvdz"):
    """
    CASSCF + CASPT2 розрахунок
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    print(f"\nCASPT2 розрахунок {symbol}")
    print(f"Активний простір: ({nelecas},{ncas})")
    print("=" * 70)

    # HF
    if spin == 0:
        mf = scf.RHF(mol)
    else:

```

```

mf = scf.UHF(mol)

mf.verbose = 0
e_hf = mf.kernel()

print(f"HF енергія: {e_hf:.10f} Ha")

# CASSCF
mc = mcscf.CASSCF(mf, ncas, nelecas)
mc.verbose = 0
mc.conv_tol = 1e-9
e_casscf = mc.kernel()[0]

print(f"CASSCF енергія: {e_casscf:.10f} Ha")
print(f"Статична кореляція: {(e_casscf - e_hf) * 1000:.6f} мHa")

# CASPT2
print("\nCASPT2 розрахунок...")
try:
    # В PySCF NEVPT2 більш стабільний ніж CASPT2
    from pyscf.mrpt import NEVPT

    nevpt2 = NEVPT(mc)
    e_corr_pt2 = nevpt2.kernel()
    e_caspt2 = e_casscf + e_corr_pt2

    print(f"NEVPT2 кореляція: {e_corr_pt2 * 1000:.6f} мHa")
    print(f"CASSCF+NEVPT2 енергія: {e_caspt2:.10f} Ha")

    # Розбиття кореляції
    total_corr = e_caspt2 - e_hf
    static_corr = e_casscf - e_hf
    dynamic_corr = e_corr_pt2

    print(f"\nРозбиття кореляційної енергії:")
    print(f"Повна: {total_corr * 1000:.6f} мHa")
    print(
        f"Статична (CASSCF): {static_corr * 1000:.6f} мHa "
        f"({abs(static_corr / total_corr) * 100:.1f}%) "
    )
    print(
        f"Динамічна (NEVPT2): {dynamic_corr * 1000:.6f} мHa "
        f"({abs(dynamic_corr / total_corr) * 100:.1f}%) "
    )

    return e_hf, e_casscf, e_caspt2

except Exception as e:
    print(f"CASPT2/NEVPT2 недоступний: {str(e)}")
    return e_hf, e_casscf, None

# Приклади
caspt2_calculation("Be", spin=0, nelecas=4, ncas=8)
caspt2_calculation("C", spin=2, nelecas=4, ncas=4)

```

6.5.8. Переваги та недоліки CASSCF

Переваги:

- Правильно описує статичну кореляцію
- Може розрахувати декілька станів одночасно

- Варіаційний метод
- Підходить для розриву зв'язків, збуджених станів
- Дає природні орбіталі та заселеності

Недоліки:

- Вибір активного простору не завжди очевидний
- Експоненційне масштабування з розміром активного простору
- Не враховує динамічну кореляцію (потрібен CASPT2)
- Може бути проблеми з конвергенцією
- Потребує досвіду для правильного використання

Коли використовувати CASSCF:

- T1 діагностика > 0.05 (сильна багатоконфігураційність)
- Перехідні метали з близькими d-орбіталами
- Збуджені стани атомів
- Діелектронні системи
- Відкрито-оболонкові синглети

6.5.9. Ієрархія методів

Post-HF методи утворюють ієрархію за точністю та обчислювальною складністю:

1. **HF** — базовий рівень, без кореляції
2. **MP2** — $\mathcal{O}(N^5)$ — найпростіша кореляція
3. **MP3, MP4** — $\mathcal{O}(N^6), \mathcal{O}(N^7)$ — вищі порядки теорії збурень
4. **CCSD** — $\mathcal{O}(N^6)$ — надійна динамічна кореляція
5. **CCSD(T)** — $\mathcal{O}(N^7)$ — "золотий стандарт" квантової хімії
6. **Full CI** — $\mathcal{O}(e^N)$ — точний розв'язок (у межах базису)

6.5.10. Важливість кореляції для різних властивостей

Таблиця 6.6. Вплив електронної кореляції на різні властивості

Властивість	Важливість кореляції	Рекомендований метод
Абсолютні енергії	Помірна	MP2, DFT
Енергії іонізації	Висока	CCSD(T)
Електронна спорідненість	Дуже висока	CCSD(T) + дифузні
Енергії збудження	Висока	EOM-CCSD, TD-DFT
Відстані між станами	Висока	CASPT2, MRCI
Дисоціація	Критична	CASSCF, MRCI
Слабкі взаємодії	Критична	CCSD(T), MP2-F12

code_12.py

```

from pyscf import gto, scf, mp, cc, ci
import numpy as np

def correlation_energy_demo(symbol, spin, basis="cc-pvdz"):
    """
    Демонстрація кореляційної енергії
    """

    mol = gto.M(atom=f"{symbol} 0 0 0", basis=basis, spin=spin, verbose=0)

    print(f"\nКореляційна енергія атома {symbol} (базис: {basis})")
    print("=" * 70)

    # HF розрахунок
    if spin == 0:
        mf = scf.RHF(mol)
    else:
        mf = scf.UHF(mol)

    mf.verbose = 0
    mf.conv_tol = 1e-10
    e_hf = mf.kernel()

    print(f"HF енергія:           {e_hf:.8f} Ha")

    # MP2
    if spin == 0:
        mp2 = mp.MP2(mf)
    else:
        mp2 = mp.UMP2(mf)

    mp2.verbose = 0
    e_mp2, t2 = mp2.kernel()
    e_total_mp2 = e_hf + e_mp2

    print(f"MP2 кореляція:           {e_mp2:.8f} Ha")
    print(f"MP2 повна енергія:       {e_total_mp2:.8f} Ha")

    # CCSD (якщо атом не дуже великий)
    if mol.nelectron <= 10:
        if spin == 0:
            mycc = cc.CCSD(mf)
        else:
            mycc = cc.UCCSD(mf)

        mycc.verbose = 0
        e_ccsd, t1, t2 = mycc.kernel()
        e_total_ccsd = e_hf + e_ccsd

        print(f"CCSD кореляція:           {e_ccsd:.8f} Ha")
        print(f"CCSD повна енергія:       {e_total_ccsd:.8f} Ha")

        # CCSD(T)
        e_t = mycc.ccsd_t()
        e_total_ccsdt = e_total_ccsd + e_t

        print(f"(T) корекція:             {e_t:.8f} Ha")
        print(f"CCSD(T) повна енергія: {e_total_ccsdt:.8f} Ha")
    else:
        print("CCSD пропущено (атом занадто великий для демо)")

    print("=" * 70)

```



```
# Аналіз
print(f"\nАналіз:")
print(f"Кореляція складає {abs(e_mp2 / e_hf) * 100:.2f}% від HF енергії")

if mol.nelectron <= 10:
    print(f"MP2 відновлює {abs(e_mp2 / e_ccsd) * 100:.1f}% CCSD кореляції")

# Приклади
correlation_energy_demo("He", spin=0)
correlation_energy_demo("Be", spin=0)
correlation_energy_demo("C", spin=2)
correlation_energy_demo("Ne", spin=0)
```

6.5.11. Порівняння методів для He

Розглянемо найпростішу багатоелектронну систему — атом Гелію:

[code_13.py](#)

```
from pyscf import gto, scf, mp, cc, fci
import numpy as np
import matplotlib.pyplot as plt

def helium_correlation_study():
    """
    Детальне дослідження кореляції у атомі He
    """

    # Різні базисні набори
    basis_sets = ["cc-pvdz", "cc-pvtz", "cc-pvqz", "cc-pv5z"]

    results = {"HF": [], "MP2": [], "CCSD": [], "CCSD(T)": [], "FCI": []}

    print("Збіжність до базисної межі для He")
    print("=" * 80)
    print(
        f"{'Базис':12s} {'HF':15s} {'MP2':15s} {'CCSD':15s} {'CCSD(T)':15s} {'FCI':15s}"
    )
    print("-" * 80)

    for basis in basis_sets:
        mol = gto.M(atom="He 0 0 0", basis=basis, verbose=0)

        # HF
        mf = scf.RHF(mol)
        mf.verbose = 0
        mf.conv_tol = 1e-12
        e_hf = mf.kernel()
        results["HF"].append(e_hf)

        # MP2
        mymp2 = mp.MP2(mf)
        mymp2.verbose = 0
        e_mp2_corr, _ = mymp2.kernel()
        e_mp2 = e_hf + e_mp2_corr
        results["MP2"].append(e_mp2)

        # CCSD
        mycc = cc.CCSD(mf)
        mycc.verbose = 0
```

```

e_ccsd_corr, _, _ = mycc.kernel()
e_ccsd = e_hf + e_ccsd_corr
results["CCSD"].append(e_ccsd)

# CCSD(T)
e_t = mycc.ccsd_t()
e_ccsdt = e_ccsd + e_t
results["CCSD(T)"].append(e_ccsdt)

# FCI (точний у даному базисі)
myfci = fci.FCI(mf)
e_fci = myfci.kernel()[0]
results["FCI"].append(e_fci)

print(
    f"{basis:12s} {e_hf:15.10f} {e_mp2:15.10f} "
    f"{e_ccsd:15.10f} {e_ccsdt:15.10f} {e_fci:15.10f}"
)

print("=" * 80)

# Експериментальне значення
e_exp = -2.90372 # Ha
print(f"\nЕкспериментальна енергія He: {e_exp:.10f} Ha")

# Похибки для найбільшого базису
print(f"\nПохибки (cc-pv5z):")
for method in results.keys():
    error = (results[method][-1] - e_exp) * 1000 # mHa
    print(f"{method:10s}: {error:8.4f} mHa")

# Графік збіжності
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Абсолютні енергії
x = np.arange(len(basis_sets))
for method, energies in results.items():
    ax1.plot(x, energies, "o-", label=method, linewidth=2, markersize=8)

ax1.axhline(y=e_exp, color="red", linestyle="--", linewidth=2, label="Експеримент")
ax1.set_xticks(x)
ax1.set_xticklabels(basis_sets, rotation=45)
ax1.set_xlabel("Базисний набір", fontsize=12)
ax1.set_ylabel("Енергія (Ha)", fontsize=12)
ax1.set_title("Збіжність енергії He", fontsize=14)
ax1.legend()
ax1.grid(True, alpha=0.3)

# Кореляційна енергія
for method in ["MP2", "CCSD", "CCSD(T)", "FCI"]:
    corr_energies = [
        results[method][i] - results["HF"][i] for i in range(len(basis_sets))
    ]
    ax2.plot(x, corr_energies, "o-", label=method, linewidth=2, markersize=8)

ax2.set_xticks(x)
ax2.set_xticklabels(basis_sets, rotation=45)
ax2.set_xlabel("Базисний набір", fontsize=12)
ax2.set_ylabel("Кореляційна енергія (Ha)", fontsize=12)
ax2.set_title("Кореляційна енергія He", fontsize=14)
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()

```

6. Пост-Хартрі-Фоківські методи

```
plt.savefig("he_correlation_convergence.pdf")
plt.show()
```

```
helium_correlation_study()
```

6.6. Практичні завдання

6.6.1. Завдання 1: Порівняння методів для He

```
"""
ЗАВДАННЯ 1: Для атома Гелію розрахуйте енергії методами
HF, MP2, CCSD, CCSD(T) та FCI з базисами cc-pVDZ, cc-pVTZ,
cc-pVQZ.

1) Побудуйте графік збіжності кожного методу до базисної межі
2) Екстраполуйте до CBS за формулою  $E(X) = E_{\text{CBS}} + A/X^3$ 
3) Порівняйте з експериментальним значенням -2.90372 Ha
4) Яка частка кореляційної енергії відновлюється кожним методом?

Базиси: cc-pVDZ, cc-pVTZ, cc-pVQZ
"""

# Ваш код тут
```

Завдання 2: T1 діагностика для другого періоду

```
"""
ЗАВДАННЯ 2: Розрахуйте T1 діагностику для всіх атомів
другого періоду (Li-Ne) з базисом cc-pVDZ.

1) Для яких атомів  $T1 > 0.02$  (багатоконфігураційні)?
2) Чи є кореляція між T1 та положенням у періоді?
3) Порівняйте T1 для основного та збудженого спінового
стану Карбону ( $^3P$  vs  $^1D$ )
4) Побудуйте графік залежності T1 від атомного номера

Базис: cc-pVDZ
"""

# Ваш код тут
```

Завдання 3: Size-extensivity тест

```
"""
ЗАВДАННЯ 3: Дослідіть size-extensivity для атомів Ne.

Розрахуйте методами HF, MP2, CISD, CCSD:
1) Енергію одного атома Ne
2) Енергію двох атомів Ne на відстані 100 Bohr
3) Обчисліть похибку:  $E(2 \times \text{Ne}) - 2 \times E(\text{Ne})$ 

Питання:
```

```
- Який метод є size-extensive?
- Яка похибка для CISD (у mHa)?
- Чому MP2 є size-extensive, а CISD - ні?
```

```
Базис: cc-pVTZ
"""
```

```
# Ваш код тут
```

Завдання 4: CASSCF для перехідного металу

```
"""
```

ЗАВДАННЯ 4: CASSCF аналіз атома Титану (Ti).

Ti: [Ar] 3d² 4s², основний стан ³F

- 1) Розрахуйте UHF, CCSD та CASSCF(4,5) енергії (4 електрони у 5 d-орбіталах)
- 2) Проаналізуйте заселеності природних орбіталей
- 3) Обчисліть статичну кореляцію (CASSCF - HF)
- 4) Спробуйте різні активні простори: (4,5), (4,6), (6,11) Який найкращий?

```
Базис: def2-SVP
"""
```

```
# Ваш код тут
```

Завдання 5: Енергії збудження

```
"""
```

ЗАВДАННЯ 5: Розрахуйте енергію збудження ³P → ¹D для атома Карбону.

Використайте методи:

- 1) ΔSCF (окремі розрахунки для кожного стану)
- 2) State-averaged CASSCF
- 3) EOM-CCSD (якщо доступно)

Порівняйте з експериментальним значенням 1.26 eV.

Який метод дає найточніший результат?

```
Базис: aug-cc-pVTZ
"""
```

```
# Ваш код тут
```

Завдання 6: Кореляційна енергія vs Z

```
"""
```

ЗАВДАННЯ 6: Дослідіть залежність кореляційної енергії від атомного номера.

Розрахуйте MP2 кореляційну енергію для:

- Благородних газів: He, Ne, Ar, Kr
- У абсолютних величинах (mHa)
- У відсотках від повної енергії

Побудуйте графіки:

- 1) $|E_{corr}|$ vs Z
- 2) $|E_{corr}|/|E_{HF}|$ vs Z

Яка тенденція спостерігається?

Базис: cc-pVTZ

""

Ваш код тут

6.7. Резюме

У цьому розділі ми детально вивчили Post-Hartree-Fock методи для врахування електронної кореляції:

6.7.1. Основні методи та їх характеристики

Таблиця 6.7. Порівняння Post-HF методів

Метод	Складність	Size-ext.	Варіац.	Точність	БК?
MP2	$O(N^5)$	Так	Ні	Помірна	Ні
MP3/MP4	$O(N^6/N^7)$	Так	Ні	Добра	Ні
CISD	$O(N^6)$	Ні	Так	Помірна	Ні
CCSD	$O(N^6)$	Так	Ні	Добра	Ні
CCSD(T)	$O(N^7)$	Так	Ні	Відмінна	Ні
CASSCF	$O(e^{n_{act}})$	Так	Так	Статична	Так
CASPT2	$O(e^{n_{act}})$	Так	Ні	Відмінна	Так
Full CI	$O(e^N)$	Так	Так	Точна	Так

БК = Багатоконфігураційний

6.7.2. Ключові висновки

1. Електронна кореляція

- Становить 1-5% від повної енергії, але критична для точних розрахунків
- Поділяється на динамічну (короткодіючу) та статичну (близькі стани)
- HF і DFT не враховують кореляцію повністю

2. MP2 — найпростіший post-HF метод

- Швидкий ($O(N^5)$), size-extensive
- Добре працює для замкнених оболонок з великим HOMO-LUMO gap
- Відновлює 80-95% кореляційної енергії
- Не виправляє забруднення спіном UHF
- Потребує великих базисів для збіжності

3. CCSD(T) — "золотий стандарт"

- Найточніший одноконфігураційний метод
- Size-extensive, відновлює >99% кореляції
- Повільний ($O(N^7)$), але надійний
- Виправляє забруднення спіном
- Обмежений розміром системи (до 20-30 атомів)

4. T1 діагностика

- $T1 < 0.02$: одноконфігураційна система (CCSD надійний)
- $0.02 < T1 < 0.05$: слабка багатоконфігураційність
- $T1 > 0.05$: потрібен CASSCF/MRCI

5. CI методи

- Варіаційні (енергія завжди вище точної)
- Full CI — точний у межах базису, але непрактичний
- CISD не є size-extensive (на відміну від CCSD)
- Корисні для аналізу хвильової функції

6. CASSCF — для багатоконфігураційних систем

- Необхідний коли $T1 > 0.05$ або близькі стани
- Вибір активного простору критичний
- Враховує статичну кореляцію
- Потребує CASPT2/NEVPT2 для динамічної кореляції
- Ідеальний для перехідних металів, збуджених станів

6.7.3. Рекомендації по використанню**6.7.4. Типові помилки**

1. Використання MP2 для систем з малим gap — може давати нефізичні результати
2. Забування перевірки T1 діагностики — CCSD може бути ненадійним
3. Занадто малий активний простір у CASSCF — не врахує всю статичну кореляцію
4. Занадто великий активний простір — неможливо розрахувати
5. Недостатній базис — post-HF методи дуже чутливі до базису

Таблиця 6.8. Вибір методу залежно від задачі

Задача	Рекомендований метод	Альтернатива
Швидка оцінка кореляції	MP2	DFT
Точні енергії (1 kcal/mol)	CCSD(T)	—
Замкнені оболонки	CCSD(T), MP2	DFT
Відкриті оболонки	UCCSD(T)	ROHF-CCSD
Енергії збудження	EOM-CCSD, CASSCF	TD-DFT
Перехідні метали	CASSCF, CASPT2	DFT+U
Багатоконфігураційні	CASSCF/CASPT2	MRCI
Еталонні розрахунки	CCSD(T)/CBS	Full CI
Великі системи	MP2, DFT	DLPNO-CCSD(T)

6. **Ігнорування size-extensivity** — CISD не підходить для енергій дисоціації

7. **CASSCF без CASPT2** — відсутня динамічна кореляція

6.7.5. Практичні поради

1. Завжди починайте з HF розрахунку та перевірте конвергенцію
2. Для нових систем спочатку протестуйте на малому базисі
3. Перевіряйте T1 діагностику після CCSD
4. Для CASSCF: починайте з малого активного простору, збільшуйте поступово
5. Аналізуйте природні орбіталі для вибору активного простору
6. Використовуйте симетрію коли можливо
7. Для екстраполяції до CBS потрібні принаймні 3 базиси
8. Зберігайте проміжні результати (checkpoint файли)

6.7.6. Обчислювальні ресурси

Таблиця 6.9. Орієнтовний час розрахунку (відносно HF)

Метод	He (DZ)	Ne (TZ)	Ar (DZ)	Пам'ять
HF	1×	1×	1×	N^2
MP2	2×	5×	10×	N^4
CCSD	10×	50×	200×	N^4
CCSD(T)	20×	200×	1000×	N^4
CASSCF(4,8)	5×	—	—	$e^{n_{act}}$

Примітка: реальний час залежить від якості початкового наближення та конвергенції

6.7.7. Подальше вивчення

Для поглибленого розуміння post-HF методів рекомендуємо:

- **Книги:**

- T. Helgaker et al., "Molecular Electronic-Structure Theory"
- I. Shavitt, R. J. Bartlett, "Many-Body Methods in Chemistry and Physics"
- R. J. Bartlett, M. Musiał, "Coupled-cluster theory in quantum chemistry"

- **Огляди:**

- CCSD(T): Reviews in Computational Chemistry, Vol. 14
- CASSCF: Chemical Reviews 2012, 112, 108
- Post-HF methods: Wiley Interdisciplinary Reviews: Computational Molecular Science

- **Документація PySCF:**

- <https://pyscf.org/user/mp.html> — MP2, MP3
- <https://pyscf.org/user/cc.html> — Coupled Cluster
- <https://pyscf.org/user/mcscf.html> — CASSCF, CASPT2
- <https://pyscf.org/user/fci.html> — Full CI

6.7.8. Що далі?

У наступному розділі ми розглянемо аналіз результатів квантово-хімічних розрахунків: орбітальні енергії, густини, заселеності, спектроскопічні властивості та візуалізацію даних.

Література

Основна література

1. *Jensen F.* Introduction to Computational Chemistry. — 3rd ed. — Wiley, 2017. — 661 p. — ISBN 1118825993.
2. *Levine I. N.* Quantum Chemistry. — 7th ed. — Pearson, 2014. — 714 p. — ISBN 978-0321803450.

Додаткові посилання

1. [Basis Sets](https://gaussian.com/basissets/). — URL: <https://gaussian.com/basissets/>.
2. *Ho M., Hernández-Perez J. M.* [Evaluation of Gaussian Molecular Integrals. I Overlap Integrals](#) // The Mathematica Journal. — 2012. — Vol. 14.
3. *Ho M., Hernández-Perez J. M.* [Evaluation of Gaussian Molecular Integrals. II. Kinetic-Energy Integrals](#) // The Mathematica Journal. — 2013. — Vol. 15.
4. *Ho M., Hernández-Perez J. M.* [Evaluation of Gaussian Molecular Integrals. III. Nuclear-Electron Attraction Integrals](#) // The Mathematica Journal. — 2014. — Vol. 16.
5. [Simple Quantum Chemistry: Hartree-Fock in Python](https://nznano.blogspot.com/2018/03/simple-quantum-chemistry-hartree-fock.html). — URL: <https://nznano.blogspot.com/2018/03/simple-quantum-chemistry-hartree-fock.html>.